

Noise and Uncertainty in Molecular Property Prediction:

Robust Models and Adaptive Representations

Adelaide Punt

St Catherine's College

University of Oxford

A thesis submitted for the degree of

Doctor of Philosophy

October 2026

Declaration of AI Use

During this doctorate I used Claude, a large language model developed by Anthropic (https://claude.ai), [model versions and dates of use]. I used it for three purposes. First, I used it to help write, test and debug software, including parts of the code that ran the experiments reported in this thesis. Second, I used it to draft and edit parts of the text and to typeset it in LaTeX. Third, I used it to search for literature and to check citations against their sources. I reviewed, edited and verified all of its output before using it. [No data, results or figures were generated by the tool; every reported number comes from code that I ran and checked.] The research questions, experimental design, analysis and conclusions are my own. [My supervisors were aware of this use.] I take full responsibility for the content of this thesis, including any errors.

Acknowledgements

I would like to thank my supervisors, Garrett Morris at the University of Oxford and Thierry Hanser and Stéphane Werner at Lhasa Limited, for their guidance, their time and their patience throughout this project.

I also thank the EPSRC (EP/L015803/1) as well as my industrial partner Lhasa for their financial support.

Thank you to my parents and grandparents for their support, and to my cat, [name].

Abstract

Quantitative structure–activity relationship (QSAR) models are widely used to screen compounds in drug discovery by predicting their molecular properties. These models train on experimental measurements which often contain inherent noise arising from assay variability, measurement error, and heterogeneous data sources. While many studies investigating label noise in cheminformatics model that noise as Gaussian error, in this thesis we use seven noise conditions that mimic the variety of possible experimental errors by varying the distribution of the artificial noise, adding more noise to specific scaffold families, and clipping labels to represent an assay limit. We then investigate how the choice of model and molecular representation affects how robust a QSAR model is to noise. We selected the HOMO–LUMO gap of QM9, as its labels are computed rather than measured, meaning they are clean. We also conducted experiments on ADMET datasets with experimentally obtained labels. We measured noise robustness as the proportion of predictive performance that was retained across increasing amounts of artificial label noise. We found that model family was the dominant factor in noise robustness, while molecular representation contributed more to predictive performance than to noise robustness. The two most reliable models in the presence of noisy labels were random forests and quantile regression forests. Other models only performed well under certain conditions. Gaussian processes had the highest predictive performance on clean and randomly distributed noised labels but struggled under noise shared within a scaffold family. Mean predicted uncertainty rose with injected label noise for most probabilistic models, but per-sample uncertainty estimates did not correlate with injected noise. The exception to this was when we clipped labels at a certain limit, and the models were most confident on those clipped labels.

We then take a turn to examine the similarity assumption and how strongly it holds under standard feature sets. We measure how many features within a standard molecular representation do not vary on a given dataset and argue that

pairwise comparisons between feature vectors are dominated by features irrelevant to the property. Similarity and distance measures, and by association the similarity assumption, are core to many downstream applications, including but not limited to learning with noisy labels and noise mitigation. We propose the adaptive hybrid representation, which selects the most important features for a given model and prediction task from several base representations under a feature budget.

We test the hybrid in three settings: as the input to a predictive model, as the space in which similarity between molecules is measured, and as the input to methods for learning with noisy labels. In preliminary runs, the hybrid matched or improved on the best single representation as a model input, and predicting a molecule's property from its nearest neighbors was more accurate in a hybrid than in the representations it was built from. Methods for learning with noisy labels also scored higher with a hybrid as input than with a one-hot encoding. Which method helped depended on the type of noise, the dataset and the feature space, but correcting labels clipped at an assay limit helped every model we tested, whereas deleting the clipped molecules made every model worse.

Before choosing a model or a method when training on noisy labels, it is important consider where the labels came from and what kind of noise they are likely to carry. We recommend choosing features for the task rather than take one representation whole.

Contents

Declaration of AI Use ii

Acknowledgements iii

Abstract iv

List of Abbreviations xii

1 Introduction 1
- 1.1 Outline of the thesis and contributions . . . 4
- 1.2 Collaborations . . . 6

2 Molecular Representations, Models and Label Noise 7
- 2.1 Overview . . . 7
- 2.2 Molecular representations in QSAR . . . 7
- 2.3 Models for molecular property prediction . . . 12
  - 2.3.1 Tree ensembles . . . 15
  - 2.3.2 Support vector machines . . . 15
  - 2.3.3 Gaussian processes . . . 15
  - 2.3.4 Bayesian neural networks . . . 16
- 2.4 Molecular similarity and activity cliffs . . . 17
- 2.5 Noise in molecular data . . . 19
  - 2.5.1 Artificial noise injection . . . 21
  - 2.5.2 Learning with noisy labels . . . 24
- 2.6 NoiseInject: a benchmarking framework . . . 26
  - 2.6.1 Package description . . . 26
- 2.7 Evaluation methodology . . . 30
- 2.8 Conclusions . . . 30

3 Noise Robustness Across Molecular Representations and Model Architectures 31
- 3.1 Introduction . . . 31
- 3.2 Methods . . . 32
  - 3.2.1 Datasets . . . 32
  - 3.2.2 Representations . . . 33
  - 3.2.3 Models . . . 34
  - 3.2.4 Noise conditions and levels . . . 36
  - 3.2.5 QM9 data pipeline . . . 36
  - 3.2.6 Uncertainty . . . 38

3.2.7 Evaluation . . . 40
3.3 Variance decomposition . . . 43
3.4 Predictive performance and noise robustness . . . 46
3.5 Artificial noise conditions . . . 47
3.6 Robustness on the three assay datasets . . . 52
3.7 Probabilistic and deterministic counterparts . . . 57
3.8 Uncertainty under increasing label noise . . . 60
3.8.1 Aleatoric and epistemic attribution . . . 61
3.8.2 Per-sample uncertainty and label noise . . . 64
3.9 Conclusions . . . 65

4 Limitations of the Similarity Assumption 70
4.1 Introduction . . . 70
4.2 Flaws in standard representations . . . 72
4.2.1 Constant and near-constant features . . . 73
4.2.2 Molecules with identical feature vectors . . . 73
4.3 Flaws in similarity and other pairwise computations . . . 78
4.3.1 Distance metrics for continuous molecular descriptors . . . 82
4.4 Consequences for similarity-based methods . . . 83
4.4.1 Exploratory study of learning with noisy labels . . . 84
4.4.2 Noise detection by neighbors . . . 85
4.5 Conclusions . . . 87

5 Building Adaptive Hybrid Representations 90
5.1 Introduction . . . 90
5.1.1 Combining representations . . . 91
5.1.2 The adaptive hybrid representation . . . 93
5.1.3 Blueprints and their chemical interpretation . . . 94
5.1.4 Antibodies and beyond . . . 95
5.2 Methods: the hybrid search . . . 96
5.2.1 From base representations to a hybrid . . . 96
5.2.2 The search space . . . 100
5.2.3 Augmentation . . . 106
5.2.4 Search grid per dataset . . . 108
5.3 Methods: experimental setup . . . 111
5.3.1 Datasets and splitting . . . 111
5.3.2 Representations by domain . . . 114
5.3.3 Models and metrics . . . 123
5.3.4 Comparators, controls and ablations . . . 124
5.3.5 Transfer, mutational scans and separate sites . . . 128
5.4 Results: comparison with single representations and ensembles . . . 131
5.5 Results: is the search needed? . . . 136
5.6 Results: sources of the gain . . . 137
5.7 Results: blueprints on specific datasets . . . 139
5.8 Results: transfer across datasets . . . 139
5.9 Results: data held at separate sites . . . 142
5.10 Discussion . . . 143
5.11 Limitations . . . 144
5.12 Conclusions . . . 145

6 Hybrid Representations for Similarity and Label Noise **146**
6.1 Introduction . . . 146
6.2 Methods . . . 147
6.2.1 Similarity in a Hybrid . . . 147
6.2.2 Noise Conditions . . . 149
6.2.3 Noise Mitigation Methods . . . 150
6.3 Results: Similarity . . . 154
6.4 Results: Noise Detection and Estimation . . . 156
6.5 Results: Learning with Noisy Labels by Noise Type . . . 157
6.5.1 Censoring: Correction of Clipped Labels . . . 157
6.5.2 Gaussian Noise: Methods for Learning with Noisy Labels . . . 158
6.5.3 Real Labels: Cleaning without Injected Noise . . . 162
6.5.4 Comparison across Noise Types . . . 166
6.6 Conclusions . . . 167

7 Discussion and Conclusions **168**
7.1 Summary . . . 168
7.2 Dataset Dependence of the Best Method . . . 170
7.3 Simple Models and Small Feature Sets . . . 171
7.4 Limitations . . . 173
7.5 Future Directions . . . 174
7.6 Conclusions . . . 175

Summary of Research Contributions **177**

List of Figures

2.1 Three representations of one molecule. 8
2.2 Nodes of a deterministic and a Bayesian neural network. 16
2.3 Label distributions under the seven noise conditions. 23
2.4 Evaluation of a noise mitigation method. 25

3.1 In-memory and memory-mapped data loading. 38
3.2 Variance decomposition of robustness and predictive performance on QM9. 44
3.3 Variance shares of clean performance, noisy performance and robustness on all four datasets. 45
3.4 Predictive performance against Gaussian label noise on QM9. 47
3.5 Clean predictive performance against robustness on QM9. 48
3.6 Robustness of each base model on each representation on QM9. 48
3.7 Robustness under every noise condition on a subset of models on QM9. 51
3.8 Random forest under each noise condition on QM9. 52
3.9 Rank of each base model at each noise level on QM9. 53
3.10 Predictive performance against noise level on every dataset. 58
3.11 Robustness on the three assay datasets. 59
3.12 Robustness rank of each model on QM9 and the three assay datasets. 60
3.13 Change from replacing a model with its probabilistic counterpart. . . 61
3.14 Aleatoric and epistemic uncertainty against label noise on QM9. . . . 63
3.15 Aleatoric uncertainty against the added noise. 64

4.1 Shares of constant, near-constant and varying features per representation. 74
4.2 Two hERG K_i stereoisomers with identical ECFP4 vectors. 77
4.3 Absolute pK_i difference against pair distance on hERG K_i. 82

5.1 Schematic overview of the hybrid search. 97
5.2 The three levels of splitting. 98

List of Tables

2.1 Sources of label noise in bioactivity data. 21
2.2 The seven noise conditions and their parameters. 24

3.1 Variance decomposition by noise condition on QM9. 45
3.2 Robustness under censoring and under Gaussian noise on QM9. . . . 50
3.3 Change in robustness from Gaussian to grouped-shifted noise on QM9. 50
3.4 Robustness under the noise distributions on the three assay datasets. 54
3.5 Rise in mean predicted uncertainty under Gaussian noise. 62
3.6 Which model to prefer under each noise condition. 69

4.1 Constant and near-constant features per representation, part 1. . . . 75
4.2 Constant and near-constant features per representation, part 2. . . . 75
4.3 Molecules that share a feature vector with another molecule. 78
4.4 All-pairs correlation, five-nearest-neighbor prediction and activity cliffs per dataset. 82
4.5 Agreement between representations per dataset. 83
4.6 Recovery with neighbor-based sample weights by representation. . . . 87
4.7 Flaws of standard representations and similarity, and the requirement each sets. 89

5.1 Methods for combining or reducing representations. 93
5.2 Settings of the importance methods. 103
5.3 The five allocation methods searched. 104
5.4 Worked example of the four fixed-share allocation methods. 104
5.5 The three augmentation conditions. 108
5.6 Values the hybrid search can take for each setting. 109
5.7 Search grid of each dataset. 109
5.8 What each cut removes from the hERG K_i search. 110
5.9 Datasets and their best single representations. 113
5.10 Representations used for small molecules. 116
5.11 Representations used for chemical reactions. 117
5.12 Representations used for antibodies. 118
5.13 Representations used for nucleotide sequences. 119
5.14 Representations used for cancer patients. 120
5.15 Base and augmentation representations per dataset. 121
5.16 Default hybrid and best single representation on hERG K_i. 133
5.17 Allocation settings and single representations on hERG K_i. 134
5.18 Allocation settings over two or three representations on hERG K_i. . . 135
5.19 Representations drawn on by the two- and three-representation hybrids. 136
5.20 Search settings against base representations on [dataset 1 136

5.21 Search settings against base representations on [dataset 2 137
5.22 Search settings against base representations on [dataset 3 137
5.23 The default hybrid against PCA, recursive feature elimination and random features on hERG K_i 138
5.24 The searched hybrid against PCA, recursive feature elimination and random features on nanobody thermostability 138
5.25 RMSE of the default hybrid against budget on hERG K_i. 139
5.26 Reusing search settings from LogD on hERG K_i. 141
5.27 Search settings found on LogD. 141
5.28 Features selected on LogD and copied to hERG K_i. 141
5.29 Settings of the hybrids searched on hERG K_i. 142
5.30 Reusing LogD settings with the DNN on hERG K_i, per fold. 142
5.31 Best single representation per fold on hERG K_i. 142
5.32 MCC with molecules kept at three hERG sites or pooled 143

6.1 Noise magnitude estimators. 152
6.2 Label smoothing and sample weighting rules. 153
6.3 Nearest-neighbor prediction on hERG K_i 155
6.4 Nearest-neighbor prediction on hERG K_i with importance weighting . 155
6.5 Activity difference to the nearest neighbors on hERG K_i 156
6.6 Correcting clipped training labels on hERG K_i with ECFP4 158
6.7 Correcting clipped labels on hERG K_i in the two hybrids 159
6.8 Correcting clipped labels on hERG K_i in single representations 159
6.9 Representations in the hybrids of the clipped-label experiment 159
6.10 Methods for LNL on nanobody thermostability 161
6.11 Pair down-weighting on the recorded hERG K_i labels 163
6.12 C-Mixup and target-noise bagging on the recorded hERG K_i labels . 164
6.13 Down-weighting suspect molecules on the recorded hERG K_i labels . 165
6.14 Cleaning training labels before the hybrid is built on humanness . . . 166

List of Abbreviations

AC Activity cliff
ADME Absorption, distribution, metabolism, and excretion
AI Artificial intelligence
ANOVA Analysis of variance
ASCII American Standard Code for Information Interchange
AT Adenine–thymine
AUC_{norm} Normalized area under the retention curve
AUROC Area under the receiver operating characteristic curve
BLOSUM Blocks substitution matrix
BNN Bayesian neural network
BRCA Breast invasive carcinoma
CASH Combined algorithm selection and hyperparameter optimization
CDR Complementarity-determining region
CGR Condensed graph of reaction
CIMP CpG island methylator phenotype
CLS Classification token
CpG Cytosine–phosphate–guanine
CPU Central processing unit
CSV Comma-separated values
DeepLIFT Deep Learning Important FeaTures
DNA Deoxyribonucleic acid
DNN Deep neural network
$\mathbf{EC_{50}}$ Half-maximal effective concentration
ECE Expected calibration error
ECFP Extended-connectivity fingerprint
ECFP4 Extended-connectivity fingerprint, diameter 4
ESM Evolutionary Scale Modeling
FCFP4 Functional-class fingerprint, diameter 4
FcRn Neonatal Fc receptor
FLuID Federated learning using information distillation
GC Guanine–cytosine
GCN Graph convolutional network
GIN Graph isomorphism network
GNN Graph neural network
GP Gaussian process
GPL GNU General Public License
GPU Graphics processing unit

H3K4me3 Trimethylation of lysine 4 on histone H3
HER2 Human epidermal growth factor receptor 2
hERG Human ether-à-go-go-related gene
HLM Human liver microsomes
HOMO Highest occupied molecular orbital
HRD Homologous recombination deficiency
IC_{50} Half-maximal inhibitory concentration
IMGT International ImMunoGeneTics information system
InChIKey Hashed International Chemical Identifier
JSON JavaScript Object Notation
KL Kullback–Leibler
kNN k-nearest neighbors
LightGBM Light Gradient Boosting Machine
LM Language model
LNL Learning with noisy labels
LOF Local outlier factor
LogD Logarithm of the distribution coefficient
LOH Loss of heterozygosity
LST Large-scale state transitions
LUAD Lung adenocarcinoma
LUMO Lowest unoccupied molecular orbital
LUSC Lung squamous cell carcinoma
MACCS Molecular ACCess System
MCC Matthews correlation coefficient
MCS Maximum common substructure
MDR1–MDCK Multidrug resistance protein 1 in Madin–Darby canine kidney cells
MHG-GNN Molecular Hypergraph Grammar Graph Neural Network
miRNA MicroRNA
ML Machine learning
MLP Multi-layer perceptron
mRMR Minimum redundancy, maximum relevance
NGBoost Natural gradient boosting
NLP Natural language processing
NN Neural network
OPIG Oxford Protein Informatics Group
PAM50 Prediction Analysis of Microarray 50
PCA Principal component analysis
PDV Physicochemical descriptor vector
PLS Partial least squares
QRF Quantile regression forest
QSAR Quantitative structure–activity relationship
RAM Random-access memory
RBF Radial basis function
ReLU Rectified linear unit
RF Random forest
RLM Rat liver microsomes
RMSE Root mean squared error

RNA Ribonucleic acid
RNA-seq RNA sequencing
RPPA Reverse-phase protein array
SALI Structure–activity landscape index
SELFIES Self-referencing embedded strings
SHAP SHapley Additive exPlanations
SMARTS SMILES arbitrary target specification
SMILES Simplified molecular-input line-entry system
SNS Sort & Slice
SVM Support vector machine
t-SNE t-distributed stochastic neighbor embedding
TAI Telomeric allelic imbalance
TCGA The Cancer Genome Atlas
Th1 T helper type 1
Th2 T helper type 2
TNF Tetranucleotide frequency
UCSC University of California, Santa Cruz
UMAP Uniform manifold approximation and projection
USR Ultrafast shape recognition
USRCAT Ultrafast shape recognition with CREDO atom types
UTF-8 8-bit Unicode Transformation Format
VBLL Variational Bayesian Last Layer
VEGF Vascular endothelial growth factor
WL Weisfeiler–Lehman
XGBoost eXtreme Gradient Boosting

Chapter 1

Introduction

New avenues in the drug discovery pipeline have opened up thanks to advances in large dataset collection, analysis, and computation. Therapeutic targets are identified, potential drug candidates are screened and optimized, and the most promising of these go on to pre-clinical and clinical trials [**?**]. However, attrition is high, and roughly 90% of drugs that enter clinical trials are never approved, often due to poor efficacy or toxicity [**?**]. Computational methods can remove poor candidates early on in the drug discovery pipeline, long before clinical trials. This is done through virtual screening, which allows for a large-scale analysis of libraries of compounds [1]. One standard screening method is the prediction of bioactivities and pharmacological properties of such candidates. There is a long history of using Quantitative Structure-Activity Relationship (QSAR) models to make these predictions, to act as a filter for compounds with undesirable properties, and thus reduce the time and cost associated with experimental screening [2]. QSAR models aim to predict the relationship between a molecule's structural and chemical features as encoded by a particular molecular representation and a desired experimental molecular property [2] of interest. A QSAR model requires a molecular representation and some form of machine learning (ML) model. An ideal molecular representation will allow a model to learn only the relevant information about each molecule with respect to the target property. Representations are either 'static' hand-crafted descriptors or circular fingerprints that encode predefined structural and physicochemical features [3–6]; or learned representations, where the encoding is derived from a predictive model, often taken from the activations of the final layer of a neural network, or a latent space embedding [2, 7].

QSAR models are of course, sensitive to the quality of the input data on which

they are trained. Experimental noise can arise from measurement errors, variability under experimental conditions, varying provenance of the data, intrinsic biological, chemical variability, and human error. Even the very measures of model performance may be flawed if test and validation sets are laden with noise. Kolmar and Grulke [8] showed that error in the test labels can make a model look worse than it is: a model trained on noisy data can predict the noise-free values more accurately than an evaluation on the noisy test set indicates. There is also a limit to how much noise the model can tolerate; Jorner et al. [9] argued that once their model's error approached the experimental error, more training data would not be expected to improve accuracy much. In the context of activity prediction, the inherent variability within experimental setups and assays, batch effects, measurement errors, and biological variability introduce additional layers of complexity [8, 10, 11]. All of this increases the likelihood of encountering noise in the data. Even the molecular representations in the training data themselves may be incorrect or misleading [12, 13], and chemical datasets contain too few repeat measurements of each compound to characterize the distribution the label is drawn from. Thus, models treat a single label as the population mean [8]. Here pIC_{50} and pK_i are the negative base-10 logarithms of the IC_{50} and K_i in molar units, so a difference of one log unit is a tenfold difference in concentration. Repeat measurements of the same compound-target pair disagree by about 0.68 log units for pIC_{50} and 0.54 log units for pK_i [10, 11], which is about a 4.8-fold and a 3.5-fold difference in concentration. Restricting to measurements from a single assay lowers the mean disagreement from 0.50 to 0.27 log units [14], or from about a 3.2-fold to a 1.9-fold difference.

This thesis addresses the challenge of conducting molecular property prediction when the underlying experimental data, or "labels" are noisy. Most work on noise in QSAR modeling assumes Gaussian label noise and evaluates either representations or model architectures in isolation. Cortes-Ciriano et al. [15] examined 12 algorithms across multiple noise levels but used fixed representations, and studies that compare representations typically use clean data. No work has systematically compared the relative importance of representation versus architecture choice for label noise robustness, asked whether probabilistic methods justify their computational overhead under noise, or tested whether findings generalize across different types of injected noise. This thesis fills that gap. I will also discuss noise mitigation, using the results of my noise robustness study. I explore why many noise mitigation techniques underperform in cheminformatcs data and methods to address this problem in the

form of novel molecular representations.

The first half of this thesis therefore asks four questions. First, how does the choice of molecular representation and model architecture explain the variance in predictive performance, and does that division change when you instead consider how much performance a QSAR model is able to retain under increasing label noise? Second, what does the choice of model provide with label noise? Within this question we compare the effects of deterministic and probabilistic modeling. Third, does the type of label noise matter, or just the quantity of it? We explore different techniques of generating and adding label noise to both clean data, as well as data with inherent experimental noise. Fourth, what is the impact of artificial label noise on a model's uncertainty estimation? Does the estimated uncertainty rise with increased label noise and can these uncertainty values point to which labels are corrupted? Together, these questions aim to identify what, if anything, makes a QSAR model robust to noise. To answer them we introduce NoiseInject, an open-source Python package for benchmarking noise robustness and uncertainty quantification (Section 2.6.1).

We hypothesized that the fault lay in this distance. On a given dataset, most features of a general-purpose representation either do not vary or do not relate to the property, yet each adds a term to the distance, so together they dominate it (Chapter 4). We propose instead to compute distances over a hybrid representation: a fixed number of task-relevant features selected from several representations. A first version showed that the selected features also work as ordinary model inputs, so the question became how to choose them (Chapter 5).

The second half asks whether the representation itself can be adapted to the task:

1. How can the best set of features be selected for a given prediction task?

2. Do hybrid representations retain more of their predictive performance under label noise than their base representations?

3. Do the features selected for a hybrid representation help stabilize distance-based noise mitigation?

4. Does weighting features by their importance improve the construction of hybrid representations, model training and noise mitigation?

5. When sample removal is suboptimal, can sample weighting or label smoothing handle Gaussian label noise better?

1.1 Outline of the thesis and contributions

Chapter 2 introduces molecular representations, model families, molecular similarity and activity cliffs, and the sources of noise in molecular data. It defines the seven noise conditions used in Chapter 3 and the methods for LNL, and closes with artifical noise injection, and evaluation metrics.

Chapter 3 looks into what makes a QSAR model robust to label noise. The primary prediction target is the HOMO–LUMO gap in QM9, whose computed labels carry negligible measurement noise. The same experiments were also run on three assay datasets: LogD, Caco-2 and hERG K_i. We compare seven different types of label noise, referred to as noise conditions. We matched the quantity of noise to the training set label spread for all but one of the noise conditions. We measure noise robustness as the proportion of predictive performance retained as artificial label noise increases. We found that model family was the main factor in noise robustness, and choice of molecular representation matters more for predictive performance than noise robustness. Strong predictive performance on clean labels does not imply robustness. The models with the highest predictive performance, LightGBM, XGBoost and the plain neural networks lost the most with increasing label noise, while random forests were the most consistent model across datasets, representations, and noise conditions. We replicated systematic noise by adding it in disproportionate amounts to select scaffold families and by clipping labels at a made-up assay limit. Models struggled the most when training on this systematic noise. Label noise mean predicted uncertainty for the probabilistic models which estimate it, however, only models with a variational head correctly attributed the rise in uncertainty to the data rather than to the model itself.

Chapter 4 takes a turn to look at another topic: molecular similarity. The natural next step to Chapter 3 is noise mitigation. However, a quick sweep of noise mitigation and various other LNL techniques showed disappointing results. None of the methods that incorporated the concept of molecular similarity showed any promising results. On a given dataset, standard representations carry many features that do not vary, and some map different molecules to the same vector. Similarity computed over these representations inherits both of these problems, and methods that rely on similarity suffer as a result. The chapter ends with what a representation would need to mitigate these problems, which Chapter 5 builds.

Chapter 5 presents the adaptive hybrid representation. The adaptive hybrid

search customises the settings of a feature selection process to a given dataset, model, and budget. It takes in several base representations and selects the most important features for that task by a fixed budget. This produces a tailor-made feature set, referred to as the hybrid representation. The actual output of the hybrid search is not the set of features, but rather the settings that define the feature selection method used to genereate that hybrid representation.

The hybrid is compared the base representations on the same data, an ensemble of one model per representation, principal component analysis, minimum-redundancy maximum-relevance selection, recursive feature elimination and random features of the same size. It also investigates properties of the hybrid representation, including whether the selected features are interpretable and stable, whether search settings transfer between datasets, and whether the method extends to other domains and to data held at separate sites.

Chapter 6 asks whether a hybrid representation repairs the failures described in Chapter 4. It compares standard and hybrid representations selected for predicting each molecule's activity from its five nearest neighbors, asks whether neighbors in a hybrid representation identify corrupted labels better than neighbors in a standard one, and tests LNL methods, including noise mitigation, one noise type at a time. Its results are also preliminary. On hERG K_i, predicting each molecule's activity from its five nearest neighbors was more accurate in a hybrid than in the best single representation, and, on ECFP4, correcting labels clipped at a limit by one shared amount recovered more than half of the lost performance, whereas deleting the clipped molecules gave a higher error than keeping them. Under Gaussian noise on nanobody thermostability, every one of twelve LNL methods scored higher on the hybrid than any of them did on one-hot, the weakest input.

Chapter 7 summarizes each chapter and argues that the noise in a dataset should be modeled before a method is chosen, since no single method fits every dataset and simple methods are often as good as complex ones. It then gives limitations, future directions and conclusions.

The software contributions are NoiseInject and the KIRBy repository, which implements the search for hybrid representations and the experiments (Additional file S.6).

1.2 Collaborations

This work was supervised by Garrett Morris and co-supervised by Thierry Hanser and Stéphane Werner at Lhasa Limited, which runs the federated learning platform Effiris [16]. Lhasa also supplied the hERG data used in Chapters 4 and 5; in Chapter 5, two of its sets and the public ChEMBL hERG K_i set are treated as data held at separate sites, as in federated learning [16, 17].

The Oxford Protein Informatics Group (OPIG) pointed us to the antibody and protein datasets used in Chapters 5 and 6; all datasets are listed in Additional file S.5. The collaboration also produced five posts on OPIG's blog, Blopig, listed under Blog Posts in the Summary of Research Contributions.

The adaptive hybrid representation, its software and the experiments in Chapters 4 to 6 are the author's own work.

Chapter 2

Molecular Representations, Models and Label Noise

2.1 Overview

2.2 Molecular representations in QSAR

The success of machine learning hinges on the quality of input data. Although the quality can refer to noise, which the majority of our work is centered around, in this case we are talking about the type of input. Molecular representations provide different features with which QSAR models and noise detection techniques can be learned (Figure 2.1). However, a potential bottleneck on larger datasets should be noted; the more complex the data structure, the greater the computational cost. For smaller datasets, the number of parameters is limited by the size of the training set, to prevent overfitting.

One of the most common and intuitive molecular representations is the Simplified Molecular-Input Line-Entry System (SMILES) string, which consists of a sequence of ASCII characters generated by a fixed set of rules to specify a given chemical identity [18]. A common way to use SMILES strings as inputs to QSAR models is to tokenize and one-hot-encode them rather than to use the strings themselves [19]. The strings are tokenized into atomic branches and symbols and mapped to indices of a vocabulary, and finally converted into binary vectors using one-hot encodings [19]. This representation, referred to as one-hot-encoded SMILES, allows models to process molecular structures as bit-vector-based fingerprints or as sequences of vectors. Sequence-based models such as recurrent neural networks and transformers

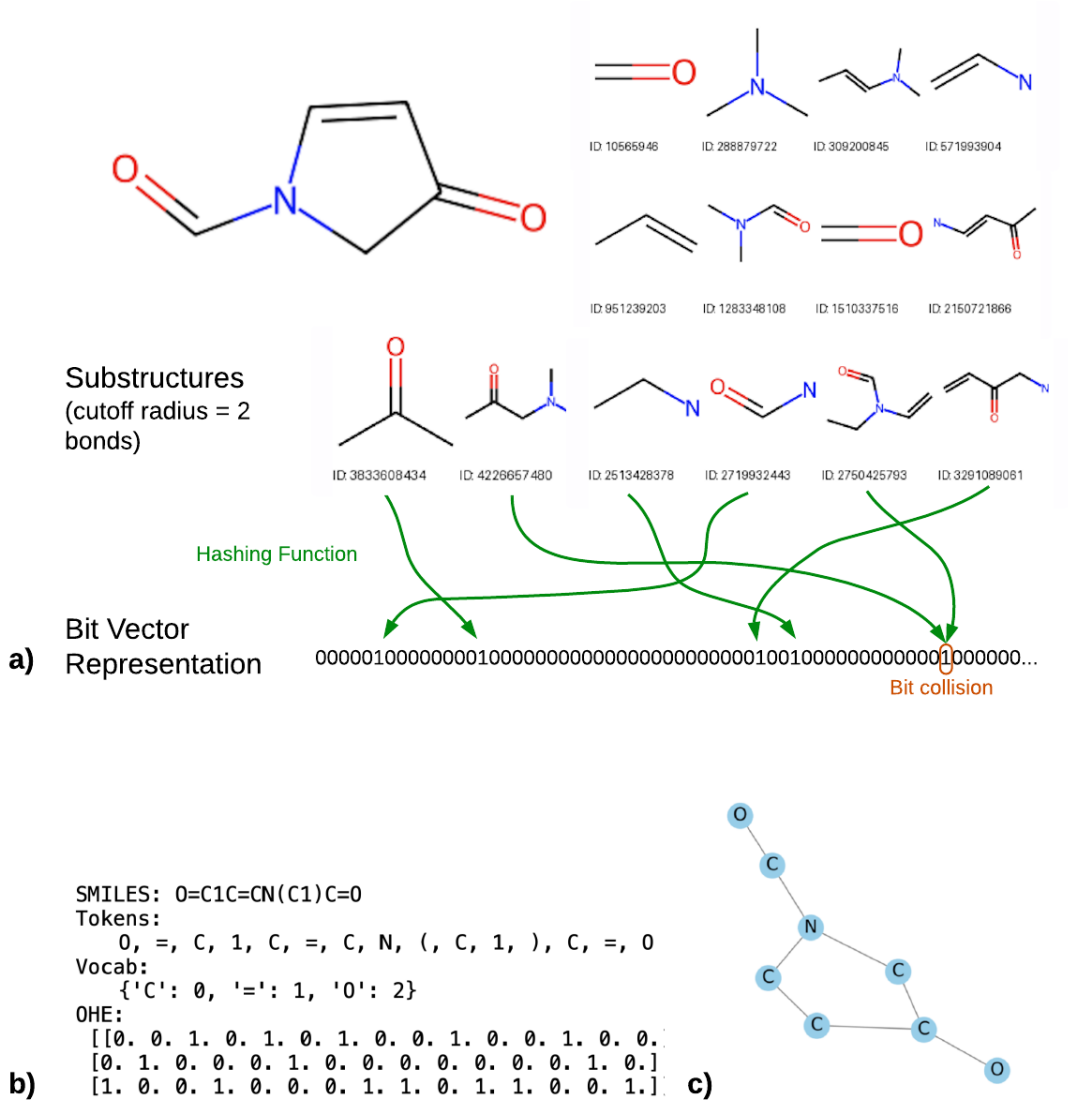


Figure 2.1: Three representations of the molecule O=C1C=CN(C1)C=O. a) ECFP4, in which circular substructures up to a radius of two bonds are hashed (green arrows) to positions in a bit vector, and two substructures can share a position (bit collision, orange). b) Tokenization and one-hot encoding of the SMILES string. c) Molecular graph, with atoms as nodes and bonds as edges.

have proven to be particularly successful in capturing text semantics within a sequence. When applied to raw or one-hot-encoded SMILES strings, these models can effectively capture the structure of the molecule in its textual representation [19].

Another common representation is the molecular fingerprint, a vector containing information about the properties of the molecule in question. In a typical bit vector-based fingerprint, each bit accounts for the presence (1) or absence (0) of a given chemical feature or substructure [20]. One common fingerprint is the Molecular ACCess System (MACCS) key, a substructure key-based fingerprint that encodes

predefined atom symbols and properties, as well as bond types [21]. Another is the Extended Connectivity Fingerprint (ECFP), which encodes local neighborhoods around each atom and bond connectivity in molecules [21]. ECFPs are typically constructed with a radius of 2, a length of 2048 bits, and active chirality flags. These existing molecular representations are well established and have been quite successful when coupled with QSAR models [2, 22]. They are relatively simple, and as the bits can be mapped back to one or more substructures, they are human-interpretable, and furthermore easy-to-compute. However, they are subject to information loss due to bit collisions, which happens when two different substructures are encoded at the same bit position [20]. They are also based on a fixed procedure and are not trainable or adaptable to a particular dataset. Alternative representations that capture different structural, chemical, or physical features are able to overcome these limitations.

Recent advances have led to the development of new molecular fingerprints. One such method is the Sort & Slice (SNS) technique, an alternative to hash-based folding for pooling ECFP substructures [5]. Sort & Slice begins by sorting all circular substructures in the training set according to their frequency of occurrence. It then "slices" away all but the N most frequent substructures, which are subsequently used to generate an N-dimensional, bit-collision-free binary fingerprint. This approach solves the ambiguity problem of hashed fingerprints, where multiple substructures might be encoded at the same bit position, making it very interpretable [5]. Sort & Slice fingerprints outperform ECFP of diameter four (ECFP4) with standard hash-based folding methods across various molecular property prediction tasks. These experiments consisted of different data splitting techniques, machine learning models, fingerprint diameters, fingerprint dimensions, and atomic invariants [5]. Furthermore, Sort & Slice outperformed advanced substructure selection schemes such as filtered fingerprints and mutual information maximization [5].

Neural fingerprints are a category of learned representations that overcome the fixed, hard-coded nature of traditional fingerprints. Unlike ECFP or MACCS keys, which rely on predefined hashing schemes or substructure catalogs, neural fingerprints train on molecular data to capture chemical information [23, 24]. These representations are the result of deep learning architectures that have learned dense, continuous embeddings which can be used for downstream tasks.

Mol2vec, which adapts the Word2vec algorithm from NLP to molecular substructures [23], is a common neural fingerprint. Mol2vec treats molecules like "sentences"

and circular substructures derived from Morgan fingerprints like "words". The model learns vector embeddings for each substructure through unsupervised training, such that chemically similar substructures are mapped to nearby points in the embedding space [23]. The embeddings of all present substructures are summed to obtain a full representation. This approach produces 300-dimensional dense vectors that avoid the sparsity and bit collisions inherent in hashed fingerprints [23]. Mol2vec has performed strongly on various QSAR benchmarks, matching or exceeding Morgan fingerprints on solubility and bioactivity prediction tasks [23].

Transformer-based architectures have also been applied to molecular representation learning, similarly stemming from NLP ideas. MolBERT applies a bidirectional transformer to learn molecular representations from SMILES strings through self-supervised pre-training [25]. The model has domain-relevant auxiliary adaptations, such as the prediction of calculated physicochemical properties alongside the masked token prediction, to improve the chemical fidelity of the learned embeddings [25]. MolBERT outperformed ECFP4, RDKit descriptors and the CDDD neural representation on virtual screening and QSAR benchmarks [25]. ChemBERTa is another transformer-based approach which pre-trains masked language models on SMILES data from PubChem [24]. ChemBERTa uses a byte-pair-encoding tokenizer by default and compares it with a SMILES-specific tokenizer [24]. Attention visualization shows neurons selective for chemically relevant functional groups and aromatic rings, and others that track bracket closures in the SMILES string [24]. ChemBERTa-2, pre-trained on up to 77 million PubChem compounds, is competitive with existing architectures on MoleculeNet [26].

Neural fingerprints provide several advantages over traditional fingerprints. They produce dense continuous vectors that avoid sparseness and information loss from bit collisions [23]. They can also be pre-trained once on large unlabeled datasets and then fine-tuned for specific downstream tasks, opening the door to transfer learning [24, 25]. However, pre-training them is computationally expensive, and they are less directly interpretable than fingerprints, although attention visualization offers some insight [24].

Beyond fingerprints, physicochemical-descriptor vectors (PDVs) are a class of continuous-value molecular descriptors that encode structural and chemical properties of molecules [27]. Unlike binary fingerprints or graph-based representations, PDVs aggregate atom and bond-level quantities into fixed-length feature vectors that capture a wide variety of information about a molecule and its structure [28].

PDVs typically consist of hundreds of properties including constitutional descriptors such as molecular weight, atom and bond counts, and heteroatom ratios; topological indices encoding molecular connectivity patterns; physicochemical properties such as logP (lipophilicity), molar refractivity, and topological polar surface area; and electronic descriptors derived from partial charges and polarizabilities [27, 29]. The `RDKit` cheminformatics toolkit provides a comprehensive suite of more than 200 molecular descriptors that cover these categories [30]. Many PDV descriptors, such as logP, molecular weight and partial charges, have a direct physicochemical meaning, although many of the thousands available have no clear interpretation [29]. They are also less computationally expensive than some 3D methods or neural fingerprints, with many descriptors simply requiring a 2D molecular structure without conformer generation or extensive pre-training. This efficiency makes PDVs particularly appropriate for high-throughput applications. Descriptors based on 3D structure can also be used [28].

However, PDVs have limitations. The dimensionality of PDV feature spaces can lead to problems with multicollinearity [27]. This can be mitigated by feature selection or dimensionality reduction techniques [29]. Additionally, PDVs encode molecular properties as aggregated global features, which may not capture fine-grained structural details such as specific substructure arrangements or local chemical environments, information important for particular endpoints.

One of the more intuitive representations is the molecular graph. These graphs can be categorized into 2D and 3D graphs. 2D graphs rely on adjacency matrices that encode topological atom- and bond-level features but leave out geometric properties such as bond angles or torsion, meaning they may fail to capture stereochemistry. 3D graphs incorporate these spatial features, but are often limited by remaining in a single conformation [31]. A standard molecular graph is constructed such that each molecule is represented as $G = (V, E)$ with node features $\mathbf{x}_i$ and edge features $\mathbf{e}_{ij}$. In one common featurization, nodes encode atom type, chirality, hybridization and aromaticity, and edges encode bond type, ring membership, direction and stereochemistry [31].

In the standard molecular graph, atoms are mapped to nodes and bonds to edges [4]. These molecular graphs are typically 2D objects that represent 3D information such as atomic coordinates, bond angles, chirality, etc. [4]. However, there are alternative molecular graph representations such as graphs with quantum 3D embeddings, weight alignment graphs, association graphs containing pairs

of molecules, and geometric interaction graph neural networks that incorporate 3D structures and physical interactions [32–34]. Du et al. [35] developed hybrid 2D and 3D graphs, combining 3D data, guaranteeing precision and completeness by providing unabridged information, and 2D data, providing chemical intuitions for guidance, in a zipper-like arrangement. Voxel-based 3D representations, in which each atom occupies one grid cell, are too sparse for convolutional networks to perform well; Kuzminykh et al. [36] instead spread each atom over nearby voxels with a wave transform.

These graph representations do not highlight specific substructures or specific arrangements of atoms and bonds within each molecule. The substructures contain functional groups that are related to a range of chemical properties and are potentially involved in downstream tasks related to the function of molecules. Bouritsas et al. [37] developed graph substructure networks, a topologically aware message passing scheme based on substructure encoding. The method involves encoding structural roles based on the appearance of specific substructures and using them in message passing updates. Since message-passing graph neural networks (GNNs) are at most as powerful as the Weisfeiler–Lehman test, they cannot isolate sufficient features from the graph structure [37]. Graph substructure networks, on the other hand, capture these substructure-related features. Encoders pretrained on unlabeled molecules, whether graph-based [7] or transformers on SMILES [38], can also produce embeddings that capture structural features.

2.3 Models for molecular property prediction

There are a wide variety of model architectures that can be used for QSAR modeling. Starting off with simpler and more explainable models, we have tree-based methods. Decision trees are non-parametric supervised learning algorithms that perform recursive binary partitioning of the feature space [39]. A decision tree builds a predictive model by learning a series of if-then rules that recursively split the data based on given feature values, creating a tree where internal nodes represent decision rules, branches represent the results of those decisions, and leaf nodes contain predicted values or class labels [39]. Each recursive split is chosen to maximize homogeneity within the resulting partitions. For classification, Gini impurity is commonly used and for regression, mean squared error reduction [39].

There are several properties that make decision trees a good foundation for

QSAR modeling. First, they handle non-linear relationships between molecular descriptors and target properties without requiring explicit feature engineering or transformation [39]. They also do not make assumptions about the underlying distribution of the data, making them robust to violations of parametric assumptions that can limit linear regression or other classical methods [40].

Decision trees handle data in a "contributory" rather than "consensus" manner. In consensus methods such as linear regression or neural networks, every data point influences the prediction at every other point in the feature space. Tree-based methods make predictions based on the subset of training samples that fall into the same leaf node as the test point [39]. This mechanism essentially quarantines outliers in their respective leaf nodes, limiting their impact on predictions for samples in other regions or chemical domains.

Individual decision trees tend to have high variance and often overfit, particularly at large depths [39]. Small modifications made to the training data can result in large differences in the structure of the tree, resulting in unstable predictions [**?**]. Ensemble methods are used to counteract this instability, aggregating predictions from many trees. Random forests (RFs) aggregate predictions from trees trained on bootstrap samples of data with random subsets of features at each split [41]. Bootstrapping ensures that each tree is trained on a slightly different subset of data, which effectively reduces the influence of any individual sample [41]. This is especially beneficial with noisy samples. The selection of random features at every node decorrelates the trees, which lowers the generalization error of the ensemble [41]. The aggregation of trees further prevents overfitting to noise by smoothing out prediction errors from individual noisy labels.

Neural networks (NNs), a very popular family of models, can also be used for QSAR, although with mixed results. An NN is a system of interconnected nodes which learn by passing information through its layers. There are weights and biases attached to the nodes and various other parameters which the model optimizes to minimize the difference between the predicted output and the actual target value. During training, the network parameters are updated using backpropagation, in which gradients of the loss function with respect to each weight are propagated backward through the network [42]. The structure of these networks and the hyperparameters used depend on the type of NN. In QSAR, one of the most commonly used architectures is the multilayer perceptron (MLP), a NN containing fully-connected layers with non-linear activation functions which transforms molecular feature vec-

tors (i.e., fingerprints, descriptors) into a property or activity prediction [43].

A typical MLP for molecular property prediction has a few fully connected hidden layers with rectified linear unit (ReLU) activations and dropout regularization to prevent overfitting. Training is performed using backpropagation with gradient descent optimizers such as Adam, to minimize a loss function (mean squared error for regression, cross-entropy for classification) on batches of training data.

However, NN performance on QSAR-related tasks has been very mixed. NNs have a long history in QSAR [44], and tuned deep networks have outperformed shallow methods such as random forests and support vector machines across diverse activity classes [45]. Their complexity and use of gradients make them prone to label memorization [46], which in the context of noise is problematic, as it means that models may memorize the noise themselves. Although models that are memorizing labels may initially appear to have strong predictive performance, they cannot generalize to out-of-distribution samples properly and often provide inaccurate uncertainty estimates. Various approaches have been proposed to improve neural network robustness to label noise, including noise-robust loss functions [47], confident learning to identify mislabeled examples [48], and regularization techniques such as mixup [49] or label smoothing [50].

All the models up to this point have assumed a vector-based input. Molecular graphs, on the other hand, require different types of predictive modeling. Two standard ML methods on molecular graphs include Graph Convolutional Networks (GCNs) and Graph Isomorphism Networks (GINs). GCNs apply the concept of convolution to graph-structured data. Instead of operating on grid-based data, such as images, GCNs aggregate information from neighbors of a node to update its representation [51]. In contrast, GINs address the limitations in GCNs' ability to differentiate between non-isomorphic graphs. Unlike GCNs, which normalize and weight the sum of neighboring node features, GINs use a simple sum followed by a trainable MLP, giving the model flexibility to learn complex transformations of node features. The GIN model's design is informed by the Weisfeiler–Lehman test for graph isomorphism, enabling it to distinguish between molecular graphs that are structurally similar but differ in subtle ways [19, 52]. GIN features have been competitive with or better than fingerprints and descriptors at classifying pairs of molecules as activity cliffs (ACs; Section 2.4), where small variations in molecular structure can lead to significant changes in chemical or biological activity [53]. Descriptor-based models still predicted the potency of cliff molecules better than

deep learning methods [54].

2.3.1 Tree ensembles

Gradient boosting methods such as eXtreme Gradient Boosting (XGBoost) sequentially fit trees to the residuals of previous iterations, progressively reducing the prediction error [55–57]. We used a handful of gradient-boosted tree ensembles in this thesis including Light Gradient-Boosting Machine (LightGBM) [58], and Natural Gradient Boosting (NGBoost) [59].

2.3.2 Support vector machines

Support vector regression fits a function that is as flat as possible while keeping training errors small [60, 61]. It uses the ϵ-insensitive loss, under which errors smaller than ϵ cost nothing and larger errors cost their size minus ϵ. A constant C trades flatness against tolerance of large errors [61, 62]. The features enter the model only through the kernel. With the radial basis function (RBF) kernel, the kernel is a function of the Euclidean distance, so every feature enters every distance. In Chapter 3 the continuous representations are standardized and the binary fingerprints are used as built. Three properties of the method suggest robustness to label noise. Errors inside the ϵ tube change nothing, the loss outside the tube grows linearly rather than quadratically, and the weight of each molecule is capped at C [61]. Under simulated experimental error, the RBF-kernel SVM degraded smoothly and linearly [15], and Chapter 3 reports how it holds up under each of our noise conditions. Its main limitation is computational cost, because training operates on a kernel matrix that grows with the square of the number of training molecules [63].

2.3.3 Gaussian processes

We employed Gaussian process (GP) models using the `Gauche` framework [64], which is optimized for cheminformatics tasks. Standardized kernels for GPs are not optimized for the chemical landscape. GAUCHE defines such kernels for strings, fingerprints, and graphs that operate on a range of widely used molecular representations [64]. We used the Tanimoto kernel [65, 66], which is defined for binary vectors $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$ for $d \geq 1$ as:

$$k_{\text{Tanimoto}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \frac{\langle \mathbf{x}, \mathbf{x}' \rangle}{\|\mathbf{x}\|^2 + \|\mathbf{x}'\|^2 - \langle \mathbf{x}, \mathbf{x}' \rangle}, \tag{2.1}$$

where $\|\cdot\|$ represents the Euclidean norm [64].

The RBF kernel is defined for $\mathbf{x}, \mathbf{x}' \in \mathbb{R}^d$ as:

$$k_{\text{RBF}}(\mathbf{x}, \mathbf{x}') := \sigma_f^2 \cdot \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right), \tag{2.2}$$

where ℓ is the lengthscale and σ_f^2 the signal variance, which scales both kernels.

2.3.4 Bayesian neural networks

Standard feed-forward networks are deterministic; however, they can be transformed into probabilistic models (Figure 2.2).

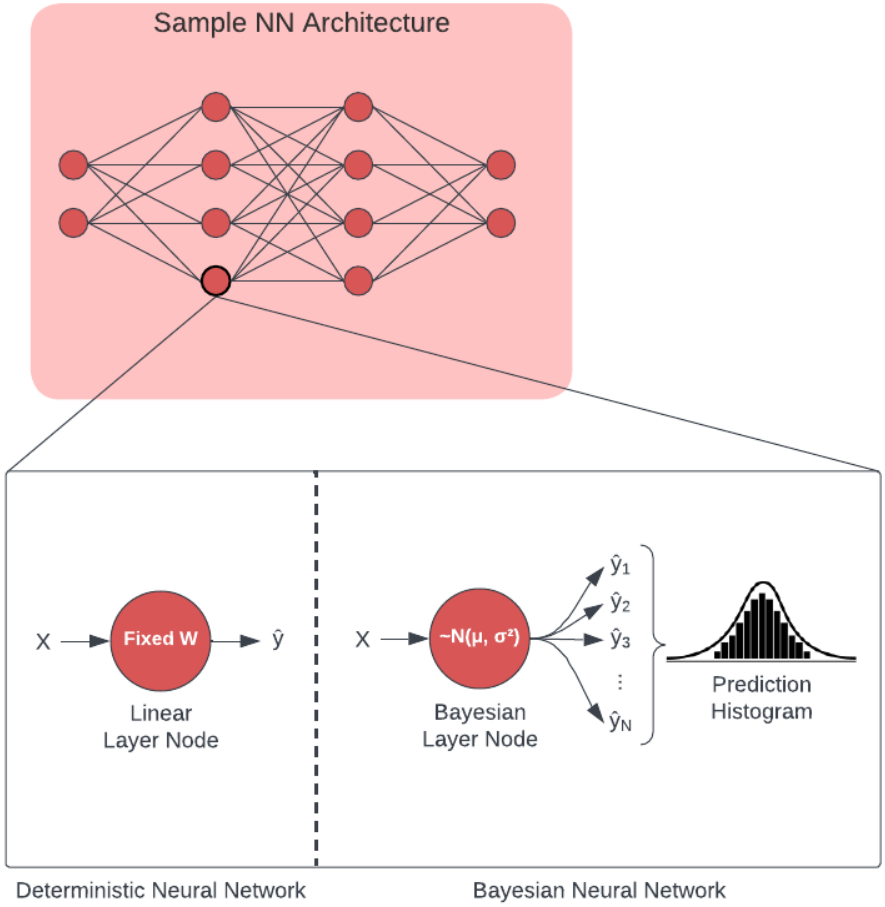


Figure 2.2: One node of a deterministic and of a Bayesian neural network. The deterministic node (bottom left) applies a fixed weight W and returns one prediction $\hat{y}$. The Bayesian node (bottom right) draws its weight from $\mathcal{N}(\mu, \sigma^2)$, so repeated predictions $\hat{y}_1, \ldots, \hat{y}_N$ form a histogram.

Unlike standard deterministic ML methods, Bayesian methods model distributions over predictions using Bayes' rule, allowing for the quantification of uncertainty within parameters and labels. Bayesian methods treat the model parameters as random variables with distributions. Uncertainty can also be estimated from other non-deterministic methods, including deep ensembles [67] and bootstrap modeling [68].

2.4 Molecular similarity and activity cliffs

Much of cheminformatics rests on the similar property principle, which states that structurally similar molecules tend to have similar properties [54, 69]. One of its clearest applications is nearest-neighbor prediction, which appears across cheminformatics workflows and, in this thesis, in the kernels used by GPs and at the basis of many noise identification algorithms.

The difficulty with this principle is that similarity is defined only relative to a similarity measure, which is fixed by the choice of representation and coefficient [70]. There is no standard definition of similarity between molecules, especially when different molecular embeddings are handled in a computational setting. Two molecules that are near-identical under a substructure fingerprint may be far apart under a PDV, and Maggiora et al. [70] caution that quantified similarity can be over-interpreted in this way.

The most common measure for binary fingerprints is the Tanimoto coefficient [71]. For two molecules with fingerprints $\mathbf{x}, \mathbf{x}' \in \{0,1\}^d$,

$$T(\mathbf{x}, \mathbf{x}') = \frac{\sum_{j=1}^{d} x_j x'_j}{\sum_{j=1}^{d} x_j + \sum_{j=1}^{d} x'_j - \sum_{j=1}^{d} x_j x'_j}, \tag{2.3}$$

which is the number of bits set in both molecules divided by the number of bits set in either. Continuous representations are instead compared with a distance, most often the Euclidean distance $\sqrt{\sum_{j=1}^{d}(x_j - x'_j)^2}$. Both measures are sums over the features, and neither weighs a feature by its relation to the property. In the Tanimoto coefficient, a bit that is off in both molecules contributes nothing, whereas every bit that is set in either molecule counts. In the Euclidean distance, every feature that differs between the two molecules counts. Chapter 4 returns to this point (Equation 4.3).

Distance-based reasoning relies on the assumption that the data embeddings contain quality information. When the labels are corrupted or the important features are scarce, similarity scores over-rely on compounding false or useless information, and the assumption that similar molecules have similar properties breaks down. Bajusz et al. [71] found that the Tanimoto, Dice and Cosine coefficients and the Soergel distance rank molecules almost identically on binary fingerprints. We take this to mean that changing the formula cannot compensate for a weak representation. This problem is further complicated by activity cliffs (defined below). These molecules, which potentially contain the most important information for model prediction, can be confused with noise. Another barrier to distance-based reasoning is persistent noise across labels. The task of trying to determine if a molecule's value matches that of its neighbors becomes a lot harder when all of the neighbors have some form of noise as well.

There are many downstream tasks which rely on the concept of similarity. The second half of this thesis relies on many such tasks, and as such focuses on identifying representations that agree with this principle.

The sharpest failure of the similar property principle is the activity cliff, a pair of structurally similar molecules with a large difference in potency that is produced by a small chemical change [72, 73]. Activity cliffs inherit the measure dependence of the principle they violate, and there is no widely accepted definition of one [54]. The MoleculeACE benchmark [54] makes this choice explicit. A pair is considered highly similar if its similarity exceeds 90% under at least one of three measures, namely the Tanimoto coefficient on ECFPs, the Tanimoto coefficient on scaffold fingerprints and a scaled Levenshtein distance on SMILES strings. Such a pair forms a cliff if the two molecules also differ tenfold or more in K_i or EC_{50} [54]. This soft consensus over three measures is used because each measure captures only part of the underlying chemistry. Across the thirty MoleculeACE targets, cliff compounds account for between 7% and 52% of a dataset [54].

Dimensionality itself poses a further difficulty. Beyer et al. [74] show that as the number of dimensions grows, the distances to the nearest and to the furthest point become increasingly similar, so that pairwise distances become progressively less discriminative. Molecular representations are routinely high-dimensional and sparse, and when a large proportion of their dimensions carry no information relevant to the property being modeled, distances computed over them are dominated by those uninformative dimensions.

2.5 Noise in molecular data

Real-world cheminformatics datasets contain label noise arising from multiple sources across the drug discovery pipeline. Although noise is often modeled as being random, or drawn from a Gaussian distribution, other varieties of noise must be considered when working with experimental data. To properly model experimental noise, we must first understand where the noise is coming from.

One key issue with QSAR models is the standard ML assumption that labels represent true values, as most chemical datasets lack sufficient replications to provide a robust statistical representation of the underlying distributions [8]. A single measurement, or even multiple measurements for a given label, will not guarantee that this label accurately reflects the population mean [8]. Most ML models rely on the assumption that the training data is, in fact, an accurate reflection of the population. Therefore, they treat these potentially noisy labels as discrete quantities rather than distributions, which opens the door to overfitting to noise. However, Bayesian methods such as Gaussian processes (GPs), and conformal prediction, do not make this assumption [8].

Studies involving noise, and specifically the quantification of noise robustness, typically approach label noise by adding independent, zero-mean, Gaussian error to the labels. However, there's evidence to suggest that noise from experimental assays takes on different forms. Krüger and Overington [75] rejected normality of the differences in binding affinity between orthologous targets by Anderson-Darling test, found them approximately Laplace, and found that differences between assays of the same ligand and target closely resemble them. Kalliokoski et al. [11] examined the widest disagreements among 16,844 pairs of repeat pIC_{50} measurements and found that in most cases, the wide gap between repeat values was due to annotation error rather than to an imprecise assay.

Although numerous studies have explored the effects of noise on ML algorithms for molecular property prediction, comparisons between different molecular representations remain limited. Kolmar and Grulke [8] examined the predictive limits of QSAR in the presence of noise, showing that by learning the underlying trends rather than fitting to noise, the models can achieve predictions more accurate than those of their training data. They assumed experimental noise could be modeled as Gaussian-distributed, using the Central Limit Theorem to argue that aggregated experimental measurements from multiple independent error sources exhibit a Gaus-

sian distribution. However, Heid et al. [76] distinguish random noise, which affects all molecules alike, from systematic noise that differs between regions of chemical space. Using synthetic noise applied only to nitrogen-containing molecules, they showed that mean-variance estimation can detect it.

Beyond just the distribution, the assumptions of independence and zero mean may fail as well. Bentz et al. [77] examined P-glycoprotein transport measurements from 23 laboratories and found that, among the laboratories using cell lines, 62% of the variance in log digoxin efflux ratio was due to differences between laboratories. When you pool data from different sources, each source has the potential to carry a shared offset. These sources usually contain families of related compounds, breaking the independence assumption. The zero-mean assumption is not assured either, especially when true values fall above or below an assay's detection limit, called censoring. Svensson et al. [78] count censored labels in 15 industrial assays, of which 13 carry censored labels and eight sit between a quarter and two thirds censored. Chapter 3 tests noise conditions that vary the distribution, the correlation, and the bias of injected error separately, at a matched quantity of noise for every condition except censoring.

Label noise enters bioactivity data at every step from the assay plate to the database. Table 2.1 lists these sources, with examples of each and the kind of error each tends to produce. The kind of error describes how the error is distributed across molecules, which is the property that the noise conditions in Section 2.5.1 vary.

These noise sources can interact with each other and present in more ways than a simple Gaussian. Real noise can be heteroscedastic, larger in some regions of chemical space than others [76], shared within a laboratory [77], and non-normal [75]. We wanted to address this complexity in our research by examining a variety of ways of both modeling and adding noise to datasets.

Although the general assumption in much of the noise analysis and mitigation literature is that noise, specifically label noise, will have a negative effect on the model, this is not necessarily the case. There are multiple instances of using noise as a method of regularization. In linear regression, appending observations of pure noise shrinks the coefficient estimates, an implicit regularization, although not an optimal one [90]. Furthermore, adding artificial noise drawn from a Gaussian distribution to the activation layers of a neural network is considered another form of regularization [91]. These studies indicate that the presence of noise may act as

Source	Example	Kind of Error
Assay	Edge, row and column effects from evaporation or temperature gradients, and errors in liquid handling [**?**]; carryover between wells [79]; reagents or compounds degrading during a run [80]	Error that varies with plate position or read time
Instrument	Higher relative error near a detection limit [81]; autofluorescence of the test compound [82]; background subtraction in radiometric assays [81]	Larger error for weak binders; a systematic bias; labels clipped at the limit
Biology	Seeding density, growth rate, cell cycle and serum properties [83]; cell passage number [84]	Variation between repeats of the same experiment
Aggregation across sources	ChEMBL, BindingDB and PubChem pool hundreds of laboratories [85, 86]; independent K_i measurements of one compound–target pair differing by 0.44 log units on average [10]	An offset shared by each laboratory or source, hard to separate from the true value without metadata [87]
Federated learning	Teacher models label a shared public set, and a student model learns from those labels [16]	The teachers' prediction errors on top of the measurement error
Annotation	Mis-registered structures, wrong stereochemistry and salt forms [12]; 0.1–3.4% of structures incorrect, depending on the database [88]; ambiguous protonation states and counterions [89]	A label attached to the wrong molecule; distinct molecules merged

Table 2.1: Sources of label noise in bioactivity data, with reported examples and the kind of error each source produces.

a form of regularization in some contexts. It is important to address this potential of regularization from label noise in QSAR and quantitative structure–property relationship (QSPR) models.

2.5.1 Artificial noise injection

Our objective is to evaluate the robustness of QSAR models against label noise. To do so, we "inject" artificial noise into the labels during training while validation and test data remain free of noise. Validation labels are given noise of the same type and quantity as the training labels, drawn independently. This artificial noise simulates real-world experimental noise. The clean target label y_i is replaced by a noisy label $\tilde{y}_i = y_i + \epsilon_i$, where ϵ_i is the artificial noise injected at the index i, the value of which is determined by a specified noise strategy. As we increased the quantity of noise, we measured how much predictive performance fell.

We explored seven ways of adding noise, which we refer to as noise conditions and which are illustrated in Figure 2.3 and summarized in Table 2.2. Three conditions vary the shape of the distribution from which the error is drawn, and three concentrate the error on some molecules or share it within Bemis–Murcko scaffold families. The seventh condition, censoring, clips labels at a limit.

For a given label, every condition except censoring delivers the same expected quantity of noise at a set noise level. These levels are defined as a fraction of the standard deviation of clean training labels. The noise added to the label of molecule i is

$$\epsilon_i = \frac{\tau}{G}\, s_i z_i, \qquad G = \sqrt{\frac{1}{n}\sum_{j=1}^{n} s_j^2}\;\; \sigma_z, \tag{2.4}$$

Here τ is the target standard deviation of the noise, in the units of the label, and z_i is drawn from the condition's distribution at unit scale. The multiplier s_i is 1 for most molecules and $\lambda = 3$ for those the condition affects, which are a random 10% of molecules under outlier and whole scaffold families covering 20% of the split under grouped-wider. The constant σ_z is the standard deviation of z, equal to 1 for the Gaussian, $\sqrt{\nu/(\nu-2)}$ for the Student-t and $\sqrt{2}$ for the Laplace. The factor G divides both of these out, so that ϵ has standard deviation τ in expectation whatever the distribution and whichever molecules are affected.

Grouped-shifted widens no molecule's error, and instead gives every scaffold family g an offset of its own,

$$\epsilon_i = \frac{\tau}{G}\left(\sqrt{\rho}\, b_{g(i)} + \sqrt{1-\rho}\, w_i\right), \tag{2.5}$$

with one draw b per family and one draw w per molecule. The two variances sum to τ^2, and the family offset carries a share ρ of that total.

We selected the parameter values from published measurement errors wherever possible, and Table 2.2 notes where no published estimate exists. We found no published estimate of how large a share of a dataset is affected by a difference between laboratories, so under grouped-wider we chose to draw scaffold families until they covered 20% of the training molecules.

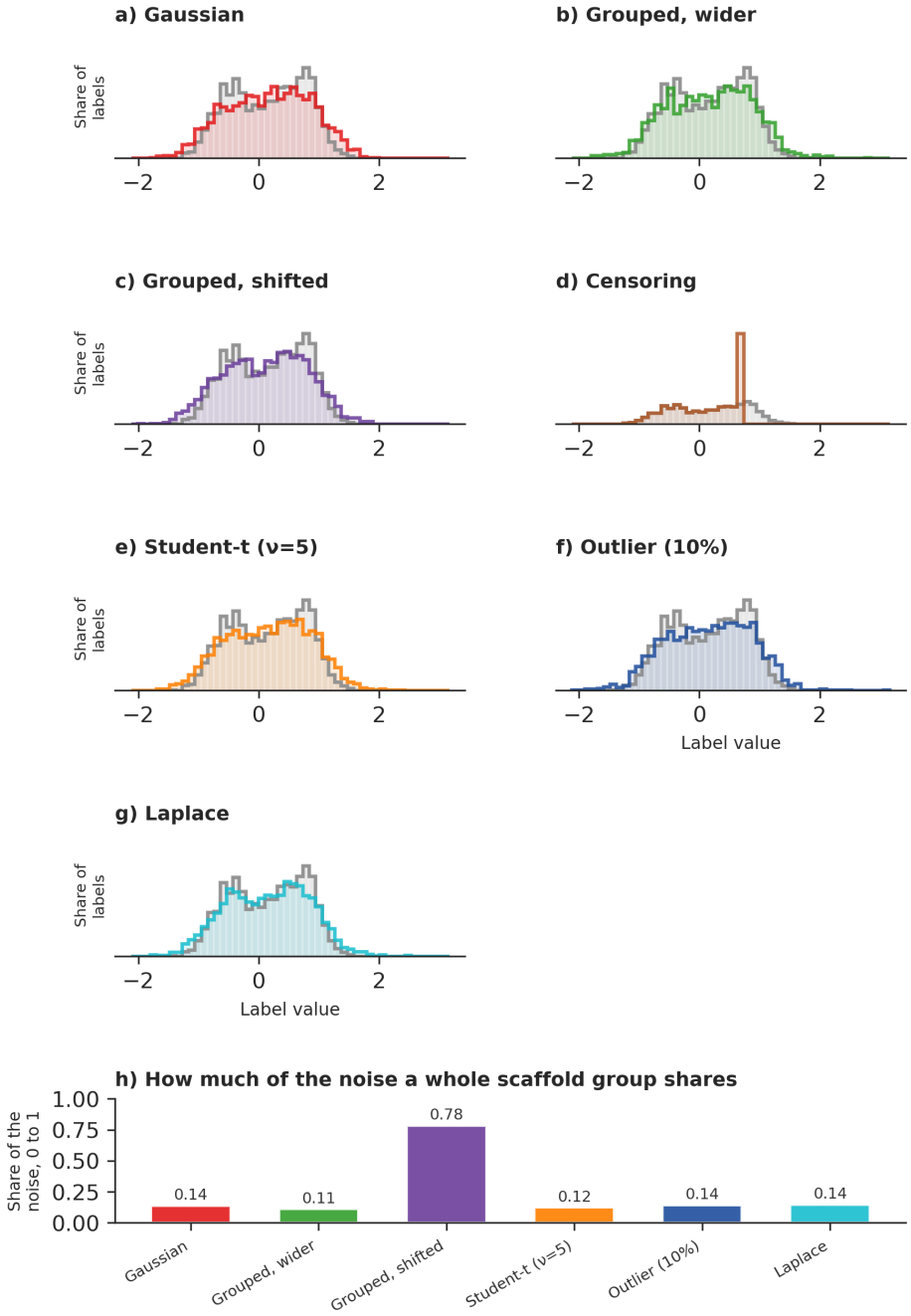


Figure 2.3: Label distributions under the seven noise conditions, for 2,000 synthetic labels in 40 random groups that stand in for scaffold families. a)–g) Clean (gray) and noised (color) labels at a noise level of 0.5 times the standard deviation of the clean labels (25% of labels clipped under censoring). h) Standard deviation of the mean error within a group divided by that of all errors, for every condition except censoring; a value near 1 means a group's labels move together, and noise that ignores the groups gives about 0.14 with about 50 labels per group.

Condition	What It Changes	Value	Source for the Value
Gaussian	Shape: every label's error is Gaussian	–	The standard assumption in noise-robustness studies
Student-t	Shape: heavier tails than Gaussian	$\nu = 5$	Differences between repeated measurements are non-normal [75]; no published estimate of ν
Laplace	Shape: sharper peak and heavier tails than Gaussian	–	Approximately Laplace for differences in pK_i between related targets, which resemble differences between repeated measurements [75]
Outlier	Error λ times wider on a random share of molecules (Huber's contamination model [92])	10% of molecules; $\lambda = 3$	1–10% of routine data contain gross errors, upper bound taken [93]; inter-laboratory error is about three times intra-laboratory error [94]
Grouped-wider	Error λ times wider on whole scaffold families	Families covering 20% of the training molecules; $\lambda = 3$	No published estimate of the share; λ as for outlier [94]
Grouped-shifted	Every scaffold family gets its own offset; no error is widened	$\rho = 0.62$	Laboratory identity explains 62% of the variance in log digoxin efflux ratio across the laboratories using cell lines [77]
Censoring	Labels past a limit are set to the limit; the level is the fraction clipped	Set by the noise level	Censored labels in industrial assays [78]

Table 2.2: The seven noise conditions and their parameter values. ν is the number of degrees of freedom of the Student-t distribution, λ the widening factor (Equation 2.4) and ρ the share of the noise variance carried by the scaffold-family offset (Equation 2.5). For censoring, the noise level is the fraction of labels clipped.

2.5.2 Learning with noisy labels

LNL covers every method for training a model when some of its labels are wrong, and we divide it into two parts. Noise mitigation changes, drops or reweights the training labels once, before any model is trained, so that any model can then be fitted to the result. Its methods include removal, down-weighting, smoothing, relabeling and correction, and Figure 2.4 shows how such a method is evaluated. Noise-robust training instead leaves the labels unchanged and alters how the model trains, through

robust losses, noise-modeling layers, regularization, or reweighting inside the training loop, as in co-teaching [95] and Meta-Weight-Net [96]. A sample-selection step that is taken out of its training loop and used only to delete molecules before a refit counts as noise mitigation. Noise detection and noise estimation, which find noisy labels or estimate how many there are, feed into noise mitigation but are not LNL methods themselves. Song et al. [47] provide a comprehensive review of Deep LNL, identifying five main approaches: (i) modifying architectures to better handle noise in training; (ii) employing regularization to reduce overfitting to noise; (iii) adjusting loss functions to be less sensitive to outliers or label flips; (iv) loss adjustment, which corrects, reweights or relabels the loss of each example during training; and (v) sample selection, which identifies examples with correct labels during training, using several networks or several rounds of training.

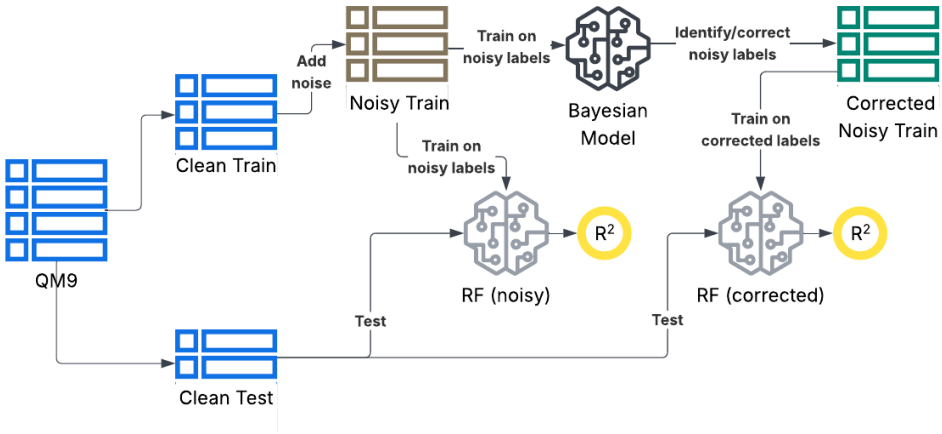


Figure 2.4: Evaluation of a noise mitigation method, with QM9 as the example. Noise is added to the training labels only. A random forest trained on the noisy labels is compared with one trained on labels corrected by a Bayesian model, and both are scored by R^2 on the clean test set.

No single method is expected to win, since averaged over all problems no learning algorithm outperforms another [97]. Cortes-Ciriano et al. [15] compared 12 algorithms on 12 bioactivity datasets with increasing Gaussian label noise; gradient boosting was the least tolerant, the other 11 degraded comparably, and two polynomial-kernel models became sensitive only at high noise. This is a recurring theme in LNL research: there is no universal solution to handling noise with experimental endpoints. Neighbor-based cleaning dates back to editing a training set by its nearest neighbors [98], and it relies on similar molecules having similar labels, an assumption that activity cliffs break. QSAR models also frequently fail to predict

activity cliffs [53]. Chapter 4 tests the assumption that similar molecules have similar labels, and Chapter 6 tests neighbor-based noise mitigation with the hybrid as the feature space.

2.6 NoiseInject: a benchmarking framework

The noise conditions described above are implemented twice. NoiseInject, an open-source Python package, injects the noise on the three assay datasets and computes the robustness and uncertainty metrics, whereas QM9 uses a separate pipeline, written in Python and Rust, that carries its own Rust implementation of the same conditions. An automated check compares the two implementations before any experiment is run on the computing cluster. Because the random number generators of Rust and NumPy produce different streams, the check compares statistics rather than individual draws. On the same labels, groups and seeds, it compares the delivered noise, the factor G, the median error, the number of badly corrupted labels and the number of molecules affected.

2.6.1 Package description

NoiseInject (version 1.0.0) evaluates model robustness and uncertainty calibration under artificial label noise, for both regression and classification. It operates on arrays of labels and on the predictions of any externally trained model, which makes it model-agnostic, and its core depends only on NumPy, pandas, SciPy and scikit-learn. Results are returned as `pandas` data frames with one row per noise level and one aggregate summary row. The software is available under an MIT license and archived on Zenodo [99].

Regression noise. The regression injector specifies a noise shape (Gaussian, Student-t or Laplace) and a targeting rule (uniform, grouped-wider, grouped-shifted, outlier or censoring) separately. Each named condition in the package is one combination of the two, together with its parameters, and this name is recorded with every result. The package defines more settings than Chapter 3 uses, namely Student-t distributions with $\nu \in \{3, 5, 10\}$ and outlier contamination of 1%, 5% and 10% of molecules. It also defines an outlier variant in which the unselected molecules receive no noise at all, rather than the base-scale noise of Equation 2.4.

The requested noise is the root-mean-square noise τ in the units of the label, that is, the noise level times the standard deviation of the clean training labels. The injector computes the per-molecule multipliers s_i and solves for the scale τ/G in closed form (Equation 2.4), so no calibration search is needed. A level of zero returns the labels exactly unchanged. The affected molecules are selected with a second random number generator, which is seeded separately from the noise draws and re-seeded on every call. As a result, the affected molecules are the same at every noise level, including zero, while the noise draws themselves change. Under the grouped-wider condition, whole scaffold families are added in random order until they cover as close as possible to 20% of the training molecules, because a fixed fraction of families gives a very different fraction of molecules when family sizes are uneven.

The censoring condition is parameterized by the fraction f of labels clipped, and its limit is the $(1-f)$ quantile of the clean training labels. On the training split, the censored set is therefore exactly the $k = \mathrm{round}(fn)$ largest labels, whereas on any other split it consists of whichever molecules lie past the training limit.

Alongside the noisy labels, each injection returns a record of how they were produced. This record contains the target amount of noise, the factor G, the solved scale, the delivered root-mean-square noise (in label units and as a fraction of the clean label standard deviation), the mean of ϵ, the fraction of molecules affected, the number of groups and the largest share held by one group, and the random seeds. The injector also checks the delivered amount of noise. Because the delivered amount is an estimate from a finite sample, it is compared against a band of three standard errors,

$$\left|\frac{\hat{\tau}}{\tau} - 1\right| \le \max\left(3\sqrt{\frac{\kappa - 1}{4\,n_{\mathrm{eff}}}},\ 0.005\right), \qquad n_{\mathrm{eff}} = \frac{\left(\sum_i s_i^2\right)^2}{\sum_i s_i^4}, \tag{2.6}$$

where $\hat{\tau}$ is the delivered root-mean-square noise and κ is the sample kurtosis of the unscaled draws z, bounded between 3 and 60. The effective count n_{eff} is the number of independent contributions over which the delivered amount of noise averages, and it falls below n when the noise is concentrated on a few molecules. Under the grouped-shifted condition it is $n_{\mathrm{eff}} = \left(\rho^2/g_{\mathrm{eff}} + (1-\rho)^2/n\right)^{-1}$, with $g_{\mathrm{eff}} = \left(\sum_g n_g\right)^2 / \sum_g n_g^2$ for family sizes n_g. At $\nu \le 4$ the Student-t distribution has no fourth moment, so the standard error is instead $1.3\,n_{\mathrm{eff}}^{2/\nu-1}$, with the constant calibrated on the measured spread. A draw outside the band raises a warning rather

than an error. At three standard errors, a correct injector falls outside the band on about 1% of draws, and the delivered amount is recorded with the result in either case.

A second function of the package returns the noise scale that each molecule's region would receive, without drawing any noise. We use it to score held-out molecules against the pattern of noise to which the training labels were exposed. Under the uniform and grouped-shifted targeting rules this scale is the same for every molecule, so the package flags the result as degenerate, because a rank correlation against a constant is undefined and must not be reported as zero.

A separate injector sizes the noise against a unit supplied by the user rather than against the label standard deviation. In the experiments implemented in the KIRBy repository, this unit is the standard deviation of a model's out-of-fold residuals on the clean training labels, so that the noise is sized against the error the model already makes. Its conditions either add Gaussian noise with variance equal to the level times the square of the unit, or move 2%, 5% or 10% of labels by the level times the unit, with a random sign or always upward. They can also add Gaussian noise whose standard deviation follows either a per-molecule covariate or a hidden log-normal draw, and both of these are scaled to the same mean variance as the first condition.

Classification noise. The package provides six label-flipping strategies, namely uniform flips, flip rates set by class frequency, asymmetric flip rates for binary tasks, a random flip rate for each sample, a separate flip rate for each class, and flips directed by a confusion matrix. Because the realized flip rate differs from the nominal flip probability under most of these strategies, a binary search finds the flip probability that produces a target fraction of changed labels.

Robustness metrics. For regression, the package computes R^2, the root mean squared error (RMSE) and the mean absolute error at every noise level. For classification, it computes accuracy, together with macro- and weighted-averaged precision, recall and F1 score and a per-class breakdown. Robustness is read from the retention curve $\mathrm{ret}(x) = m(x)/m(0)$, where m is a bounded metric for which higher values are better (R^2 for regression; accuracy, F1, precision or recall for classification) and x is the noise level. The primary summary is the normalized area under this curve, $\mathrm{AUC}_{\mathrm{norm}}$ (Equation 3.2 in Chapter 3), evaluated by the trapezoid rule. It is the

mean fraction of clean performance retained across the noise levels, lies approximately within $[0, 1]$, and is less directly correlated with clean-label R^2 than other robustness metrics we tested. A secondary summary fits $\mathrm{ret}(x) \approx \exp\left(-(x/\tau_W)^\beta\right)$ to the curve. The shape parameter β separates curves that hold and then collapse ($\beta > 1$) from curves that fall early and then plateau ($\beta < 1$), while τ_W sets the scale of the decay, and the fit requires at least four noise levels. These two summaries replace the linear slope of performance against noise used in earlier versions of NoiseInject, which misdescribed a nonlinear curve and was coupled to clean performance. The package also reports the clean baseline and the retention at the highest noise level. A flag marks any curve that falls below zero retention, and an optional threshold withholds the curve summaries for models whose clean baseline is too low for a meaningful degradation.

Uncertainty metrics. For a model that returns a predicted standard deviation u_i for each sample, the package computes several metrics at each noise level. These are the Spearman correlation between u_i and the absolute error $|y_i - \hat{y}_i|$, the empirical coverage $\frac{1}{n}\sum_i \mathbb{1}[|y_i - \hat{y}_i| \le k u_i]$ at $k = 1$ and $k = 2$ against Gaussian targets of 68.3% and 95.4%, the mean interval width $\frac{1}{n}\sum_i 2u_i$, and the expected calibration error (ECE),

$$\mathrm{ECE} = \sum_{b=1}^{10} \frac{|B_b|}{n} \left| \bar{u}_b - \overline{|y - \hat{y}|}_b \right|, \tag{2.7}$$

computed over ten equal-count bins B_b of u. All of these metrics are computed on the clean test labels. The Spearman correlation between u_i and the injected noise $|\epsilon_i|$ can only be computed where the noise is known, which is on the corrupted molecules, so it requires a separate pair of inputs. It is reported within each noise level, and a correlation pooled across all levels is returned separately. The pooled value mostly reflects the fact that uncertainty and noise both rise with the level, so it does not measure whether a model can identify which molecules are corrupted.

Three optional wrappers produce u_i from a point predictor. The first applies split-conformal prediction to any scikit-learn-compatible model, optionally scaled by the label spread of each molecule's nearest training neighbors. The second fits an exact Gaussian process with a Tanimoto or RBF kernel through `Gauche` [64], and the third applies Monte Carlo dropout to any `PyTorch` network [100]. Example notebooks and scripts cover regression and classification from start to finish.

2.7 Evaluation methodology

Chapter 3 reports predictive performance as R^2 on held-out molecules and robustness as $\mathrm{AUC}_{\mathrm{norm}}$ (Equation 3.2). Chapters 4 to 6 state the metric used for each dataset where it is introduced.

2.8 Conclusions

This chapter has introduced both halves of a predictor, namely the representation and the model. Because real assay noise is not independent, identically distributed or symmetric, our artificial noise model comprises seven conditions. We have also set out the methods for LNL, of which noise mitigation is the subset that acts on the labels, and noted that no single method wins on every dataset. NoiseInject injects these noise conditions and reports the robustness and uncertainty metrics, and the section on evaluation methodology fixes the metrics. Chapter 3 asks whether the representation or the model architecture governs how strongly a predictor degrades under label noise, through the four questions set out in Chapter 1. The representations, similarity definitions and noise conditions introduced here are reused in Chapters 4 to 6.

Chapter 3

Noise Robustness Across Molecular Representations and Model Architectures

3.1 Introduction

While there have been many investigations into the effect of noise on molecular property prediction, direction comparisons between how different molecular representations responds to label noise remains limited. The choice of molecular representation certainly affects predictive performance. Deng et al. [101] evaluated 62,820 models and found that fixed representations, in most datasets RDKit 2D descriptors, outperformed representations learned from SMILES and molecular graphs. Whether these conclusions translate to noise robustness is our question. Between different types of label noise, Heid et al. [76] found no difference in model performance between error distributions with the same mean and standard deviation. We expand on this by considering alternative patterns of noise, namely noise concentrated in structural groups or on a sparse subset of samples.

This chapter addresses the questions set out in Chapter 1, specifically how the choice of representation and model architecture explain the variance in predictive performance and in noise robustness, what is the impact of the type of label noise, and whether uncertainty estimates track label noise.

3.2 Methods

We looked at six representations paired with thirteen models, and several variants models, across four prediction tasks. Every pairing was run under Gaussian, grouped-wider and grouped-shifted noise at seven noise levels, and the other four noise conditions (Section 2.5.1) were run on selected pairings. The hyperparameters of every model are given in Additional file 1.

3.2.1 Datasets

Although we are investigating the effects of label noise stemming from experimental settings, for the purposes of testing we inject controlled amounts of artificial noise into clean labels. Data were extracted from the QM9 molecular property dataset distributed by PyTorch Geometric [102], which contains 130,831 small organic molecules with pre-computed quantum-mechanical properties obtained from density functional theory (DFT) calculations at the B3LYP/6-31G(2df,p) level of theory. We selected the gap between the highest occupied and the lowest unoccupied molecular orbital (HOMO–LUMO gap) as the prediction target in QM9. We assumed that QM9 labels were free of noise, notwithstanding approximations in the level of theory. Each experiment was replicated 10 times with a distinct random seed and a randomly selected subset of $N = 10{,}000$ molecules, split 80/10/10 into 8,000 training, 1,000 validation and 1,000 test molecules. All models were fitted on those 8,000 training molecules, except the Gaussian processes, which were fitted on a random subset of 5,000 of them; the validation and test molecules were unchanged.

We repeated these experiments on three molecular property datasets with experimentally determined endpoints. From the OpenADMET initiative [103], we used LogD and Caco-2 efflux permeability, with $N = 5{,}039$ and $N = 2{,}161$ respectively. We also selected hERG K_i data from ChEMBL [104], following a protocol inspired by Landrum and Riniker [14]: filter for binding assays, deduplicate by median pChEMBL value, and remove compounds with an inter-assay standard deviation greater than 1.0 log unit, resulting in $N = 1{,}415$ compounds. Each of the three was partitioned into five folds by grouped k-fold cross-validation (CV), and each fold's training block gave up a further grouped fifth as a validation set for early stopping. A model therefore fits between 3,194 and 3,364 of the 5,039 LogD molecules, depending on the fold, so the Gaussian-process subsample never applies to these datasets. Unlike QM9's 10 replicates, which are 10 independent draws from

one pool, each assay dataset is a single fixed set in which every molecule is tested exactly once and nothing is redrawn.

All splits and CV folds were grouped by the chirality-blind Bemis–Murcko scaffold, implemented such that each distinct acyclic molecule forms its own group, and groups are filled in random order. Prior to modeling, all molecular structures were sanitized and canonicalized using `RDKit` to remove invalid valence states and standardize atom and bond typing [30]. Prediction targets were standardized to zero mean and unit variance. Experiments that track uncertainty were run once rather than in replicates.

3.2.2 Representations

In this study, we evaluated six molecular representations: extended-connectivity fingerprints of diameter four (ECFP4), Avalon, Sort & Slice (SNS), physicochemical descriptors (PDV), ChemBERTa and Molecular Hypergraph Grammar GNNs (MHG-GNNs). The same six were used on QM9 and on the three assay datasets. ECFP4 is a Morgan fingerprint with radius $r = 2$ with $d = 2048$ bits [3, 30]. Sort & Slice takes the same substructures as ECFP4, but it ranks them by prevalence and selects the top L ($L = 1024$) [5]. We fitted the Sort & Slice vocabulary on the training molecules alone, refitting it for each QM9 replicate and for each CV fold. Avalon is a 2,048-bit fingerprint, generated with the Avalon toolkit through RDKit [6, 30]. The PDV representation is a vector of 200 physicochemical descriptors computed with RDKit's `MolecularDescriptorCalculator` [29, 30]; the 200 are listed in Additional file 1, and the list is fixed in our code rather than taken from RDKit's default descriptor set, which grows between releases. ChemBERTa embeddings are the mean over the non-padding token embeddings of the `DeepChem/ChemBERTa-77M-MTR` checkpoint, producing 384 features [26]. MHG-GNNs are a GIN-based autoencoder pre-trained on about 1.38 million PubChem molecules using β-VAE loss, producing 1024-dimensional embeddings through iterative message passing and found in the published `mhggnn_pretrained_model_0724_2023` checkpoint [7]. We z-score normalized PDV, ChemBERTa and MHG-GNN per feature, using the training molecules' mean and standard deviation; ECFP4, Avalon and Sort & Slice reach the model as built.

3.2.3 Models

To understand model behavior in the presence of label noise, we explored a range of model families. We fitted tree ensembles, a support vector machine, Gaussian processes and neural networks. Several are variants of one another, differing in the probabilistic machinery they add, and Additional file 1 gives every configuration with its settings.

Random Forests (RFs) are one of the most common choices for QSAR modeling thanks to their robustness and interpretability [40]. Quantile Regression Forests (QRFs) hold the distribution of training labels in each leaf and compute quantiles over all trees [105]. While we used 100 trees for RF, we observed a noisy quantile estimate with 100 trees for QRF, prompting us to use 300 instead. To observe the effect of the quantile mechanism, we then fit a second RF at 300 trees so the two models were identical in all other settings.

We also used eXtreme Gradient Boosting (XGBoost) [55], Light Gradient-Boosting Machine (LightGBM) [58], and Natural Gradient Boosting (NGBoost) [59]. NGBoost extends deterministic gradient boosting by treating the parameters of a chosen parametric distribution as regression targets and learns them through boosting with a natural gradient update rule [59]. NGBoost relies on scoring off validation data, which differs by dataset. On QM9 the validation split is already held out of every fit. However, on the three assay datasets the validation set is a scaffold-grouped fifth of the rows NGBoost is handed, which already exclude the validation molecules.

Support vector machines (SVMs) are a well-established baseline in QSAR modeling [40, 60]. We used a radial basis function (RBF) with $C = 1.0$ and $\gamma = \texttt{scale}$ as the kernel, as it accommodates continuous vectors.

Gaussian processes (GPs) are a non-parametric class of models that use a kernel to produce a Gaussian predictive distribution over every data point [106, 107]. We fitted three variants of GPs, each with a constant mean and a scaled kernel. The first uses an RBF kernel and runs on all six representations. The second uses the Tanimoto kernel [65, 66] and was run on ECFP4 alone. The third is a heteroscedastic variant in which the RBF process is given a second network that predicts the observation noise of each molecule. One of the major limitations of using GPs in practice is computational scaling; inference has $O(N^3)$ time complexity and $O(N^2)$ memory, where N is the number of training points [107]. This can be mitigated

with sparse GP approximations, but these introduce approximation error and can be more difficult to tune [108]. As such, we used a standard GP and only fit on at most 5,000 training molecules, drawn by a seeded sample. Due to the size of the datasets, this cap only affected QM9. We implemented these GP models in `GPyTorch` [109], with the Tanimoto kernel taken from the `Gauche` framework [64].

We tested two deterministic feed-forward architectures, both fully connected and both implemented in PyTorch [110]: (i) NN-α with two hidden layers of sizes 128 and 64, and dropout after each hidden layer and (ii) NN-β with two hidden layers, each of size 128, with dropout applied before the output layer. Each model was trained using the Adam optimizer with early stopping on validation loss. We used Bayesian transformations to convert NNs into Bayesian neural networks (BNNs), and implemented them using `PyTorch` [110] with ReLU activations and dropout regularization ($p = 0.2$). Additional file 1 gives their full widths, dropout fractions, learning rates and batch size.

To get these networks producing uncertainty estimates, we applied four probabilistic transformations to each architecture, producing eight probabilistic networks. The first replaces every linear layer with a Bayesian layer carrying a Gaussian prior $\mathcal{N}(0, 0.1^2)$ on its weights, producing BNN-α and BNN-β. They are trained on an evidence lower bound whose Kullback–Leibler (KL) divergence term is divided by the number of training molecules. While this is a straightforward approach, it is computationally expensive. We explored alternative techniques like our second variant, where we add an output head to the standard NN and fit the negative log likelihood of a Gaussian with that variance [111]. The third, Variational Bayesian Last Layer (VBLL), replaces the output layer with a Bayesian last layer that produces mean-field variational posteriors $q(\mathbf{W}) = \mathcal{N}(\boldsymbol{\mu}_W, \mathrm{diag}(\boldsymbol{\sigma}_W^2))$ over the last-layer weights and trains by maximizing the evidence lower bound. This can be thought of as minimizing the reconstruction loss plus KL divergence $D_{\mathrm{KL}}(q(\mathbf{W})\|p(\mathbf{W}))$ from a standard normal prior, scaled by $1/N$ [112]. Because only its output layer keeps a learned observation-noise parameter, VBLL networks report only epistemic uncertainty. The fourth makes observation noise a function of the input. All eight probabilistic networks estimate their predictive distributions from 100 stochastic forward passes. Each network and its Bayesian counterpart were tuned to the same set of parameters so that NN-α against BNN-α and NN-β against BNN-β differ by the Bayesian layers alone.

The thirteen base models are RF, QRF, XGBoost, LightGBM, NGBoost, SVM,

the RBF-kernel GP, and NN, BNN and VBLL in each of the two architectures. The six variant models are the random forest with 300 trees, RF (300 trees); the heteroscedastic GP, GP (het.); the Bayesian networks with a variance head, BNN-α (var. head) and BNN-β (var. head); and the heteroscedastic VBLL networks, VBLL-α (het.) and VBLL-β (het.).

3.2.4 Noise conditions and levels

The seven noise conditions used in this chapter are defined in Section 2.5.1.

We controlled noise quantity using noise levels, representing a fraction of the standard deviation of clean training labels. We ran experiments across seven levels, 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5. We added the noise to the raw label and standardized the target afterwards, using the clean training mean and spread. To control the quantity of noise for censoring, we recorded every label above the kth largest clean training label as that limit and used this level as the fraction clipped, at 0, 0.1, 0.2, 0.25, 0.3, 0.4 and 0.5. Censoring is common in pharmaceutical assays, where a third or more of labels can be censored [78].

We ran Gaussian, grouped-wider and grouped-shifted noise on every model and representation pairing. The remaining four conditions ran on selected pairs.

On QM9, Laplace, Student-t and outlier noise were run on eight models, namely RF, NGBoost, SVM, the GP, GP (het.), BNN-α (var. head), VBLL-α (het.) and VBLL-β (het.), each on ECFP4, PDV and ChemBERTa. Censoring was run on five pairings, namely RF and GP (het.) on ECFP4, and NGBoost, the GP and BNN-α (var. head) on PDV. On the three assay datasets, $\mathrm{AUC}_{\mathrm{norm}}$ under censoring was computed for the same five pairings, and the uncertainty statistics under censoring for QRF and the other models that give an uncertainty estimate, on ECFP4, PDV and ChemBERTa.

3.2.5 QM9 data pipeline

QM9 [102], loaded through PyTorch Geometric, was processed by a separate pipeline where the Python code loads and splits the data, and computes several representations. The data is then passed to the Rust portion of the pipeline which injects the label noise, standardizes the labels, computes the remaining representations, and sends the processed data back to Python which trains and scores the models.

Each replicate, using its own random seed, randomly selects a subset from QM9, split using scaffold splits.

. Each of these subsets is split new scaffold split (Datasets subsection), and the same grouping defines the scaffold families targeted by the grouped noise conditions.

The six representations described above are computed in Python. The run halts if any molecule has an ECFP4 or Avalon fingerprint that is entirely zero, so that a molecule that failed featurization cannot enter training.

Traditional approaches load entire datasets into memory from CSV or Parquet files, creating RAM bottlenecks at scale and redundant deserialization overhead across experimental replicates. Our pipeline instead serializes preprocessed data into custom memory-mapped binary files (`.mmap`) that enable on-demand access with constant low memory footprint regardless of dataset size (Figure 3.1). Python writes one file per split, the Rust program reads each file and writes a new one in which the processed labels replace the raw ones, and Python then reads the new files back for training.

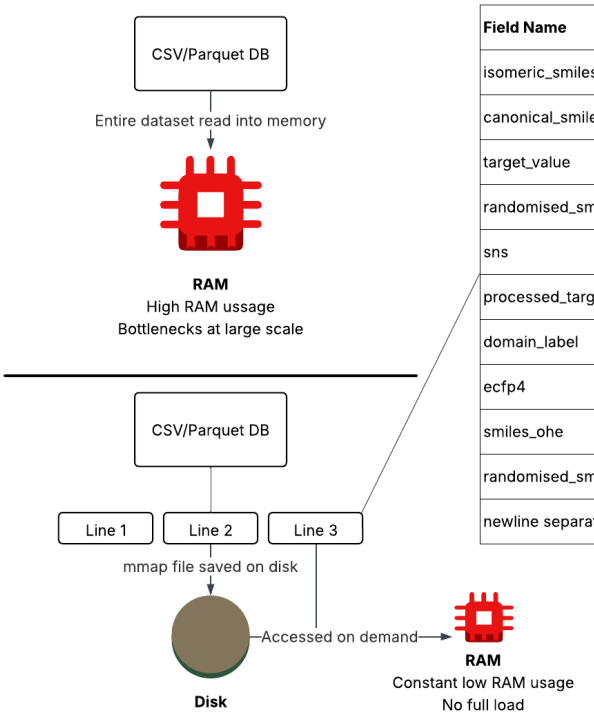
Each memory-mapped file stores a sequence of fixed-format records. A record holds the isomeric and canonical SMILES strings in UTF-8 with a length prefix, the raw target value as a 4-byte float, the requested representations as fixed-width blocks, and the processed target value, which includes the noise and normalization, as a 4-byte float. The blocks occupy 2,048 bytes for Sort & Slice (1,024 16-bit integers), 800 bytes for PDV (200 32-bit floats), 1,536 bytes for ChemBERTa (384 32-bit floats), 4,096 bytes for MHG-GNN (1,024 32-bit floats), and 256 bytes each for Avalon and ECFP4. Only the requested representations are written, so the byte offset of every field depends on the list of representations. This format permits zero-copy memory mapping, in which Python and Rust access the same disk-backed memory pages without deserialization.

The Rust program draws the noise for each split separately, following the order in which that split's records are stored. The noise parameters are set on the training split, since the amount of noise, the censoring limit and the affected scaffold families are all computed from the clean training labels. Validation labels receive an independent draw, from a seed derived from the training seed, with the same amount of noise in label units, the same limit and the same affected families, and the program verifies that validation received as much noise as training. Test labels are never corrupted, although every split records the per-molecule noise scale its region would have received, so that held-out molecules can be scored against the pattern applied

in training.

For every molecule in every split, the Rust program records its canonical SMILES, clean label, injected noise, noise scale, the noise pattern applied to its region, noisy label and the standardized value written to file. It also writes a summary of each run that lists the parameters of the noise condition and the amount of noise delivered. The run halts if the record at any position holds a different molecule when it is written than when its noise was drawn. A separate self-test mode applies the same checks on the amount and targeting of noise to a label file, without running the rest of the pipeline.

The pipeline then trains the models described in the Models subsection, with the default and tuned hyperparameters given in Additional file 1.



Field Name	Required?	Data Type & Size	Size
isomeric_smiles	Yes	UTF-8	Variable length
canonical_smiles	Yes	UTF-8	Variable length
target_value	Yes	float32	4 bytes
randomised_smiles	Optional	UTF-8	Variable length
sns	Optional	uint8[16]	16 bytes
processed_target	Yes	float32	4 bytes
domain_label	Optional	uint8	1 byte
ecfp4	Optional	uint8[32]	32 bytes
smiles_ohe	Optional	bit-packed uint8[]	Variable size
randomised_smiles_ohe	Optional	bit-packed uint8[]	Variable size
newline separator	Yes	ASCII '\n' character	1 byte

Figure 3.1: In-memory (top left) and memory-mapped (bottom left) data loading. Memory-mapped loading keeps each molecule as one record of a binary file on disk and reads records into memory on demand. The table on the right lists the fields of one record.

3.2.6 Uncertainty

Up to this point we have been concerned with how noise affects the predictive performance of QSAR models. However, label noise should also affect the uncertainty

these models report. Some models return only a point prediction, while others also return an estimate of uncertainty, either as a predicted distribution or as the spread across an ensemble. Furthermore, some report a single value of uncertainty while others report one that can be decomposed into an aleatoric and an epistemic component [111]. The aleatoric component describes underlying noise the model associates with the data, while the epistemic component represents the uncertainty within the model's fit. We sought to determine if, as expected, the aleatoric component rises with injected label noise while the epistemic component does not.

For the BNN and VBLL transformations, we took the two components over $T = 100$ stochastic forward passes [111]:

$$\hat{u}^2_{\text{ale}}(x) = \frac{1}{T}\sum_{t=1}^{T} \hat{v}_t(x), \qquad \hat{u}^2_{\text{epi}}(x) = \frac{1}{T}\sum_{t=1}^{T} \big(\hat{\mu}_t(x) - \bar{\mu}(x)\big)^2, \tag{3.1}$$

where $\hat{\mu}_t(x)$ and $\hat{v}_t(x)$ are the mean and the variance returned for molecule x on pass t, and $\bar{\mu}(x)$ is the mean over passes. Here $\hat{v}_t(x)$ is the predicted observation noise variance. It varies per molecule in the variance-head and heteroscedastic variants, and is one number for the whole fit in the VBLL transformation. A full-BNN without a variance head returns no variance, so its uncertainty is epistemic alone.

The other models follow the same split. For GPs, the epistemic component is the posterior variance of the latent function and the aleatoric component is the learned observation noise [107]. All variants of the GP produce one number for the whole fit except heteroscedastic. For QRF models, each tree holds multiple training molecules. So for a given molecule, the aleatoric component is the variance the labels the molecule's given leaf. The total aleatoric uncertainty are those variances averaged over the trees. The epistemic component is the variance of the leaf means across the trees. NGBoost predicts one variance for each molecule from a single fit, which is aleatoric alone [59]. In every model, the total uncertainty is the sum of the aleatoric and epistemic components.

No held-out label carries injected noise, so every comparison against the injected noise was made on training molecules. Each training set was divided into five folds grouped on Murcko scaffolds, and each molecule was scored by a refit that excluded its own fold. For NGBoost on QM9, three of the five folds were scored to limit its cost. No uncertainty was calibrated after fitting.

3.2.7 Evaluation

We compute statistics on the held-out molecules, for QM9 the test split and for three assay datasets the held-out fold of the scaffold CV. Different combinations of dataset, model, representation, noise condition, and noise level are examined in each scenario. We used the coefficient of determination (R^2) throughout. For molecule i, let y_i be the clean label, $\hat{y}_i$ the prediction, ϵ_i the noise injected into its label and u_i the predicted uncertainty.

To evaluate the effect of label noise, we tracked predictive performance retention under increasing artificial noise. For each QSAR pairing we recorded $R^2(\tau)$ at each noise level τ, and normalized it to obtain the retention ratio $R^2(\tau)/R^2(0)$. To quantify robustness, we used the normalized area under this retention curve,

$$\mathrm{AUC}_{\mathrm{norm}} = \frac{1}{\tau_{\max}} \int_0^{\tau_{\max}} \frac{R^2(\tau)}{R^2(0)}\, d\tau, \tag{3.2}$$

evaluated using the trapezoidal rule over the seven noise levels, $\tau \in \{0, 0.2, 0.3, 0.5, 0.75, 1.0, 1.5\}$, in the units of Equation 2.4. While we tested other metrics of robustness, we found $\mathrm{AUC}_{\mathrm{norm}}$ to be less directly correlated to clean-label R^2 predictions. For censoring, we sweep over a set of fractions to clip the labels prior to normalization.

An $\mathrm{AUC}_{\mathrm{norm}}$ value near 1 indicates that the QSAR model in question has retained predictive performance with artificial noise, meaning it is more robust to noise. Since we normalize the curve by its own clean-label predictive performance, a model with a low clean R^2 has less to lose, so $\mathrm{AUC}_{\mathrm{norm}}$ is reported beside clean R^2. We excluded replicates with a clean $R^2 < 0.3$ from the robustness analysis (Additional file 2), as performance retention ratios become unstable when the clean-label denominator approaches zero. On QM9 one $\mathrm{AUC}_{\mathrm{norm}}$ is computed per replicate, and we report the median over the 10 replicates. On the three assay datasets one value is computed per scaffold fold, and the median over the five folds is reported.

To compare the relative effect of the 13 base models across datasets and noise conditions, we compared their rankings using Spearman's rank correlation coefficient (ρ) between clean R^2 and $\mathrm{AUC}_{\mathrm{norm}}$. We compared probabilistic models with its deterministic counterpart using a two-sided Wilcoxon signed-rank test paired on the replicate, at $\alpha = 0.05$, separately for each representation and noise condition. This test was run only on QM9, as with five scaffold folds the smallest attainable p is

0.0625.

We conducted a separate analysis of variance (ANOVA) for each noise condition to identify the relative contributions of molecular representation and model architecture to both predictive performance and noise robustness. For a given metric y, either R^2 at a noise level of 1.0 or $\mathrm{AUC}_{\mathrm{norm}}$:

$$y_{fmjr} = \mu + s_r + \alpha_f + \gamma_{m(f)} + \beta_j + (\alpha\beta)_{fj} + (\gamma\beta)_{m(f)j} + e_{fmjr}, \tag{3.3}$$

where s_r represents the effect of replicate r, α_f represents the effect of model family f, $\gamma_{m(f)}$ represents the effect of model architecture m within its family, β_j represents the effect of molecular representation j, $(\alpha\beta)_{fj}$ and $(\gamma\beta)_{m(f)j}$ are the interaction terms, and e_{fmjr} is the residual. The proportion of variance explained by each term was calculated as the effect size

$$\eta^2_{\mathrm{term}} = \frac{SS_{\mathrm{term}}}{SS_{\mathrm{total}}}, \tag{3.4}$$

using Type I (sequential) sums of squares, entered in the order replicate, model family, model architecture within family, molecular representation, interaction. The 10 QM9 replicates provide the residual. The uncertainty on each share is the 95% interval of η^2 over 1,000 bootstrap resamples of whole replicates. These shares describe the models and representations tested here, not the choice of model or representation in general.

ANOVA relies on the assumption that observations both within and across groups are independent. Two features of this study break that assumption. First, each QM9 replicate uses the same data split and the same noise draw for every model and representation, so observations from one replicate are not independent of each other; we therefore entered the replicate as a block. Second, across the whole set of models in this study, some pairs produced similar predictions that are not independent, so a method present in several near-identical versions would count several times towards the share of model architecture. To handle this issue, we grouped the models into families. We considered grouping pairs of models that share a method or an architecture, and compared each pair on clean R^2 and on $\mathrm{AUC}_{\mathrm{norm}}$ under the three noise conditions run on every representation, separately on each of the six representations, which gives 24 comparisons per pair. A pair was grouped when the mean difference between the two models was smaller than their standard deviation

across replicates in at least two thirds of these comparisons (Additional file 3). Five families met this criterion: RF with quantile regression forests (QRF; 16 of 24), XGBoost with LightGBM (23 of 24), NN-α with BNN-α (20 of 24), NN-β with BNN-β (18 of 24), and VBLL-α with VBLL-β (20 of 24). VBLL did not group with the plain or Bayesian network of its own architecture (4 to 10 of 24), as its clean R^2 was lower by 0.02 to 0.09, and NGBoost did not group with XGBoost or LightGBM (2 and 3 of 24). The two architectures of the plain networks, and of the Bayesian networks, also met the criterion (18 and 17 of 24); we grouped by architecture instead, which the evidence supports equally well. The 13 base models therefore form eight families. Within these families, the differences between models accounted for at most 1.0% of the variance in any outcome and noise condition on QM9, and up to 15.1% of predictive performance on clean labels on the three assay datasets. We also ran Tukey's honestly significant difference test between the members of each family, separately for each representation (Additional file 4). With 10 replicates it detects differences smaller than the variation between replicates, so we report it beside the size of each difference rather than use it to form the families.

Across representations, we considered grouping SNS with the ECFP4 fingerprint, as both encode overlapping circular substructure information. The two differed by more than their variation across replicates on clean R^2 for every model (0 of 13), so all six representations were kept as separate levels (Additional file 5).

To assess the interaction between uncertainty estimation and label noise, we posed three questions. First, whether uncertainty rise with label noise, as measured by the mean u_i over held-out molecules at the highest noise level divided by its value with no added noise, in each replicate or fold. The second pertains to models who report separate aleatoric and epistemic components. We divided the increase of mean epistemic uncertainty between the highest level of noise and clean labels by the mean of the aleatoric component to determine how each responds to label noise. A ratio near 0 means the added noise was attributed to the labels while a ratio of 1 or above means the epistemic component grew as much or more. We also compared the mean aleatoric component with the standard deviation of the injected noise. The third question we had was whether per-sample uncertainty values track injected label noise, as measured by the Spearman correlation between u_i and $|\epsilon_i|$ over out-of-fold training molecules, per fold and per noise level. We also report the Spearman correlation between u_i and the absolute error of the prediction. Finally, we asked whether dividing a molecule's out-of-fold absolute error by u_i ranks the

corrupted labels better than the error alone, and report the difference between the two Spearman correlations with $|\epsilon_i|$.

3.3 Variance decomposition

Considering the two main components of a QSAR model, the molecular representation and the model architecture, we sought to understand how each contributes to both predictive performance and noise robustness. ANOVA decomposition divides the variance between model family, model architecture within its family, molecular representation, the pairing of the two, the replicate, and a residual. We ran this decomposition for Gaussian noise, the standard model, as well as methods whose injected noise is related to the feature space, grouped-wider and grouped-shifted (Figure 3.2, Table 3.1). For predictive performance measured in R^2 at a noise level of 1.0, model family and molecular representation account for comparable shares of the variance, 26.6% and 29.0% under Gaussian noise. However, for noise robustness, quantified by the normalized area under the R^2 retention curve (AUC_{norm}; higher values indicate greater robustness), model family accounts for 47.2% of the variance while molecular representation for 8.1%. This pattern is observed across noise conditions and makes sense, as label noise does not impact the features. The interaction between model and molecular representation explains 19.9% of the variance in AUC_{norm}, more than twice the 8.1% explained by representation alone, meaning representation acts on noise robustness mainly through the model it is paired with. As such, we primarily compare representations within each model.

We compare the variance decomposition on both clean and noisy labels (Figure 3.3a). For predictive performance, the artificial label noises lowers the impact of the choice of model family, while the replicate and residual take more. This means that the variance is left more up to chance and to the underlying variation within the data itself with label noise, which is the expected behavior. Robustness divides the variance a bit differently. Representation explains 8.1% of the variance in AUC_{norm}, against 31.1% of clean R^2, while model family rises to 47.2%. The replicate nearly disappears from AUC_{norm}, likely because each replicate's curve is divided by that replicate's own clean R^2, so a difficult split lowers both and cancels out.

We include two terms which stem from the data rather than the model, the residual and replicate terms. The replicate measures how much of the variance can be attributed to the train and test split, which in this case is a scaffold split. On QM9

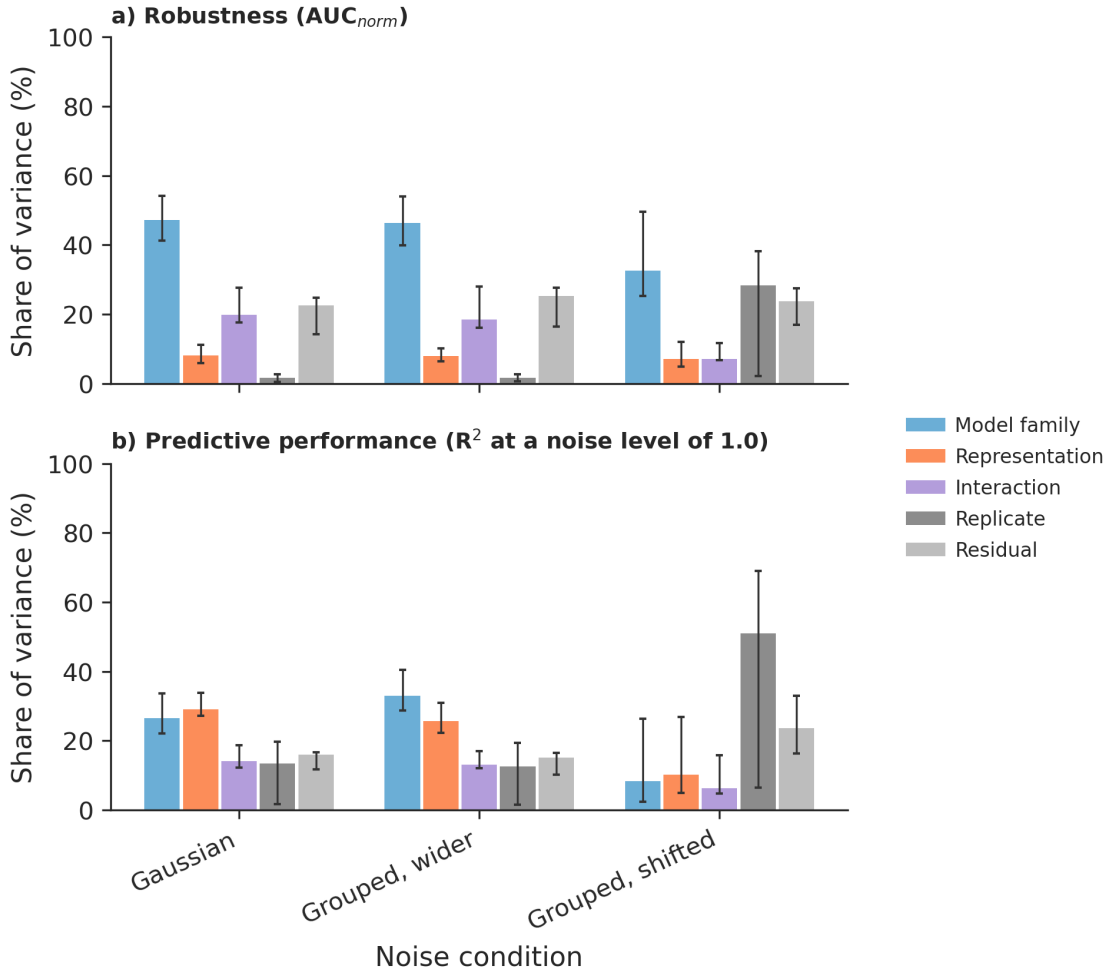


Figure 3.2: Share of the variance explained by each term, on the QM9 HOMO–LUMO gap: a) robustness (AUC_{norm}); b) predictive performance (R^2 at a noise level of 1.0). Whiskers are 95% bootstrap intervals over replicates. Model within family, at most 1.0% in every decomposition on QM9, is not drawn.

we replicated each experiment 10 times with different random draws of $N = 10{,}000$, each time giving a new scaffold split. On the assay datasets, a replicate refers to one of the five scaffold CV folds. A large replicate term in ANOVA indicates that the folds differ from one another. The residual is variance that no other term accounts for.

While we observe similar patterns between noise conditions, the predictive performance of grouped-shifted noise in particular varies significantly between replicates, resulting in a larger share for the replicate. We find that 28.4% of the variance in AUC_{norm} and 51.0% of the variance in R^2 are due to the replicate, with model family falling to 32.7% and 8.4%. This indicates that the offset applied to each scaffold may account for more of the variance in both R^2 and AUC_{norm}.

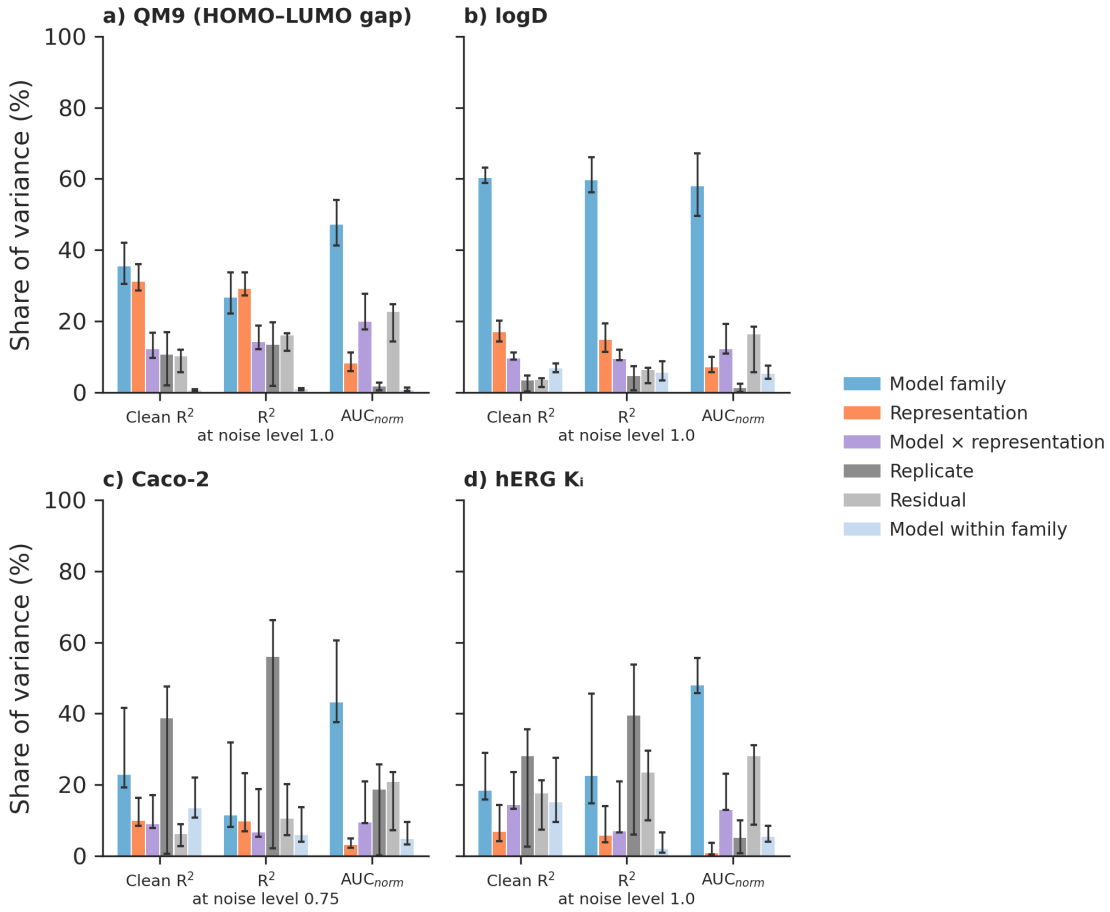


Figure 3.3: Share of the variance explained by each term on a) the QM9 HOMO–LUMO gap, b) logD, c) Caco-2 and d) hERG K_i, for clean R^2, and for R^2 at a noise level of 1.0 (0.75 on Caco-2) and AUC_{norm} under Gaussian noise. Whiskers are 95% intervals from 1,000 bootstrap resamples of replicates: 10 on QM9 and five scaffold folds on each assay dataset. The values and their intervals are in Additional file 6.

Condition	Outcome	Replicate	Model family	Representation	Interaction	Residual
None	Clean R^2	10.6	35.4	31.1	12.1	10.2
Gaussian	AUC_{norm}	1.7	47.2	8.1	19.9	22.6
Grouped, shifted	AUC_{norm}	28.4	32.7	7.0	7.2	23.7
Grouped, wider	AUC_{norm}	1.7	46.3	7.9	18.5	25.3
Gaussian	R^2 at level 1.0	13.4	26.6	29.0	14.2	16.0
Grouped, shifted	R^2 at level 1.0	51.0	8.4	10.3	6.2	23.7
Grouped, wider	R^2 at level 1.0	12.7	33.0	25.7	13.1	15.1

Table 3.1: Share of the variance (%) explained by each term on the QM9 HOMO–LUMO gap, for 13 base models and six representations over 10 replicates. Model within family, at most 1.0% in every row, is left out, so a row does not sum to 100%. Intervals are in Additional file 6.

3.4 Predictive performance and noise robustness

The variance decomposition tells us what aspect of QSAR modeling affects predictive performance and robustness, however it does not tell us about the characteristics of individual models. All model's performance degrades as the labels get increasingly corrupted with noise (Figure 3.4). The loss remains small until the artificial noise reaches about half the spread of the clean training labels, and then it begins to drop. The rate it drops and the quantity of noise retained is not necessarily correlated to the model's predictive performance on clean labels. Across the 13 base models under Gaussian noise at ECFP4, clean R^2 and AUC_{norm} rank in opposite directions at a Spearman correlation of -0.10 ($p = 0.73$) (Figure 3.5). If anything, the lower a model's predictive performance on clean labels, the less it has to lose as the labels get noisier.

The link between predictive performance and noise robustness can be examined by model family (Figure 3.5). The forests, boosted trees, SVM, and GP all retain relatively high predictive performance on clean labels while maintaining relatively strong robustness to noise. The plain NNs and BNNs are among the highest in predictive performance on clean labels, and are the four least robust. The two variational networks sit between these groups, with lower predictive performance on clean labels and middling noise robustness. Standard feed-forward networks can make strong predictions under clean labels, however they are not a safe bet under noisy ones.

NGBoost is a standout case in the sense it is highly dependent on its paired molecular representation. Under Gaussian noise it has the highest AUC_{norm} on every one of the six representations, yet also the lowest in predictive performance on four of the six representations. It is one of the worst performing models with respect to R^2 on clean data. Its high robustness is partially due to the fact that when you start with a low predictive performance, you don't have as much to lose. However, it maintains shallow robustness curves across representations. On ECFP4 NGBoost's predictive performance ranks lowest up to a noise level of 0.75 and stays in the lower half at every noise level. However, on PDV it starts second lowest and ends highest, surpassing the other models between noise levels 1.0 and 1.5. So when paired with the right features, NGBoost has the potential to make better predictions under higher quantities of label noise. On group-shifted noise, RF has higher AUC_{norm} values than NGBoost on ECFP4, Avalon and ChemBERTa. Even then, NGBoost

reports the highest noise robustness for PDV, MHG-GNN and Sort & Slice. While NGBoost performs well under higher label noise, LightGBM performs well under smaller quantities of noise. Its performance drops sharply after noise level 1.0.

As we've seen, the choice of representation can have an impact on noise robustness, but that impact is determined by model family, represented by the interaction term between model and representation in Figure 3.6. Some models are more impacted by representation than others, though. In general, networks span a wider range of AUC_{norm} across the six representations than any of the other models. Between ECFP4 and PDV under Gaussian noise, the four plain and Bayesian networks differ by between 0.037 and 0.065 of AUC_{norm}, while no tree or kernel model shifts by more than 0.011. VBLLs are an exception, as no two variational networks differ by more than 0.019.

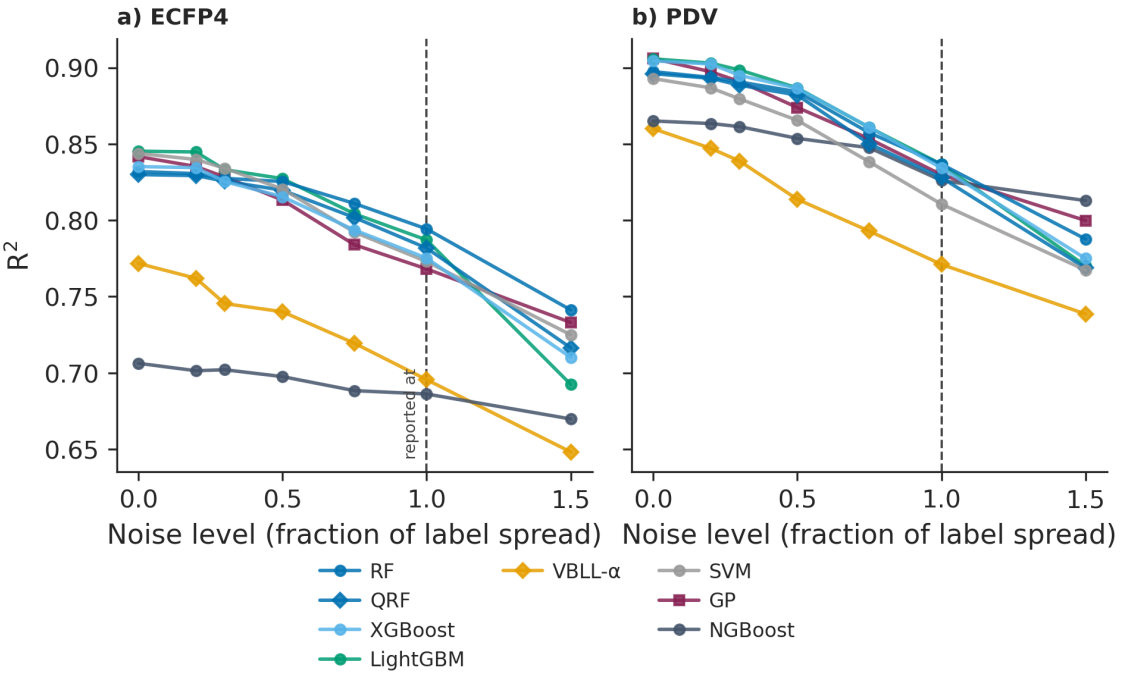


Figure 3.4: R^2 against noise level on the QM9 HOMO–LUMO gap under Gaussian noise, for the eight base models with the highest AUC_{norm} on ECFP4: a) ECFP4; b) PDV. Medians over 10 replicates. The dashed line marks the noise level of 1.0 at which R^2 is reported.

3.5 Artificial noise conditions

Many studies involving experimental label noise rely on the assumption that noise can be modeled by a Gaussian distribution. We decided to vary the distribution

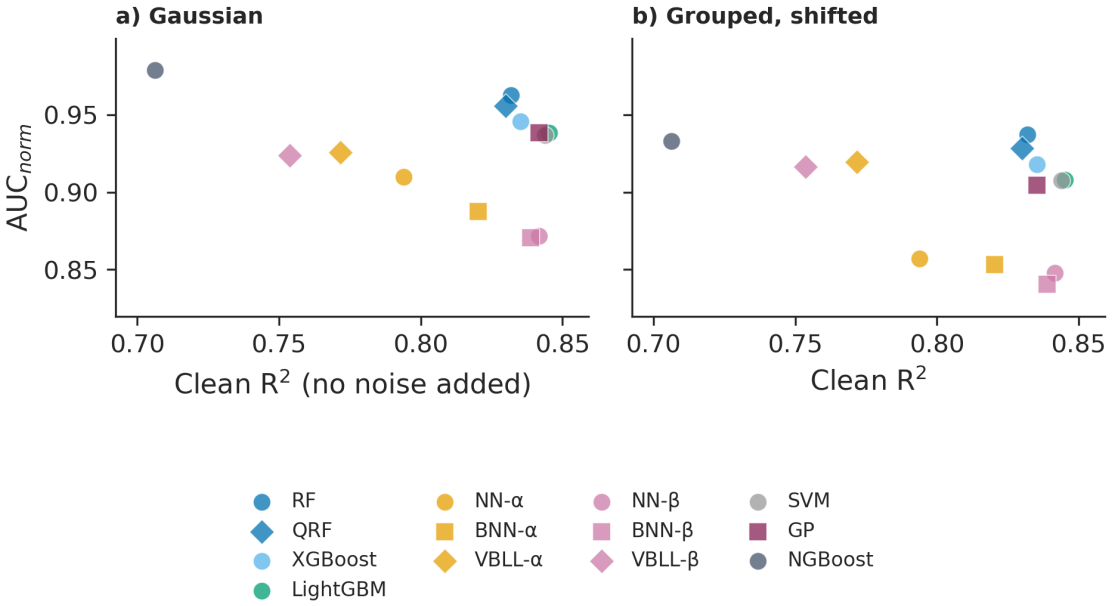


Figure 3.5: Clean R^2 against $\mathrm{AUC}_{\mathrm{norm}}$ for each base model on the QM9 HOMO–LUMO gap with ECFP4, under a) Gaussian and b) grouped-shifted noise.

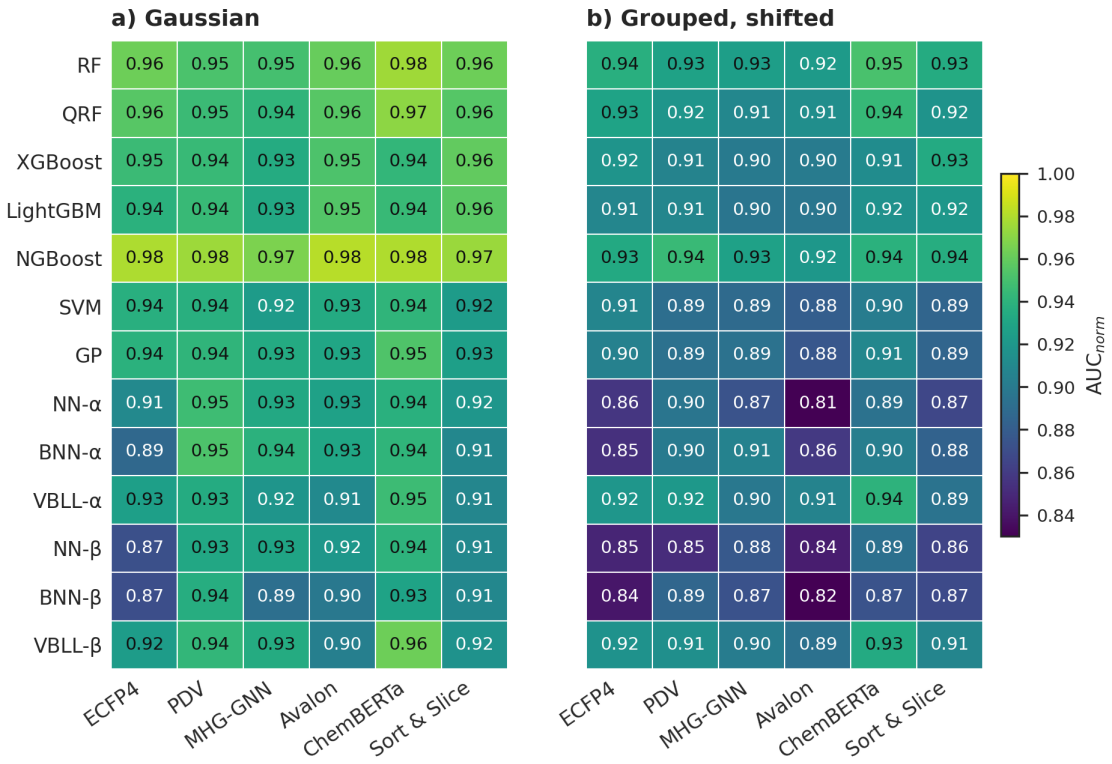


Figure 3.6: $\mathrm{AUC}_{\mathrm{norm}}$ of the 13 base models on each representation, on the QM9 HOMO–LUMO gap under a) Gaussian and b) grouped-shifted noise. Medians over 10 replicates, on one color scale. Excluded configurations are in Additional file 2; grouped-wider noise is in Additional file 7.

which noise is drawn from, as well as the methodology by which it's injected to specific labels. Five of the seven conditions draw each label's error on its own, so the error is random. Grouped-shifted gives a whole scaffold family a shared offset, and censoring pulls every label above a limit down to that limit, so in both the error is systematic. We controlled the quantity of noise by adding it in proportion to the spread of the clean labels on the training set, so a difference between two conditions is a difference in pattern rather than amount. One condition, censoring, could not be held to a set amount as its levels count the fraction of labels clipped.

We compare Gaussian noise on the HOMO–LUMO gap to Laplace, Student-t and outlier noise in Figure 3.7. The shape of the distribution from which the noise was drawn from made little difference. SVM reported higher $\mathrm{AUC}_{\mathrm{norm}}$ values on non-Gaussian label noise, 0.937 under Gaussian noise against 0.947, 0.952 and 0.955 under Student-t, outlier and Laplace noise on PDV, while NGBoost lost $\mathrm{AUC}_{\mathrm{norm}}$ under every one. NGBoost's relative stronger performance on Gaussian makes sense as it fits a per-molecule scale under a Gaussian likelihood, which the other distributions violate. SVM gains $\mathrm{AUC}_{\mathrm{norm}}$ and NGBoost loses it under all three distributions across all datasets and representations. The size of the shift stays small, which is why these results coincide with Heid et al. [76], who found no difference between error distributions drawn at a matched mean and standard deviation. The shape of the noise does not change how much performance a model keeps, but it does change which model keeps the most.

While alternative shapes of noise may be accurately modeled by the Gaussian distribution, we do not find that is the case for systematic label noise. Relative to Gaussian, grouped-shifted noise lowered the $\mathrm{AUC}_{\mathrm{norm}}$ of across all models and representations (Table 3.3). The models struggle to make predictions when the noise is isolated on molecules of specific scaffold families instead of being applied randomly across every sample. The variational networks retained the most predictive performance, while the plain, deterministic NNs lost the most. The predictive abilities of BNN-α degraded at a slower rate compared to its deterministic counterpart on every representation except PDV, while BNN-β did so only on MHG-GNN and PDV. Grouped-wider noise changed no model beyond its replicate spread except QRF on Avalon and MHG-GNN, which makes predictions particularly well under this noise condition. Censoring created the largest drop in predictive performance, $\mathrm{AUC}_{\mathrm{norm}}$ fell by 0.12 to 0.18 for the same QSAR models relative to Gaussian noise (Table 3.2).

Most of these observations are being made at the upper end of the noise range.

With ECFP4, six of the seven most robust models start within 0.015 of one another in R^2 for clean labels, stay together up to a noise level of 0.5 and separate only above 1.0, finishing between 0.692 for LightGBM and 0.741 for RF (Figure 3.4). We ranked the models by predictive performance at each noise level. Of the 13 base models, RF started ranked seventh, but by level 1.0 it was ranked first. SVM on the other hand falls from first to fifth (Figure 3.9). Because AUC_{norm} weights the whole range from 0 to 1.5 equally, and the first half of it separates little, it has the potential to understate just how much the pattern of the noise costs. However, choosing to look at performance of a set level has its drawbacks, as the ranks shift at a different pace between noise conditions.

Model	Representation	Clean R^2	Gaussian	Censoring
RF	ECFP4	0.832	0.963	0.817
GP (het.)	ECFP4	0.840	0.937	0.813
NGBoost	PDV	0.865	0.975	0.809
GP	PDV	0.906	0.941	0.815
BNN-α (var. head)	PDV	0.890	0.951	0.771

Table 3.2: AUC_{norm} under censoring and under Gaussian noise on the QM9 HOMO–LUMO gap, for the five model and representation pairs run under censoring. Medians over 10 replicates. A censoring level is the fraction of labels clipped, so the two AUC_{norm} columns are on different scales.

Model	ECFP4	PDV	MHG-GNN	Avalon	ChemBERTa	Sort & Slice	Exceeds spread
RF	-0.025	-0.024	-0.025	-0.038	-0.025	-0.028	6 of 6
QRF	-0.027	-0.027	-0.032	-0.042	-0.027	-0.038	5 of 6
XGBoost	-0.028	-0.039	-0.035	-0.052	-0.030	-0.025	5 of 6
LightGBM	-0.030	-0.037	-0.037	-0.058	-0.028	-0.037	5 of 6
NGBoost	-0.046	-0.030	-0.039	-0.062	-0.041	-0.039	6 of 6
SVM	-0.029	-0.045	-0.034	-0.053	-0.040	-0.038	5 of 6
GP	-0.034	-0.048	-0.040	-0.054	-0.045	-0.041	6 of 6
NN-α	-0.053	-0.051	-0.060	-0.115	-0.044	-0.053	4 of 6
BNN-α	-0.034	-0.053	-0.034	-0.073	-0.039	-0.025	2 of 6
VBLL-α	-0.006	-0.014	-0.020	-0.005	-0.007	-0.013	0 of 6
NN-β	-0.024	-0.080	-0.052	-0.076	-0.047	-0.045	4 of 6
BNN-β	-0.030	-0.050	-0.019	-0.075	-0.055	-0.041	2 of 6
VBLL-β	-0.007	-0.028	-0.033	-0.010	-0.028	-0.009	0 of 6

Table 3.3: Change in AUC_{norm} from Gaussian to grouped-shifted noise on the QM9 HOMO–LUMO gap, per base model and representation; negative is a loss. The last column counts the representations on which the loss exceeds the replicate spread, the highest minus the lowest AUC_{norm} over 10 replicates under Gaussian noise.

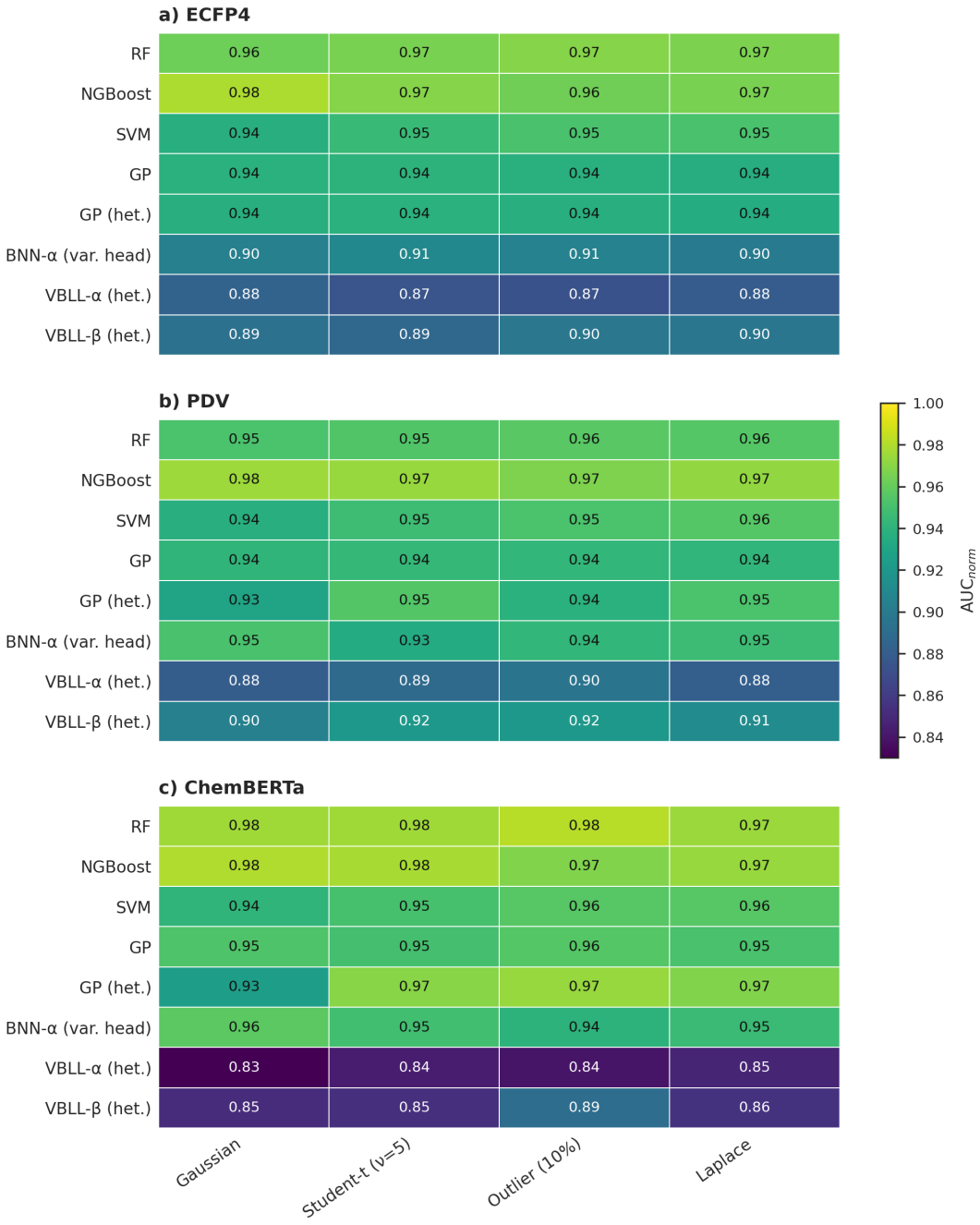


Figure 3.7: $\mathrm{AUC}_{\mathrm{norm}}$ under the noise conditions run on a subset of models, on the QM9 HOMO–LUMO gap: a) ECFP4; b) PDV; c) ChemBERTa. Medians over 10 replicates, on one color scale. Censoring is in Table 3.2.

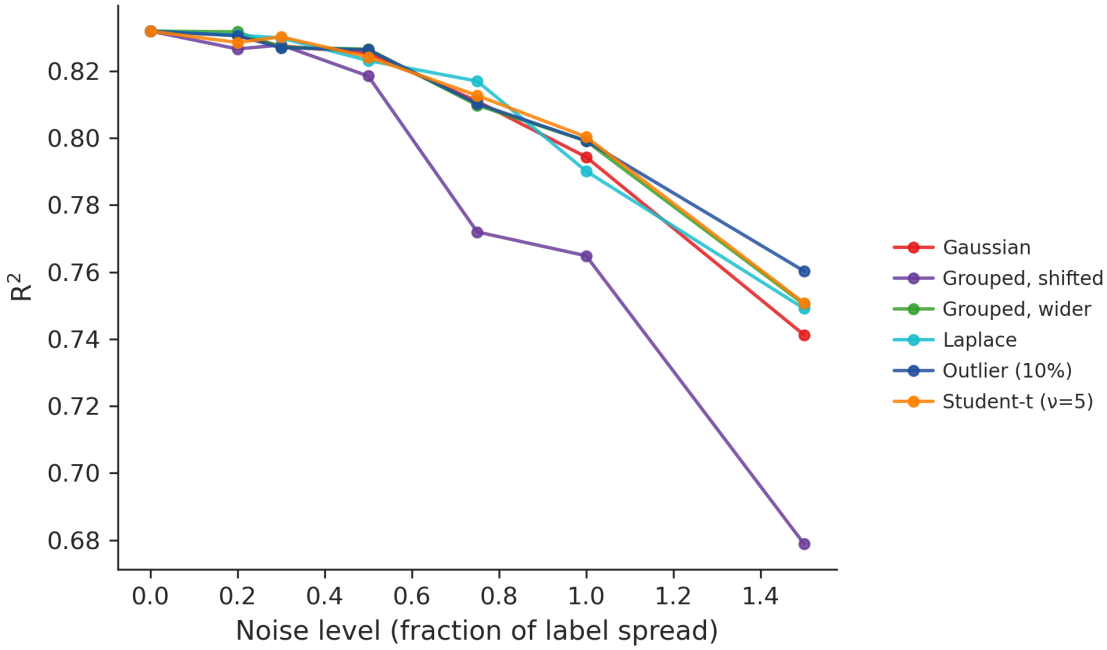


Figure 3.8: R^2 of the random forest against noise level under each noise condition, on the QM9 HOMO–LUMO gap with ECFP4. Medians over 10 replicates. Censoring is not shown, as its level is a fraction of labels clipped.

3.6 Robustness on the three assay datasets

So far, we have only discussed results coming on HOMO–LUMO gap from QM9, a computed property whose labels carry no measurement error. We repeated these experiments on three experimentally-obtained datasets, LogD, Caco-2 [103], and hERG K_i [104], replicating our process of adding artificial noise. Unlike QM9, the experimental endpoints carry their own measurement noise in both training and test labels; the injected artificial noise therefore adds on top of an unknown noise floor. Repeat measurements of the same compound disagree by about 0.54 log units for pK_i [10, 11]. The hERG K_i labels have a spread of 0.915 log units, so a noise level of 0.6 of that spread is about one unit of that estimated laboratory error. Each model's $\mathrm{AUC}_{\mathrm{norm}}$ under each noise condition on the three endpoints is given with its clean R^2 beside it (Figure 3.11).

As on QM9, the noise distribution does not change the predictive capabilities of a model (Table 3.4). Laplace, Student-t and outlier noise were run on seven models, RF, the two GPs, SVM, NGBoost, BNN-α (var. head) and VBLL-β (het.), on ECFP4, PDV and ChemBERTa. Each delta below is measured from the same model's $\mathrm{AUC}_{\mathrm{norm}}$ under Gaussian noise. RF and the two GPs never lose more than 0.03 of $\mathrm{AUC}_{\mathrm{norm}}$ under these three conditions or under grouped-wider noise, on any

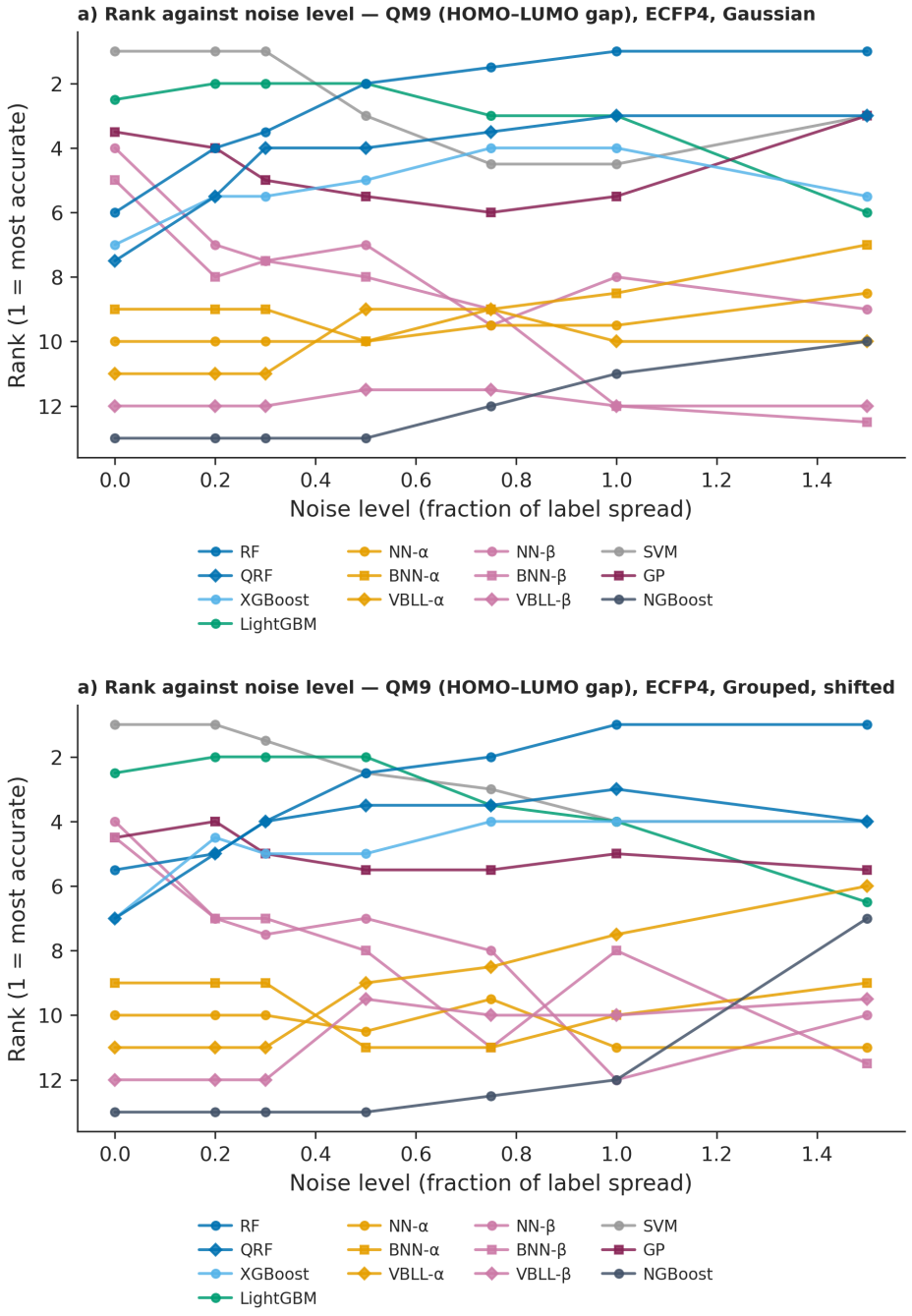


Figure 3.9: Median rank of each base model by R^2 at each noise level, on the QM9 HOMO–LUMO gap with ECFP4, under a) Gaussian and b) grouped-shifted noise. Models are ranked within each replicate; tied medians give half ranks.

dataset or representation. SVM and NGBoost again move in opposite directions, SVM never loses AUC_{norm} under any of the four, while NGBoost loses under all four on Caco-2 and hERG K_i. The two networks move furthest in both directions. It's worth remembering that while we are injecting artificial noise, the assay measurements carry their own source of noise whose properties are unknown, so no "clean" label is truly clean.

Dataset	Model	Gaussian	Laplace	Student-t (ν=5)	Outlier (10%)	Grouped, wider
logD	RF	0.935	+0.018	+0.006	+0.007	+0.021
logD	NGBoost	0.924	0.000	-0.028	-0.032	-0.009
logD	SVM	0.879	+0.044	+0.023	+0.027	+0.044
logD	GP	0.891	+0.004	+0.003	-0.001	+0.006
logD	GP (het.)	0.896	0.000	-0.002	-0.007	-0.003
logD	BNN-α (var. head)	0.881	-0.013	-0.013	-0.024	-0.029
logD	VBLL-β (het.)	0.796	+0.010	+0.012	+0.020	+0.068
Caco-2	RF	0.865	+0.035	+0.028	+0.026	+0.053
Caco-2	NGBoost	0.878	-0.069	-0.038	-0.060	-0.147
Caco-2	SVM	0.841	+0.049	+0.051	+0.031	+0.018
Caco-2	GP	0.809	+0.039	+0.034	+0.020	+0.023
Caco-2	GP (het.)	0.815	+0.035	+0.032	+0.011	+0.019
Caco-2	BNN-α (var. head)	0.787	-0.110	+0.092	-0.043	-0.017
Caco-2	VBLL-β (het.)	0.707	+0.036	+0.025	+0.015	+0.032
hERG K_i	RF	0.898	+0.015	-0.006	+0.016	-0.012
hERG K_i	NGBoost	0.842	-0.031	-0.027	-0.042	-0.093
hERG K_i	SVM	0.859	+0.051	+0.035	+0.025	+0.055
hERG K_i	GP	0.831	+0.022	+0.034	+0.025	+0.030
hERG K_i	GP (het.)	0.843	+0.010	+0.036	+0.015	+0.021
hERG K_i	BNN-α (var. head)	0.809	-0.034	-0.032	-0.063	-0.009
hERG K_i	VBLL-β (het.)	0.836	+0.007	-0.047	+0.005	-0.002

Table 3.4: AUC_{norm} under Gaussian noise, and its change under each other noise condition, for a subset of models on the three assay datasets with ECFP4; negative is a loss. Medians over five scaffold folds.

Across all datasets, models struggle to fit with group-shifted noise the most, producing lower AUC_{norm} values across the board. For grouped-shifted, RF is the dominant model on hERG K_i and logD, where it ranks first or second for AUC_{norm} with every representation. For predictive performance at under higher label noise (noise level 1.0), which is more impacted by the clean label performance, SVM ranks first to third on logD while RF ranks fifth to ninth across datasets. Both are strong contenders for systematic label noise.

LightGBM has particularly varied results across the datasets. On LogD, it has relatively high predictive performance and middle-of-the-pack noise robustness.

However, on hERG and Caco-2, it loses the most with increased noise, particularly on Gaussian and grouped-shifted noise. The effects are not as pronounced on grouped-wider, although it still ranks among the least robust there. XGBoost degrades in the same way on hERG and Caco-2.

While the results are not identical between the datasets, there are some patterns that emerge (Figure 3.12). In general, the relative AUC_{norm} model rankings match between QM9 and logD, as well as between Caco-2 and hERG K_i with every representation except ChemBERTa. RF is the most consistent, among the 13 base models under Gaussian noise it ranks between first and seventh for AUC_{norm} on every dataset and representation, second or third on QM9 and first on logD at five of six. The GP has the highest predictive performance on clean labels on all three assay datasets with nearly every representation, and ranks third to ninth for AUC_{norm}. NGBoost, the most robust model on QM9, ranks first to third on logD, second to tenth on Caco-2 and sixth to ninth on hERG K_i. LightGBM falls furthest, ranking 10th to 13th on Caco-2 and hERG K_i at every representation while being among the highest in predictive performance on clean labels. VBLL-α is the opposite, ranking first or second for AUC_{norm} on hERG K_i and last on logD at every representation, from one of the lowest clean R^2 values. This fits the pattern of the NNs being exceptionally varied.

We consider the impact of label noise on each model family. While the forests, RF and QRF, are typically middle-ground in terms of predictive performance on clean labels, they are the most reliably robust to noise across all datasets. Their rank barely changes between noise conditions, so there is no pattern of noise they are better suited to. Caco-2 is the exception, where they rank third to seventh rather than first or second.

Other models perform conditionally well, and in some cases that condition is the representation they are paired with. The GP has some of the higher predictive capabilities on clean labels and under Gaussian noise across assay datasets. It is the highest on logD and Caco-2 under grouped-wider noise as well, but under grouped-shifted noise it falls. The choice of representation has little impact on GP, moving its AUC_{norm} less than repeating the split does on every dataset.

There is a slight consideration with using both AUC_{norm} and predictive performance side-by-side. AUC_{norm} measures the fraction of its own clean R^2 that a model keeps, so predictive performance on clean labels works against it by construction. A model that already predicts poorly has little to lose and often scores a higher

AUC_{norm}, while a model that starts with strong performance has more to lose. The GP and NGBoost are the clearest cases. The GP has among the highest predictive performance on clean labels and a middling AUC_{norm}, while NGBoost has the highest AUC_{norm} across the board from one of the lowest clean R^2 values. While NGBoost's predictive performance is highly dependent on its representation, its robustness is not. On QM9 it is both the most robust model and the lowest ranked model for predictive performance up to a noise level of 0.75 with every representation. But with PDV it's able to retain some predictive performance under noise, finishing highest across all the models at noise level 1.5.

Upon first glance, SVM does not seem like a standout, however, it is the model whose standing is better with non-Gaussian noise. Among RF, the GP, NGBoost and SVM, the four models run under every condition, SVM has the lowest AUC_{norm} of the four under Gaussian noise on almost every dataset and representation, and rises to second under Laplace, Student-t and outlier noise and under grouped-wider noise. Gaussian noise is actually the one condition it is worst at. Its AUC_{norm} stays in the middle, partially because it starts high, and it ranks among the five highest for predictive performance at a noise level on every assay dataset under every noise condition.

The feed-forward networks are the most variable, although in general they have some of the highest predictive performance under clean labels and are generally not robust to noise. They are also dependent on the choice of representation. VBLL networks perform poorly on clean labels, are the least robust to noise on logD, yet are the most robust on Caco-2 and hERG K_i. The gradient-boosted trees, LightGBM and XGBoost, were also quite variable, with strong predictive performance on clean labels and poor noise robustness on Caco-2 and hERG K_i.

The quantity of noise is an important factor as well. At a noise level of 0.5 the models still rank by predictive performance much as they do on clean labels, while by 1.0 the order has changed, most of all on hERG K_i and then QM9. The order of models by AUC_{norm} is broadly similar across noise conditions, with the forests near the top under each.

While it is clear that QSAR model performance drops with label noise, different models drop faster than others. How much noise a dataset tolerates before anything happens is itself a property of the dataset. Up to a noise level of 0.3, three tenths of the spread of the clean labels, the models are largely unaffected by the noise on the HOMO–LUMO gap and on logD, where every base model with ECFP4 still keeps at

least 93% of its clean R^2; on Caco-2 one of the 13 and on hERG K_i four of the 13 are already below 90% at that level. By a noise level of 0.5 the lowest retained fraction is 90% on the HOMO–LUMO gap against 76% on hERG K_i (Figure 3.10). Boosted tree ensembles are some of the first to fall. On Caco-2 with ECFP4, LightGBM has among the highest predictive performance on clean labels. It is also the first tree ensemble to fall below 90% of that performance, at a noise level of 0.5. By level 1.5 it retains 26% of its original R^2, compared to 63% for RF on ECFP4. XGBoost drops off at the next recorded noise level. However, NGBoost is the opposite, having some of the lowest predictive performance on clean labels and the flattest curve with injected label noise across all models. On the HOMO–LUMO gap it never falls below 90% of its clean R^2 at any level tested. The networks drop at an inconsistent rate. VBLL-α is the earliest to fall on logD, dropping below 90% of its clean R^2 at a level of 0.5 and reaching 38% by 1.5. Yet on Caco-2 and hERG K_i it is among the flattest models.

The variance decomposition of predictive performance tells us how much the choice of model and representation matters under clean labels (Figure 3.3; Additional file 6). Extending that concept, we are also interested how these shares change under increasing label noise and compare to the variance decomposition of $\mathrm{AUC}_{\mathrm{norm}}$. The replicate and residuals account for more variance across all datasets. On Caco-2 and hERG K_i the replicate, which on these datasets is one of five scaffold folds, is the largest share of predictive performance before and after noise is injected. Conversely, it falls well below the choice model family for robustness. On every dataset the interaction between model and representation explains more of the variance in $\mathrm{AUC}_{\mathrm{norm}}$ than representation alone.

3.7 Probabilistic and deterministic counterparts

To further explore the impact of model architecture on noise robustness, we examined deterministic and probabilistic variants of the same core model architecture. The hypothesis was that a model that reports a distribution rather than a point value may be more robust to small amounts of injected label noise. We compared variants across three sets of base models: two NNs labeled NN-α and NN-β and RF (Figure 3.13). For each NN, we transform a standard deterministic network into a BNN. The BNNs with a variance head run at different settings from their Bayesian bases, so they are compared only for uncertainty. We compared RF to

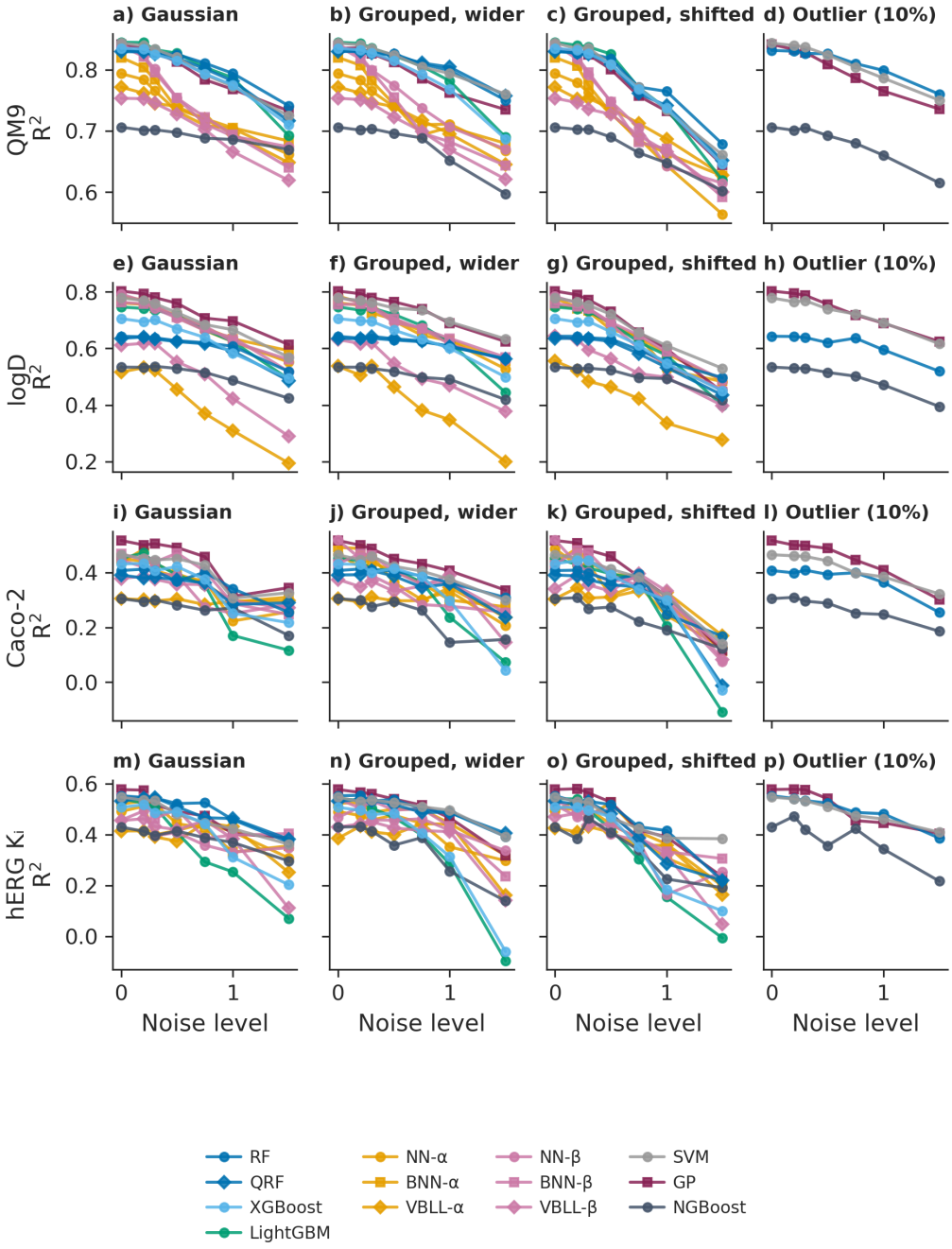


Figure 3.10: R^2 against noise level for the 13 base models on every dataset with ECFP4. Rows are datasets and columns noise conditions: a)–d) QM9 HOMO–LUMO gap; e)–h) logD; i)–l) Caco-2; m)–p) hERG K_i. Medians over 10 replicates on QM9 and five scaffold folds on the assay datasets.

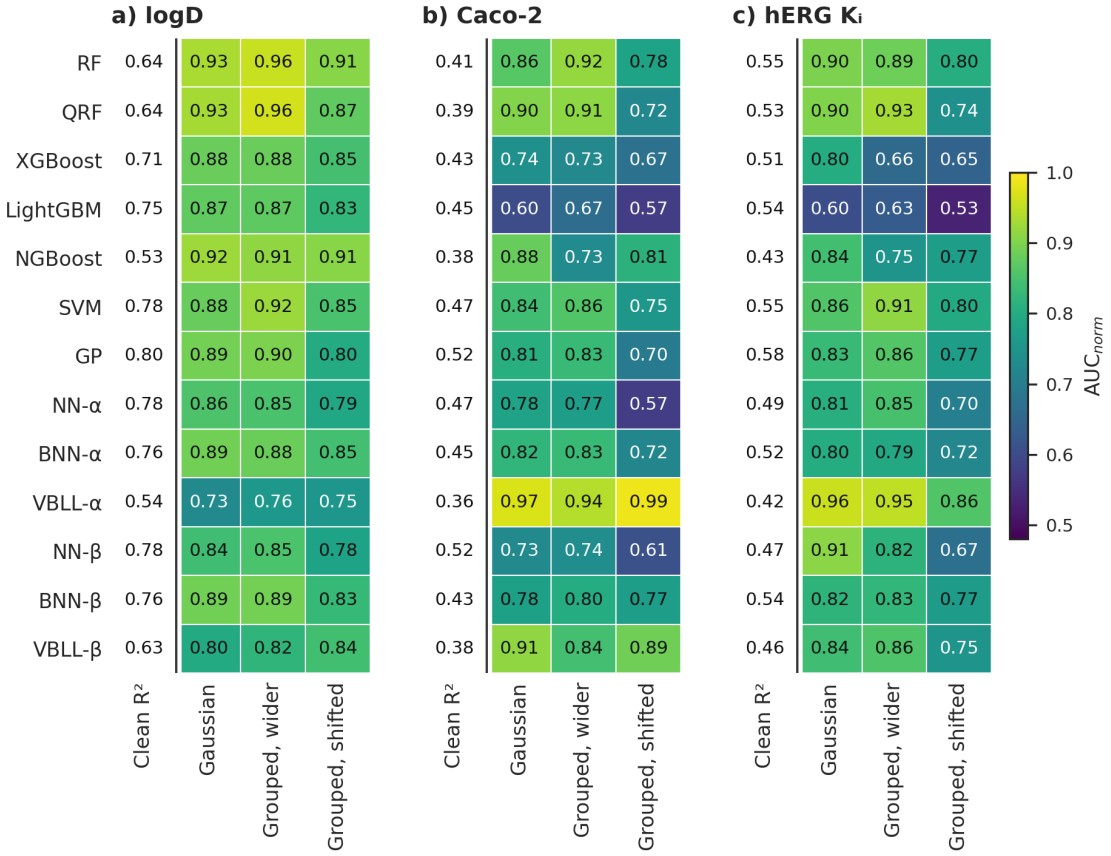


Figure 3.11: AUC_{norm} of the 13 base models with ECFP4 on a) logD, b) Caco-2 and c) hERG K_i, under the three noise conditions run on every model. The first column is clean R^2. Medians over the scaffold folds that pass the clean-R^2 threshold (Additional file 2), on one color scale. The variant models are in Additional file 8.

its probabilistic counterpart, QRF, with the same baseline architecture and hyperparameters. Comparisons of this nature are only possible when the variants carry the same underlying settings (Additional files 1 and 1). NN-α and BNN-α therefore share one setting, as do NN-β and BNN-β, and QRF is compared with a random forest of the same 300 trees (Additional file 1).

We compare RF and QRF directly with the same underlying architecture. QRF had lower AUC_{norm} on QM9 under Gaussian and grouped-shifted noise on every representation, and changed it little under grouped-wider noise, with almost no change in clean R^2 (Figure 3.13). On the assay datasets QRF was less robust than RF under grouped-shifted noise on every dataset and representation, and under Gaussian noise on logD and hERG K_i (Additional file 9). Under grouped-wider noise it was more robust on Caco-2 and mixed elsewhere.

The Bayesian transformations that we used to transform feed-forward deterministic NNs into probabilistic produced a slightly larger AUC_{norm}, but only because

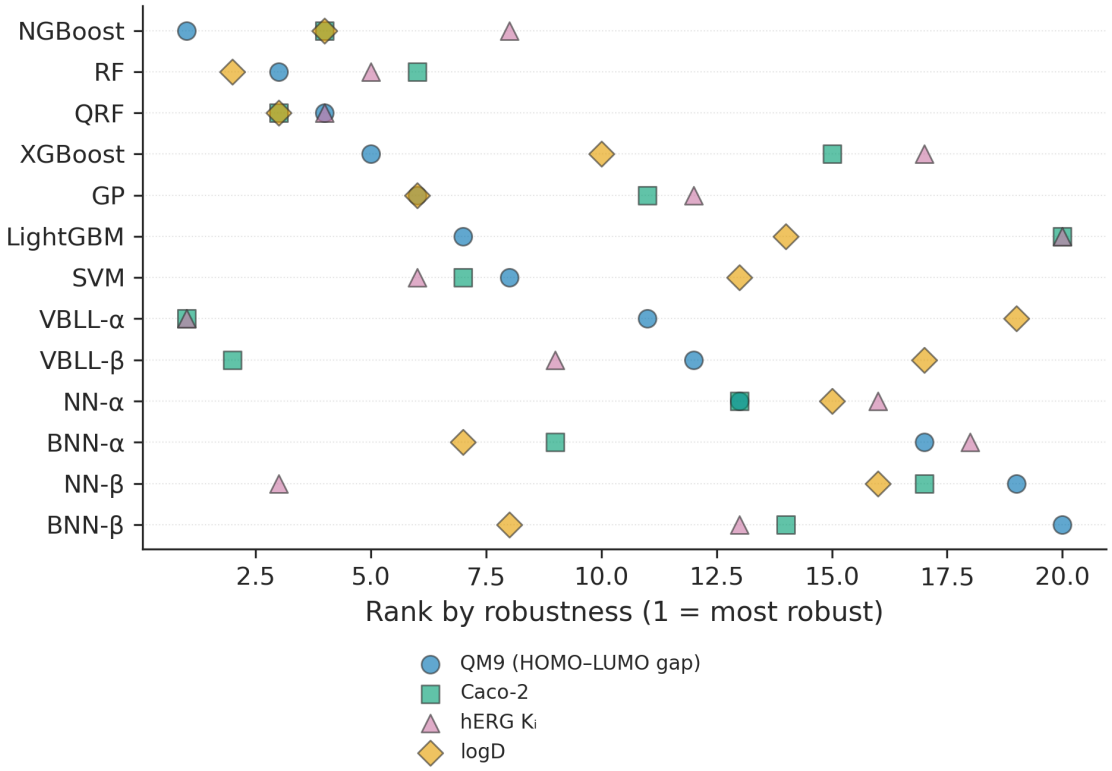


Figure 3.12: Rank by $\mathrm{AUC}_{\mathrm{norm}}$ of each base model on every dataset, with ECFP4 under Gaussian noise. Models are ordered by their rank on the QM9 HOMO–LUMO gap. Ranks are among all 19 models; the six variant models are not drawn.

they lowered predictive performance on clean labels. BNN-β kept more $\mathrm{AUC}_{\mathrm{norm}}$ than NN-β in most combinations on every assay dataset, and BNN-α on all assay datasets. On QM9 the Bayesian transformations made no significant change on either predictive performance or $\mathrm{AUC}_{\mathrm{norm}}$ for either network.

3.8 Uncertainty under increasing label noise

Until now we've discussed how much predictive capabilities a model is able to retain while its training labels are corrupted with noise. Now, we consider how confident the model reports its own predictions are in the presence of noise, and whether or not those estimates match the increased label noise. Kolmar and Grulke [8] found that the mean predicted uncertainty derived from the GPs increases with the amount of label noise in the training data. We go beyond the population level and instead ask if a model's per-sample uncertainty tracks label noise. One key sign that a model is able to handle noise is its ability to track it, represented by the per-sample Spearman correlation between predicted uncertainty and noise magnitude.

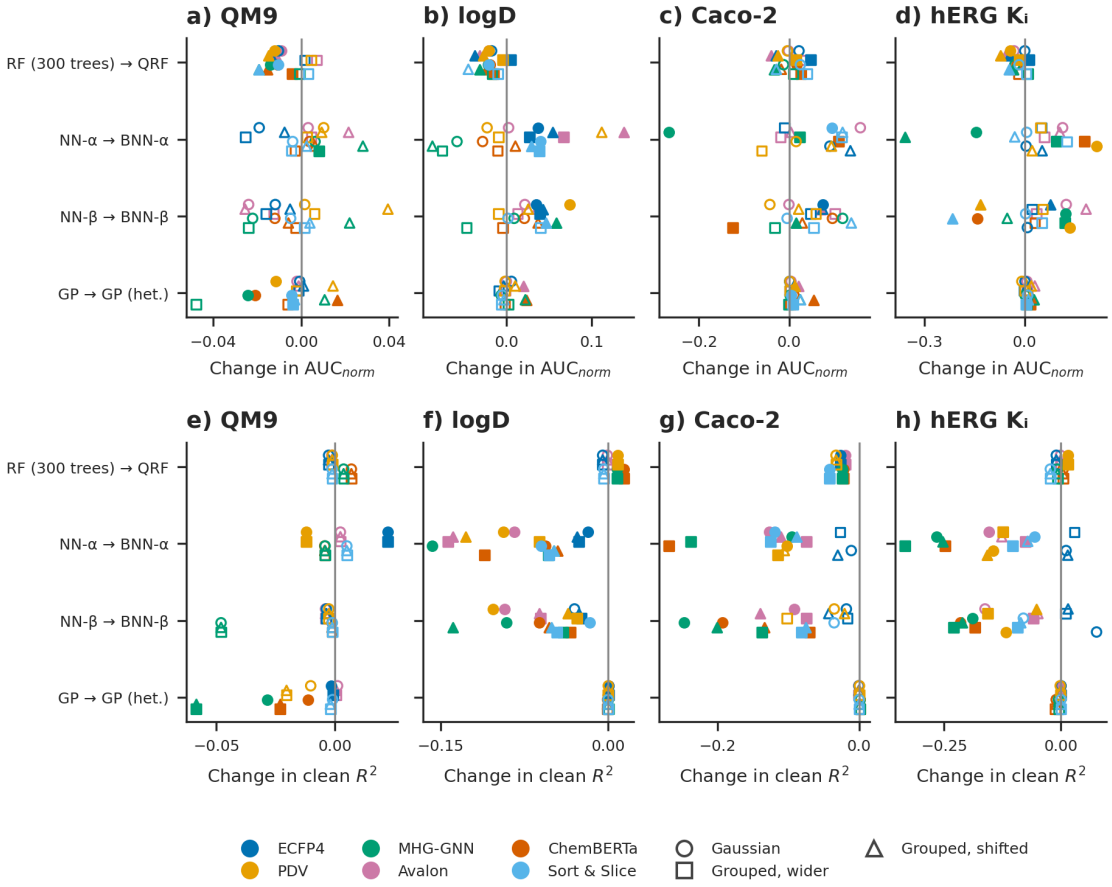


Figure 3.13: Change in AUC_{norm} (top) and clean R^2 (bottom) from each model to its probabilistic counterpart at the same settings, on every dataset: counterpart minus base, the median of paired differences over 10 replicates on QM9 and five scaffold folds on the assay datasets. Color is the representation and shape the noise condition. A filled mark changed in the same direction in every replicate or fold.

As expected, mean predicted uncertainty is positively correlated with Gaussian label noise for most models and representations (Table 3.5). There are a few exceptions, namely BNN-β on PDV and VBLL-β (het.) on Avalon for QM9. BNN-α fails to rise in at least one fold on every assay dataset, and both full-BNNs with a variance head fail on MHG-GNN for hERG K_i, and BNN-β with a variance head also fails on ChemBERTa and on Caco-2. Kolmar and Grulke [8]'s finding therefore holds for almost every probabilistic model on the computed property, and for most of them on measured labels.

3.8.1 Aleatoric and epistemic attribution

While this tells us that the models are reportedly less certain under noisy data, we don't know what the model is attributing this uncertainty to. To answer this

Model	QM9	logD	Caco-2	hERG K_i	Aleatoric	Epistemic
QRF	4.9	2.7	2.4	2.3	per molecule	per molecule
NGBoost	4.8	3.2	2.6	2.6	per molecule	none
GP	5.3	4.1	2.5	2.6	one per fit	per molecule
GP (het.)	—	4.3	2.7	2.8	per molecule	per molecule
BNN-α	3.4	1.4†	1.0†	1.4	none	per molecule
BNN-α (var. head)	4.1	2.7	1.9	2.1	per molecule	per molecule
VBLL-α	3.5	2.3	1.4	1.2	one per fit	per molecule
VBLL-α (het.)	3.0	1.9	2.8	3.5	per molecule	per molecule
BNN-β	2.7†	1.2†	1.1	1.2	none	per molecule
BNN-β (var. head)	4.3	2.5	2.4	1.7	per molecule	per molecule
VBLL-β	3.3	2.4	1.3	1.2	one per fit	per molecule
VBLL-β (het.)	2.6	2.0	2.7	3.1	per molecule	per molecule

Table 3.5: Mean predicted uncertainty on held-out molecules at a noise level of 1.5, divided by its value with no added noise, with PDV under Gaussian noise. †: the uncertainty fails to rise in at least one fold. A dash: not run. Aleatoric and Epistemic: whether the model reports that component per molecule, as one value per fit, or not at all.

question, we decompose uncertainty estimates into their aleatoric and epistemic components wherever possible. We expect to see aleatoric uncertainty rise with injected label noise, while the epistemic component should remain relatively constant, as it is the aleatoric component that describes uncertainty in the underlying data. The models whose frameworks allowed for this decomposition include full-BNNs with a variance head, the heteroscedastic GP, and QRF (Table 3.5). The two heteroscedastic VBLL networks also report both components, and they separate them more often than any other model, though not on every dataset and representation. They are not drawn in the figures, as they were not scored out of fold.

The full-BNNs with a variance head behave as expected (Figure 3.14), the epistemic component grows by at most 0.43 of the aleatoric growth in every configuration where the ratio can be computed. Despite QRF's strong predictive performance and noise robustness, both of its aleatoric and epistemic components rise with noise and on some representations, the epistemic component actually rises at a faster rate.

Unlike the predictive performance and robustness, the representation actually dictates whether decomposition works. For QRF, the epistemic growth is 0.61 to 1.14 times the aleatoric growth on the three fingerprints, 1.02 to 1.44 times on PDV, and 1.29 to 2.04 times on ChemBERTa and MHG-GNN. Under Gaussian noise the heteroscedastic GP keeps its epistemic growth below a quarter of its aleatoric growth on PDV, ChemBERTa and MHG-GNN except MHG-GNN on QM9, but not on ECFP4 under grouped-shifted noise, where the two grow by similar amounts.

The noise conditions follow the pattern found in robustness, the three conditions which alternate the noise distribution show similar decompositions to those under Gaussian noise, while group-shifted raises the epistemic share of the heteroscedastic GP and of QRF in every pairing of dataset and representation. This scaffold family-based offset may be appearing as structure in the data rather than label noise to the models.

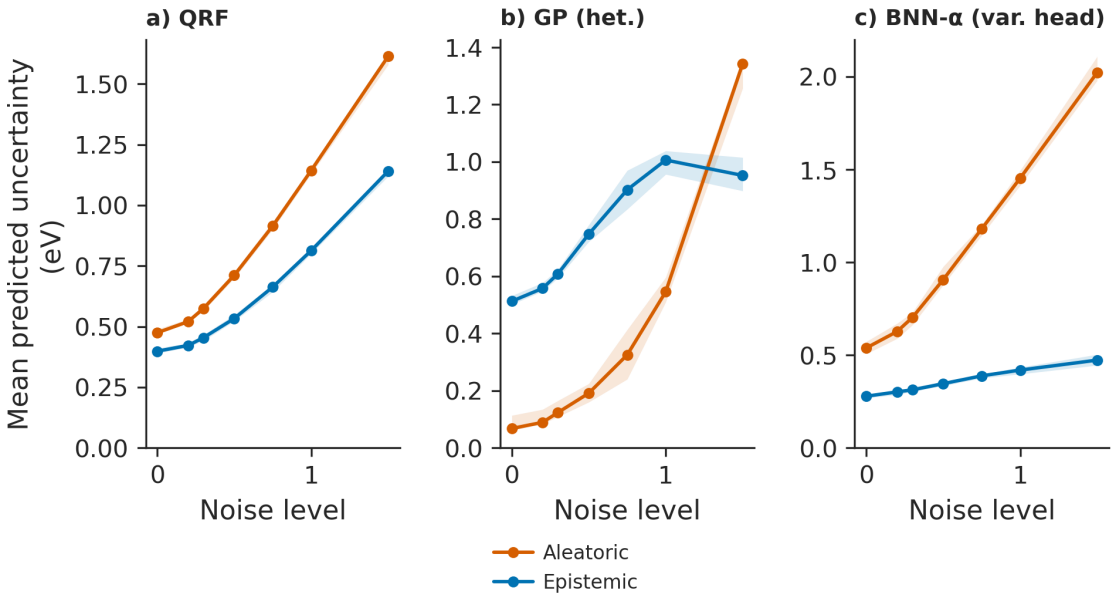


Figure 3.14: Aleatoric and epistemic components of the predicted uncertainty against noise level, one panel per model, on the QM9 HOMO–LUMO gap with ECFP4 under Gaussian noise. Each value is the mean over the molecules of one out-of-fold pass; lines are medians over folds and bands their range. A component that is one value per fit is not drawn (Table 3.5).

The aleatoric component indicates that label noise increased, but it doesn't tell us by how much (Figure 3.15). For each unit of added noise it rises by roughly 0.4 to 1 unit, depending on model, dataset and representation. The aleatoric component can therefore compare noise between datasets or noise levels, but it does not estimate a dataset's internal error.

Uncertainty estimation breaks under censoring as the process of clipping the labels narrows their spread, which in turn shrinks the total uncertainty. None of the models tested treat a clipped label as a noisy one, so they actually grow more confident while losing information.

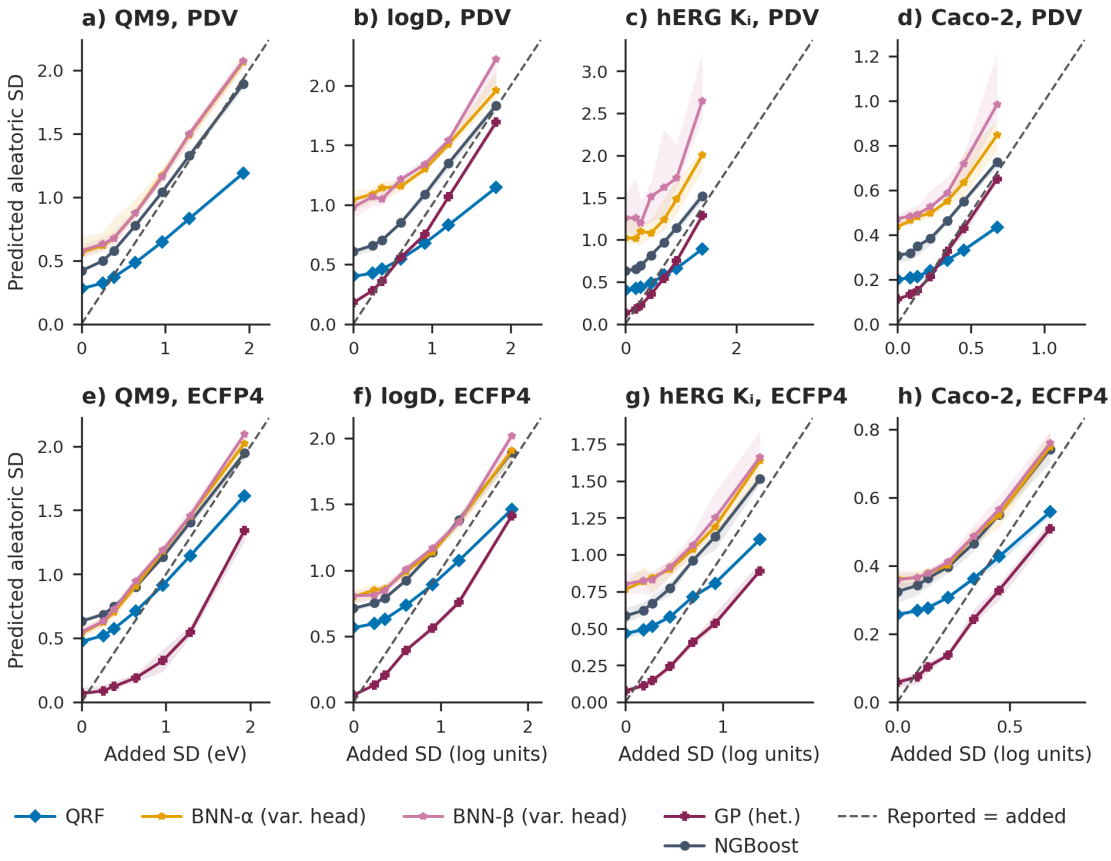


Figure 3.15: Predicted aleatoric standard deviation against the standard deviation of the added noise, under Gaussian noise. Rows are PDV (top) and ECFP4 (bottom); columns are the QM9 HOMO–LUMO gap, logD, hERG K_i and Caco-2. Each line is one model: the median over folds of the mean over the molecules of one out-of-fold pass, with a band spanning the folds. The dashed line is equality. The heteroscedastic Gaussian process was not run on QM9 with PDV, and the two heteroscedastic variational networks were not scored out of fold.

3.8.2 Per-sample uncertainty and label noise

Finally, we ask whether a model's uncertainty on one molecule tracks the noise on that molecule's label. We assess this using the Spearman correlation between predicted uncertainty and the size of the injected noise over out-of-fold training molecules.

Since Gaussian, Laplace, Student-t and grouped-shifted conditions add noise from the same scale to every label, there is not much for uncertainty to find. It's the same situation with outlier noise, under which a tenth of the data is chosen at random for extra noise, and with grouped-wider in which each scaffold family falls in a single fold, so a model makes predictions based on unseen scaffolds and has no basis for estimating uncertainty. The only case where per-sample uncertainty

tracking is possible is censoring, as the noise is dependent on the value of the label itself. The correlations are then large and mostly negative, meaning the model is least uncertain on the labels that were clipped most. On QM9 with half the labels clipped, on PDV, they run from -0.84 to -0.80 across folds for NGBoost and from -0.47 to -0.41 for the GP, and on logD with half the labels clipped every fold of every model except the two GPs is at -0.29 or lower. We divided each molecule's error by its uncertainty and found that the division ranked the clipped labels better than the error alone on QM9 and on most of logD. Models are most confident on the clipped labels, probably because the clipping piles many labels into one value, which the model learns to tightly predict. Ironically, per-molecule uncertainty can be used in reverse to identify clipped labels as the labels a model is more certain about, especially when they fall in a certain range, are most likely to have been clipped.

3.9 Conclusions

We investigated how a variety of molecular representations and models perform under increasing label noise. We used ANOVA variance decomposition across three noise conditions on four datasets to show that molecular representation accounts for a larger share of the variance in predictive performance than in robustness. Robustness to label noise, as measured by $\mathrm{AUC}_{\mathrm{norm}}$, the normalized area under the R^2 retention curve, is more impacted by the choice of model. The representation each model is paired with explains more variance than the representation alone. As such, a representation should not be chosen independently of the model it will be used with, and some models benefit from alternative molecular representations across both predictive performance and noise robustness. Model family was the primary factor in every decomposition except one, Caco-2 under grouped-shifted noise, where the scaffold fold took the larger share.

Grouped-shifted and censored noise, the two conditions in which the error is systematic, corrupt QSAR model predictions the most. Changing the shape of the distribution from which noise is drawn barely has an impact on model performance, a conclusion which is in agreement with Heid et al. [76]. Low quantities of noise do not make much of an impact, on the HOMO–LUMO gap every base model keeps at least 93% of its clean R^2 up to a noise level of 0.3, and grouped-shifted noise separates from the other conditions only from a level of 0.75 (Figure 3.8). Models' performance

begins to degrade more rapidly on the assay datasets than on the HOMO–LUMO gap, which makes sense as assay data contain inherent experimental noise. Tree and kernel models maintained consistent $\mathrm{AUC}_{\mathrm{norm}}$ across all representations, while the plain NNs and BNNs depended on the representation they were given, as did the predictive performance of NGBoost. Across datasets and noise conditions, RF and QRF were reliably robust, although their predictive performance on clean labels was middling and on Caco-2 they struggled under Gaussian noise. QRF was the slightly less robust of the two, falling below RF under Gaussian and grouped-shifted noise. Other models performed well only under some conditions. The GP had the highest predictive performance on clean labels and under independent noise but fell behind when the noise was shared within a scaffold family, NGBoost predicted well only with the right representation, and SVM led in predictive performance under scaffold-shared noise on some datasets. Transforming a feed-forward network into a Bayesian one increased $\mathrm{AUC}_{\mathrm{norm}}$, but the Bayesian variants never predicted better at any noise level, so switching to a tree-based model is the stronger move under label noise.

With regard to uncertainty estimation, mean predicted uncertainty rose with label noise for almost every probabilistic model on QM9 and for the majority of models on the assay datasets. Of the models whose aleatoric and epistemic components can be split per molecule, the BNNs with a variance head, the heteroscedastic GP and the heteroscedastic VBLL networks correctly attributed error from the injected noise as aleatoric, while QRF read that error as epistemic. The success of a split is also dependent on the representation, which is not necessarily the case for predictive performance or noise robustness. The aleatoric component rises by roughly 0.4 to 1 unit for each unit of noise added, so it can compare noise between datasets or between noise levels, but it does not measure a dataset's internal error. Outside censoring, no model's per-sample uncertainty identified which labels had been corrupted. Under censoring the models' most confident predictions are the clipped labels themselves, and dividing a molecule's error by its uncertainty picked out the clipped labels better than the error alone.

QM9 served as a relatively clean baseline as its properties are computed. The assay datasets containing experimental measurements on the endpoints LogD, Caco-2 [103], and hERG K_i [104] are smaller and cover a more narrow range of the chemical space. As the labels in these data are experimentally-derived and pooled from different laboratories, they contain some unknown amount of noise. We are then

adding artificial noise on top of the existing experimental noise. These validation datasets supported the QM9 findings for the forests, with the boosted trees XGBoost and LightGBM the notable exceptions, degrading on the external datasets. Future work should extend this type of noise robustness benchmarking to classification tasks and larger experimental datasets with better-known uncertainty estimates.

When training on noisy experimental data in the context of QSAR, it is important to keep in mind that a model's robustness to noise is primarily driven by its own mechanism of training, and to an extent the molecular representation the input data comes in, oftentimes with respect to the model. Of the models tested, RF was the most consistent choice across the four datasets, ranking between first and eighth for $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset, representation and noise condition. Where a predictive distribution is wanted, its probabilistic counterpart QRF maintains RF's predictive performance and nearly all of its noise robustness. What QRF cannot do is say where that uncertainty came from. Beyond the forests, the better choice depends on how much noise there is. LightGBM and XGBoost predict well on clean labels and lose the most as noise rises, so they suit data you trust, while NGBoost and the VBLL networks start lower and hold what they have.

In this research, we define a noise level as a fraction of the spread of the clean training labels. It is therefore relative to the dataset rather than an absolute measurement error, and cannot be mapped onto a given dataset exactly. There are other mechanisms used to estimate the noise in a dataset directly. When an assay's controls are available, the Z′-factor can be used to estimate noise [81]. You can also use screening data to test for row, column and plate effects, which produce systematic rather than independent error [113]. When you lack controls, replicates, and metadata, a label can be checked against the data itself. Confident learning flags labels by comparing each label to what a model predicts for that molecule when it was left out of training [48]. You can use the point at which the model error matches the data's error to generate a predictive limit [8, 114]. Beyond formal methods, you should consider how a dataset was assembled. How many independent replicates were done, how many laboratories contributed, whether assay protocols were consistently applied, and how varied are the compounds in chemical space?

The first limitation of this study is data availability. While HOMO–LUMO gap from QM9 is a good baseline as its computed labels carry no measurement error, it is only a single task. As the noise is meant to replicate experimental data, this analysis would be improved by more quality-controlled, relatively clean and well-replicated

experimental data across different targets. However, data of that nature are hard to find publicly.

Second, $\mathrm{AUC}_{\mathrm{norm}}$ is a ratio to clean R^2, so a model that starts from a low clean baseline has less to lose, and a few replicates scored above 1.05. We tried other measures of noise robustness, but they were either too strongly correlated with clean R^2 or did not capture the right quantity. The change in R^2 at a single noise level, for example, misses the shape of the curve. $\mathrm{AUC}_{\mathrm{norm}}$ is not a perfect metric, but it describes the whole curve, and across the 13 base models it is unrelated to clean R^2.

Third, it is hard to draw conclusions that hold across so many conditions. No model or representation wins everywhere, although RF comes closest. When working with cheminformatics data, you don't know in advance how a model and representation will perform on the given prediction task. The underlying data is probably noisy, but you don't know how noisy it is or what shape that noise takes.

Given that, our recommendation is to test a handful of models against a handful of representations rather than tuning a single pairing, since on every dataset the pairing explains more of the variance in robustness than the representation does alone. The pattern of the noise should also guide that choice (Table 3.6). The same preferences held on all four datasets. Under Laplace, Student-t and outlier noise, RF was the most robust model and SVM was the second. NGBoost lost $\mathrm{AUC}_{\mathrm{norm}}$ under all three. Under grouped-wider noise, QRF was the most robust model and SVM was close behind. Of the boosted trees, XGBoost lost the most. Grouped-shifted noise was the hardest condition. Every model lost $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset and representation. RF lost the least of the models that also predicted well, and the GP lost the most. Gaussian noise was the only condition with a clear specialist. NGBoost was the most robust model on the HOMO–LUMO gap under Gaussian noise with every representation, but not on Caco-2 or hERG K_i.

These results follow from the loss function each model fits. NGBoost fits a per-molecule scale under a Gaussian likelihood, so it loses ground when the noise is not Gaussian. SVM ignores residuals that fall inside its margin, so it gains under the same conditions. QRF fits conditional quantiles, so a noisy scaffold family widens that family's interval without shifting the fit. Each boosting stage fits the residuals of the stage before it, and those residuals are largest in the noisy families. The GP is the most affected by error that is correlated with structure, because its kernel measures structure. None of this matters if the data are clean, as little separated

Noise condition	What it looks like	Prefer	Avoid	Extent of the evidence
Gaussian	Independent and symmetric, at the same scale on every label	NGBoost, on the HOMO–LUMO gap	—	NGBoost is the most robust model on the HOMO–LUMO gap with every representation, but not on Caco-2 or hERG K_i
Laplace, Student-t, outlier	Independent but heavier-tailed, or concentrated on a random subset of labels	RF first, SVM second	NGBoost, which loses $\mathrm{AUC}_{\mathrm{norm}}$ under all three	All four datasets
Grouped-wider	Whole scaffold families receive a wider error than the other molecules	QRF first, SVM close behind	XGBoost, which loses the most of the boosted trees	All four datasets
Grouped-shifted	Every scaffold family carries its own offset, so the error is systematic	RF, which loses the least of the models that also predict well	The GP, which loses the most; every model loses $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset and representation	All four datasets; the hardest condition
Censoring	Labels above a limit are pulled down to that limit	—	Every pair tested loses more robustness than under Gaussian noise	The HOMO–LUMO gap only, on the five model and representation pairs run under censoring

Table 3.6: Which model to prefer under each noise condition. One row is one noise condition. A dash means no model is named.

the models below a noise level of 0.3.

In this chapter each model was given a standard molecular representation, and the choice of representation explained relatively little of the variance in noise robustness. We observe that this choice does impact the performance of different models to varying degrees, and the representations seem to capture different signals on each dataset that we tested. Chapter 4 asks how well these representations compare molecules on a single dataset.

Chapter 4

Limitations of the Similarity Assumption

4.1 Introduction

The previous chapters have been focused on the impact of label noise. Before we delve into techniques that are meant to train on, identify, and mitigate such label noise, we first focus on a tangential point, molecular similarity. All of the aforementioned noise-adjacent methods, as well as the QSAR modeling itself, rest on the similar property principle (SPP; Section 2.4), which posits that structurally similar compounds display similar chemical properties [69]. This serves as the conceptual foundation for QSAR modeling. The concept of molecular similarity is also used in various contexts to compare neighboring molecules. This comes into play when identifying noisy molecules that in theory stick out from their neighbors. There are some important exceptions to the SPP, namely activity cliffs [54, 73]. Furthermore, there is no single way to define similarity between compounds. This is where the principle begins to break down in practice. When defining similarity between two molecules, we typically rely on some pairwise mathematical function, or coefficient, that takes in two molecular representations as input. Two factors vary here, the representation and the coefficient. Bajusz et al. [71] compared eight coefficients and found Tanimoto, Dice, Cosine and Soergel the best and largely equivalent, while similarities derived from Euclidean and Manhattan distances were not recommended on their own. In this chapter we ask if the standard methods of comparing molecules satisfy the SPP, insofar as they do not sufficiently quantify how similar two molecules actually are.

General-purpose molecular representations are designed to work across chemical domains. Consequently, when considering data from a specific domain, many of their features show little to no variance. Some representations assign separate molecules identical feature vectors. Since common fingerprints do not encode stereochemistry [115], they are often unable to differentiate stereoisomers, which can have substantially different measured activity [116]. As these coefficients compare every feature, irrelevant features dilute the result. Different representations also disagree about which molecules are close based on the information they carry.

The problem of whether neighboring molecules can predict chemical activity has been measured before. The modelability index of Golbraikh et al. [117] scores a binary dataset by the proportion of nearest-neighbor pairs that share an activity class, producing a single value for how strongly the similarity assumption holds on that particular combination of representation and data. It is not clear whether a low value is due to the data being intrinsically hard to model or to the representation not capturing adequate features for the task. Other work in this area relates to activity cliffs. Sheridan et al. [118] compared experimental error, activity distribution and activity-cliff metrics as predictors of how well a dataset can be modeled, and found the cliff metrics the strongest of the three. However, they found that these metrics cannot separate real cliffs from those produced by uncertain activity measurements. Medina-Franco [119] point out that some of these activity cliffs are artifacts of the chosen representation, similarity threshold, or noise in the data. Activity cliffs are distinct from noise, as they carry some of the most valuable information about the interaction between structure and activity. None of these measures ask specifically about whether the specific features within these representations matter for the property at hand.

We first consider how much the representation impacts similarity measures. Riniker and Landrum [120] benchmarked 12 fingerprints and two simple baseline fingerprints for similarity-based virtual screening on 88 targets from three public collections, and found that the screening performance of the 12 fingerprints was highly correlated. Only the two baselines differed significantly from the rest. Neural embeddings were found to have a negligible improvement over ECFP across 25 pretrained models and 25 datasets [121]. However, when looking into the differences between the structure of fingerprints, Huber and Pollmann [122] found that fingerprints which store raw counts are more effective at defining similarity than folded fingerprints when compared against a graph-based structural reference. Folding in-

troduces bit collisions, which disproportionately distort similarity for molecules with many set bits [122]. ACs follow the same pattern: across 15 configurations of representation and similarity measures on 3 datasets, the representations disagreed about which pairs form cliffs [123], supporting the point Medina-Franco [119] make about ACs being artifacts of their representation.

Any method that incorporates similarity measures, such as AC detection, or cleaning a training set by comparing each sample's label with its nearest neighbors' labels, whether by removing the samples that disagree [98] or by smoothing their labels, inherits this choice, as do the noise estimation and mitigation methods of this thesis. Coupry and Pogány [124] learn a molecular similarity from graph embeddings, but independently of any endpoint. Zheng and Tropsha [125] did choose the similarity for the task. They predict each molecule's activity as the average over its k most similar molecules by Euclidean distance over descriptors, and choose the descriptors by simulated annealing to maximize the leave-one-out q^2. Itskowitz and Tropsha [126] also optimize the number of neighbors and their weighting. Both select features for a nearest-neighbor model of one endpoint, whereas this thesis selects features across several representations.

The purpose of this chapter is to establish the problem that Chapter 5 sets out to solve. Section 4.2 asks, for a given dataset, how much the features of standard representations vary and how many molecules map to identical feature vectors. Section 4.3 asks what happens when these features are used to quantify similarity, and Section 4.4 follows the consequences for methods built on these similarity measures, including an exploratory study of LNL methods (Section 4.4.1).

4.2 Flaws in standard representations

General-purpose representations are built to span a broad range of chemical domains. When your dataset covers only a small region of the chemical space, a lot of the features within these generalized representations become irrelevant, describing chemistry which is absent from the data at hand. Pairwise comparisons of these representations rely on these irrelevant features.

4.2.1 Constant and near-constant features

To investigate similarity estimation, we first look at a simple measure: how many features vary within a dataset. A single dataset often contains a set of relatively similar molecules. Many features within a generalized representation contain information that describes all, or almost all, of the molecules in the set, failing to capture the variation between them. We define a constant feature to be one that takes a single value in every molecule of a dataset and a near-constant feature to be one that carries the same value in at least 95% of the molecules. We counted constant and near-constant features from a set of representations (described in Table 5.15) across hERG K_i, Caco-2, LogD, QM9, MDR1–MDCK efflux, human liver microsome (HLM) clearance, rat liver microsome (RLM) clearance and the Lhasa hERG blocker training set. HLM clearance and the Lhasa hERG set are not in Table 5.15; their base representations are those with counts in Table 4.2.

The substructure-based fingerprints have the most near-constant features. The learned embeddings, which are taken from the last layers of a NN, have none except MHG-GNN (Tables 4.1 and 4.2, Figure 4.1). ECFP4 is the worst offender. On hERG K_i (1,412 molecules) 1,815 of the 2,048 features are near-constant. That jumps to 1,997 out of 2,048 near-constant features for the HOMO–LUMO gap on QM9. 292 of MHG-GNN's 1,024 features are zero in every molecule across all seven datasets that used it. They come directly from the saved encoder output rather than the data, and we have not looked into why.

These constant counts are sometimes filtered out by the distance. Tanimoto similarity (Equation 2.3; the distance is $1 - T$) ignores bits off in both molecules, but a bit on in every molecule raises every molecule's measured similarity. It does not use the 105 constant ECFP4 bits on hERG K_i. Constant features add zero to a Euclidean or Manhattan distance; however, near-constant features are amplified. That's great behavior if those features define an AC. If not, they are contributing disproportionately to the measure of similarity.

4.2.2 Molecules with identical feature vectors

A standard fingerprint- or descriptor-based representation can produce the same feature vector for two separate molecules. In turn, models using this representation will produce a single prediction for the molecules with identical features. Identical feature vectors do not always mean that the representation has merged two different

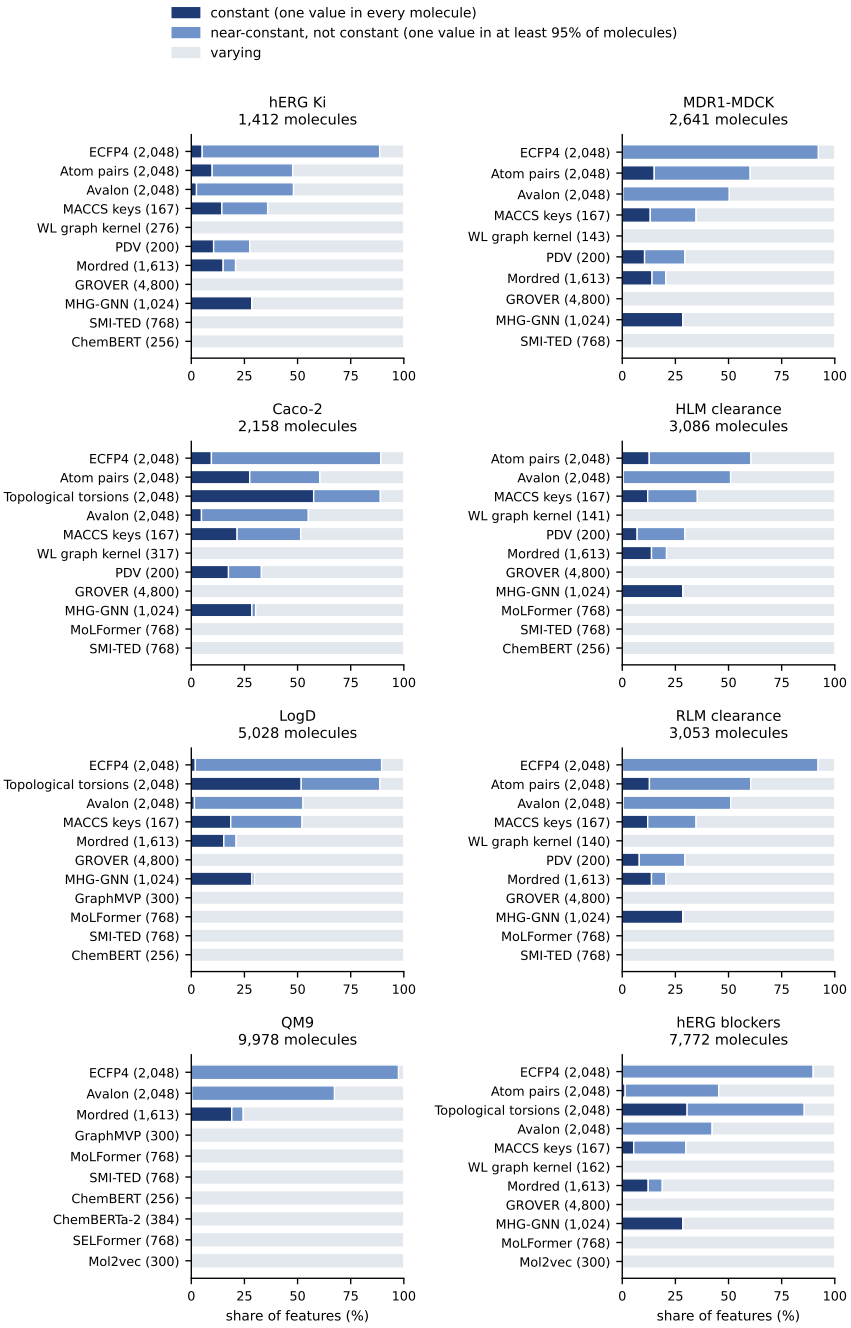


Figure 4.1: Shares of constant (dark blue), near-constant (mid blue) and varying (gray) features of each representation on eight datasets. The number of features of each representation is in parentheses.

Representation	Features	hERG K_i (1,412)	Caco-2 (2,158)	LogD (5,028)	QM9 (9,978)
ECFP4	2,048	105 / 1,815	194 / 1,826	39 / 1,834	0 / 1,997
Atom pairs	2,048	201 / 977	563 / 1,239	–	–
Topological torsions	2,048	–	1,179 / 1,822	1,059 / 1,818	–
Avalon	2,048	50 / 985	98 / 1,128	30 / 1,076	4 / 1,380
MACCS keys	167	24 / 60	36 / 86	31 / 87	n/a
WL graph kernel	140–317	0 / 0 / 276	0 / 0 / 317	–	–
PDV	200	21 / 55	35 / 66	–	–
Mordred	1,613	242 / 336	–	247 / 340	307 / 394
GROVER	4,800	0 / 0	0 / 0	0 / 0	–
MHG-GNN	1,024	292 / 292	292 / 310	292 / 304	–
GraphMVP	300	–	–	0 / 0	0 / 0
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	0 / 0
ChemBERT	256	0 / 0	–	0 / 0	0 / 0
ChemBERTa-2	384	–	–	–	0 / 0
SELFormer	768	–	–	–	0 / 0
Mol2vec	300	–	–	–	0 / 0

Table 4.1: Constant and near-constant features of each representation on hERG K_i, Caco-2, LogD and QM9, with the number of molecules under each dataset name. Each entry gives the number of constant features and the number of near-constant features (one value in at least 95% of molecules); WL graph kernel entries add its number of features, which varies by dataset. A dash marks a representation outside the dataset's base representations, and n/a one that was not computed.

Representation	Features	MDR1–MDCK (2,641)	HLM clearance (3,086)	RLM clearance (3,053)	hERG blockers (7,772)
ECFP4	2,048	1 / 1,890	–	1 / 1,887	0 / 1,840
Atom pairs	2,048	308 / 1,233	260 / 1,240	262 / 1,238	28 / 929
Topological torsions	2,048	–	–	–	625 / 1,754
Avalon	2,048	5 / 1,032	8 / 1,044	8 / 1,045	0 / 866
MACCS keys	167	22 / 58	20 / 59	20 / 58	9 / 50
WL graph kernel	140–317	0 / 0 / 143	0 / 0 / 141	0 / 0 / 140	0 / 0 / 162
PDV	200	21 / 59	14 / 59	16 / 59	–
Mordred	1,613	226 / 331	222 / 336	222 / 331	197 / 304
GROVER	4,800	0 / 0	0 / 0	0 / 0	0 / 0
MHG-GNN	1,024	292 / 292	292 / 293	292 / 293	292 / 292
MoLFormer	768	–	0 / 0	0 / 0	0 / 0
SMI-TED	768	0 / 0	0 / 0	0 / 0	–
ChemBERT	256	–	0 / 0	–	–
Mol2vec	300	–	–	–	0 / 0

Table 4.2: Constant and near-constant features of each representation on MDR1–MDCK efflux, HLM clearance, RLM clearance and the Lhasa hERG blocker training set of 7,772 molecules. Entries read as in Table 4.1.

molecules, since a dataset pooled from several sources can hold two measurements of the same molecule.

Many fingerprints are computed without stereochemistry [115]. In this research, ECFP4 is built with stereochemistry off, and the 2D descriptor sets (PDV, Mordred) also ignore it, so almost every molecule they produce identical representations for is a stereoisomer of another (Table 4.3). Any representation that reads from these subsequently merges stereoisomers. Additionally, methods using the Weisfeiler–Lehman relabeling scheme are bounded by the 1-Weisfeiler–Lehman test (Chapter 2), so some non-isomorphic graphs also receive the same vector. This is seen in the graph-kernel features which tie 105 of 1,412 hERG K_i molecules, 56 of which differ only in stereochemistry and 49 in more than that.

Whether or not a learned embedding is able to differentiate stereoisomers is dependent on its input as well as its individual implementation. SMI-TED, a pre-trained SMILES encoder, ties 56 hERG K_i, 633 LogD and 177 Caco-2 molecules, all stereoisomers. A tokenizer can drop stereochemistry prior to training. The tokenizer of the ChemBERTa-77M-MTR checkpoint (ChemBERTa-2 in Table 4.1, called ChemBERTa in Chapter 3) gives 72 of the 1,415 hERG K_i molecules of Chapter 3 the same token sequence as another molecule. Since stereoisomers can differ in measured activity [116], these representations lack potentially useful chemical information.

We counted these ties on hERG K_i, LogD and Caco-2, for each dataset's base representations and augmentation features (small feature sets added to the hybrid after it is built, Chapter 5, "Augmentation"). Two molecules are tied when a representation gives them exactly the same feature vector. No tie came from one compound listed twice (checked by InChIKey) or from a row of zeros, so every tie merges two different molecules (Table 4.3).

Apart from the MACCS keys, most tied molecules are stereoisomers. Atom pairs, PDV, Mordred and SMI-TED tie only stereoisomers, and ECFP4, Avalon and topological torsions add a few molecules that differ in more than their stereochemistry. On hERG K_i, ECFP4, atom pairs, PDV, Mordred and SMI-TED tie the same 56 molecules, so joining any of these five representations leaves all 56 tied. The WL graph kernel also ties 49 hERG K_i and 26 Caco-2 molecules that are not stereoisomers, and the MACCS keys tie the most molecules on every dataset, mostly not stereoisomers. The 3D augmentation features and the learned embeddings ChemBERT, GROVER, MHG-GNN, MoLFormer and GraphMVP tie none, and joining

all of a dataset's representations leaves no molecule tied.

A model has to give every molecule in a tied group the same prediction. The best single prediction for a group is its mean label, so ties set an error that no model can go below,

$$\mathrm{RMSE}_{\min} = \sqrt{\frac{1}{n}\sum_{G}\sum_{i \in G}(y_i - \bar{y}_G)^2}, \tag{4.1}$$

where n is the number of molecules, G runs over the tied groups and $\bar{y}_G$ is the mean label of group G. The matching best R^2 is $1 - \mathrm{RMSE}^2_{\min}/\mathrm{Var}(y)$. On hERG K_i this lowest error is at most 0.134 pK_i (MACCS keys), small next to the label standard deviation of 0.915 pK_i.

Single groups can still disagree by a lot. Independent pK_i measurements of the same compound against the same target differ by 0.44 log units on average, with a standard deviation of 0.54 [10], so a gap larger than 0.54 inside a tied group is larger than typical measurement noise. On hERG K_i, 6 of the 26 ECFP4 groups and 38 of the 153 MACCS groups differ by more than 0.54. The widest ECFP4 group is a pair of stereoisomers 1.14 pK_i apart (Figure 4.2). An ECFP4 model can at best predict their mean, 5.59 pK_i, which leaves each molecule 0.57 pK_i from its measured value.

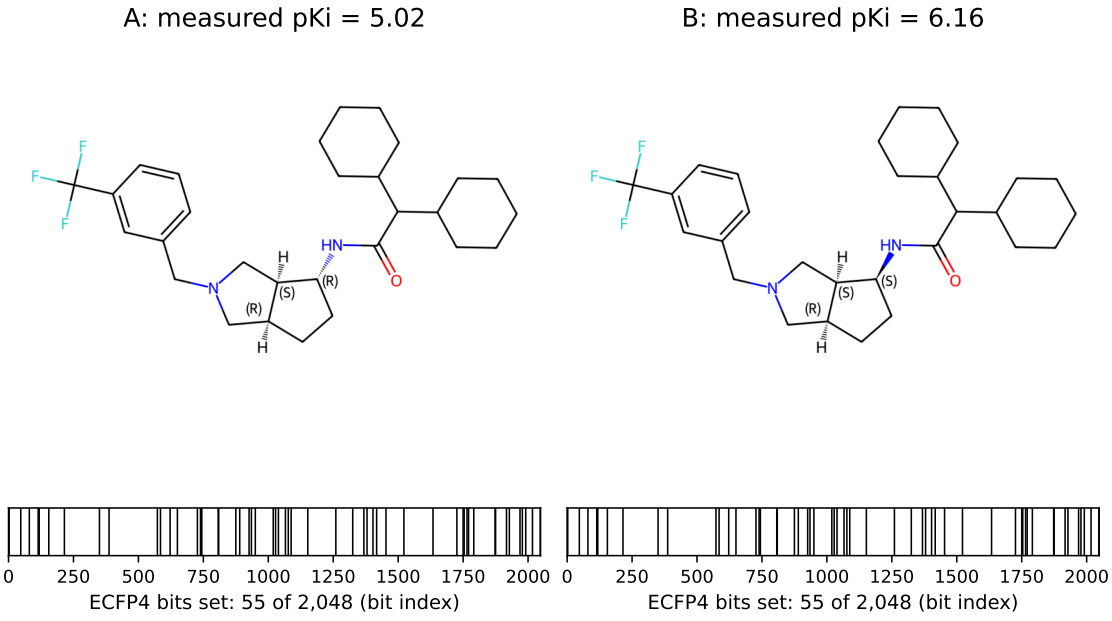


Figure 4.2: Two hERG K_i stereoisomers with identical ECFP4 vectors, which differ at the carbon bearing the amide nitrogen (R in A, S in B). The top row shows the structures with their measured pK_i values, and the bottom row shows the 55 of 2,048 ECFP4 bits set in both.

A tied pair with different labels is also the most extreme activity cliff possible.

The structure–activity landscape index of molecules i and j,

$$\mathrm{SALI}_{ij} = \frac{|y_i - y_j|}{1 - \mathrm{sim}(i, j)} \tag{4.2}$$

[127], is infinite when their similarity is 1 and their labels differ, and here the cliff is made by the representation rather than the chemistry. A method that judges a label by its nearest neighbors sees a tied neighbor at distance 0 with a different label, and may flag a correct label as noise (Section 4.4.2).

Dataset	Representation	Molecules	Tied	Stereo-only	Best RMSE	Best R^2	Largest spread
hERG K_i (pK_i)	ECFP4	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Atom pairs	1,412	56 (4.0%)	56	0.041	0.998	1.14
	PDV	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Mordred	1,412	56 (4.0%)	56	0.041	0.998	1.14
	SMI-TED	1,412	56 (4.0%)	56	0.041	0.998	1.14
	Avalon	1,412	66 (4.7%)	56	0.049	0.997	1.20
	WL graph kernel	1,412	105 (7.4%)	56	0.071	0.994	1.49
	MACCS keys (aug.)	1,412	377 (26.7%)	30	0.134	0.978	1.80
LogD (log units)	ECFP4	5,028	641 (12.7%)	633	0.044	0.999	1.00
	Mordred	5,028	633 (12.6%)	633	0.043	0.999	1.00
	SMI-TED	5,028	633 (12.6%)	633	0.043	0.999	1.00
	Avalon	5,028	647 (12.9%)	629	0.047	0.998	1.40
	Topological torsions	5,028	675 (13.4%)	631	0.047	0.998	1.10
	MACCS keys (aug.)	5,028	2,343 (46.6%)	437	0.220	0.965	2.80
Caco-2 (log units)	ECFP4	2,158	179 (8.3%)	177	0.027	0.996	0.74
	Atom pairs	2,158	177 (8.2%)	177	0.027	0.996	0.74
	PDV	2,158	177 (8.2%)	177	0.027	0.996	0.74
	SMI-TED	2,158	177 (8.2%)	177	0.027	0.996	0.74
	Avalon	2,158	181 (8.4%)	177	0.027	0.996	0.74
	Topological torsions	2,158	186 (8.6%)	177	0.029	0.996	0.74
	WL graph kernel	2,158	203 (9.4%)	177	0.028	0.996	0.74
	MACCS keys (aug.)	2,158	804 (37.3%)	139	0.111	0.935	1.72

Table 4.3: Molecules that share their feature vector with at least one other molecule, one row per representation that ties any molecules. Tied gives the number and share of such molecules, and Stereo-only those tied only with their own stereoisomers. Best RMSE and best R^2 are the best any model can reach given the ties. Best RMSE and Largest spread, the largest label range within one tied group, are in the units beside the dataset name, and (aug.) marks an augmentation representation.

4.3 Flaws in similarity and other pairwise computations

Section 4.2 described some of the limitations of standard representations. Here we ask what happens when two molecules are compared through one of them, as these

pairwise comparisons (Chapter 2) can violate the SPP.

The Tanimoto coefficient is the most popular similarity metric in cheminformatics [128], and is commonly computed on ECFPs, although the coefficient applies to any bit vector. It has been described as the "coefficient of choice" for molecular similarity [**?**], while the Morgan fingerprint, ECFP4 in this research, ranks among the strongest fingerprints for virtual screening and target prediction [129]. ECFP4 and ECFP6 specifically, alongside topological torsion, rank among the best of 28 fingerprints tested for ordering diverse structures by similarity [130]. However, it is not a one-size-fits-all solution. Across 96,456 human metabolites, 72.9% had an ECFP4 nearest neighbor at Tanimoto distance 0 [129]. Stereochemistry had been removed before that comparison, so these are constitutionally different molecules that ECFP4 cannot tell apart.

A similarity coefficient cannot tell you which features matter to the prediction task at hand. And the SPP holds only relative to such a measure. To understand the impact of irrelevant features on similarity measures, we make the following computation. First, we categorize a representation's features into those relevant to the task, R, and those irrelevant to the task, I, and take the squared Euclidean distance as a sum over both:

$$d^2(\mathbf{x}, \mathbf{y}) = \sum_{j \in R} (x_j - y_j)^2 + \sum_{j \in I} (x_j - y_j)^2. \tag{4.3}$$

Every non-constant irrelevant feature adds one further term, so the irrelevant portion grows linearly with $|I|$. After standardization each feature contributes the same amount on average over all pairs, so the share of the distance carrying signal is about $|R|/(|R| + |I|)$. When we use a general-purpose representation like ECFP4 on a dataset spanning a small portion of the chemical space, $|I|$ can be a lot larger than $|R|$. Constant features contribute nothing, while near-constant features have a strong impact. The worst-case scenario involves a single rare bit set in only a handful of molecules. After standardization, uncommon bits take on larger values, outweighing common features that may be more relevant. If this bit is irrelevant to the task at hand, the similarity measure suffers.

Larger representations compound the problem. As the number of features grows, the nearest and farthest molecules end up at nearly the same distance, meaning that these pairwise distances are less able to discriminate, as explained in Chapter 2. Two measures of this effect are the relative contrast, defined as the difference between the

largest and smallest measured distance divided by the smallest [131], and hubness, a measure of how unevenly the molecules appear in other molecules' nearest neighbors [132]. These errors propagate along larger data pipelines, starting from the representation into similarity, from similarity into the set of neighbors a molecule is compared against, and from there into a prediction of some kind.

We measured four things on our datasets: how many activity cliffs a representation finds, whether the distance between two molecules tracks the difference in their labels, whether two representations agree on each molecule's nearest neighbors, and whether a fingerprint distance agrees with a distance based on shared structure. The datasets are the six small-molecule regression datasets of Table 5.15 (Chapter 5) and human liver microsome (HLM) clearance, a third dataset from Biogen not used elsewhere in this thesis. Binary representations were compared by Tanimoto distance, and all others by Manhattan distance on standardized features (next subsection), with constant features removed.

Activity cliffs. Chapter 2 gives the MoleculeACE definition of an activity cliff, which combines three similarity measures. Here we use one. Two molecules are similar if their ECFP4 Tanimoto similarity is at least 0.55, and a molecule is a cliff if a similar molecule has a label that differs from its own by more than a set gap. We chose the gap on each dataset so that 15% of the molecules are cliffs, so the gap follows the spread of the labels (Table 4.4). On hERG K_i the gap is 1.76 pK_i, and 211 of the 1,412 molecules are cliffs. On QM9 only 1,093 pairs of the 9,978 molecules are similar, and the gap is 0.05 eV, 0.06 times the median absolute deviation of the labels. On the three Biogen datasets (RLM clearance, MDR1–MDCK efflux and HLM clearance) it is 0.20 to 0.34 log units, 0.50 to 0.60 times the median absolute deviation.

The count of cliffs depends on the representation. We kept each dataset's gap and replaced ECFP4 with each other base representation, calling similar the same number of closest pairs as ECFP4 does. On hERG K_i the number of cliff molecules then ranges from 138 (atom pairs) to 267 (WL graph kernel). Of the pairs that ECFP4 calls similar, only 57% to 74% are also among another representation's closest pairs, and on QM9 only 1% to 24% (Additional file S.2).

Distance and label difference. We asked whether distance tracks label difference in two ways. Over all pairs of molecules, we took the Spearman correlation

between the distance of a pair and the absolute difference of its labels, a one-number summary of the neighborhood-behavior plot of Patterson et al. [133]; datasets above 2,000 molecules were sampled down to 2,000. Over each molecule's closest pairs only, we predicted each label as the mean label of the molecule's five nearest neighbors, taken from other scaffold groups, and took the Spearman correlation between predicted and measured labels.

Across the 70 combinations of dataset and representation, the all-pairs correlation ranged from -0.04 to 0.23 and was below 0.10 in 44 of them. The five-neighbor prediction reached 0.29 to 0.89, higher than the all-pairs correlation in all 70 (Table 4.4; every representation is in Additional file S.2). On hERG K_i with ECFP4, the all-pairs correlation is 0.06, while the five-neighbor prediction reaches 0.72, with an RMSE of 0.63 pK_i against a label standard deviation of 0.92 pK_i.

Figure 4.3 shows why the two differ. Of the 996,166 hERG K_i pairs, 93% have an ECFP4 Tanimoto similarity below 0.20, and only 1.1% reach 0.55. Over all pairs the median label difference is 0.81 pK_i, but above a similarity of about 0.4 it is 0.20 to 0.30 pK_i, although 8.3% of the 774 pairs at 0.8 or above still differ by more than 1 pK_i. Mordred shows the same pattern: the closest 5% of pairs have a median difference of 0.44 pK_i, and every other 5% has 0.78 to 1.11 pK_i. The useful information sits in the few closest pairs. The all-pairs correlation is set by the other 95%, whereas the five-neighbor prediction uses only the closest pairs.

Agreement on neighbors. We compared the five nearest neighbors of each molecule, from other scaffold groups, under ECFP4, SMI-TED and Mordred (PDV on Caco-2, where Mordred is not a base representation; on HLM clearance ECFP4 is not a base representation and its stored features were used). On hERG K_i any two of the three share on average 0.47 to 0.52 of each molecule's five neighbors, and on the three Biogen datasets and QM9 only 0.13 to 0.21 (Table 4.5).

Agreement with shared structure. For 500 random pairs per dataset we found the maximum common substructure (MCS) with RDKit [30], allowing 2 seconds per pair and leaving out pairs that ran out of time. The MCS distance is $1-n/(a+b-n)$, where n is the number of atoms in the common substructure and a and b are the numbers of heavy atoms in the two molecules. Its Spearman correlation with ECFP4 Tanimoto distance ranged from 0.20 (hERG K_i and HLM clearance) to 0.51 (Caco-2 and LogD).

		All pairs			Five neighbors			Cliffs	
Dataset	Molecules	ECFP4	Desc.	SMI-TED	ECFP4	Desc.	SMI-TED	Molecules	Gap
hERG K_i	1,412	0.06	0.07	0.09	0.72	0.72	0.69	211	1.76 pK_i
LogD	5,028	0.12	0.06	0.10	0.76	0.80	0.76	731	2.90 log
Caco-2	2,158	0.08	0.18	0.14	0.59	0.62	0.57	324	1.21 log
RLM clearance	3,053	0.02	0.07	0.04	0.36	0.49	0.46	458	0.34 log
MDR1–MDCK efflux	2,641	0.02	0.16	0.10	0.51	0.58	0.59	397	0.20 log
HLM clearance	3,086	−0.00†	0.07	0.03	0.37†	0.49	0.45	462	0.27 log
QM9	9,978	0.19	0.18	0.13	0.88	0.84	0.85	1,495	0.05 eV

Table 4.4: All-pairs correlation, five-nearest-neighbor prediction and activity cliffs for each dataset. All pairs gives the Spearman correlation between pair distance and absolute label difference, and Five neighbors the Spearman correlation between each label and the mean label of its five nearest neighbors. Cliffs gives the number of cliff molecules and the label gap that defines a cliff. Desc. is Mordred, or PDV on Caco-2, and † marks ECFP4 where it is not a base representation of the dataset.

Every method in Section 4.4 takes a similarity as input and inherits the dilution described here.

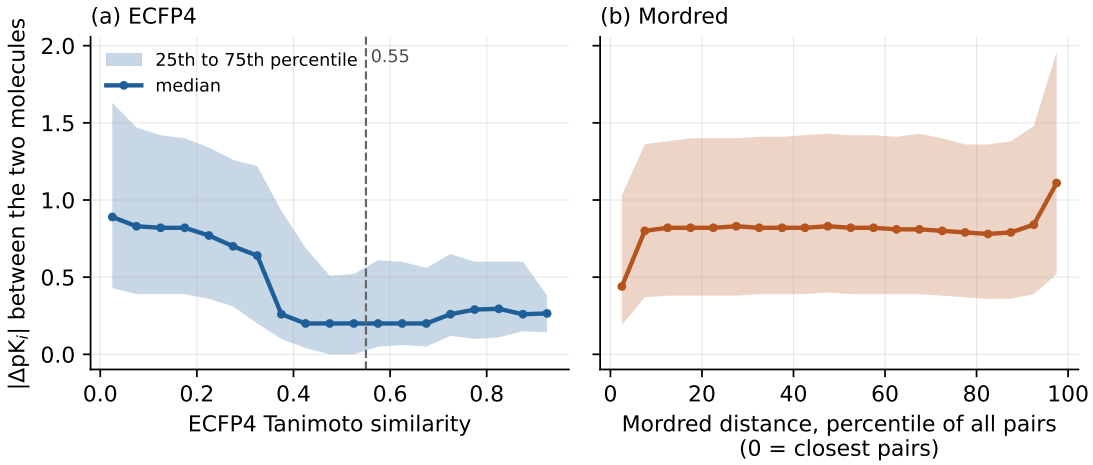


Figure 4.3: Absolute pK_i difference against distance for all pairs of hERG K_i molecules, with the median in each bin (markers) and the interquartile range (shaded). a) ECFP4 Tanimoto similarity in bins of 0.05 (bins with fewer than 20 pairs omitted), with the activity-cliff threshold of 0.55 dashed. b) Manhattan distance on standardized Mordred features, as a percentile of all pairs.

4.3.1 Distance metrics for continuous molecular descriptors

While binary fingerprints are compared with the Tanimoto coefficient (Chapter 2), we need distance metrics for continuous feature vectors. The research in Section 4.4.1 used the following four:

- Euclidean (L_2): standard geometric distance, $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$

Dataset	Desc.	Five-neighbor overlap			ECFP4 against MCS		
		ECFP4–Desc.	ECFP4–SMI-TED	Desc.–SMI-TED	Spearman	Pairs	Timed out
hERG K_i	Mordred	0.52	0.50	0.47	0.20	495	5
LogD	Mordred	0.38	0.41	0.41	0.51	494	6
Caco-2	PDV	0.38	0.41	0.38	0.51	492	8
RLM clearance	Mordred	0.15	0.21	0.18	0.28	500	0
MDR1–MDCK efflux	Mordred	0.15	0.20	0.17	0.26	500	0
HLM clearance	Mordred	0.15	0.21	0.18	0.20	500	0
QM9	Mordred	0.13	0.18	0.21	0.46	500	0

Table 4.5: Agreement between representations for each dataset. Five-neighbor overlap is the mean fraction of a molecule's five nearest neighbors that two representations share. ECFP4 against MCS gives the Spearman correlation between ECFP4 Tanimoto distance and MCS distance, the number of pairs used, and the number left out because the MCS search timed out.

- Manhattan (L_1): sum of absolute differences, $d(x, y) = \sum_i |x_i - y_i|$, more robust to outlier features
- Cosine: angle-based, $d(x, y) = 1 - \frac{x \cdot y}{\|x\|\|y\|}$, invariant to magnitude
- Mahalanobis: covariance-adjusted distance, $d(x, y) = \sqrt{(x - y)^\top S^{-1}(x - y)}$, where S is the feature covariance estimated on the training molecules [134]

All features are standardized to zero mean and unit variance prior to distance computation.

The later chapters use a different set, chosen after that study. The hybrid search's nearest-neighbor objective (Chapter 5) uses Manhattan distance only. Chapter 6 compares Manhattan, cosine and Minkowski distance with $p = 0.5$, $d(x, y) = \left(\sum_i |x_i - y_i|^{0.5}\right)^2$, on datasets of up to 5,000 molecules, and Manhattan and cosine on larger ones. The Minkowski distance with $p = 0.5$ is a dissimilarity rather than a metric, because it breaks the triangle inequality.

4.4 Consequences for similarity-based methods

We've identified some issues within the representations and similarities computed over them. While this calls into question the predictive limits of QSAR and other structure-based modeling, it also calls into question downstream applications of molecular similarity. Nearest-neighbor methods, where a molecule's neighbors, as measured by similarity, are the inputs to several other processes, are by definition

sensitive to the quality of similarity measurements. They are used within other processes like predicting a molecule's activity using its neighbors' labels, detecting outliers, either ACs or noise, defining an applicability domain by distance to the training set [135], grouping molecules into clusters for data splits [136], identifying activity cliffs, and, in federated learning, weighting each site's vote on a molecule by that molecule's distance to the site's own data [137]. If this measure of similarity, or equivalently distance, is flawed, then all of these downstream applications will feel the effects of it.

Section 4.4.1 reports the exploratory LNL study, in which the 20 configurations with the lowest retention under noise all relied on a pairwise distance measurement somewhere in their process. Section 4.4.2 focuses directly on neighbor-based noise detection, the most clear-cut use of similarity.

4.4.1 Exploratory study of learning with noisy labels

This section reuses an exploratory study of LNL carried out earlier in this project. In Chapter 3, model family explained the largest share of the variance in robustness to label noise, and representation at most 11.2%, so we asked whether training methods built for noisy labels improve robustness, and whether the representation still matters under them.

LNL methods and the five groups of Song et al. [47] are described in Section 2.5.2, and such methods have been studied across machine learning, including QSAR. Most LNL methods were built for image classification with flipped labels [47], so they do not transfer automatically to continuous labels.

The study tested 12 LNL methods on the QM9 HOMO–LUMO gap (10,000 molecules, scaffold splits), adding Gaussian noise with σ from 0 to 1.0 in steps of 0.1 to the training labels only. The representations were ECFP4, Sort & Slice [5], PDV, canonical SMILES and randomized SMILES. Depending on the method, a configuration could add the model's own uncertainty, a distance between molecules (Tanimoto, Euclidean, Manhattan, cosine or Mahalanobis), an alternative GP kernel or an alternative loss function, giving 221 configurations. Four methods, small-loss selection, confident learning, distance-based selection and multistage cleaning, remove suspected molecules and retrain. These are noise mitigation methods; the other eight are noise-robust training methods.

We measured noise robustness by retention, R^2 under noise divided by R^2 with

clean labels, as a percentage, which is the retention ratio of Chapter 3 ("Evaluation"). Methods that used a distance did worse than those that did not, and in an analysis of variance over the adapted components, whether a distance was used explained the most variance in R^2. All 20 configurations with the lowest retention used a distance, and none with a distance reached both a clean R^2 above 0.6 and a retention above 80%.

The methods that held up estimate noise from the model's own loss or uncertainty. DivideMix [138] retained the most, 86.7% (standard deviation 0.4%), followed by small-loss selection [95] at 84.2% (2.1%) and the uncertainty curriculum, which orders molecules by the model's own loss [139**?**], at 82.7% (3.5%). The heteroscedastic GP, a GP whose noise is predicted per molecule by a second model [140], retained 79.0%, averaged over 15 configurations, with R^2 of 0.71 on clean labels and 0.56 at $\sigma = 0.6$, where its retention was measured. The exploratory study does not record this noise level for the other methods. Regularization alone retained less, 73.0% for Mixup and 74.0% for sharpness-aware minimization. Under these methods, the choice of representation explained even less variance than in Chapter 3.

The study used one dataset, one property and one noise condition (Gaussian noise on every training label), and Chapter 6 repeats the question on more datasets and under other noise conditions (Section 2.5.1). It also added the distance step to methods designed without one, so poor adaptation is an alternative explanation.

The distances may have failed for two reasons, which the evidence here does not separate. First, a single representation may lack the information a distance needs, with and possibly even without noise. PDVs encode global molecular properties, ECFP4 contains substructure information and MHG-GNNs encode the molecular graph [7], and any one of these sources alone may not be sufficient for robust similarity estimation. Second, some representations are sparse, with many features irrelevant to the molecules of a given dataset (Section 4.3). We sought to address the second with the adaptive hybrid representation (Chapter 5), which concatenates the top-ranked features by importance from several base representations.

4.4.2 Noise detection by neighbors

Neighbor-based noise detection treats a label far from its neighbors' labels as probably wrong, an idea that goes back to editing nearest-neighbor training sets [98]. The

wider literature on filtering noisy labels concerns classification [141, 142]. Martín et al. [143] adapt fourteen classification noise filters to regression, seven of them nearest-neighbor filters that use the neighbors' mean label. They find that the filters improve regression methods on 20 general regression benchmarks with injected label noise, none of them QSAR datasets. The neighbor-based noise scores, sample removal, sample weighting, label smoothing and the recovery rate are defined in Chapter 6, "Noise mitigation methods".

The method needs a distance that picks the right neighbors, and neighbors with clean labels. Earlier sections of this chapter argue that the first condition fails, and the second fails whenever noise is spread over the whole training set. A large gap from the neighbors' labels can mean a noisy label, a noisy neighbor or the wrong neighbors, and the score cannot tell which. Activity cliffs also look like noise to such a detector (Chapter 2, "Molecular similarity and activity cliffs").

The exploratory study also used neighbors to clean labels, on 5,000 randomly chosen QM9 molecules (HOMO–LUMO gap, scaffold split) with Gaussian noise on the training labels at $\sigma = 0.3$ and $\sigma = 0.6$. Each cleaning method was judged by retraining a random forest on ECFP4 with clean, noisy and cleaned labels, giving the recovery rate, the share of the R^2 lost to noise that cleaning recovers (Equation 6.5). Neighbor-based scores compared each label with those of the molecule's k nearest training molecules under Euclidean, Manhattan or cosine distance. Table 4.6 holds only the configurations built on a single representation; those built on an early hybrid are described below. This study is separate from the sample-removal study of Chapter 6.

With the neighbor-based scores used as sample weights at strength $\alpha = 0.1$, mean recovery was negative for every representation at both noise levels (Table 4.6). Of all 860 cleaning configurations, 8 had positive recovery at both noise levels; 6 of these used a single representation, and 4 of those 6 were neighbor-based. Label smoothing toward neighbor or model estimates gave positive recovery in 5.0% of configurations at $\sigma = 0.3$ and 0.8% at $\sigma = 0.6$. Mean recovery was negative at every weighting and smoothing strength tried, from 0.1 to 0.7.

Judging a label by its neighbors usually made the model worse and rarely helped at both noise levels, but it did not always fail. The best configuration at $\sigma = 0.6$ was neighbor-based (PDV, Euclidean distance, local variance ratio score) and recovered +17.2%. Because noise was added to every training label, the neighbors' labels are noisy too; the exploratory study gave this as the reason label smoothing failed.

	Mean recovery (%)		Positive (% of configurations)	
Representation	$\sigma = 0.3$	$\sigma = 0.6$	$\sigma = 0.3$	$\sigma = 0.6$
ECFP4	−3.0	−7.3	33.3	16.7
PDV	−2.4	−3.4	50.0	27.8
MHG-GNN	−0.1	−7.2	38.9	5.6

Table 4.6: Recovery rate in R^2 on the QM9 HOMO–LUMO gap when neighbor-based noise scores are used as sample weights, for each representation used to find the neighbors, at two levels σ of Gaussian label noise. Recovery is the share of the R^2 lost to noise that cleaning regains, so negative values mean cleaning made the model worse. Positive is the share of configurations with recovery above zero.

Uncertainty-based methods, which use no neighbors, showed potential at lower noise but degraded at higher noise. Sample weighting did better than label smoothing but usually still hurt, with positive recovery in 14.2% of configurations at $\sigma = 0.3$ and 4.7% at $\sigma = 0.6$, and mean recoveries of −22.9% and −20.9%.

The study also found neighbors in an early hybrid of 100 PDV and MHG-GNN features, built by greedy forward selection, which is no longer part of the method. Its mean recovery with neighbor-based scores was −0.9% at $\sigma = 0.3$ and −3.0% at $\sigma = 0.6$, with 44.4% and 27.8% of configurations positive. Chapter 6 repeats the measurement in a hybrid and under noise conditions other than Gaussian.

4.5 Conclusions

On a single dataset, often very little of a standard representation varies. On hERG K_i, 1,815 of the 2,048 ECFP4 features take one value in at least 95% of the 1,412 molecules (Section 4.2.1). Several representations also give different molecules, mostly stereoisomers, the same feature vector, and no model can separate those molecules, whatever their labels. ECFP4 gives 56 of the 1,412 hERG K_i molecules a vector shared with another molecule, all of them stereoisomers ("Molecules with identical feature vectors").

A distance sums over every feature, so when only a few features matter for the property, the rest dominate the sum (Section 4.3). Across 70 combinations of dataset and representation, the Spearman correlation between pair distance and label difference was below 0.10 in 44. Predicting each label from its five nearest neighbors gave a higher Spearman correlation in all 70, so the useful information sits in a few close pairs rather than in the distance as a whole.

In the exploratory LNL study, configurations that used a distance did worst under label noise (Section 4.4.1), and judging a label by its neighbors' labels usually made the model worse (Section 4.4.2). It did not always fail, since one neighbor-based configuration gave the best recovery at the higher noise level. That noise was Gaussian on every training label, so each molecule's neighbors were noisy too; Chapter 6 turns to the other noise conditions of Chapter 2.

A representation that avoids these flaws would need five properties (Table 4.7). Its features would be chosen for the task, using only each fold's training labels. It would be compact, dropping features that do not vary or do not matter, with the number kept searched rather than fixed. It would draw on several representations, since each misses information the others capture; plain concatenation does this but is not compact (Chapter 5, "Combining representations"). It would work both as a model input and as a space for similarity. Finally, it would be traceable: each chosen feature leads back to its representation and can be read chemically wherever the base feature can.

Feature selection cannot fix every flaw. Molecules identical in every base representation stay identical in the hybrid, and a selected hashed bit still merges every substructure folded into it. Selection also needs enough labeled molecules to estimate which features matter.

These flaws are where the thesis began (Chapter 1), and Chapter 5 builds the method they led to, the adaptive hybrid representation. The settings that method searches over are listed in Chapter 5, "The search space".

Flaw (where shown)	Requirement	Where handled
Features that do not vary ("Constant and near-constant features")	Features chosen for the task, within a budget	"Feature importance scoring"
Molecules merged into one vector ("Molecules with identical feature vectors")	Features drawn from several representations	"Combining representations"
Distance dominated by irrelevant features ("Flaws in similarity and other pairwise computations")	Compact, with the number of features searched	"The search space"
Poor performance of neighbor-based methods ("Consequences for similarity-based methods")	Usable as a space for similarity	"Similarity in a hybrid" (Chapter 6)
Features that are hard to read chemically ("Flaws in standard representations")	Traceable, so that each chosen feature leads back to its representation and can be read chemically wherever the base feature can	"Blueprints and their chemical interpretation"

Table 4.7: Flaws of standard representations and similarity, the requirement each sets for a representation, and where the requirement is addressed, in Chapter 5 unless stated.

Chapter 5

Building Adaptive Hybrid Representations

5.1 Introduction

So far, Chapter 4 explained how the standard methods of estimating molecular similarity, or conversely distance are flawed and potentially break the SPP. Specifically, the representaions used in these similarity calculationsc contains irrelevant features that either do not vary or more generally do not pertain to the prediction task at hand. This chapter explores a solution. With the understanding that each representation contains a set of features which contain a certain type of information about that data. We aim to select the most informative features across a larger set of representations, while filtering out the irrelevant ones. We create an adaptive hybrid representation, a set of features bounded by a budget that is selected from several base representations by a search using one dataset and downstream model. The goal is to select a set of features that is customized for a given model and prediction task. As discussed in Chapter 3, performance can vary signficantly across datasets and models. Rather than hope the features you give a model will be useful, this process let's you prescreen them.

In this chapter we build the adaptive hybrid representation, referred to as the hybrid representation, first reporting on when and why it helps. The questions this chapter and Chapter 6 address are set out in Chapter 1 (Section 1). This chapter measures predictive performance under both standard labels and labels with injected artificial noise. Distance meseasures between hybrids, noise detection and LNL are covered in Chapter 6.

5.1.1 Combining representations

Not every molecular representation is equal. In fact, they hold very different types of information. RDKit2D descriptors matched or outperformed features derived from embedded GNNs and pretrained language models on many datasets across 62,820 models on a variety of molecular property prediction benchmarks [101]. Descriptor-based models held 73% of top-three positions on 11 benchmarks [144]. In 1,310 ChEMBL assays, feedforward networks on ECFP fingerprints outperformed graph convolutions and recurrent networks trained on SMILES [145]. We know that the choice of representation matters, as is discussed in Chapter 3, yet this choice is usually decided by trial-and-error. Beyond just molecules, general protein language models predict antibody expression better than antibody-specific ones, while the antibody-specific models are better on binding [146], and no pathology foundation model is best on every clinical task [147]. We turn our attention to combining features from these different representations.

Most research in this field relies on a single representation, and few combine features from different representations beyond concatenation. One of the reasons just pooling features from different representations together doesn't work is dimensionality. If you add a standard ECFP4 ($d = 2,048$) with Morderd descriptors ($d = 1,613$) and an MHG-GNN embedding ($d = 1,024$), you end up with 4,685 features, which is far too many considering many datasets from experimental assays contain only a few hundred labeled samples. With such ratios, more features mean weaker predictions [148, 149]. Therefore, we consider ways of combining representations.

The most straightforward method of combining representations is concatenation. This however degrades wiht non-homogenous feature sets and obscures which reprsentation is responsible for the score [150]. Deng et al. [101] found that appending RDKit2D descriptors to GROVER lowered RMSE on 9 of 24 datasets tested. Late fusion trains one model per source and aggregates the predictions [151]. MOGONET is of this kind, combining per-source predictions in label space [152]. While this is effective, it loses interactions between the source representations, an issue which is addresed by intermediate fusion, for example cross-attention between sources [153]. Multiple kernel learning does this with kernels rather than whole models [154, 155], and produces no interpretable feature sets. Laufkötter et al. [156] found that combining bioactivity-based fingerprints with structure-based ones like ECFP4 improved both predictive performance and scaffold-hopping. Similarly,

Turkina et al. [157] combined features from six non-hashed fingerprints with feature importance scores and showed that the additional features consistently improved predictive accuracy over the original fingerprints across a range of models. Most of this work involves limitation combinations, adding together standard fingerprints and linear representations [21, 156]. Although the features derived from embeddings are not as conceptually-straightforward, they contain another axis of information.

Beyond simply combining features, there are a range of other methods that handle training on multiple representatinos or excessive feature sets. Ensembling fits one model per representation and aggregats their predictions, essentially combining the information carried by each representation after tarining rather than before. Consensus QSAR works this way [158]. One drawback is that no model in an ensemble sees features from other representations so their interaction is never measure, and every representation and every model must be computed for each new

When you want to select filter a feature set for the most important to the prediction task, you can use a dimensionality reduction technique or feature selection methodology. Principal component analysis (PCA) uses linear combinations of the original features, ordered by the variance they explain [159]. Their combined features are dependant on the original feature units unless each feature is z-scored first which matters when data from different modalities like fingerprint bits, descriptors and embeddings are joined [159]. PCA never sees the labels, and components with small variance can still be important in prediction [160]. Partial least squares (PLS), the supervised counterpart in chemometrics, builds components that covary with the property [161]. Neither gives components that can be traced to one substructure, descriptor or representation, making them difficult to interpret.

Feature selection keeps a subset of the original features, unlike PCA or PLS which builds new ones. Of its three families [162], filter methods score features against the labels before any model is fitted, as minimum-redundancy maximum-relevance selection (mRMR) ranks features by their mutual information, defined as the label minus its redundancy with already-chosen features [163]. Wrapper methods score subsets of features by fitting the model [164]. Embedded methods select features while the model is fitted, as the does lasso [165]. Recursive feature elimination repeatedly fits a model and drops the lowest-ranked features [166].

When selecting features, the question is not only what features are most important, but also, what features does the model prefer? One of the ways to answer this is to consider how many features relative to the size of a dataset a given model works

best with. Distance-based models such as k-nearest neighbors (kNN) and SVM use every feature in every distance computation (Chapter 4), whereas tree ensembles choose one feature per split and can essentially ignore irrelevant ones. Dablander et al. [5], who created a substructure-based fingerprint called Sort & Slice, suggest that using labels to select features may result in overfitting. Table 5.1 compares these alternatives with the adaptive hybrid methodology developed in this thesis.

Method	Uses the labels	Keeps original features	Several representations	Retrain to add one
Concatenation [150]	no	yes	yes	yes
Late fusion and ensembling [151, 152, 158]	only if pooling weights are learned	yes	no	no, one more model
Intermediate fusion [153]	yes	no	yes	yes
Multiple kernel learning [154, 155]	yes	no, kernels only	yes	yes
PCA and PLS [159, 161]	PCA no, PLS yes	no	yes, on a concatenation	yes
Flat feature selection [162]	yes	yes	yes, on a concatenation	yes
Group lasso [167]	yes	yes, whole groups	yes	yes
Adaptive hybrid representation	yes	yes	yes	search rerun, method unchanged

Table 5.1: Methods for combining or reducing representations, compared by whether they use the labels, keep the original features, let one model see several representations, and must refit to add a representation.

5.1.2 The adaptive hybrid representation

The task of selecting the best features and combining them from a set of representations and combining them is not simple. How do you measure what features are "best"? Are these features "best" relative to the representation they came from or to the whole pool of features? Should you select more features from repesentations that seem to have more signal to begin with, or take a set number of featurs from every representaton? How many base reprsentations should you select features from? How many features should you select? When scoring potential combinatinos of featurse, what type of evaluator should you use?

In a dream world, there would be a single answer to each one of these questions that applies to every dataset. Unfortunately we live in reality, and as I discovered, the answer to every single one of these questions is "depends on the data." The task of selecting the best features then became a search space optimisation challenge. The question actually became for a given prediction task, defined by the dataset, prediction target, and model, what is the best methodology for selecting features? The hybrid search does not output just a list of features, rather it outputs an ideal search configuration to find features for your data. It tells you how to score features (feature importance score), what rules you should use to select them (allocation method), how to score potential hybrid feature combinations (internal evaluator). In this research, I run this hybrid search multiple times across different maximum number of features (budgets), number of input representations, and models (downstream evaluators) on each dataset. While it is not formal "training" in the ML sense, the hybrid search is essentially a feature selection method that is fit on the training data. The representations used in this thesis are listed in Tables 5.10, 5.11, 5.12, 5.13 and 5.14.

Formally, an adaptive hybrid representation is a set of fixed number of features, capped at a fixed budget, selected from several base representations of the same molecules. The output of a hybrid search is itself a feature selection method that when run on the given dataset, produces the hybrid repesentation. This search is run on a single datast and downstream model, and the resulting hybrid differs by model, dataset, and even split. Its intended use case is when there are multiple base representations that describe different aspects of a molecule, and you want to combine these different features for a single training task. The hybrid ranks features by a fitted model's importance scores, as embedded methods do, and keeps a fixed number of the top-ranked ones, as a filter does. Classical methods treat all features as one flat list. The nearest classical relative, the group lasso, keeps or drops predefined groups of variables whole [167], whereas the hybrid splits the budget among representations (Section 5.2.2). Because only original features are kept, each traces back to one representation, which makes blueprints possible.

5.1.3 Blueprints and their chemical interpretation

As explained above, the adaptive hybrid representation relies on a search. A completed search is saved as a blueprint, which records the settings needed to select the

features that make the hybrid representation. Because each selected feature traces back to its original base representation and relative importance rank, the search can tell you about your own prediction task. The best configuration is saved for each downstream model and budget, giving one blueprint per pair, allowing us to compare blueprints and see if different models prefer different features.

Depending on the origin of the feature, some are chemically interpretable. By definition, the PDV contains values that relate back to chemical propertie like molecular weight or logP. An ECFP4 bit can be shown as the atom environment that sets it, and a MACCS key stands for a known substructure. Beyond small molecules, the readable unit of an antibody is a sequence position and its residue, and for gene expression a named gene set, such as a pathway or a chromosome arm. This feature-level explainability fails when features are derived from learned embeddings. That will only tell you if the learned embedding itself is useful as a representation, but nothing about the selected feature.

While a set of thousands of features may seem overwhelming and irrelevant, identifying substructures or descriptors that recur among the selected features can point to parts of the molecule that may be involved in binding or activity. If the selected featues make sense chemically for that endpoint, it's a good sign that the selection mechanism is finding real signal instead of fitting nose. Conversely, if the hybrid search is done over a range of budgets or models, and the blueprints are showing a vastly different set of features, that may be an indication that the features at hand are struggling to provide information about the prediction task. A predictor that scores proposed molecules in generative design [168] could also take a hybrid as input, which we leave to future work.

5.1.4 Antibodies and beyond

The hybrid search is a generalised search that takes in multiple feature matrices fro the same samples and a supervised prediction task, and outputs a set of feature selection method that produces a filtered set of features from those original vectors. The default search objective scores prediction only, meaning the methodology applies to tabular tasks or cases where you have different measurements of the same inputs. Although this was developed with small molecules and QSAR modelling in mind, it is by no means limited to that use case. While there are some functionalities in this research that are tailored to chemical tasks, such as blueprint mappings features

back to their chemical interpretations and the use of nearest-neighbor objectives Chapter 6 to aid in the computation of molecular similarity, the core functionality of the hybrid process is domain-independent.

One caveat concerns sparse representations such as one-hot encoded sequences. Each of their features is set in only a few samples, so each scores low importance on its own. Such encodings are usually passed to a network that learns a representation from them, but they can also be used directly. Here, the one-hot encoding is a base representation on nanobody thermostability, and the blocks substitution matrix (BLOSUM) encoding on humanness and trastuzumab–HER2 binding. One-hot was among the selected representations on **[RESULT NEEDED: antibody datasets where one-hot was selected, with score]**.

While this setup was initially intended for small molecules, we extend this research to antimodies. Four antibody dataset are used in this chapter: Koenig expression, nanobody thermostability, humanness and trastuzumab–HER2 binding (Table 5.9). Reactions (two Buchwald–Hartwig yield datasets) and nucleotide sequences (the H3K4me3 histone-mark and TATA promoter tasks, Table 5.13) are used to test applications of this framework in entirely separate domains.

Another caveat of this setup is that the hybrid can help only on datasets with enough samples (see Section 5.11) and with base representations whose features carry importance. When all the representations score poorly, there are not enough useful features to pull from. Additionally, if there is only a single base representation that carries signal, there is nothing for the hybrid to improve upon, which is demonstrated in Section 5.4.

5.2 Methods: the hybrid search

Figure 5.1 outlines the hybrid search.

5.2.1 From base representations to a hybrid

The input to the search is K frozen representation matrices $\mathbf{X}^{(k)} \in \mathbb{R}^{n \times d_k}$, $k = 1, \ldots, K$, for the same n samples, whose widths d_k range from tens to thousands of features, as well as a label vector $\mathbf{y}$, one group label per sample for cross-validation (for example, its scaffold group), the evaluation metric, the task type, one or more downstream models and a list of budgets. It returns a hybrid of at most B features

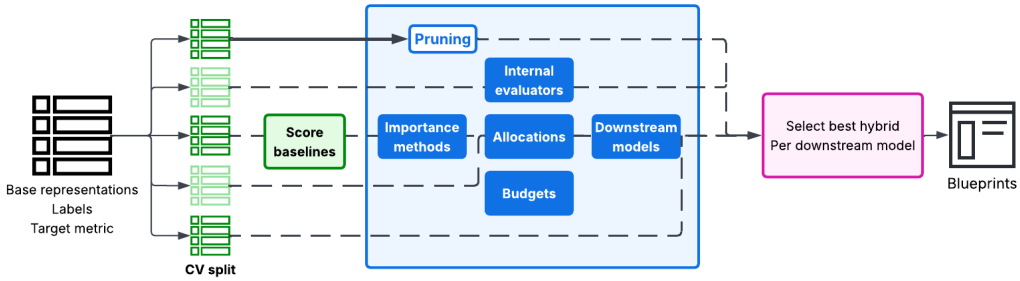


Figure 5.1: Schematic overview of the hybrid search. The inputs (left) are split into five cross-validation folds, and the single representations are scored as baselines. Each fold is searched over importance methods, allocation methods, budgets, internal evaluators and downstream models (center). The best hybrid for each downstream model is stored as a blueprint (right).

before any augmentation features added outside the budget (Section 5.2.3), where the budget B is a maximum rather than a target, and a blueprint that records the settings necessary to rebuild the hybrid representation on another set of features, presumably another subset within the same dataset.

When combining representations, we are careful to avoid data leakage. Because the feature selection process trains on the features and their labels, pooling all the features prior to selection would leak validation-level information into the training, which is a well-documented bias in feature selection [169, 170]. As such, we run each hybrid search within a cross-validation fold. When a single representation is score, we impute missing values with training-fold medians. A representation wider than 5,000 features is scored on its own using only its top 500 features (200 on datasets under 1,000 samples). These are chosen in each fold by a LightGBM model fitted on the training part. This affects only that representation's score, not the features the hybrid can select from it.

The data are split at three levels within CV runs. Every split keeps each group whole, a scaffold for small molecules, a sequence cluster for antibodies, and some pre-defined grouping for datasets from other domains (Section 5.3.1). With fewer than five groups, the inner and internal splits use as many folds as there are groups (Additional file 16). Figure 5.2 shows the three levels as nested boxes.

For each representation k, the importance method gives every feature a score $s_j^{(k)} \geq 0$, computed on the 80% part of the internal split (Figure 5.2). These scores rank features only within their own base representation. The standalone score r_k is the score of the internal evaluator, trained on representation k in the 80% part and scored on the 20% part, in the dataset's metric. These scores cannot be compared

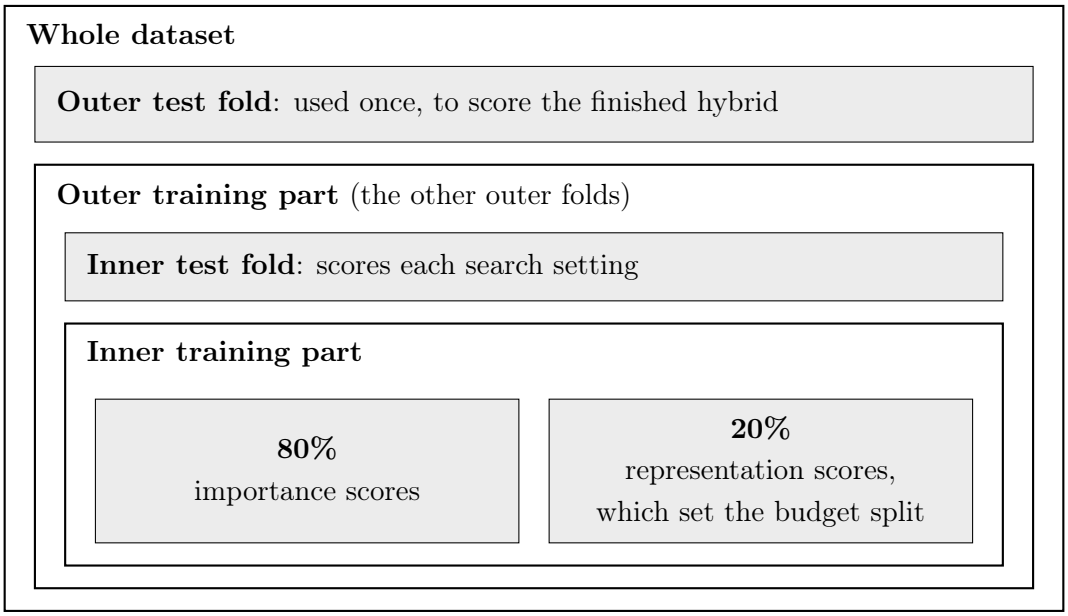


Figure 5.2: The three levels of splitting, each inside the one before. Every split keeps each group whole.

across metrics as they stand: a lower RMSE is better and it has no fixed upper limit, whereas a Spearman correlation is higher when better and lies between -1 and 1. Before the budget is split, each r_k is therefore converted to a score $\tilde{r}_k$ on which 0 means the representation predicts no better than a trivial predictor and 1 means it predicts perfectly. The trivial predictor depends on the metric. For Spearman, Pearson, the Matthews correlation coefficient (MCC) and R^2, $\tilde{r}_k = \max(0, r_k)$. For AUROC and the concordance index, $\tilde{r}_k = \max(0, 2(r_k - 0.5))$. For RMSE, $\tilde{r}_k = \max(0, 1 - \mathrm{RMSE}_k/\mathrm{RMSE}_0)$, where RMSE_0 is the error of predicting the training mean, and mean absolute error is rescaled likewise against the training median. For accuracy and weighted F1, $\tilde{r}_k = \max(0, (r_k - r_0)/(1 - r_0))$, where r_0 is the score of always predicting the majority training class.

An optional representation count N keeps the N representations with the highest standalone scores on the fold and discards the rest for that iteration of the search. Each remaining representation receives a weight $w_k = g(\tilde{r}_k)$, with g set by the allocation method, and a share $b_k^* = B\, w_k / \sum_l w_l$ features, rounded by largest remainder and capped at its width. A representation left out by the representation count, or whose standalone fit fails, receives no features. Thus $\sum_k b_k = \min(B, \sum_k d_k)$ and $b_k \le d_k$, and a representation with $w_k > 0$ keeps at least one feature. The global pool does not use these shares (see below).

Let S_k be the top b_k features of representation k by $s_j^{(k)}$, among those with

$s_j^{(k)} > 0$; if none scores above 0, its full b_k is taken. The cut exists because tree importance gives exactly 0 to every feature no tree split on, and filling the budget with those would pick them by their position in the matrix. The hybrid is $\mathbf{H} = [\mathbf{X}^{(1)}_{:,S_1}, \ldots, \mathbf{X}^{(K)}_{:,S_K}]$, with $\sum_k |S_k| \leq B$, so it can be narrower than its budget.

The hybrid search returns a list of settings for feature selection, referred to here as a configuration. It is scored by fitting the downstream model on the hybrid in each fold of the search and averaging the held-out scores. Each pair of downstream model and budget has its own search, and thus its own winner. Because this score is chosen over a range of selected configurations, it is optimistic [171, 172]. However, so is its direct comparison with the highest-performing base representation on the dataset. The whole search runs inside each outer training fold, and the winning settings are rebuilt there and scored once on the untouched outer test fold, both for the winner at each budget and for each downstream model's best across budgets. The inner search score minus the outer test score is **[RESULT NEEDED: inner minus outer test score, per dataset, in its own metric]**.

Chapter 6 uses two variants of this search. Under label noise, training quantities use noisy labels while held-out scoring uses clean labels unless. Under the nearest-neighbor objective, each molecule's activity is predicted from its five nearest neighbors in the feature space, as discussed in Chapter 6.

Blueprints. As explained above, a hybrid search returns a list of settings that function as a feature selection method to generate the hybrid representation itself. These settings are recorded in a blueprint that can be used on new folds of the same dataset or even new data. The blueprint contains additional information, including some fields for analysis, meta-data, and scores. These scores are used only for comparative purposes as in this research, the hybrids are rebuilt per-fold and are scored as such.

Listing 5.1 lists every field of a blueprint and its type.

The blueprint does not record the specific features that are returned by the hybrid, as it is meant to be re-run using the same settings on new, labeled data. Alternatively, you can reuse the features of a labeled fit and apply them to data with no labels under the assumption that the same features matter between sets.

The blueprint is extended by an additional explainbility layer. Since it does not store feature names, this additional step runs on a completed hybrid, records where each selected feature came from and, wherever possible, what that feature means.

```
Blueprint
  # What the search chose. A rebuild reads all of these.
  reps                  : list[str]           # every representation searched, not only those used
  rep_count             : int or None         # how many of them may contribute; None = all
  budget                : int                 # most features the hybrid may have
  allocation_method     : str
  importance_method     : str
  internal_evaluator    : str
  objective             : str                 # 'predictive' or 'similarity'

  # How the search scored configurations.
  metric                : str
  lower_is_better       : bool
  task_type             : str                 # 'regression' or 'classification'
  n_folds               : int
  seed                  : int

  # Augmentation settings. A rebuild reads all of these.
  augmentation_strategy : str
  aug_per_source_budget : int, or {source: int}, or None
  augmentation_margin   : float
  augmentation_fraction : float or None
  candidate_ranking     : str
  augmentation_threshold: float or None
  augmentation_budget   : int
  aug_max_candidates    : int

  # Stored for inspection.
  rep_scores            : {representation: float}
                          # each representation's best score over every molecule the search saw;
                          # the rebuild uses it to choose which representations to keep only when
                          # rep_count is set and no per-fold scores are given
  allocation_per_fold   : {fold: {representation: int}}      # features each representation got
  kept_augmentations    : {fold: {source: list[int]}}         # extra features kept, by index
```

Listing 5.1: Fields of a blueprint and their types. Comment lines group the fields, and each field is given with its type and, where needed, a comment on its meaning.

An ECFP4 bit is named by the atom environment that sets it, written as a SMILES fragment from a molecule with the bit set. MACCS keys are named by their SMILES arbitrary target specification (SMARTS) pattern and a short name, and descriptor features by descriptor name. Protein composition, descriptor and k-mer features, antibody composition, one-hot, BLOSUM, structural and inverse-folding features, gene-expression pathway sets and PCA components are named too. MAP4 and DRFP features get only a bit index, and Avalon, Mordred, atom-pair, graph-kernel and learned-embedding features only the representation name and dimension index.

5.2.2 The search space

To run a hybrid search, we first measure feature importance, select how many features to take from each contributing base representation, and how to score a candidate feature set. And for each of those steps, there's no one right way to do it. Different prediction tasks prefer different methods of scoring feature importance or candidate features, different sets of input hybrids, and so forth. The search tries

to find the ideal settings for the hybrid feature selection process, tailor to given dataset and downsream model. All combinations of settings values are explored, as the settings interact. The best budget depends on the downstream model, as shown below for nanobody thermostability. This search grid also tests the chapter's claim that no single setting suits every dataset.

Six settings in total are searched, namely the downstream model, the budget, the representation count, the importance method, the allocation method and the internal evaluator. Table 5.6 gives default search settings, organised by runtime speed, and Table 5.7 the smaller set searched on each dataset in this chapter (Section 5.2.4). The importance methods tested are LightGBM gain, TreeSHAP or DeepLIFT (Section **??**). Feature-level importance depends on the model as well as on the data [173], and as such we observe that within the search, a different ranking of features produces a different hybrid output.

The downstream model is trained on the hybrid representation. This model can be a supervised model of any form, classification or regression. We used random forest, XGBoost, LightGBM, SVM, a deep neural network (DNN) and a Gaussian process. Each dataset in this chapter uses two of them (Section 5.3.3).

The budget is the maximum number of features in the hybrid, taken from eight values (10, 25, 50, 100, 250, 500, 1,000 and 2,000 features). When the budget is too small, informative features can be left out, yet when it is too large it will pick up on too many uninformative ones, especially ones that give small datasets room to overfit.

[RESULT NEEDED: best budget per downstream model on nanobody thermostability, full runs]

We select a set of up to 10 base representations per dataset, chosen prior to the search (Table 5.15). The selections were made based on each representation's relative performance on the prediction task. We took into account what type of information each representation contained, and tried to span a wide set across the selected 10. For example, while there are many high-performing graph-based embeddings in QSAR modelling, for the purposes of testing a feature selection process which takes features from different sources of information, it is better to have a mix of graph-based embeddings, as well as other types of molecular representations like substructure fingerprints, property vectors, sequential embeddings, etc. The representation count (2, 3, 5 or all) sets how many may contribute features, keeping those with the highest standalone scores on each fold. The number of representation

counts to test is searched in the sweep as datasets differ in how many representations have features worth selecting, and pooled feature selection over all representations has no such setting. While we tested various methods of pooling the features, we found computing feature importance scores and concatenation techniques on a global pooled feature set was far less effective than when the feature scoring is kept on a per-representation basis. Since the features contain different types of information at different scales, pooling the features too early makes comparisons within in the pool challenging. We found that it's better to determine which features are important to each representation first, and go from there.

Every allocation method is paired with every count. In preliminary runs, some datasets performed best with 2 or 3 representations and others with all 10. **[RESULT NEEDED: datasets best with 2 or 3 representations and with all, full runs]**

Feature importance scoring

We score run a feature importance scoring method on each base representation which gives each features a single positive score. These scores remain relative within a given representation, so they are not compared or normalised between representations. Split-based tree importance is biased toward features with many possible split points [174], which matters here because we use binary fingerprint bits alongside continuous embeddings. No single importance method works best across datasets and downstream models. After testing a wider variety of methods found across literature, we settled three methods whose rankings did not correlate strongly with each other and whose selected features performed well as model input (Additional file 13).

We score features using LightGBM gain importance, SHapley Additive exPlanations (SHAP) values for tree ensembles (TreeSHAP) and DeepLIFT (Deep Learning Important FeaTures) [175]. LightGBM gain measures how much a feature reduced the training loss where trees split on it [58]. TreeSHAP gives each feature's exact Shapley value for each molecule's prediction from a tree model [176]. DeepLIFT measures how much a feature moves the output of a small network, trained with Adam [177], away from its output on an all-zero reference input, which after standardization places every feature at its training mean [175, 178, 179]. On Caco-2, the median share of features scoring above 0 was 30% for LightGBM gain, 70%

for TreeSHAP and 90% for DeepLIFT (preliminary, over 10 representations and 3 folds). Table 5.2 gives the settings of each method, including the pre-screen of representations wider than 2,000 features.

Method	Model fitted per representation	Score of one feature	Classification labels	Representations over 2,000 features
LightGBM gain	LightGBM, 50 trees, depth 6	Summed loss reduction of its splits; 0 if never split on	One model on the class labels	2,000 highest-variance features fitted; the rest score 0
TreeSHAP	RF, 50 trees, depth 10	Mean absolute SHAP value over training molecules	Binary: positive class; multiclass: mean over classes	As for LightGBM gain
DeepLIFT	NN, ReLU layers of 64 and 32 units, 100 epochs of Adam, learning rate 0.001	Mean absolute attribution against an all-zero reference (Rescale rule)	Binary: positive class; multiclass: mean over classes	Every feature scored

Table 5.2: Settings of the importance methods, fitted per representation and fold. Unlisted settings are library defaults.

Each dataset searches one, two or all three importance methods (Table 5.7).

Feature allocation methods

The allocation method decides how many of the B features each representation contributes; the importance method decides which. Of the five methods searched, four give each representation a fixed share computed from its rescaled standalone score (linear, squared, softmax and rank-decay allocation). The fifth, the global pool, lets the candidate features offered by every representation compete in one importance ranking.

The four fixed-share methods compute shares from the standalone score on the internal validation part. Only linear allocation makes the share proportional to that score. All four assume that representations with stronger predictive performance contribute more valuable features.

The assumption fails when a strong representation owes its score to a few features, so that a large share fills with weak ones. Fixed shares also ignore complementarity, so two representations carrying the same information both receive a share. Only the

global pool ranks features from different representations against each other.

The fixed-share methods differ only in how w_k is computed from $\tilde{r}_k$ (Table 5.3), with settings fixed for every dataset. Under linear and squared allocation, a representation with rescaled score 0 receives 0 features. Under softmax and rank decay, no representation receives 0 features because of its score. Since every rescaled score lies between 0 and 1, softmax cannot give one representation most of the budget, and with 10 representations the best takes at most 23.2% of it. Rank decay uses only the order, so with 10 representations the best receives 27.3% and the tenth 1.8%, whatever the gaps between their scores. Table 5.4 gives a worked example.

Method	Weight w_k	Fixed setting	How features are chosen	When a representation receives 0 features
Linear	$\tilde{r}_k$	none	Top b_k of its own ranking	$\tilde{r}_k = 0$
Squared	$\tilde{r}_k^2$	power 2	Top b_k of its own ranking	$\tilde{r}_k = 0$
Softmax	$\exp\big((\tilde{r}_k - \max_l \tilde{r}_l)/T\big)$	$T = 1$	Top b_k of its own ranking	never from its score
Rank decay	$e^{-0.3\rho_k}$, ρ_k = rank by score, 0 for the best	decay 0.3	Top b_k of its own ranking	never from its score
Global pool	$\tilde{r}_k^2$, sizing its offer to a pool of $2B$ candidates	power 2; pool $2B$	Top B of one joint ranking over all offered candidates	$\tilde{r}_k = 0$, or none of its candidates in the joint top B

Table 5.3: The five allocation methods. w_k is the weight computed from representation k's rescaled standalone score $\tilde{r}_k$, B is the budget, b_k is the share of representation k, ρ_k is its rank by score and T is the softmax temperature.

Method	$\tilde{r}_1 = 0.6$	$\tilde{r}_2 = 0.5$	$\tilde{r}_3 = 0.2$
Linear	46	39	15
Squared	55	39	6
Softmax	39	35	26
Rank decay	44	32	24

Table 5.4: Worked example of the four fixed-share allocation methods for a budget of 100 features and three representations. Each entry is the number of features a representation receives, with shares rounded by largest remainder.

The global pool sizes each representation's offer with squared weights $\tilde{r}_k^2$, as shares of a pool of $2B$ candidate features, each offer capped at the representation's width and holding only features scoring above 0. The same importance method is

fitted once on the joined offers, and the hybrid keeps the top B features that score above 0. Each representation's count is therefore an outcome of the joint ranking, with no minimum of one feature, and a feature's rank depends on what the other representations offered. The cost is one extra importance fit per configuration.

The five methods were chosen in a preliminary comparison (Additional file 13).

Search configuration and internal evaluation

The most useful features for an RF may not be the most useful for an SVM, and vice versa. The relative importance of a feature is dependent on the model [173]. Within the search grid, we us an internal evaluator to fit on one candidate feature set at a time. This process is separate from the final downstream model evaluation. In a standard search, four internal evaluators are used: RF, DNN, LightGBM, and the matched evaluator. When the internal evaluator is set to `match`, the search scores each candidate hybrid on the same model type as the downstream model, for each of the twelve model types used in this thesis. This means that internal evaluators outside the standard sweep may be tested. For a downstream model outside of the pre-defined set, if no matched evaluator exists in the code, and the search uses the other internal evaluators that were requested. Each internal evaluator is fitted on the internal training fold and scores on the internal validation fold within that CV iteration using the dataset's specified evaluation metric.

A random forest, LightGBM or DNN downstream model is searched with three internal evaluators, because its matched evaluator is already one of the other three. An XGBoost, SVM or Gaussian process downstream model is searched with four, because the matched evaluator adds one of its own family. Only the DNN evaluator shares its downstream model's architecture and training settings, and even it draws its 20% early-stopping molecules at random, whereas the downstream DNN splits them by the outer split's groups. Additional file 10 gives every evaluator setting.

The internal evaluator's scores determine how the budget is split and which representations are kept. The internal evaluator ranks candidate feature sets, it does explicity not rank features within a representation. This process of selecting a feature set by fitting a model on it, and keeping the set that scores the best is known as the wrapper method [164]. This is distinct from a filter method which scores features without fitting a model.

Section 5.2.2 gives how each allocation method turns these scores into a number

of features per representation.

The search tests every combination of the six settings on a dataset under 5-fold cross-validation. Over every value of Table 5.6, this is 8,160 configurations with five downstream models; with each dataset's own values it is 63 to 3,360 (Table 5.7).

Of the six settings, the representation count is the only one that acts on whole representations. At each count (2, 3, 5 or all), only the representations with the highest internal evaluator scores on that fold contribute features, so the internal evaluator decides which representations are kept as well as how the budget is split among them (Section 5.2.2).

Importance scores are computed once per search fold and importance method, and each representation's internal evaluator score once per search fold and internal evaluator. Every configuration reuses them, so the cost grows more slowly than the number of configurations.

Before the full runs, we ran preliminary design experiments on 1 to 9 datasets to choose the values each setting may take. A value was kept if it gave the best score for some dataset, downstream model and budget. A setting was fixed at one value when the spread between its values was smaller than the spread of one value across folds. For example, on hERG K_i with linear allocation at 300 features, five ways of rescaling each representation's score gave RMSE values from 0.5870 to 0.5924 ($\mathrm{p}K_i$), while a single rescaling ranged from 0.5475 to 0.6294 across 5 folds, so the rescaling is one fixed rule per metric.

Preliminary design experiments set the values each setting may take (Additional file 13).

5.2.3 Augmentation

Some feature sets are too small or too specialized to take part in budget allocation, such as MACCS fingerprints (167 features) or log-likelihood ratios (3 features). Others, such as mutation features that record differences from a parent antibody, carry meaning only beside a full representation. Excluding them would lose potentially useful signal, but including them would distort the budget split, since they cannot contribute many features. We therefore add them as augmentation, calling the finished hybrid **H** the base and the feature sets that supply extra features the donors.

Donors are set per dataset (Augmentation column of Table 5.15). They range from small fingerprints such as MACCS to structural sets such as 3D pharma-

cophores and Coulomb matrix eigenvalues, and include features that describe only differences between variants, such as antibody mutation features. MACCS is a donor on four of the six small-molecule datasets and a base representation on MDR1–MDCK efflux. On QM9, MACCS is not offered and the topological torsion fingerprint (2,048 features) is a donor, the only wide one. Bahia et al. [180] found that combining 2D and 3D descriptors gave many more significant models across six protein–ligand series (461 structures), which they attribute to complementary properties. Their 3D descriptors came from bioactive conformations in the Protein Data Bank, whereas our 3D donors use one generated conformer per molecule.

Donor features are ranked by what the base gets wrong. Inside each outer training fold, an 80/20 internal split keeps each scaffold or sequence group on one side. A small random forest (50 trees, depth at most 10) is fitted on the base features, and 3-fold group-aware cross-validation gives out-of-fold residuals $r_i = y_i - \hat{y}_i^{\text{oof}}$ for the internal-training molecules, where for binary tasks $\hat{y}$ is the positive-class probability. In-sample residuals would shrink toward zero. All donor features are pooled, each is scored by its absolute Pearson correlation with the residuals, $c_j = |\rho_{\text{P}}(\mathbf{x}_j, \mathbf{r})|$, and the top $M = 50$ go forward. Ranking against $\mathbf{y}$ would favor features that repeat what the base captures; ranking against $\mathbf{r}$ favors features that explain its errors, as in gradient boosting [181] and least angle regression [182].

Blocks of the top $k \in \{0, 5, 10, 20, 40\}$ ranked features, capped at the number of candidates, are each added to the base and scored with the downstream model on the internal-validation molecules. The chosen k^* has the best score and must beat $k = 0$, which wins ties, so keeping no extras is the default. Five block choices instead of about 50 accept-or-reject decisions leave less room to overfit a small validation set [172]. The chosen donors and feature positions are stored and re-extracted for the outer fold.

In uncapped mode, extras are appended outside the budget, so the final width is $|\mathbf{H}| + k^*$, at most $B + 40$ features. In capped mode, they displace base features and the width stays at most B. The base features are ranked once by random forest importance (50 trees, depth at most 10) and the k^* weakest are dropped, after any unspent budget is filled, keeping at least one base feature. Uncapped augmentation and no augmentation differ in width at the same B, so their comparison mixes better features with more features. Capped mode, where an extra must beat a base feature, removes this difference.

Augmentation is a separate condition rather than a searched setting, so a dataset

with donors is searched three times, with no, uncapped and capped augmentation. In an earlier search of each whole dataset, 14 of the 16 datasets ran all three, while humanness and trastuzumab–HER2 binding have no donors. On none of the 16 did either augmented condition score better than no augmentation by more than 1% (3% on datasets under about 1,000 samples), so the searches in this chapter run without augmentation (Section 5.2.4). In a second use, a user keeps one strong representation at full width as the base and adds a few extras, with B its width, so only uncapped mode makes sense. This use matters for adoption, because a chemist can keep the representation they already trust and learn which few outside features help, and how many. Augmentation also asks directly whether another representation holds signal the base lacks (Chapter 4).

The residual ranking has four limitations. $|\rho_{\mathrm{P}}|$ captures only linear relations, one feature at a time. It ignores redundancy, so two near-identical donor features can both rank high. Donors are pooled with no per-donor limit, so a 2,048-feature donor gets more chances than a 49-feature one. Finally, the margin over $k = 0$ is zero, so a near-tie on one internal split can admit extras.

Table 5.5 summarizes the three conditions.

Condition	Final width (features)	What an extra feature must beat	Datasets that run it
No augmentation	at most B	nothing is offered	all 16
Uncapped	$\lvert\mathbf{H}\rvert + k^*$, at most $B + 40$	keeping no extras	none (cut after the earlier search)
Capped	at most B	keeping no extras, and the weakest base feature it replaces	none (cut after the earlier search)

Table 5.5: The three augmentation conditions. B is the budget and $|\mathbf{H}|$ the width of the base hybrid, both in features, and k^* is the chosen number of extra features, which are drawn from a small or specialized feature set offered to the hybrid.

5.2.4 Search grid per dataset

Searching every value of Table 5.6 in every fold of every dataset was too slow, so each dataset searches a smaller grid (Table 5.7). The grid was chosen from an earlier search of each whole dataset. On each dataset we kept the two downstream models with the best scores, and the budgets whose ten best configurations scored within 1% of the best budget (3% on datasets under about 1,000 samples), widened to at

least three consecutive budgets. Importance methods, allocation methods, internal evaluators and representation counts that scored clearly worse than the best were dropped.

Four datasets, QM9, trastuzumab–HER2 binding, H3K4me3 and TATA promoter, were too slow for the earlier search over every value. They were searched over a shorter fixed list, called the fast tier (LightGBM gain importance; linear, squared and rank-decay allocation; three or all representations), and their grid stays within it. The search with the nearest-neighbor objective has its own grid, given in Chapter 6.

Setting	Values
Downstream models	Random forest, XGBoost, LightGBM, SVM[a], DNN, Gaussian process[b]
Internal evaluators	Matched evaluator, random forest, LightGBM, DNN
Feature budgets (features)	10, 25, 50, 100, 250, 500, 1,000, 2,000
Allocation methods	Linear, squared, softmax, rank decay, global pool
Importance methods	LightGBM gain, DeepLIFT, TreeSHAP
Representation counts	2, 3, 5, all representations

[a]Not fitted above 10,000 molecules (5,000 when augmentation features are added outside the budget). [b]Not fitted above 1,000 molecules.

Table 5.6: Values the hybrid search can take for each setting. The matched evaluator belongs to the same model family as the downstream model.

Dataset	Downstream models	Budgets (features)	Importance	Allocation	Representation counts	Configurations
hERG K_i	SVM, DNN	250–2,000	G, T	L, Q, S, R	all	224
LogD	DNN, SVM	500–2,000	G	L, Q, R	all	63
Caco-2 efflux	DNN, SVM	500–2,000	G, T, D	L, Q, S, R	all	252
Rat liver microsome clearance	SVM, DNN	500–2,000	G, T, D	all five	all	315
MDR1–MDCK efflux	SVM, DNN	500–2,000	G, T, D	all five	all	315
QM9†	DNN, LightGBM	500–2,000	G	L, Q, R	3, all	108
Buchwald–Hartwig yield[a]	Random forest, LightGBM	500–2,000	T, D	all five	2	120
Buchwald–Hartwig plates	Random forest, LightGBM	100–500	G, D	all five	2, 3	360
Koenig expression	SVM, DNN	250–2,000	G, T, D	all five	all	420
Nanobody thermostability	DNN, SVM	50–2,000	G, T	all five	3, 5, all	1,260
Humanness	XGBoost, random forest	10–2,000	G, T, D	all five	2, 3, 5, all	3,360
Trastuzumab–HER2 binding†	XGBoost, LightGBM	100–500	G	L, Q, R	3, all	126
H3K4me3†	DNN, LightGBM	250–2,000	G	L, Q, R	3, all	144
TATA promoter†	LightGBM, SVM	500–2,000	G	L, Q, R	all	63
Lung adenocarcinoma vs squamous	LightGBM, XGBoost	10–2,000	G, T, D	all five	2, 3, 5, all	3,360
Breast cancer PAM50 subtype	DNN, SVM	25–2,000	G, T, D	all five	3, 5, all	2,205

†Fast tier. [a]Internal evaluators are the matched evaluator, random forest and LightGBM; every other dataset uses all four of Table 5.6.

Table 5.7: Search grid of each dataset. One row is one dataset. Budgets are consecutive values of Table 5.6. Importance: G, LightGBM gain; T, TreeSHAP; D, DeepLIFT. Allocation: L, linear; Q, squared; S, softmax; R, rank decay; all five adds the global pool. Configurations is the number scored in one search, over both downstream models.

Table 5.8 follows one dataset, hERG K_i, through the cuts. Its earlier search

scored 8,220 configurations without augmentation, over six downstream models including kNN. The best had an RMSE of 0.5472 pK_i (SVM, 1,000 features, LightGBM gain, softmax allocation, random forest evaluator, all representations), with a standard deviation over the five folds of 0.0443 pK_i. Keeping only the SVM and the DNN removed 71% of the recorded fitting time, mostly XGBoost (35%) and LightGBM (25%), whose best configurations scored 0.5842 and 0.5707 pK_i. The budget, importance, allocation and representation-count cuts then removed most of the rest. The best configuration removed by each of these four cuts was 0.0051 to 0.0154 pK_i behind the best kept, less than the spread over folds. The final grid keeps 224 configurations and 1.4% of the fitting time, and still contains the best configuration. The cuts were chosen on this same search, so keeping its best configuration is expected; the walk-through shows how much each cut removes and how close the best removed configuration came.

Step	Configurations kept	Fitting time kept (%)	Best kept	Best removed
Full search	8,220	100.0	0.5472	–
Downstream models: SVM and DNN only	3,360	28.7	0.5472	0.5707
Budgets: 250 to 2,000 features	1,680	20.0	0.5472	0.5626
Importance: DeepLIFT removed	1,120	16.3	0.5472	0.5523
Allocation: global pool removed	896	4.6	0.5472	0.5536
Representation count: all only	224	1.4	0.5472	0.5545

Table 5.8: The hERG K_i search after each cut, applied in turn. One row is one cut. Fitting time is the share of the full search's recorded fitting time that remains. Best kept and best removed are the lowest mean RMSE over five folds, in pK_i (lower is better), among the configurations kept and among those the cut removed.

Adaptive pruning

The search also has an adaptive pruning option, which we did not use in this research: because of time constraints, the per-dataset cuts above, which remove far more configurations, were used instead. Pruning scores every configuration on the first two of the five folds and stops scoring those that are clearly behind. Budgets are scored from smallest to largest, and once every budget up to 100 features is scored, pairs of allocation and importance method with no configuration in the better half of their group are not searched at larger budgets. Each remaining setting is then voted on within groups of configurations that share every other setting, and a value is removed if it is among the best in too few groups. The survivors are scored on the other three folds, and the best is picked as without pruning. Unlike the cuts above, pruning decides from the dataset's own fold scores during the search, so it

needs no earlier search. Its cut-offs were set on five datasets so that it never removed the best configuration, and it cut the estimated run time on LogD from 79 to 44 hours. Successive halving [183] and Hyperband [184] likewise drop configurations after partial evaluations.

5.3 Methods: experimental setup

Every experiment splits each dataset into outer folds as set out in Section 5.3.1, except one preliminary analysis of Koenig expression, which uses three outer folds of unequal size. The search sees only the training samples of each outer fold, and the selected features are rebuilt on them and applied unchanged to the held-out samples. Four of the experiments below (budget overfitting, the internal evaluator, the learning curves and robustness to label noise) run only the first three outer folds, except on hERG K_i and trastuzumab–HER2 binding, where their searches are fast enough to run all five.

The hybrid is compared with the best single base representation at its full number of features, which is what a user would otherwise run. This comparator is chosen per downstream model and per fold from the search's scores on the outer training samples, then refit on those samples and scored on the same held-out samples as the hybrid, so it can differ between folds. Choosing it on the held-out samples would give it an advantage that the hybrid does not get. The hybrid is held to its budget, plus any augmentation features, so the comparator is often many times wider, and both widths are recorded for every fold, since a difference between the two scores is partly a difference in width as well as in method. The main quantity per fold is the hybrid's score minus that of the best single representation, signed so that a positive value means the hybrid scored better (for RMSE the sign is flipped). Unless stated otherwise, an experiment takes from each fold's search, for each of the dataset's two downstream models, the two best single representations and the two best hybrids.

Chapter 6 reuses this protocol with noise added to the training labels of each outer fold, while the held-out labels stay clean.

5.3.1 Datasets and splitting

The hybrid experiments use 16 datasets from six domains (Table 5.9), whose curation is given in Additional file S.5 and whose representations are listed in Table 5.15.

Folds keep each group whole, with groups chosen to match what an unseen sample means in each field.

Pre-processing. Additional file S.5 gives the shared and per-dataset pre-processing steps, with the counts after each step. The LogD and Caco-2 labels are clipped at their 1st and 99th percentiles, so on Caco-2, for example, 22 molecules share the highest label and 23 the lowest.

Splitting rules. As in Chapter 3 (Section 3.2), no group is split between folds. Classification folds are also stratified with a fixed seed. Every dataset uses five outer folds. No split is random, because random splitting inflates performance estimates for molecular data by placing structurally similar compounds on both sides of the train/test boundary [200, 201]. On a given dataset, every representation, model and hybrid is scored on the same folds.

Groups by domain. The small-molecule and QM9 datasets are grouped by Bemis–Murcko scaffold (Section 3.2), here keeping stereochemistry. On QM9 the largest group is the empty scaffold of acyclic molecules, with 1,041 of the 9,978 molecules. The two datasets from Biogen [185] (rat liver microsome clearance and MDR1–MDCK efflux) have 1.2 and 1.1 molecules per scaffold, respectively, so holding out a scaffold there is close to holding out single molecules. Chapter 3 used a different scaffold key and different curation for its versions of LogD, Caco-2, hERG K_i and QM9, and Additional file S.5.2 lists the differences.

The reaction datasets are Buchwald–Hartwig C–N couplings from high-throughput plates. Following Ahneman et al. [188], they are grouped by additive, so each held-out fold holds four or five additives unseen in training. The two sets share 3,955 reactions (Additional file S.5).

In the Koenig deep mutational scan of the anti-vascular endothelial growth factor (VEGF) antibody G6, each variant carries one mutation. Variants are grouped by their mutated position, giving 225 groups, so all mutations at one position fall on the same side of a split and a model cannot look up a per-position effect at test time [202]. The nanobody and humanness sets are grouped by single-linkage clusters at 95% sequence identity, the identity threshold Kenlay et al. [146] used to cluster their pretraining sequences. The trastuzumab variants differ from the wild type only in the ten residues of CDR-H3 and are grouped by how many of them are mutated.

Dataset	Label (unit)	Task, metric	n	Folds grouped by (groups)	Best single representation (score)
Small molecules: ADME and toxicity					
hERG K_i [104]	pK_i	regression, RMSE ↓	1,412	scaffold (769)	**[RESULT NEEDED: name, score]**
LogD [103]	LogD (log units)	regression, RMSE ↓	5,028	scaffold (2,413)	**[RESULT NEEDED: name, score]**
Caco-2 efflux [103]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,158	scaffold (1,086)	**[RESULT NEEDED: name, score]**
Rat liver microsome clearance [185, 186]	$\log_{10}$ CL_{int} (mL/min/kg)	regression, RMSE ↓	3,053	scaffold (2,625)	**[RESULT NEEDED: name, score]**
MDR1–MDCK efflux [185, 186]	$\log_{10}$ efflux ratio	regression, RMSE ↓	2,641	scaffold (2,318)	**[RESULT NEEDED: name, score]**
Quantum chemistry					
QM9 [102, 187]	HOMO–LUMO gap (eV)	regression, RMSE ↓	9,978	scaffold (3,452)	**[RESULT NEEDED: name, score]**
Chemical reactions					
Buchwald–Hartwig yield [188]	yield (%)	regression, RMSE ↓	3,955	additive (22)	**[RESULT NEEDED: name, score]**
Buchwald–Hartwig plates [188]	yield (%)	regression, RMSE ↓	4,312	additive (24)	**[RESULT NEEDED: name, score]**
Antibodies					
Koenig expression [189, 190]	expression enrichment ratio (no unit)	regression, Spearman ↑	4,275	mutated position (225)	**[RESULT NEEDED: name, score]**
Nanobody thermostability [190, 191]	melting temperature (°C)	regression, Spearman ↑	672	95% identity cluster (309)	**[RESULT NEEDED: name, score]**
Humanness [192]	human or not (198 : 366)	binary, AUROC ↑	564	95% identity cluster (461)	**[RESULT NEEDED: name, score]**
Trastuzumab–HER2 binding [193]	binder or not (1,639 : 3,361)	binary, AUROC ↑	5,000	mutations from wild type (10)	**[RESULT NEEDED: name, score]**
Nucleotide sequences					
H3K4me3 [194]	mark present or not (9,122 : 9,122)	binary, MCC ↑	18,244	chromosome (23)	**[RESULT NEEDED: name, score]**
TATA promoter [194]	promoter or not (2,637 : 2,637)	binary, MCC ↑	5,274	chromosome (23)	**[RESULT NEEDED: name, score]**
Cancer genomics (TCGA)					
Lung adenocarcinoma vs squamous [195–197]	tumor type (515 : 502)	binary, AUROC ↑	1,017	tissue source site (68)	**[RESULT NEEDED: name, score]**
Breast cancer PAM50 subtype [197–199]	subtype (421 : 192 : 141 : 67)	4 classes, weighted F1 ↑	821	tissue source site (19)	**[RESULT NEEDED: name, score]**

Table 5.9: The 16 datasets of the hybrid experiments after curation, grouped by domain, with the best single base representation of each and its mean score over the five outer folds. n is the number of samples, and the fold groups are given with the number of groups in parentheses. For classification tasks, the class counts are in parentheses in the label column. CL_{int} is the intrinsic clearance, and arrows show whether lower or higher is better.

The two nucleotide tasks, from the revised Nucleotide Transformer benchmark [194], are grouped by chromosome, so each held-out fold holds two to six of the 23 chromosome groups (Additional file S.5).

The TCGA datasets use RNA sequencing (RNA-seq) expression and other measurements of primary tumors, one per patient, from UCSC Xena [197]. Samples are grouped by tissue source site, the institution that supplied the sample, so a model cannot score well by recognizing the site.

Consequences of the grouping. Some groupings give very unequal folds. The trastuzumab held-out folds range from 13 to 3,035 variants, with binder shares from 15.5% to 92.3% against 32.8% overall, and the breast cancer folds from 3 to 403 tumors. None of the 68 lung sites supplied both tumor types, so one lung fold holds 20 adenocarcinomas and 113 squamous tumors. Per-fold scores on these datasets spread more widely, so results are read per dataset and never pooled.

5.3.2 Representations by domain

Any representation that gives one fixed-length vector per sample can serve as a base representation (Section 5.2.1). The first version of the method used three molecular representations: PDV, ECFP4 and MHG-GNN. These three are now one small-molecule example among many. Every base representation is frozen: it is computed once from the raw input (a SMILES string, a sequence, a reaction or a patient profile) and is never trained on the labels.

For each dataset we pre-selected at most ten base representations that carry different information and each showed some positive result on its own for the task.

To choose them, every candidate representation was first scored on its own with a gradient-boosted tree model [58] under the dataset's grouped folds. Candidates were then compared by the Spearman correlation between their out-of-fold predictions, since two representations whose predictions correlate highly carry much the same signal. At most ten were then chosen by hand, looking at each candidate's own score next to these correlations and favoring representations that encode the input in different ways. Two nucleotide datasets have fewer. H3K4me3 had 12 candidates, and four of them predicted almost the same as a better-scoring candidate, with a Spearman correlation of 0.95 or above between their out-of-fold predictions; eight remain. TATA promoter had 10 candidates, one was dropped by the same rule,

and nine remain. Augmentation features do not count toward the ten. They are added after the hybrid is built (see the Augmentation subsection above), and the same representation can be a base representation on one dataset and augmentation on another. No search in this chapter uses augmentation features (Section 5.2.3). The most and least correlated pairs of base representations on each dataset, by the Spearman correlation of their out-of-fold predictions, are **[RESULT NEEDED: most alike pair]** and **[RESULT NEEDED: least alike pair]**.

If an input cannot be parsed or embedded, it gets an all-zero row, so that every representation of a dataset keeps the same rows. In computed descriptors, a missing or infinite value is set to 0, and values are stored unscaled. A pretrained model gives one vector per input, the mean of its last-layer token vectors with padding excluded, unless the tables say otherwise. Scaling and filling of missing values happen inside each fold, on the training rows only, as described in the construction of the hybrid.

Small molecules. The small-molecule representations (Table 5.10) are the fingerprint, descriptor, graph and language-model families described in Chapter 2. The only additions are four sets of 3D shape and pharmacophore features, used as augmentation only. All four are computed from a single conformer per molecule, embedded with a fixed seed and optimized with the Merck molecular force field. A single conformer may misdescribe flexible molecules.

Chemical reactions. Reaction datasets (Table 5.11) use three kinds of representation. The condensed graph of reaction (CGR) merges reactants and products into one graph in which the bonds that change are marked [203**?**]; here it is hashed into a circular fingerprint after atom mapping [204]. Reaction language models read the reaction SMILES directly [205, 206]. Any molecular representation can also be applied to the reactants, to the product, or to their difference. A one-hot encoding of which aryl halide, ligand, base and additive was used carries no chemistry. It is kept because on plate data such identity features can predict yield about as well as chemical descriptors [207].

Antibodies. Antibody representations (Table 5.12) start from the heavy- and light-chain amino-acid sequences. Besides one-hot and BLOSUM62 encodings [208], most are frozen language models, either trained on all proteins or on antibody repertoires only. Most models embed each chain separately and join the two vectors,

so their width doubles on datasets with both chains. Several representations read one model in different ways, for instance averaging over earlier layers or over the CDR residues only, with CDRs found by IMGT numbering [209] using ANARCI [210]. Datasets of point variants of one parent antibody also use features that describe each mutation.

Nucleotide sequences. A k-mer frequency vector counts how often each DNA word of length k occurs in a sequence, divided by the total number of words [211]; the canonical variant merges each word with its reverse complement [**?**]. DNA language models are networks pretrained on genomes to predict masked or next tokens [194, 212, 213]. Table 5.13 lists both kinds, along with the small sequence summaries used as augmentation.

Cancer patients. For the two TCGA datasets (Table 5.14), each representation is a different measurement of the same patient's tumor, not a different encoding of one input. The measurements are RNA-seq gene expression (read through foundation models or summarized per chromosome arm), DNA methylation on the HM450 array, gene-level copy number, microRNA abundance, protein levels from reverse-phase protein arrays, and embeddings of the diagnostic whole-slide image. Data were obtained from TCGA through UCSC Xena [197, 214]. The most-variable selections, the z-scoring and the principal components for methylation, copy number, microRNA and protein were fitted once on all patients in a cohort, held-out patients included. No labels are used in these steps. A patient with no measurement for one data type gets an all-zero row for that representation.

Tables 5.10 to 5.14 list the representations of each domain, and Table 5.15 gives those used on each dataset.

Table 5.10: Representations used for small molecules. In the Role column, B marks a base representation, A an augmentation and B/A a representation that is a base representation on some datasets and an augmentation on others.

Representation	What it encodes	Features	Source	Role
ECFP4	hashed circular substructures, radius 2	2048	[3]	B
Atom pairs	hashed atom-type pairs with bond distance	2048	[215]	B
Topological torsions	hashed paths of four bonded atoms	2048	[216]	B/A
Avalon	hashed paths, rings and atom environments	2048	[6]	B
MACCS keys	166 predefined substructure keys	167	[217]	B/A

Representation	What it encodes	Features	Source	Role
WL graph kernel	Weisfeiler–Lehman label counts, 5 rounds, labels present in at least 5% of molecules	140–317	[218]	B
PDV	RDKit 2D physicochemical descriptors	200	[30]	B
Mordred	2D molecular descriptors	1613	[219]	B
GROVER	frozen pretrained graph transformer	4800	[220]	B
MHG-GNN	frozen pretrained graph encoder	1024	[7]	B
GraphMVP	frozen 2D graph encoder pretrained with 3D views	300	[221]	B
MoLFormer	frozen SMILES language model; public checkpoint trained on 10% of its corpus	768	[38]	B
SMI-TED	frozen SMILES encoder–decoder	768	[222]	B
ChemBERT	frozen SMILES language model; community checkpoint trained on ChEMBL following the cited method	256	[223]	B
ChemBERTa-2	frozen SMILES language model	384	[26]	B
SELFormer	frozen SELFIES language model	768	[224]	B
Mol2vec	summed substructure word vectors	300	[23]	B
3D pharmacophore	counts of 7 pharmacophore types, and mean and spread of distances for the 21 type pairs, one conformer	49	[30]	A
USR	shape moments from 4 reference points, one conformer	12	[225]	A
USRCAT	USR per pharmacophore atom type, one conformer	60	[226]	A
Coulomb matrix	sorted eigenvalues of the Coulomb matrix, one conformer	50	[227]	A

Table 5.11: Representations used for chemical reactions. Role is read as in Table 5.10, and the suffix in parentheses states whether a molecular representation is computed on the reactants, on the product or as their difference.

Representation	What it encodes	Features	Source	Role
CGR fingerprint	circular fingerprint, radius 2, hashed from the condensed graph of reaction	2048	[203**?**]	B
rxnfp	frozen reaction language model	256	[205]	B
ReactionT5	frozen reaction language model	768	[206]	B
Component one-hot	which aryl halide, ligand, base and additive	45	[188]	B
GROVER (reactants)	GROVER averaged over reactants	4800	[220]	B
Mordred (reactants)	Mordred averaged over reactants	1613	[219]	B
Mol2vec (reactants)	Mol2vec averaged over reactants	300	[23]	B
ECFP4 (reactants)	ECFP4, bitwise OR over reactants	2048	[3]	B

Representation	What it encodes	Features	Source	Role
Avalon (difference)	product minus reactant Avalon bits	2048	[6]	B
MHG-GNN (difference)	product minus reactant embedding	1024	[7]	B
MACCS (difference)	product minus reactant MACCS keys	167	[217]	B
Mol2vec (difference)	product minus reactant Mol2vec	300	[23]	B
MACCS (reactants)	MACCS keys, bitwise OR over reactants	167	[217]	A
MACCS (product)	MACCS keys of the product	167	[217]	A
3D pharmacophore (reactants)	3D pharmacophore averaged over reactants	49	[30]	A
3D pharmacophore (product)	3D pharmacophore of the product	49	[30]	A

Table 5.12: Representations used for antibodies. Feature counts marked /chain double on the datasets with heavy and light chains, Koenig expression and trastuzumab–HER2 binding. A dash in the Source column marks a standard encoding or one built for this thesis, and Role is read as in Table 5.10.

Representation	What it encodes	Features	Source	Role
One-hot	one-hot residues, both chains padded to fixed length	5600	–	B
BLOSUM62	BLOSUM62 row per residue, both chains padded	5600	[208]	B
Amino-acid composition	fraction of each amino acid, per chain	40	–	A
ESM-2	protein LM, last layer, mean over residues	1280/chain	[228]	B
ESM-2 (joined chains)	ESM-2 on heavy and light chain joined into one string	1280	[228]	B
ESM-2 early layers	ESM-2, mean over layers 0–16	1280/chain	[228]	B
ESM-2 middle layers	ESM-2, mean over layers 20–33	1280/chain	[228]	B
ESM-2 CDR	ESM-2, mean over CDR residues only	1280/chain	[228]	B
ESM-2 per CDR	ESM-2, CDR1/2/3 pooled separately then joined	3840/chain	[228]	B
ESM-2 variable positions	ESM-2, mean over the 20 highest-entropy positions	1280/chain	[228]	B
ESM-2 shift from parent	ESM-2 variant minus parent embedding	1280/chain	[228]	B
ESM-2 log-likelihood ratio	masked mutant vs parent log-probability: sum, mean, min	3	[229]	A
ESM3	protein LM	1536/chain	[230]	B
ESM3 CDR	ESM3, mean over CDR residues	1536/chain	[230]	B
ESM C	protein LM	1152/chain	[231]	B
ProtT5	protein LM	1024/chain	[232]	B
ProtT5 CDR	ProtT5, mean over CDR residues	1024/chain	[232]	B

Representation	What it encodes	Features	Source	Role
SaProt	structure-aware protein LM	1280/chain	[233]	B
AbLang-2	paired antibody LM, both chains read together	480	[234]	B
IgT5	paired antibody LM, both chains read together	1024	[146]	B
AntiBERTy	antibody LM	512/chain	[235]	B
AntiBERTy CDR	AntiBERTy, mean over CDR residues	512/chain	[235]	B
Ab-RoBERTa	antibody LM	768/chain	[236]	B
Ab-RoBERTa CDR	Ab-RoBERTa, mean over CDR residues	768/chain	[236]	B
Mutation features	wild-type and mutant residue, substitution score, changes in hydrophobicity, weight and charge, position; averaged over mutations	48	–	B/A

Table 5.13: Representations used for nucleotide sequences. A dash in the Source column marks a simple summary built for this thesis, and Role is read as in Table 5.10.

Representation	What it encodes	Features	Source	Role
4-mer counts	normalized counts of every 4-letter word	256	[211]	B
5-mer counts	normalized counts of every 5-letter word	1024	[211]	B
6-mer counts	normalized counts of every 6-letter word	4096	[211]	B
Canonical TNF	4-mer counts merged with reverse complements	136	[?]	B
DNABERT-2	DNA LM	768	[212]	B
Nucleotide Transformer v2 250M	DNA LM	768	[194]	B
Nucleotide Transformer v2 500M	DNA LM	1024	[194]	B
HyenaDNA	long-context DNA LM	256	[213]	B
GROVER (DNA)	DNA LM (not the molecular GROVER)	768	[237]	B
Mistral-DNA	mixture-of-experts DNA LM	768	[238]	B
GC content	GC fraction, GC and AT skew, mean and spread of GC over 100-base windows	5	–	A
Dinucleotide frequency	frequency of each 2-letter word	16	[211]	A
Codon usage	relative usage of each of the 61 sense codons, read in frame from the first base	61	[239]	A
Sequence complexity	entropy of 1-, 2- and 3-letter words, linguistic complexity, fraction of alphabet used	5	–	A

Representation	What it encodes	Features	Source	Role
DNA shape	minor groove width, propeller twist, helix twist and roll from a dinucleotide lookup table; mean and spread	8	–	A

Table 5.14: Representations used for cancer patients from TCGA. A dash in the Source column marks a representation computed for this thesis from the patient's own data, and Role is read as in Table 5.10.

Representation	What it encodes	Features	Source	Role
Geneformer (2,048 genes)	single-cell foundation model (V2-316M) on the 2,048 top-ranked genes, [CLS] vector	1152	[240**?**]	B
Geneformer (4,096 genes)	the same model on the 4,096 top-ranked genes, [CLS] vector	1152	[240**?**]	B
BulkRNABert	bulk RNA-seq foundation model	256	[241]	B
Top-variance methylation probes	beta values of the 5,000 most variable CpG probes	5000	[197]	B
Methylation PCA	50 principal components of the standardized CpG probes above median variance	50	[197]	B
Copy number PCA	50 principal components of gene-level copy number	50	[242]	B
miRNA	200 most variable miRNAs, z-scored	200	[197]	B
Protein (RPPA)	reverse-phase protein array, z-scored	258	[243]	B
Chromosome-arm expression	mean expression of the genes on each chromosome arm	42	–	B/A
TITAN	whole-slide image foundation model	768	[244]	B
Prov-GigaPath	whole-slide image foundation model	512	[245]	B
CTransPath	histopathology image encoder	512	[246]	B
Phikon	histopathology image encoder	512	[247]	B
H-optimus-0	histopathology image encoder	512	[248]	B
HRD score	homologous recombination deficiency: LOH, TAI, LST and their sum	4	[249]	A
Tumor mutation burden	non-silent, missense and truncating mutation counts, log count, rate per megabase	5	[250]	A
Tumor purity	purity, ploidy, genome doublings, subclonal genome fraction	4	[251]	A
Expression statistics	summaries of the patient's expression distribution	15	–	A
Immune signatures	10 CIBERSORT cell fractions and Th1 and Th2 signature scores	12	[252]	A
CIBERSORT	immune cell fractions deconvolved from bulk expression	21	[252, 253]	A

Representation	What it encodes	Features	Source	Role
Stemness index	stemness indices from expression and from epigenetically regulated expression	2	[254]	A
Hypermethylation summary	fraction of all methylation probes with beta above 0.3, and mean beta; a genome-wide stand-in for CIMP status [255]	2	–	A

Table 5.15: Base and augmentation representations of each dataset, grouped by domain. The hybrid search selects features from the base representations. Augmentation features would be added after the hybrid is built, but no search in this chapter uses them (Section 5.2.3). A dagger marks datasets searched at the fast tier.

Dataset	Task	Base representations	Augmentation
Small molecules: ADME and toxicity			
Caco-2 efflux [103]	efflux ratio (log), regression	WL graph kernel, MHG-GNN, ECFP4, PDV, GROVER, Avalon, Topological torsions, SMI-TED, Atom pairs, MoLFormer	MACCS keys, 3D pharmacophore, USRCAT, Coulomb matrix, USR
LogD [103]	LogD, regression	MHG-GNN, Mordred, GROVER, ECFP4, SMI-TED, MoLFormer, Avalon, Topological torsions, ChemBERT, GraphMVP	MACCS keys, 3D pharmacophore, USRCAT
hERG K_i [104]	hERG inhibition, pK_i, regression	MHG-GNN, Mordred, WL graph kernel, Avalon, GROVER, PDV, ECFP4, Atom pairs, ChemBERT, SMI-TED	MACCS keys, 3D pharmacophore, Coulomb matrix, USR, USRCAT
MDR1–MDCK efflux [185]	efflux ratio (log), regression	Mordred, PDV, MHG-GNN, Avalon, GROVER, SMI-TED, Atom pairs, WL graph kernel, MACCS keys, ECFP4	3D pharmacophore, Coulomb matrix
Rat liver microsome clearance [185]	intrinsic clearance (log), regression	PDV, MHG-GNN, SMI-TED, Mordred, MoLFormer, GROVER, WL graph kernel, Atom pairs, Avalon, ECFP4	3D pharmacophore, MACCS keys, USRCAT, Coulomb matrix, USR
Quantum chemistry			
QM9† [102]	HOMO–LUMO gap, regression	Mordred, Avalon, SMI-TED, ECFP4, SELFormer, ChemBERTa-2, Mol2vec, ChemBERT, GraphMVP, MoLFormer	Topological torsions, 3D pharmacophore, USRCAT, Coulomb matrix
Chemical reactions			

Dataset	Task	Base representations	Augmentation
Buchwald–Hartwig yield [188]	yield (%), regression	CGR fingerprint, GROVER (reactants), Avalon (difference), Mordred (reactants), MHG-GNN (difference), ReactionT5, Component one-hot, Mol2vec (reactants), rxnfp, MACCS (difference)	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Buchwald–Hartwig plates [188]	yield (%), regression	ECFP4 (reactants), GROVER (reactants), MHG-GNN (difference), CGR fingerprint, Component one-hot, Avalon (difference), ReactionT5, Mol2vec (difference), Mordred (reactants), rxnfp	MACCS (product), MACCS (reactants), 3D pharmacophore (product), 3D pharmacophore (reactants)
Antibodies			
Koenig expression [189]	expression enrichment of variants, regression	ProtT5, ESM3 CDR, SaProt, IgT5, ESM-2 early layers, ESM C, AbLang-2, ESM-2 (joined chains), ESM-2, ESM-2 middle layers	Mutation features, ESM-2 log-likelihood ratio, Amino-acid composition
Nanobody thermostability [191]	melting temperature, regression	SaProt, ESM3, ESM C, ESM-2 variable positions, ESM-2 CDR, ESM-2 early layers, ESM-2 shift from parent, ProtT5 CDR, IgT5, One-hot	Mutation features
Humanness [192]	human vs non-human, binary	IgT5, AntiBERTy CDR, Ab-RoBERTa, AbLang-2, BLOSUM62, ESM C, ESM-2 per CDR, ESM-2 early layers, ProtT5, ESM-2 middle layers	none
Trastuzumab–HER2 binding† [193]	binder vs non-binder, binary	BLOSUM62, ESM-2 per CDR, ESM-2 middle layers, ProtT5, ESM3, ESM C, Ab-RoBERTa CDR, AntiBERTy, Mutation features, AbLang-2	none
Nucleotide sequences			
H3K4me3† [194]	histone mark present vs absent, binary	GROVER (DNA), Mistral-DNA, Canonical TNF, Nucleotide Transformer v2 250M, Nucleotide Transformer v2 500M, HyenaDNA, 6-mer counts, DNABERT-2	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content
TATA promoter† [194]	promoter vs non-promoter, binary	Mistral-DNA, GROVER (DNA), HyenaDNA, 6-mer counts, 5-mer counts, Canonical TNF, 4-mer counts, Nucleotide Transformer v2 500M, Nucleotide Transformer v2 250M	Codon usage, Sequence complexity, Dinucleotide frequency, DNA shape, GC content

Dataset	Task	Base representations	Augmentation
Cancer genomics (TCGA)			
Breast cancer PAM50 subtype [198]	breast cancer subtype, 4 classes	Geneformer (4,096 genes), BulkRNABert, Protein (RPPA), Copy number PCA, TITAN, H-optimus-0, Prov-GigaPath, CTransPath, Phikon, miRNA	HRD score, Tumor mutation burden, Tumor purity, Expression statistics, Chromosome-arm expression, Immune signatures
Lung adenocarcinoma vs squamous [195]	lung cancer type, binary	Geneformer (2,048 genes), Top-variance methylation probes, BulkRNABert, TITAN, Methylation PCA, Chromosome-arm expression, Copy number PCA, miRNA, CTransPath, Protein (RPPA)	Immune signatures, Expression statistics, Stemness index, CIBERSORT, Hypermethylation summary, Tumor mutation burden, HRD score, Tumor purity

5.3.3 Models and metrics

The downstream models, introduced in Chapter 2, range from methods that degrade in high dimensions (SVMs and Gaussian processes) to tree ensembles that tolerate irrelevant features.

The downstream models are random forest, XGBoost, LightGBM, an SVM, a DNN and a Gaussian process [107], and each dataset uses the two that scored best in the earlier search of the whole dataset (Table 5.7). Across the 16 datasets, all but the Gaussian process are used. Several models are used because the best set of features depends on the model, and a hybrid that helped only one model family would reflect that model rather than a better input. Tree ensembles are the baseline to beat, because a hybrid is a tabular feature matrix and tree-based models still lead on medium-sized tabular data of about 10,000 samples [256].

By default, every model uses the same settings on every dataset, so that score differences come from the input. An optional per-fold random search tries 20 draws of a model's settings on 3 group-aware inner folds of the outer training fold; the DNN relies on early stopping instead. **[RESULT NEEDED: whether tuning pays on base representations]**. The Gaussian process is skipped above 1,000 molecules, because it factorizes the full kernel matrix, and the SVM above 10,000 molecules (5,000 under augmentation beyond the budget), because a fit no longer finishes in useful time. Other models left out on named datasets are listed in Additional file 11,

and all model settings are in Additional file 10.

The kNN model, which is not a default downstream model, takes $k = 5$ neighbors by default, lowered to the square root of the number of training molecules when a fold has fewer than 25. It is a third downstream model in the search with the nearest-neighbor objective (Chapter 6), on every dataset except the two nucleotide ones.

Each dataset is scored in the metric its field normally uses, so that it can be compared with published work (Table 5.9). Regression on small molecules, QM9 and reaction yields uses RMSE, in the units of the label. Antibody regression uses Spearman ρ, because these assays rank candidates and their scales differ between laboratories. Binary classification uses AUROC, which does not depend on a decision threshold. The two nucleotide datasets are the exception and use MCC, which is high only when all four entries of the confusion matrix are good [257]. The four PAM50 subtypes, of 67 to 421 tumors each, are scored with support-weighted F1. Metrics are never averaged across datasets.

Each hybrid is paired by fold with the best single representation, and each comparison is reported as its per-fold differences on outer test folds and their spread across the 5 folds. We use no significance tests and no confidence intervals, because 5 fold scores are correlated through shared training molecules. No unbiased estimator of the variance of k-fold cross-validation exists, and estimates that ignore the overlap underestimate it [258].

A full search takes **[RESULT NEEDED: hours per dataset]** on **[RESULT NEEDED: hardware]**.

5.3.4 Comparators, controls and ablations

The hybrid is the best of many configurations, so its gain may be no more than the reward for a large search. Each experiment below asks one question about this or a related issue.

Every experiment below runs the search inside each outer fold, on that fold's training molecules, and scores what it chose on the held-out fold (nested cross-validation).

The first experiment uses nested cross-validation to ask whether the search overfits as the budget grows. At every budget B, the winning configuration is rebuilt on the outer training molecules and scored on the outer held-out fold. The selection

bias is the inner-search score minus the held-out score, positive when the search promised more than the held-out fold gave. The budgets are the standard list from 10 to 2,000 features, plus 4,000 and 8,000, plus 0.5, 0.75, 1, 1.25, 1.5 and 2 times the smallest outer training set in molecules, which place a budget where the number of features equals the number of molecules. For the dataset's two downstream models, the budget does not set the number of fitted parameters, so their curves show whether the search's choice of budget overfits. Per model, we report the budget picked in each outer fold and the budget with the best held-out score. A pick far above the latter means the search overfit. **[RESULT NEEDED: held-out score and selection bias against budget]**.

The ensemble fits one model per representation and pools their predictions (late fusion), whereas the hybrid fits one model on one fused feature vector (early fusion) [150]. Every choice for the ensemble is made on an inner 3-fold group split of the outer training molecules. The ensemble uses the same model on every representation or each representation's best model, and its members are the top 3 representations or all of the dataset's base representations. Predictions (probabilities, for classification) are pooled in one of seven ways, from the mean and median to trimmed, outlier-removed and score-weighted means. Three controls are added. They are the best single representation, picked by its mean over the held-out folds, all representations joined and reduced by PCA to each budget, and the best pair of model and representation read off the held-out fold. The first and last use the held-out labels, so they are optimistic rather than fair comparators, and an ensemble or hybrid can still score above the last. We also report the number of representations and models needed at prediction time, and the fit and prediction time. The hybrid saves cost only if it draws on fewer representations than the ensemble. **[RESULT NEEDED: held-out score and cost per method]**.

The next experiment asks whether the gain comes from drawing on several representations or only from having fewer features. On regression datasets, every method receives B features from all base representations, fitted on the fold's training molecules. The methods are the two best hybrids at budget B from the fold's search, PCA on the joined, z-scored representations, mRMR [163], recursive feature elimination [166] and B random features. mRMR stops at 200 features for run time, so it is not run at larger budgets. Each of the dataset's two downstream models scores every method, so only the features differ. **[RESULT NEEDED: held-out score against B per method]**.

This experiment asks whether the internal evaluator must match the downstream model, using nested cross-validation with one search per outer fold for each evaluator setting. The matched evaluator belongs to the downstream model's family. A near-family evaluator pairs tree models with each other, the SVM with a Gaussian process, and the DNN with a narrower DNN. A far-family evaluator pairs tree models with an SVM, the SVM with LightGBM, and the DNN with k-nearest neighbors. A fourth setting scores the DNN with a wider DNN, and a fifth searches the dataset's own internal evaluators (Table 5.7). A Gaussian process downstream model is paired with an SVM as near family and LightGBM as far family. Each setting runs on the dataset's two downstream models. We report each setting minus the matched evaluator, paired per outer fold. Each setting's search picks its own budget, so the number of features can differ between settings, and we report it beside each difference. The answer determines whether one cheap evaluator is enough or whether the search must be rerun for each model. **[RESULT NEEDED: each setting minus matched, per downstream model]**.

Selection on each of the F folds gives a binary vector $\mathbf{z}_f \in \{0,1\}^d$, where d is the number of features in all base representations joined, not the size of the union of the selected features. We measure the stability of the selection with the index of Nogueira et al. [259],

$$\hat{\Phi} = 1 - \frac{\frac{1}{d}\sum_{f=1}^{d} s_f^2}{\frac{\bar{k}}{d}\left(1 - \frac{\bar{k}}{d}\right)}, \qquad s_f^2 = \frac{F}{F-1}\hat{p}_f(1-\hat{p}_f).$$

Here $\hat{p}_f$ is the fraction of folds that selected feature f, and $\bar{k}$ is the mean number of features selected. $\hat{\Phi} = 1$ means the same features on every fold, and $\hat{\Phi} \approx 0$ is what random selection gives. Unstable feature sets would weaken any chemical reading of the blueprints. **[RESULT NEEDED: $\hat{\Phi}$ per dataset and model]**.

The learning curves draw subsamples of 10, 25, 50, 75 and 100% of the molecules as whole groups, never fewer than 50 molecules, and above 5,000 molecules use 250, 500, 1,000 and 2,000 molecules instead. Each size below 100% is drawn 3 times. The search runs on each subsample with the dataset's grid, and the gain is the best hybrid minus the best single representation, both picked by the search on that subsample's training molecules. **[RESULT NEEDED: gain per dataset, model and size]**.

This experiment asks whether importance also helps during training. It uses the

best hybrid from each fold's search with the dataset's two downstream models, with importance computed on the training molecules by the method the search picked. Molecule weighting needs a model that accepts per-molecule weights, and is not run on multiclass tasks. The five conditions are:

1. No weighting: the hybrid as selected.

2. Feature weighting: standardized features multiplied by the square root of their importance, with the weights normalized to average 1.

3. Molecule weighting: molecules weighted by how well their label agrees with their 10 nearest neighbors in the rescaled space.

4. Combined weighting: feature and molecule weighting together.

5. Above-median selection: only the features strictly above the median importance.

The weight of molecule i is $1 - |y_i - \bar{y}_{N(i)}|/(\max y - \min y)$, clipped to lie between 0.1 and 1, where $\bar{y}_{N(i)}$ is the mean label of its 10 nearest neighbors by Manhattan distance. Tree splits depend only on the order of values [260], so for tree models feature weighting differs from no weighting only in that zero-importance features drop out. The rescaling does change which molecules are nearest neighbors, and so changes molecule weighting. Molecule weighting down-weights rather than removes a suspect label, unlike neighbor-based filtering (Section 4.4.2). We found no QSAR study that does this, and learned per-sample weights for noisy labels come from general machine learning [96]. **[RESULT NEEDED: one result line]**.

This experiment asks whether hybrids are more robust to label noise than their base representations, on the regression datasets. Noise of each condition in Table 2.2 is added to the training labels of each outer fold, and held-out labels stay clean. A first screen of every dataset uses noise levels of 0 and 1.0 times the spread of the fold's clean training labels, and censoring of 0 and 25% of the labels. Datasets taken further use levels of 0, 0.5, 1.0 and 1.5 and censoring of 0, 10, 25 and 40% (0 and 1.0, and 0 and 25%, above 5,000 molecules). For each of the dataset's two downstream models, the two best hybrids and the two best single representations of the fold's search on clean labels have their features selected again on the noisy labels. Where four levels exist, robustness is measured by $\mathrm{AUC}_{\mathrm{norm}}$ (Equation 3.2), with the retention curve in each dataset's own metric, inverted for an error metric so that 1 still means no loss; otherwise we report the noisy score minus the clean score. A

separate search on the noisy labels asks whether noise changes which configuration wins.

5.3.5 Transfer, mutational scans and separate sites

The first study asks whether search settings (importance method, allocation method, budget B and internal evaluator) found on a large dataset can be reused on a small one instead of a new search. The source is LogD (lipophilicity), with 5,028 molecules, and the target is hERG pK_i, with 1,412 molecules. Both searches use the representations listed for both datasets at matching width. Before the width check, the current lists share seven: Avalon, ECFP4, Mordred, GROVER, MHG-GNN, SMI-TED and ChemBERT **[RESULT NEEDED: the shared set in the full run]**. If settings transfer, the expensive search becomes a one-off cost. The closest prior work, Meta-QSAR [261], chose the learning method and representation for a new QSAR dataset from results on over 2,700 others. It beat random forest on a fingerprint by up to 13% on average and, like this study, reused decisions rather than selected features. We have not found a paper that applies feature indices selected on one QSAR dataset unchanged to another, although our literature search was not complete.

Four comparisons run inside every held-out fold of the target. They are the best single representation, a hybrid searched on the target's training molecules as the upper reference, the source settings with features re-selected on the target, and the source's selected features applied unchanged. The last two ask separately whether the search decisions and the chosen features transfer. We use five scaffold-group folds, RMSE in pK_i units and the two downstream models decided for both datasets, the DNN and SVM. **[RESULT NEEDED: hybrid minus best single representation per comparison and model]**.

A deep mutational scan measures a property, here expression or binding, for many single- or few-point mutants of one antibody. We use four such scans from FLAb [190]: the expression and binding scans of the anti-VEGF antibody G6 [189] and of the anti-fluorescein antibody 4-4-20 [**?**]. The three scans other than Koenig expression are not among the 16 datasets of Table 5.9, and their sizes, groups and representations are **[RESULT NEEDED: sizes, groups and representations of the G6 binding and the two 4-4-20 scans]**. The candidates are the representations of Section 5.3.2 plus two representations of the change from the wild

type. The first sums, over the mutations, the changes in five amino-acid properties (hydrophobicity, charge, isoelectric point, molecular weight and aromaticity), each z-scored over the 20 amino acids; the second places the same five features at the mutated position. The binding scan of 4-4-20 is binarized at $pK_d > 5.0$ and scored by AUROC, and the others by Spearman correlation. We report the hybrid minus the best single representation per scan. This difference says nothing about the wild-type-difference representations, so we also report how many of each hybrid's features they supply. **[RESULT NEEDED: features taken from the two wild-type-difference representations, per scan and fold]**. **[RESULT NEEDED: hybrid minus best single representation per scan and model]**.

Federated learning trains one model across data held by several parties without moving the data [262], and it has been run across pharmaceutical companies [263], whose structures and assay results are commercial assets that cannot be pooled [16, 17]. Our collaborator, Lhasa Limited, uses the teacher-student process of Chapter 2 (Table 2.1) [137]. We use their code for the fingerprints, site models and site clustering, and re-implement from it the step that combines the site models' predictions.

The site models' predictions are combined in one of two ways. Under distance weighting, each site's vote on a molecule is weighted by how close the molecule is to that site's training molecules, so a site counts most on chemistry it has seen. The distance is measured to the 8 nearest molecules in a random 20% sample of that site's training molecules, on the features the site model uses. Distances are Manhattan by default, whereas the collaborator's code uses Euclidean. Under score weighting, each site has one weight, its own cross-validated MCC. Only predictions on shared molecules cross a site boundary. The published process then trains a student model on the most confident combined labels. In the hERG comparison below, no student is trained, and the combined predictions score the held-out molecules directly. There, every site model is fitted after the classes are balanced by random undersampling.

Our addition is that feature selection, as well as model fitting, takes place at the sites. One search on the training molecules of all sites together chooses the settings (allocation method, importance method and internal evaluator). Each site then selects features with these settings on its own molecules and reports only feature indices, importance values and one score. A coordinator splits the total budget B across sites in proportion to their scores, with at least one feature for each site whose selection succeeded. Only this proportional split is run, so we cannot say

what the site scores buy. A feature already taken by another site is replaced by the site's next-ranked feature, so all features in the hybrid are distinct. Indices and importances are smaller, and harder to invert, than model weights or gradients. In related work, each client runs mutual-information feature selection on its own data and a server aggregates the results [264].

hERG channel blockade is a cardiotoxicity liability screened early in drug discovery, with public data from many laboratories [265]. Three real sources act as sites: the public ChEMBL hERG K_i set used elsewhere in this thesis, and the collaborator's training set (7,772 molecules) and test set (4,383 molecules). The two collaborator sets were labeled by different curation rules, so the sites differ in curation as well as in chemistry, whereas the process is usually simulated by clustering one dataset. Molecules held by several sources keep only the public copy. A public molecule is a blocker if $\mathrm{p}K_i \geq 6$. We use five scaffold-group folds over the pooled molecules. The search is scored by AUROC, which is also the reported number, and MCC is recorded beside it.

The comparison crosses two choices: molecules pooled or kept at their sites, and all features or a selected set. Pooled molecules with all features are the upper reference, not possible in practice. Pooled molecules with selected features measure what the selection is worth on its own. Site-held molecules with all features are federated training as usually done, and site-held molecules with selected features are the setting in which, apart from the shared choice of search settings, only indices, importances, one score and predictions cross between sites. Selected minus all features is what selection buys, and pooled minus site-held is what pooling buys. Four context comparisons are added: the single best source picked on training molecules, both ways of weighting the site models, the same search on one fingerprint alone, and one site model per representation. The budget B takes the values 100, 250, 500, 1,000 and 2,000 features, and the downstream models are LightGBM and XGBoost. The all-features settings fit one 2,048-bit Morgan fingerprint of radius 2 with pharmacophoric atom types, whereas the selected settings draw on the base representations chosen for the collaborator's hERG data. Selected minus all features therefore differs in two ways at once, width and source. The closest control for source is the same search run on ECFP4 only.

Four further separate-site experiments are run. In the first, the sites carry different amounts of label noise. Selection at the sites is compared, at several noise levels, with the ensembles of site models and with one model on the pooled molecules, in

one version after each site removes the molecules it flags as noisy. In the second, every site holds the same molecules as a different representation, and fusing features is compared with pooling predictions. In the third, the sites hold different families of representations (substructure, physicochemical and pretrained). In the fourth, the sites are three antibody sources: a binding-energy-change set and a deep mutational scan of binding, which are not among the 16 datasets of Table 5.9, and nanobody thermostability **[RESULT NEEDED: sizes and representations of the first two sources]**. It skips the ensemble comparisons, because each site's features are reduced separately and the labels are in incompatible units. **[RESULT NEEDED: outcome of each of the four]**.

The separate-site work has four limitations. One source is proprietary, so the comparison cannot be reproduced from public data alone (Additional file S.6). It depends on the collaborator's implementation. The published process trains a student on a large separate public set; where a student is trained here, the training molecules take that role as a compromise, and held-out molecules are excluded from it. Finally, the sites differ in size and in how their labels were derived, so a site's score reflects its curation as well as its chemistry.

5.4 Results: comparison with single representations and ensembles

Preliminary results from local runs on hERG K_i with 3 folds. These may change when the full runs finish.

The results in this and the following results sections of this chapter are preliminary. Most come from preliminary runs on a workstation, called local runs below. The full runs of the complete search on the computing cluster are not yet finished. Most of these local runs are on hERG K_i with three scaffold-group folds instead of five. The rest come from an earlier set of local runs on nanobody thermostability, hERG K_i and LogD. The separate-sites result instead uses the three hERG sources described in the Methods, with 13,231 molecules in all. The full search chooses among five allocation methods, three importance methods, four internal evaluators, four representation counts and eight budgets for each dataset and downstream model (Table 5.6). The local runs on hERG K_i did not run this search. They built one default hybrid in each fold, with linear allocation, LightGBM gain importance, a

random forest internal evaluator, five representations and a budget of 250 features. They scored it with random forest, SVM and LightGBM only. The five representations (MHG-GNN, ECFP4, atom pairs, WL graph kernel and PDV) were the five of the dataset's ten with the best random forest scores over all molecules. Those scores included the held-out molecules, and the effect of this on the comparisons was not measured. Because the settings were not searched for the dataset and the downstream model, the hybrid scores reported here are not the best the method can reach. Some experiments also scored other budgets and allocation settings on the same folds. Where we report the best of these, it was picked on the held-out folds, as was the single representation it is compared with. Both sides of that comparison are therefore optimistic.

Table 5.16 compares the default hybrid with the best single representation on hERG K_i for three downstream models, in RMSE ($\mathrm{p}K_i$), where lower is better. For each held-out fold, the single representation is the one of the five with the lowest mean RMSE on the other two folds with the same downstream model. It is used at its full width, 2,048 features for ECFP4 and 1,024 for MHG-GNN. The hybrid is one fixed configuration and is not picked. The random forest has 100 trees and considers the square root of the number of features at each split. With SVM the hybrid has a mean RMSE of 0.5784 against 0.5979 for MHG-GNN, and it is lower on all three folds. With LightGBM it has 0.5870 against 0.6120, and it is also lower on all three folds. With random forest the two are level, at 0.6038 for the hybrid against 0.6032 for ECFP4. The hybrid is lower on folds 1 and 3 and higher on fold 2.

A separate local experiment on the same three folds built the hybrid with 48 allocation settings. It crosses four ways of weighting the representations (linear, squared, softmax and rank decay) with two ways of taking features. Under the first, each representation contributes its weighted share from the top of its own ranking. Under the second, each representation offers candidates in proportion to its weight to a pool of twice the budget, and the budget is filled from one LightGBM gain ranking of the pool. Squared weights with a joint ranking is the global pool of Table 5.3; the other three weightings with a joint ranking were run only in this experiment. Features are ranked by LightGBM gain on each fold's training molecules. Each representation's weight is computed from its score with the downstream model over three inner folds of the training molecules. Each setting draws on the two, three or five representations that score best with the downstream model on each fold's

		Best single representation		Default hybrid
Model	Fold	Representation	RMSE	RMSE
Random forest	1	ECFP4	0.6258	0.6040
	2	ECFP4	0.5852	0.6161
	3	ECFP4	0.5986	0.5913
	Mean	–	0.6032	0.6038
SVM	1	MHG-GNN	0.6052	0.5876
	2	MHG-GNN	0.6137	0.5865
	3	MHG-GNN	0.5748	0.5612
	Mean	–	0.5979	0.5784
LightGBM	1	MHG-GNN	0.6255	0.5945
	2	MHG-GNN	0.5917	0.5733
	3	ECFP4	0.6187	0.5930
	Mean	–	0.6120	0.5870

Table 5.16: RMSE in pK_i (lower is better) of the default hybrid and the best single representation on hERG K_i, over three scaffold-group folds. One row is one held-out fold, or the mean over the three folds, for one downstream model; both scores in a row use that model. Mean rows average the three fold rows above them.

training molecules (Table 5.19), at 500 and 1,000 features. The internal evaluator is the downstream model itself (the matched evaluator), whereas the default hybrid uses a random forest. Unlike the default hybrid, these hybrids fill the budget with features of zero importance, so each holds exactly the budget on every fold. None of the 48 settings is the default hybrid, which uses linear weights and its own ranking at 250 features; its scores are in Table 5.25.

Table 5.17 gives the 16 settings over all five representations, next to the five single representations scored in the same experiment. Table 5.18 gives the 32 settings over two or three representations. For all three models, the lowest RMSE comes from own-ranking shares over five representations at 500 features. With softmax weights it is 0.5976 with random forest and 0.5663 with SVM, and with squared weights it is 0.5773 with LightGBM. The single representation with the lowest mean RMSE over the three held-out folds is ECFP4 with random forest (0.6032) and MHG-GNN with SVM (0.5979) and LightGBM (0.6079). Picked per fold on the training molecules instead, as in the last row of Table 5.17, the single representation scores 0.6176, 0.5979 and 0.6240. It is then ECFP4, MHG-GNN and atom pairs on folds 1 to 3 with random forest, MHG-GNN on every fold with SVM, and MHG-GNN, MHG-GNN and atom pairs with LightGBM. The hybrids have 500 features and the single representations 200 to 2,048. This comparison therefore changes two

things at once, the number of features and the number of representations. The hybrid was picked on the held-out folds from 48 settings, and the lowest-mean single representation from five, so both are optimistic and the hybrid more so. Over two or three representations, the lowest mean RMSE with random forest is 0.6018 (two representations, 500 features, linear weights, own ranking). With SVM it is 0.5825 (three, 500, rank decay, own ranking), and with LightGBM 0.5903 (three, 500, softmax, joint ranking). All three are higher than the lowest five-representation setting (0.5976, 0.5663 and 0.5773).

			Mean RMSE (pK_i), lower is better		
Budget (features)	Weights	Features taken from	Random forest	SVM	LightGBM
Hybrids of all five representations					
500	Linear	Own ranking	0.5987	0.5664	0.5824
500	Squared	Own ranking	0.6017	0.5678	**0.5773**
500	Softmax	Own ranking	**0.5976**	**0.5663**	0.5829
500	Rank decay	Own ranking	0.5998	0.5696	0.5917
500	Linear	Joint ranking	0.6026	0.5762	0.5926
500	Squared	Joint ranking	0.6079	0.5726	0.5925
500	Softmax	Joint ranking	0.6058	0.5790	0.5904
500	Rank decay	Joint ranking	0.6088	0.5737	0.5974
1,000	Linear	Own ranking	0.6076	0.5744	0.5896
1,000	Squared	Own ranking	0.6026	0.5729	0.5861
1,000	Softmax	Own ranking	0.6019	0.5752	0.5803
1,000	Rank decay	Own ranking	0.5984	0.5709	0.5959
1,000	Linear	Joint ranking	0.6075	0.5851	0.5928
1,000	Squared	Joint ranking	0.6045	0.5843	0.5931
1,000	Softmax	Joint ranking	0.6126	0.5830	0.5980
1,000	Rank decay	Joint ranking	0.6119	0.5803	0.5968
Single representations at full width					
ECFP4 (2,048 features)			0.6032	0.6841	0.6087
MHG-GNN (1,024 features)			0.6103	0.5979	0.6079
WL graph kernel (276 features)			0.6120	0.6307	0.6220
Atom pairs (2,048 features)			0.6246	0.6436	0.6137
PDV (200 features)			0.6255	0.6428	0.6226
Best single representation, picked per fold			0.6176	0.5979	0.6240

Table 5.17: Mean RMSE in pK_i over three scaffold-group folds on hERG K_i, lower is better. One row is one allocation setting of the hybrid over all five representations, or one single representation; one column is one downstream model. The weights are those of Table 5.3, and the last row uses on each fold the single representation that scores best with that downstream model on the fold's training molecules. Bold marks each model's lowest RMSE.

				Mean RMSE (pK_i), lower		
Representation count	Budget (features)	Weights	Features taken from	Random forest	SVM	L
2	500	Linear	Own ranking	0.6018	0.5850	
2	500	Squared	Own ranking	0.6039	0.5844	
2	500	Softmax	Own ranking	0.6042	0.5853	
2	500	Rank decay	Own ranking	0.6098	0.5846	
2	500	Linear	Joint ranking	0.6125	0.5922	
2	500	Squared	Joint ranking	0.6133	0.5926	
2	500	Softmax	Joint ranking	0.6106	0.5874	
2	500	Rank decay	Joint ranking	0.6126	0.5927	
2	1,000	Linear	Own ranking	0.6130	0.5871	
2	1,000	Squared	Own ranking	0.6110	0.5875	
2	1,000	Softmax	Own ranking	0.6091	0.5891	
2	1,000	Rank decay	Own ranking	0.6098	0.5874	
2	1,000	Linear	Joint ranking	0.6079	0.5962	
2	1,000	Squared	Joint ranking	0.6126	0.5973	
2	1,000	Softmax	Joint ranking	0.6111	0.5963	
2	1,000	Rank decay	Joint ranking	0.6115	0.5978	
3	500	Linear	Own ranking	0.6029	0.5846	
3	500	Squared	Own ranking	0.6057	0.5834	
3	500	Softmax	Own ranking	0.6035	0.5843	
3	500	Rank decay	Own ranking	0.6041	0.5825	
3	500	Linear	Joint ranking	0.6131	0.5896	
3	500	Squared	Joint ranking	0.6170	0.5856	
3	500	Softmax	Joint ranking	0.6064	0.5868	
3	500	Rank decay	Joint ranking	0.6192	0.5859	
3	1,000	Linear	Own ranking	0.6097	0.5867	
3	1,000	Squared	Own ranking	0.6096	0.5868	
3	1,000	Softmax	Own ranking	0.6052	0.5870	
3	1,000	Rank decay	Own ranking	0.6067	0.5865	
3	1,000	Linear	Joint ranking	0.6146	0.5938	
3	1,000	Squared	Joint ranking	0.6152	0.5930	
3	1,000	Softmax	Joint ranking	0.6110	0.5943	
3	1,000	Rank decay	Joint ranking	0.6103	0.5921	

Table 5.18: Mean RMSE in pK_i over three scaffold-group folds on hERG K_i, lower is better, for the hybrids of the experiment in Table 5.17 that draw on two or three representations. One row is one allocation setting; one column is one downstream model. The representations used on each fold are listed in Table 5.19.

Model	Fold	First	Second	Third
Random forest	1	ECFP4	WL graph kernel	MHG-GNN
Random forest	2	MHG-GNN	ECFP4	WL graph kernel
Random forest	3	Atom pairs	WL graph kernel	PDV
SVM	1	MHG-GNN	WL graph kernel	Atom pairs
SVM	2	MHG-GNN	WL graph kernel	PDV
SVM	3	MHG-GNN	Atom pairs	PDV
LightGBM	1	MHG-GNN	WL graph kernel	PDV
LightGBM	2	MHG-GNN	WL graph kernel	ECFP4
LightGBM	3	Atom pairs	MHG-GNN	PDV

Table 5.19: Representations drawn on by the hybrids of Table 5.18. One row is one downstream model on one fold. On each fold, the representations are ranked by their score with that downstream model on the fold's training molecules. Two-representation hybrids use the first two; three-representation hybrids use all three.

5.5 Results: is the search needed?

This section asks whether the search is needed, or whether one fixed set of settings would do as well. On each of three datasets, we compare every base representation at full width with three kinds of hybrid. The first uses the search settings chosen most often over all 16 datasets. The second reuses the settings selected on each other dataset, with the features selected again on this dataset. The third uses the settings the search selected on this dataset. All are scored on the same held-out folds with the dataset's two downstream models, in the dataset's own metric (Tables 5.20 to 5.22).

[RESULT NEEDED: the three tables below from the full runs]

	[Downstream model 1]		[Downstream model 2]	
Representation or hybrid	Mean	Fold SD	Mean	Fold SD
[Base representation 1]	–	–	–	–
[Base representation 2]	–	–	–	–
Most common settings	–	–	–	–
Settings from [other dataset 1]	–	–	–	–
Settings from [other dataset 2]	–	–	–	–
Searched on this dataset	–	–	–	–

Table 5.20: .
][Metric, units, direction] on [dataset 1], over 5 scaffold-group folds. One row is one base representation at full width, or one hybrid; both scores in a column use the same downstream model. Fold SD is the standard deviation over the 5 held-out folds.

	[Downstream model 1]		[Downstream model 2]	
Representation or hybrid	Mean	Fold SD	Mean	Fold SD
[Base representation 1]	–	–	–	–
[Base representation 2]	–	–	–	–
Most common settings	–	–	–	–
Settings from [other dataset 1]	–	–	–	–
Settings from [other dataset 2]	–	–	–	–
Searched on this dataset	–	–	–	–

Table 5.21: .
][Metric, units, direction] on [dataset 2], over 5 scaffold-group folds. One row is one base representation at full width, or one hybrid; both scores in a column use the same downstream model. Fold SD is the standard deviation over the 5 held-out folds.

	[Downstream model 1]		[Downstream model 2]	
Representation or hybrid	Mean	Fold SD	Mean	Fold SD
[Base representation 1]	–	–	–	–
[Base representation 2]	–	–	–	–
Most common settings	–	–	–	–
Settings from [other dataset 1]	–	–	–	–
Settings from [other dataset 2]	–	–	–	–
Searched on this dataset	–	–	–	–

Table 5.22: .
][Metric, units, direction] on [dataset 3], over 5 scaffold-group folds. One row is one base representation at full width, or one hybrid; both scores in a column use the same downstream model. Fold SD is the standard deviation over the 5 held-out folds.

5.6 Results: sources of the gain

Preliminary results from local runs on hERG K_i with 3 folds and an earlier local run on nanobody thermostability. These may change when the full runs finish.

Table 5.23 compares the default hybrid with PCA, recursive feature elimination and random features on hERG K_i, in RMSE ($\mathrm{p}K_i$), where lower is better. Each method reduces the same five representations, 5,596 features when joined, to 250 features, and the same random forest scores all four. PCA is fitted to the five representations joined and z-scored. This random forest has 100 trees at scikit-learn's default settings and is fitted on z-scored features. Recursive feature elimination ranks features with a random forest of 50 trees of depth 5 and drops 559 features per round. The random features are one draw of 250 of the 5,596 features, the same on every fold. The default hybrid has a mean RMSE of 0.5992, against 0.6553 for

PCA and 0.6404 for random features, and it is lower than both on all three folds. It scores 0.5992 here and 0.6038 in Table 5.16 because the random forest differs. Here the forest considers every feature at each split, and there the square root of the number of features; we have not checked whether this is the only difference. Recursive feature elimination has 0.5922 and is lower than the default hybrid on all three folds. mRMR has no score, because its ranking is capped at 200 features. Table 5.24 makes the same comparison on nanobody thermostability, from an earlier local run, in Spearman correlation, where higher is better. In that run each method reduces the same representations on each fold's training antibodies, and the hybrid was not the default. Its configuration was chosen from 48 for the random forest at each number of features, using all antibodies. Its score is therefore optimistic, and PCA, recursive feature elimination and random features had no such choice. Which configuration won at each number of features was not recorded, and mRMR was not run. The hybrid has the highest Spearman correlation at 10, 50 and 250 features. At 250 features, recursive feature elimination has 0.6004 against 0.6029 for the hybrid.

Method	Fold 1	Fold 2	Fold 3	Mean
Default hybrid	0.5921	0.6100	0.5955	0.5992
Recursive feature elimination	0.5842	0.5972	0.5950	0.5922
PCA	0.6595	0.6464	0.6600	0.6553
Random features	0.6620	0.6462	0.6131	0.6404

Table 5.23: RMSE in pK_i (lower is better) on hERG K_i, on three scaffold-group folds. One row is one way of reducing the same five representations to 250 features or, for PCA, 250 components. Every row is scored with the same random forest.

Number of features	Searched hybrid	PCA	Recursive feature elimination	Random features
10	0.5945	0.4860	0.4682	0.2145
50	0.6131	0.5225	0.5689	0.5094
250	0.6029	0.4824	0.6004	0.5636

Table 5.24: Spearman correlation (higher is better) on nanobody thermostability, mean over the held-out folds. One row is one number of features, the same for all four methods; for PCA, the number of components. Every method is scored with a random forest.

Table 5.25 gives the RMSE on hERG K_i when the default hybrid is rebuilt at eight budgets from 10 to 2,000 features, with all other settings unchanged. The random forest is that of Table 5.16. RMSE falls steeply up to about 100 features for random forest and LightGBM, and up to 250 features for SVM. Between 250 and

2,000 features it changes by 0.008 or less for all three models. The lowest RMSE is at 100 features for random forest (0.5978), at 250 features for LightGBM (0.5870) and at 1,000 features for SVM (0.5733). For LightGBM, the values at 250 and 500 features are 0.58695 and 0.58699 before rounding. At 500 features the hybrid held 500, 500 and 499 features on the three folds; at 1,000 features it held 793, 771 and 790; and at 2,000 features 1,184, 1,136 and 1,165. It held fewer features than the budget because features of zero importance are not taken. As in the earlier run on nanobody thermostability (Section 5.2.2), SVM does best at a larger budget than the tree ensembles. Because the best budget is read off the held-out folds, these numbers describe the curves and do not reflect a choice made by the search. The single representations at full width are compared with the default hybrid in Tables 5.16 and 5.17.

	RMSE (pK_i), lower is better		
Budget (features)	Random forest	SVM	LightGBM
10	0.6704	0.7139	0.6846
25	0.6135	0.6470	0.6270
50	0.6061	0.6280	0.6083
100	**0.5978**	0.5957	0.5944
250	0.6038	0.5784	**0.5870**
500	0.6074	0.5737	0.5870
1,000	0.6062	**0.5733**	0.5920
2,000	0.6118	0.5762	0.5898

Table 5.25: RMSE of the default hybrid on hERG K_i at eight budgets. One row is one budget, and one column is one downstream model. Each value is the mean RMSE in pK_i over three scaffold-group folds; lower is better, and bold marks the lowest value in each column.

5.7 Results: blueprints on specific datasets

No results yet; awaiting the full runs.

5.8 Results: transfer across datasets

Preliminary results from an earlier local run on one pair of datasets with 5 folds. These may change when the full runs finish.

We searched LogD (5,028 molecules) for the best settings for each downstream model and reused them on hERG K_i (1,412 molecules). This run used five scaffold-

group folds and all five downstream models. Both datasets were offered the same six representations: atom pairs, Avalon, ECFP4, MACCS keys, Mordred and PDV. These are not the representation lists of Table 5.15, and the search grid was smaller than that of Table 5.6 (Table 5.27). Every stored setting, on LogD and in every hERG K_i fold, uses LightGBM gain importance, a random forest internal evaluator and a budget of 100 or 500 features. The full list of settings searched was not recorded. Table 5.26 gives the RMSE in pK_i on hERG K_i, where lower is better. The best single representation of each fold is the one with the lowest RMSE in the search run on that fold's training molecules, refit on those molecules and scored on the held-out fold (Table 5.31). The hybrid for hERG K_i itself was searched on each fold's training molecules (Table 5.29). Its budget is 500 features on every fold except random forest fold 3, XGBoost fold 3 and DNN fold 4, where it is 100 features, and its representation count was not recorded.

We first reused the LogD settings, including which representations the representation count keeps, and selected the features again on hERG K_i. The RMSE was then lower than that of the best single representation with SVM (0.5678 against 0.6616), the DNN (0.5943 against 0.6594) and LightGBM (0.6019 against 0.6318). With the LogD features applied unchanged, it was lower with SVM (0.5824) and LightGBM (0.6005). With the DNN, the copied features had a mean RMSE of 0.6815. They scored 1.0014 on fold 5, against 0.5372 for re-selected features on the same fold (Table 5.30). The copied features were built with each downstream model's LogD settings, except that all six representations were offered whatever the representation count. The DNN's copied features therefore come from five representations, whereas the DNN setting found on LogD keeps three (Table 5.28). For random forest, whose setting keeps five, we have not checked whether the copied features differ from those of the LogD setting. The copied DNN and LightGBM features are identical, because both settings use squared allocation at 500 features and all six representations were offered. Where a hybrid holds fewer features than its budget, the missing features had zero importance and were not taken. The hybrid searched on hERG K_i itself had a higher RMSE than the best single representation with random forest (0.6040 against 0.5994) and XGBoost (0.6411 against 0.6226). This is one pair of datasets, so it does not show how often search settings carry over.

	Mean RMSE (pK_i), lower is better			
			LogD settings reused	
Downstream model	Best single	Searched on hERG K_i	Features re-selected	Features copied
SVM	0.6616	0.5693	0.5678	0.5824
DNN	0.6594	0.6064	0.5943	0.6815
LightGBM	0.6318	0.5925	0.6019	0.6005
Random forest	0.5994	0.6040	0.6030	0.6197
XGBoost	0.6226	0.6411	0.6447	0.6277

Table 5.26: RMSE in pK_i on hERG K_i (lower is better), mean over five scaffold-group folds. One row is one downstream model; all four columns of a row use that model and the same five folds. Best single is the best single representation of each fold, and Searched on hERG K_i is a hybrid searched on each fold's training molecules. The last two columns reuse the settings found on LogD for that downstream model, with features selected again on each fold's training molecules (Features re-selected) or with the features selected on all LogD molecules applied unchanged (Features copied).

Downstream model	Budget (features)	Allocation method	Representation count
SVM	500	Linear	all
DNN	500	Squared	3
LightGBM	500	Squared	all
Random forest	100	Global pool	5
XGBoost	500	Softmax	all

Table 5.27: Settings found by the search on all 5,028 LogD molecules, one row per downstream model. Budget is the maximum number of features, and allocation methods are defined in Table 5.3. The representation count is the number of the six representations allowed to contribute features; "all" means all six.

Downstream model	Atom pairs	Avalon	ECFP4	MACCS keys	Mordred	PDV	Total
SVM	65	86	90	0	139	118	498
DNN	40	70	75	0	180	118	483
LightGBM	40	70	75	0	180	118	483
Random forest	5	6	7	0	63	19	100
XGBoost	81	83	84	73	91	88	500

Table 5.28: Number of features from each representation in the hybrid built on all 5,028 LogD molecules, one row per downstream model. These are the features that the Features copied column of Table 5.26 applies unchanged. A 0 means no features from that representation were selected.

Downstream model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
SVM	Linear	Linear	Linear	Squared	Rank decay
DNN	Linear	Linear	Rank decay	Rank decay	Rank decay
LightGBM	Rank decay	Global pool	Softmax	Squared	Linear
Random forest	Linear	Softmax	Squared	Softmax	Linear
XGBoost	Softmax	Rank decay	Squared	Rank decay	Softmax

Table 5.29: Allocation method of the hybrid searched on each fold's training molecules of hERG K_i (the Searched on hERG K_i column of Table 5.26), one row per downstream model and one column per scaffold-group fold.

	RMSE (pK_i), lower is better			
Fold	Best single	Searched on hERG K_i	Features re-selected	Features copied
1	0.6515	0.6037	0.5720	0.5871
2	0.7437	0.6440	0.6279	0.6154
3	0.6766	0.6318	0.6753	0.6561
4	0.6038	0.5898	0.5590	0.5477
5	0.6213	0.5626	0.5372	1.0014

Table 5.30: RMSE in pK_i on hERG K_i (lower is better) with the DNN as the downstream model, one row per scaffold-group fold. Columns are as in Table 5.26.

Downstream model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
SVM	ECFP4	ECFP4	ECFP4	Atom pairs	ECFP4
DNN	Atom pairs	Avalon	Mordred	Atom pairs	ECFP4
LightGBM	Atom pairs	Avalon	ECFP4	Mordred	Mordred
Random forest	ECFP4	ECFP4	ECFP4	ECFP4	ECFP4
XGBoost	Atom pairs	ECFP4	ECFP4	Avalon	ECFP4

Table 5.31: The single representation used in the Best single column of Table 5.26, one row per downstream model and one column per scaffold-group fold.

5.9 Results: data held at separate sites

Preliminary results from a local run on three hERG sources with 3 folds. These may change when the full runs finish.

Table 5.32 gives the MCC of the four settings on the three hERG sites, with three scaffold-group folds. After molecules held by several sources were kept only in the public set, the public ChEMBL hERG K_i set holds 1,412 molecules and the collaborator's training and test sets hold 7,606 and 4,213. The only downstream model is the collaborator's random forest, with 150 trees and a maximum depth of 15. With molecules kept at their sites, one random forest is fitted per site and the site models' predictions are combined by distance weighting (Section 5.3.5). With

molecules pooled, one random forest is fitted on all training molecules. The FCFP4 rows use the 2,048 bits of the collaborator's Morgan fingerprint of radius 2 with pharmacophoric atom types (functional-class fingerprint, FCFP4). The selected features come from the default hybrid of the local runs, built at each site with linear allocation, LightGBM gain importance and a random forest internal evaluator, the only settings this run allowed, from MHG-GNN (1,024 features), ECFP4 (2,048), atom pairs (2,048), WL graph kernel (148) and PDV (200). A budget of 250 features is split across the sites in proportion to each site's AUROC, and the selected features totaled 248 in every fold. With molecules kept at their sites, the site models on the selected features reached an MCC of 0.5264 against 0.4690 on the 2,048 FCFP4 bits. They were ahead on all three folds. With molecules pooled, the selected features gave 0.5598 against 0.4995, also ahead on all three folds. With molecules pooled, the two settings differ in two ways at once. The features are selected, and they come from the five hERG K_i representations rather than from FCFP4. With molecules kept at their sites, the two settings differ in three ways at once, not one. The features are selected, and they come from the five hERG K_i representations of the local runs rather than from FCFP4. The distances that weight the site models' predictions are also measured on the selected features in one setting and on the FCFP4 bits in the other. The sites also differ in curation. Of the 336 molecules held by both the public set and a collaborator set, 113 fall on opposite sides of the pK_i 6 threshold. Each held-out fold mixes molecules labeled by three sets of curation rules.

			MCC, higher is better			
Molecules	Features	Number of features	Fold 1	Fold 2	Fold 3	Mean
Kept at their sites	FCFP4	2,048	0.4845	0.4514	0.4710	0.4690
Kept at their sites	Selected	248	0.5184	0.5060	0.5549	0.5264
Pooled	FCFP4	2,048	0.5220	0.4832	0.4934	0.4995
Pooled	Selected	248	0.5490	0.5393	0.5910	0.5598

Table 5.32: MCC on the held-out molecules of three hERG sites (higher is better), over three scaffold-group folds; a molecule is labeled a blocker if its pK_i is at least 6. One row is one combination of where the molecules are held and which features the downstream model, a random forest, is fitted on. FCFP4 is the 2,048-bit Morgan fingerprint of radius 2 with pharmacophoric atom types, and Selected is the features chosen at the sites.

5.10 Discussion

Ambroise and McLachlan [169] showed the extreme case of overfitting through fea-

ture selection. On colon-cancer data with permuted class labels, an SVM with recursive feature elimination reached a cross-validated error close to zero when genes were selected on the full dataset first. With selection redone inside each fold, the error was between 0.40 and 0.45, close to chance. The bias from pre-selection can exceed that from hyperparameter tuning [170]. Cawley and Talbot [172] showed that over-fitting the model-selection criterion biases performance estimates, and called this bias analogous to the feature-selection bias of Ambroise and McLachlan [169]. The hybrid search computes all importance scores within each fold, on training data only, but searching over many feature subsets still adds complexity even when each individual selection is unbiased [266]. The budget caps the number of features the downstream model sees, which limits how far it can fit noise in the rest. The selected features are also fixed inputs, fingerprint bits, descriptors or frozen embeddings, rather than fitted parameters, and we consider choosing among them less expressive than learning new features. Finally, overfit selection would give feature sets that differ widely between folds, which the Nogueira index measures (Section 5.3.4).

5.11 Limitations

The adaptive hybrid representation has several limitations. Below about 200 samples, the search is not expected to beat the best single representation reliably **[RESULT NEEDED: sample size below which the hybrid stops beating the best single representation]**. We suspect this is because the internal validation set in each fold shrinks to about 20 samples, too few for the evaluator to tell real improvements from noise, and tuning cannot fix this. Frozen embeddings also have a ceiling, since no signal can be added beyond what the inputs contain. Integrating 20 to 50 foundation models, each with its own dependencies and GPU requirements, is difficult engineering, and the burden scales with the number of models. A downstream model supplied by a user from outside the fixed list of model families gets no matched evaluator, and no warning. Finally, on endpoints with high experimental noise such as public K_i data (Chapter 1), even a well-constructed hybrid may not improve on simpler baselines, because the noise sets an accuracy ceiling. The budget may limit overfitting, but it cannot push past this noise floor.

5.12 Conclusions

No conclusions yet; awaiting the full runs.

Chapter 6

Hybrid Representations for Similarity and Label Noise

6.1 Introduction

Chapter 4 showed where the similarity assumption fails on standard representations. Any method that trusts a molecule's neighbors inherits this error, including noise detection that compares each label with its neighbors' labels. Chapter 5 introduced the adaptive hybrid representation. The original idea behind it was to select the features relevant to the task first and then to compute distances on those features alone, and this chapter returns to that idea.

Methods that handle noisy labels by comparing each label with its neighbors or with held-out predictions, such as nearest-neighbor editing [98], filtering by classifiers trained on other folds [141] and confident learning [48], each depend on the feature space in which they work. We ask three research questions, each answered in one results section:

1. Is predicting each molecule's activity from its five nearest neighbors more accurate in a hybrid than in the single representations from which the hybrid is built?

2. Are noisy labels easier to find, or the size of their errors easier to estimate, in a hybrid?

3. Does a hybrid change how well noise mitigation (label correction and cleaning) and noise-robust training perform (Section 2.5.2)?

This chapter is planned as a second publication on hybrids; its methods are final and its results preliminary. On hERG K_i, predicting each molecule's activity from its five nearest neighbors was more accurate in a similarity hybrid than in the best single representation. Noise detection has no result yet. With 10% of the hERG K_i training labels clipped at a detection limit, raising every clipped label by one shared amount gave each of four models on ECFP4 an RMSE between its clipped and never-clipped values, and closer to the never-clipped one. Under Gaussian noise on nanobody thermostability, every one of twelve methods scored higher on the hybrid than any of them did on one-hot.

6.2 Methods

Datasets, grouped folds and metrics are those of Chapter 5. Each downstream model is compared with its own top two single representations, ranked by its own score, not with a fixed list or one chosen fingerprint.

In every experiment, the hybrid's features are rebuilt in each fold from that fold's training labels, whether noisy, clipped or uncleaned. The search settings (representations, budget and allocation method) come from a different search in each experiment. In the survey of twelve methods for LNL, the search is repeated in each fold and at each noise level on the noisy labels, scored by the neural network the methods use, over budgets of 50, 100 and 250 features. In the censoring experiment, the settings come from the search on the clean, unclipped labels, and only the feature ranking and budget split in each fold see the clipped labels. When real labels are cleaned, the settings come from a search on those same labels. Settings searched on clean labels give the hybrid information a real user would not have, an advantage that the survey avoids and the clipped-label experiment does not.

We keep apart three tasks often treated as one: finding noisy labels, estimating the size of the noise and improving the model's held-out score. A method can succeed at the first and fail at the third. **[FIGURE NEEDED: flow from noisy data, to per-fold hybrid and single representations, to scores]**

6.2.1 Similarity in a Hybrid

We use hybrids for similarity in two ways. In a hybrid, the search of Chapter 5 selects features that help a downstream model, and distances are computed on those

features. A similarity hybrid comes from the same search, but each candidate is ranked by how accurately each molecule's activity is predicted from its five nearest neighbors in the hybrid, and no downstream model is fitted. The two searches share importance methods, allocation methods, budgets and folds, and differ only in the score that ranks candidates. The neighbor score, computed on a validation set from the fold's training molecules, also replaces the internal evaluator in splitting the budget. All of the dataset's representations are candidates, and the number of representations is not searched. Chapter 4 showed that the representation determines which molecules count as neighbors. The similarity hybrid tests whether choosing features for the quality of the neighbors addresses this problem more directly than choosing them for prediction. **[FIGURE NEEDED: D6.1, one search, two scores]**

The distance is the Manhattan distance of Chapter 4 on features z-scored as in Chapter 5. Each z-scored feature adds a term of similar size to this distance, so a representation's weight in the distance, set by the allocation method, is roughly its number of features divided by the budget. The neighbor score applies no importance weighting.

To rank whole candidates, the score is computed on the held-out molecules of each fold of the search, inside the outer training fold, and these molecules also serve as each other's neighbors. Their fold groups (scaffolds for molecules) are split into up to 5 parts, and each molecule's 5 nearest neighbors are drawn from the other parts, because a neighbor with the same scaffold is an easy match that would inflate the score. Each neighbor is weighted by the inverse of its distance:

$$\hat{y}_i = \frac{\sum_{j \in N_5(i)} y_j / d_{ij}}{\sum_{j \in N_5(i)} 1 / d_{ij}}. \tag{6.1}$$

A neighbor at distance 0 receives all of the weight. On regression data, the score is the Spearman correlation $\rho(\hat{\mathbf{y}}, \mathbf{y})$. On binary data, $\hat{y}_i$ is the distance-weighted fraction of neighbors in class 1, and the score is $2(\text{AUROC} - 0.5)$, where AUROC is the area under the receiver operating characteristic curve. Both scores put chance at 0, which the budget split needs because it divides by the score. The modelability index of Golbraikh et al. [117] (Chapter 4) is a related published measure, but it uses a single nearest neighbor and a hard class match, whereas our score retains the graded fraction of neighbors. The score thus measures how far the similar property principle (Chapter 2) holds in the hybrid.

A fold with fewer than 10 held-out molecules, fewer than 2 scaffold groups, or constant labels or predictions is left out of the mean rather than scored 0. The similarity hybrid is not searched on two of the 16 datasets. AUROC needs two classes, which excludes breast cancer PAM50 subtype (4 classes). The folds of trastuzumab–HER2 binding are grouped by the number of mutations from the wild-type CDR-H3, so a held-out block can hold a single group, with no neighbor from another group. All other datasets are searched, including the antibody and other non-molecule datasets.

To see how close in activity the neighbors are, we also report, for each held-out molecule, the mean absolute difference between its label and those of its five nearest training molecules, averaged over the held-out molecules.

Feature importance weighting. Beyond simply using the feature importance scores for allocation, we investigate whether feature weighting by importance during distance computation improves nearest-neighbor prediction and noise detection. For a feature importance vector w, rescaled to average 1, weighted distances are computed by scaling the standardized features: $x' = x \odot w$ under Manhattan distance and $x' = x \odot \sqrt{w}$ under Euclidean distance.

6.2.2 Noise Conditions

The noise conditions are defined in Section 2.5.1 (Figure 2.3, Table 2.2). Censoring, Gaussian noise and real labels drive the three experiments on LNL. Outlier and grouped-wider noise grade detection, and grouped-shifted noise grades estimation only, because it moves every training label. Noise is added only to each outer fold's training labels. Its size, the censoring limit and the scaffold families that grouped-wider noise widens come from that fold's clean training labels.

The censoring levels, the fraction of training labels clipped, are 0, 0.10, 0.20, 0.25, 0.30 and 0.40. The survey and the detection results use 0, 0.10, 0.25 and 0.40, with 0 as the negative control. The clip involves no random draw, so each level is run once. The error is one-sided, a bias rather than a scatter, and its direction and location (the limit) are known. Censoring applies to regression only, because a class label has no value to clip.

Under Gaussian noise, the levels are 0, 0.2, 0.3, 0.5, 0.75, 1.0 and 1.5 times the standard deviation of the fold's clean training labels, as in Chapter 3, with three

noise draws per level. On classification data, the level is instead the fraction of training labels flipped to the wrong class: 0, 0.05, 0.10, 0.20, 0.30 or 0.40. Gaussian noise, which represents the random error of a repeated measurement and is also the standard assumption in studies of noise robustness, is used, together with censoring at the four levels above, in the survey of twelve methods.

Outlier noise represents annotation and transcription errors [11, 93]. We apply it to 1%, 5% and 10% of training molecules, where Chapter 3 used 10% only. The grouped conditions need a scaffold per molecule, so they run only on datasets with SMILES strings.

With real labels, nothing is injected and no list of wrong labels exists, so success can only be measured as a better held-out score after cleaning. That shows cleaning helped the model, not that it found errors; it may have removed difficult molecules instead. QM9, the computed dataset of Chapter 3, is a control: its labels come from a quantum-chemical calculation with no experimental error [102], so a cleaning method that helps on QM9 is not removing measurement error. **[RESULT NEEDED: datasets per noise condition]**

6.2.3 Noise Mitigation Methods

Sample removal has limits when Gaussian noise affects every label, so we also use sample weighting and label smoothing, which act on a continuous estimate of noise size instead of a noisy-or-clean decision (Section 4.4.2).

Correcting clipped labels. Labels clipped at a detection limit are corrected by raising every training label at the limit by one shared amount. The limit L is read from the clipped training labels as their largest value, and the molecules at the limit are those whose label equals it. A model fitted on the clipped training labels gives an out-of-fold prediction p_i for each training molecule, and σ is the standard deviation of $y_i - p_i$ over the molecules below the limit. Each molecule at the limit contributes

$$\ell_i = \sigma \frac{\phi(a_i)}{1 - \Phi(a_i)}, \qquad a_i = \frac{L - p_i}{\sigma}, \tag{6.2}$$

where ϕ and Φ are the standard normal density and distribution function. The shared lift is the mean of ℓ_i over the molecules at the limit. It is not the distance of the truncated normal mean above the limit, which is $p_i - L + \ell_i$. Each result states which model gave the predictions p_i.

Sample Removal Methods

Distance-based approaches rely on the assumption that similar molecules have similar properties [29]. Presumably, noisy data would break this pattern, making it a method for detecting anomalous data. k-nearest-neighbor filtering examines the variance of a sample's neighborhood. For sample x_i with k nearest neighbors $\mathcal{N}_k(x_i)$, we compute the standard deviation of neighbor labels:

$$\sigma_i = \text{std}(\{y_j : x_j \in \mathcal{N}_k(x_i)\}) \tag{6.3}$$

Samples with σ_i exceeding the 90th percentile are marked as potentially noisy. We test $k \in \{5, 10, 20\}$.

Local Outlier Factor (LOF) compares the local density around a sample with the local densities around its k nearest neighbors [267]; a sample in a sparser region than its neighbors receives a high LOF. We flag the 10% of training molecules with the highest LOF, using 20 neighbors.

We also tested the following noise removal methods: distance-weighted prediction filtering, neighborhood consensus filtering, activity-cliff detection, Mahalanobis outlier detection, cross-validation disagreement, bootstrap ensemble filtering, a filter adapted from co-teaching [95], in which two random forests each train on the other's lowest-residual 70%, and a filter adapted from DivideMix [138], which fits a two-component Gaussian mixture to out-of-bag residuals.

Noise Magnitude Estimation

The central part of the noise mitigation work is estimating the size of the noise, which lets each label be handled in proportion to its error instead of being removed outright. Table 6.1 lists three families of estimators. Neighbor-based estimators compare a molecule's label with those of its k nearest training neighbors, model-based estimators use the disagreement between predictions made without the molecule, and probabilistic estimators use a model's predicted uncertainty.

The heteroscedastic network predicts a mean $\mu(x)$ and a variance $\sigma^2(x)$ and is trained with

$$\mathcal{L} = \frac{1}{2}\left(\frac{(y-\mu)^2}{\sigma^2} + \log \sigma^2\right) \cdot \sigma^{2\beta}, \tag{6.4}$$

where $\beta = 0.5$ prevents the variance from hiding the mean errors.

NGBoost provides the probabilistic estimates μ_i and σ_i used by the confidence

Estimator	Family	Score of molecule i
Neighbor residual	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \sigma_y$
Local variance ratio	Neighbors	$\lvert y_i - \bar{y}_{\mathcal{N}} \rvert / \max(s_{\mathcal{N}}, c)$, with $c = \max(0.01\,\sigma_y, 10^{-3})$ so that a neighborhood sharing one label gives a finite score
Activity-cliff-aware	Neighbors	$\frac{\lvert y_i - \bar{y}_{\mathcal{N}} \rvert}{\sigma_y} \cdot \frac{1}{1 + s_{\mathcal{N}}/\sigma_y}$: a large deviation among neighbors that agree suggests noise; among neighbors that disagree, an activity cliff [53]
Cross-validation residual	Model	Absolute residual of the out-of-fold prediction
Ensemble disagreement	Model	Standard deviation of several models' out-of-fold predictions, plus the absolute residual of their mean
Gaussian process	Probabilistic	Predictive standard deviation plus the absolute out-of-fold residual, with an RBF kernel and a learned noise level [107]
Heteroscedastic network	Probabilistic	Predicted variance $\sigma^2(x)$, trained with the β-NLL loss of Equation 6.4 [268]
Conformal prediction	Probabilistic	Width of the normalized split-conformal interval, $2q\,\hat{\sigma}(x)$, with $\hat{\sigma}$ the spread of the forest's trees [269**?**]
Deep ensemble	Probabilistic	M networks: variance of their means (epistemic) plus mean of their variances (aleatoric) [67]
Random forest spread plus residual	Probabilistic	Standard deviation of the trees' predictions, plus half the absolute out-of-fold residual
Quantile regression	Probabilistic	Width between the fitted quantiles
NGBoost	Probabilistic	Predicted standard deviation σ [59]

Table 6.1: Noise magnitude estimators by family. The last column gives the noise score of molecule i, where $\bar{y}_{\mathcal{N}}$ and $s_{\mathcal{N}}$ are the mean and standard deviation of the labels of its k nearest training neighbors, σ_y is the standard deviation of all training labels and M is the number of networks in the deep ensemble. A higher score marks a label as more likely to be noisy.

rule (Table 6.2). It was chosen because, of the models in Chapter 3, its predicted uncertainty had the highest rank correlation with its prediction error under Gaussian noise, 0.252 on the QM9 HOMO–LUMO gap with PDV. Chapter 3 also found that, outside censoring, no model's per-molecule uncertainty tracked which training labels had been corrupted (Section 3.8.2). Under censoring, the models were instead most confident on the labels that had been clipped.

Label Smoothing and Sample Weighting

Label smoothing moves a suspect label toward a target, and sample weighting gives less weight to suspect molecules during training. Both use the noise score min-max normalized to $[0, 1]$, written $\tilde{s}_i$ (Table 6.2).

Rule	Changes	New label or weight of molecule i
Proportional	Label	$y_i + \alpha\,\tilde{s}_i\,(\text{target} - y_i)$
Threshold	Label	Only above the $100(1-\alpha)$th percentile of $\tilde{s}$: $(1-\alpha)\,y_i + \alpha \cdot \text{target}$, so α sets both how many labels move and how far
Soft threshold	Label	Correction strength $\alpha \cdot \text{sigmoid}\big(5(\tilde{s}_i - 0.5)\big)$
Confidence	Label	$(1-c_i)\,y_i + c_i\,\mu_i$, with $c_i = \alpha\sigma_i/(1+\alpha\sigma_i)$; for estimators that output σ, such as NGBoost
Inverse	Weight	$1/(1+\alpha\,\tilde{s}_i)$
Exponential	Weight	$\exp(-\alpha\,\tilde{s}_i)$
Soft	Weight	$1 - \alpha\,\tilde{s}_i^2$, clipped to $[0.1, 1]$

Table 6.2: Label smoothing and sample weighting rules. $\tilde{s}_i$ is the noise score of molecule i scaled to $[0, 1]$, α sets the strength, the target is the mean label of the molecule's nearest neighbors, and μ_i and σ_i are the mean and standard deviation predicted by a probabilistic model. Weights are rescaled to a mean of 1.

Four further methods are tested on the recorded labels with nothing injected. C-Mixup [**?**], a form of mixup [49], adds one training molecule for each existing one by mixing its z-scored features and label with those of a partner that has a similar label. The partner j of molecule i is drawn with probability proportional to $\exp(-(y_i - y_j)^2/2h^2)$, with h equal to 0.2 standard deviations of the training labels, and the mixing weight is drawn from Beta(2, 2). Target-noise bagging averages the predictions of 5 models, each fitted on the labels plus fresh Gaussian noise. The noise standard deviation is 0.5 times that of the residuals of out-of-fold LightGBM predictions, made on 5 scaffold-group folds of the training molecules in the same representation. Pair down-weighting takes the closest 1% of training pairs and gives weight 0.25 to both molecules of any pair whose labels differ by more than $3\sqrt{2}\,\hat{\sigma}$, where $\hat{\sigma}$ is the noise standard deviation estimated from the label differences of close pairs. Down-weighting suspect molecules lowers the weight of training molecules with a high noise score (Table 6.1), over 11 settings of the score and the strength.

Detection Scores and Recovery Rate

Noise detection is scored against the injected noise as ground truth, by precision, recall, F1 score and average precision.

Each downstream model m is trained on the clean, noisy and corrected training labels, giving held-out scores s^m_{clean}, s^m_{noisy} and s^m_{denoised} in the dataset's own metric, averaged over folds. The recovery rate is the share of the lost performance that correction recovers,

$$\text{Recovery}_m = 100 \times \frac{s^m_{\text{denoised}} - s^m_{\text{noisy}}}{s^m_{\text{clean}} - s^m_{\text{noisy}}} \ \%, \tag{6.5}$$

which reads the same whether higher or lower values of the metric are better. When a model's gap between clean and noisy score is no more than 2% of the clean score, the noise did too little damage to be recovered, and the recovery rate is left undefined. A recovery rate of 100% fully restores clean performance, and a negative value means the corrected labels scored worse than the noisy ones.

6.3 Results: Similarity

Preliminary results from local runs on a few datasets with 3 folds. These may change when the full runs finish.

The preliminary results in this chapter come from the local runs described in Section 5.4, with 3 scaffold-group folds. These runs used fixed hybrid settings instead of a search, so the hybrid scores below are not the best the method can reach. The settings of each run are given with its results below.

Table 6.3 compares nearest-neighbor prediction on hERG K_i in the two hybrids and in the five single representations, on the same 3 scaffold-group folds of 1,412 molecules. Each held-out molecule's pK_i was predicted as the unweighted mean of its five nearest training molecules, by Euclidean distance on z-scored features with every feature weighted equally. This prediction, scored by RMSE, is simpler than the neighbor score of the Methods, which weights neighbors by inverse Manhattan distance and is scored by Spearman correlation. The single representations were used at full width: 2,048 features for ECFP4 and atom pairs, 276 for the WL graph kernel, 200 for PDV and 1,024 for MHG-GNN. These five representations were chosen using all molecules, held-out ones included. Both hybrids draw 250 features from them, use linear allocation and LightGBM gain importance, and are rebuilt on each fold's training molecules. The hybrid splits the budget by the scores of a nearest-neighbor internal evaluator, which is not one of the internal evaluators of Chapter 5; it is the same five-neighbor prediction applied to validation molecules.

The similarity hybrid splits the budget by the neighbor score of Section 6.2.1.

The similarity hybrid had the lowest mean RMSE, 0.6128 pK_i, against 0.6197 for the hybrid and 0.6452 for atom pairs, the lowest single representation. The similarity hybrid was lower than atom pairs on all 3 folds. Table 6.4 repeats the comparison with each feature weighted by its importance: each z-scored feature is multiplied by the square root of its LightGBM gain, computed on each fold's training molecules, before Euclidean distances are computed. For a single representation, the gain comes from a LightGBM model on that representation; for a hybrid, it is the importance the hybrid used to select its features. Features with zero gain get weight zero and so drop out of the distance. With importance weighting, the hybrid had a mean RMSE of 0.6145 pK_i, the similarity hybrid 0.6167 and the WL graph kernel, the lowest single representation, 0.6246. In both comparisons, the lowest single representation was picked on the same folds on which it is scored.

Feature space	Fold 1	Fold 2	Fold 3	Mean
Similarity hybrid	0.6156	0.6052	0.6176	0.6128
Hybrid	0.6179	0.6146	0.6264	0.6197
ECFP4	0.7299	0.6619	0.6695	0.6871
Atom pairs	0.6497	0.6136	0.6723	0.6452
WL graph kernel	0.6661	0.6389	0.6812	0.6621
PDV	0.6510	0.6371	0.6722	0.6535
MHG-GNN	0.6549	0.6424	0.6527	0.6500

Table 6.3: RMSE (pK_i, lower is better) of nearest-neighbor prediction (mean pK_i of the five nearest training molecules) on hERG K_i, per fold and as the mean of the 3 folds. One row is one feature space: the similarity hybrid (Section 6.2.1), the hybrid, or one of five single representations.

Feature space	Equal feature weights	Importance-weighted features
Similarity hybrid	0.6128	0.6167
Hybrid	0.6197	0.6145
ECFP4	0.6871	0.6387
Atom pairs	0.6452	0.6565
WL graph kernel	0.6621	0.6246
PDV	0.6535	0.6355
MHG-GNN	0.6500	0.6686

Table 6.4: Mean RMSE over 3 folds (pK_i, lower is better) of nearest-neighbor prediction on hERG K_i, with equal feature weights and with each feature weighted by the square root of its LightGBM gain. One row is one feature space. The first column is the mean column of Table 6.3.

Table 6.5 gives the mean absolute pK_i difference between each held-out molecule

and its five nearest training molecules (Section 6.2.1), with neighbors found by Manhattan distance on z-scored features and every feature weighted equally. This comparison comes from a separate run on 3 scaffold-group folds of the same 1,412 molecules, with the single representations of Table 6.3 and the two hybrids rebuilt with the settings given there (250 features each). The similarity hybrid had the lowest mean, 0.5602 pK_i, against 0.5622 for the hybrid and 0.5811 for PDV. The similarity hybrid was lower than PDV on all 3 folds. PDV was chosen as the lowest single representation in this table, on the same folds on which it is scored.

Feature space	Fold 1	Fold 2	Fold 3	Mean
Similarity hybrid	0.5330	0.5775	0.5702	0.5602
Hybrid	0.5389	0.5819	0.5659	0.5622
ECFP4	0.5762	0.5871	0.5922	0.5852
Atom pairs	0.5562	0.5892	0.6014	0.5823
WL graph kernel	0.5877	0.6193	0.5914	0.5995
PDV	0.5678	0.5870	0.5885	0.5811
MHG-GNN	0.5682	0.5896	0.6083	0.5887

Table 6.5: Mean absolute difference in pK_i (lower is better) between a held-out molecule and its five nearest training molecules on hERG K_i, averaged over the held-out molecules of each fold, per fold and as the mean of the 3 folds. One row is one feature space.

In a separate run on hERG K_i, the five representations of Table 6.3 were joined side by side and z-scored, and each held-out molecule's pK_i was predicted as the mean of its ten nearest training molecules by Euclidean distance. The held-out R^2 (higher is better, mean of 3 scaffold-group folds) was 0.4638 with equal feature weights and 0.5235 with each feature multiplied by the square root of its LightGBM gain on the training molecules. This run reports R^2, not the RMSE used for hERG K_i elsewhere in this chapter. No control shuffles the weights across features, so these two scores cannot separate the effect of which features receive large weights from the effect of LightGBM giving many features zero weight, which removes them from the distance.

6.4 Results: Noise Detection and Estimation

Results pending: noise detection and estimation need the full runs.

[RESULT NEEDED: average precision per noise score and feature space, with chance level; the three noise-size estimates beside the model's mean squared error]

6.5 Results: Learning with Noisy Labels by Noise Type

6.5.1 Censoring: Correction of Clipped Labels

Preliminary results from local runs on a few datasets with 3 folds. These may change when the full runs finish.

In an earlier exploratory run on ECFP4 alone (Table 6.6), we clipped the hERG K_i training labels at 6.86 pK_i and left the held-out labels unchanged. This limit is the 90th percentile of all hERG K_i labels, held-out molecules included, and was the same in every fold. The clip changed 94, 88 and 100 of the 941, 941 and 942 training labels, and the folds held out 471, 471 and 470 molecules. Every model was trained on ECFP4 alone (2,048 bits, standardized on the training molecules), with no hybrid. The random forest had 100 trees, LightGBM had 100 trees and otherwise default settings, kNN used 5 neighbors, and SVM was support vector regression with an RBF kernel and default settings. Every training label at the limit was raised by the shared lift of Section 6.2.3, 0.89, 0.99 and 0.93 pK_i in the three folds, with predictions from a random forest on 3 inner scaffold-group folds of the training molecules. The lift and the deletion act on every training label equal to 6.86 pK_i, 95, 90 and 103 molecules, which includes 1, 2 and 3 molecules whose measured label was already 6.86 pK_i. For every model, the lifted labels gave an RMSE between the clipped and never-clipped values and closer to the never-clipped one; for example, LightGBM scored 0.6499 pK_i clipped, 0.6248 lifted and 0.6087 never clipped. Deleting the molecules at the limit gave the highest RMSE of the four versions of the labels for every model.

A second run clipped 10% of the hERG K_i training labels at a limit read from each fold's training labels: 6.86, 6.82 and 6.99 pK_i in folds 1 to 3. The clip changed 94, 93 and 95 of the 941, 941 and 942 training labels. The held-out labels were not changed. The lift and the deletion act on every training molecule at the limit, 95, 97 and 95 molecules. In this run, the predictions behind the lift came from LightGBM on 5 group-aware inner folds of the training molecules, in the same feature space as the downstream model. Only the hybrid's five representations were chosen by a search on each fold's unclipped training labels; the budget (250 features), allocation (linear) and importance method (LightGBM gain) were fixed, not searched as the Methods describe. The representations were chosen on the unclipped labels, but

	RMSE in pK_i (lower is better)			
Training labels	Random forest	LightGBM	kNN	SVM
Never clipped	0.6032	0.6087	0.6871	0.6841
Clipped	0.6588	0.6499	0.7158	0.7148
Clipped, then lifted	0.6118	0.6248	0.6937	0.6847
Clipped, deleted	0.7706	0.7631	0.8037	0.8162

Table 6.6: Held-out RMSE in pK_i (lower is better) on hERG K_i with ECFP4, mean over 3 scaffold-group folds, after the training labels above 6.86 pK_i were clipped to 6.86 pK_i. One row is one version of the training labels, and one column is one downstream model. Never clipped: the original training labels. Clipped, then lifted: every label at the limit raised by the shared lift of Section 6.2.3. Clipped, deleted: the molecules at the limit removed before training.

the features within them were selected from the clipped labels, for every version of the training labels. So the hybrid trained on the original labels is not a hybrid that never saw clipping. The hybrid's search used a random forest internal evaluator. The similarity hybrid's search fitted no model and scored how well each training molecule's pK_i is predicted from its five nearest neighbors. Random forest, LightGBM and SVM were scored on the hybrid and the similarity hybrid (Table 6.7) and on each model's two best single representations (Table 6.8). These two were ranked by cross-validated RMSE on the unclipped labels of all molecules, held-out ones included. For all three models they were ECFP4 and MHG-GNN, used at full width (2,048 and 1,024 features). Table 6.9 lists the representations in each fold's hybrids. In each fold, all three models used the same hybrid and the same similarity hybrid. The lifted labels gave a lower mean RMSE than the clipped labels for all 12 pairs of model and feature space, and deleting the molecules at the limit gave the highest mean RMSE for all 12. For random forest on ECFP4, one shared lift gave 0.6139 pK_i, a separate lift ℓ_i for each clipped molecule gave 0.6203, and the same separate lifts given to the clipped molecules in one random order gave 0.6199.

6.5.2 Gaussian Noise: Methods for Learning with Noisy Labels

Preliminary results from local runs on a few datasets with 3 folds. These may change when the full runs finish.

The survey of twelve methods comes from an earlier preliminary run on nanobody thermostability (672 nanobodies), scored by the Spearman correlation between pre-

Model	Feature space	Training labels			
		Original labels	Clipped	Clipped, then lifted	Clipped, deleted
Random forest	Hybrid	0.6121	0.6602	0.6174	0.7603
	Similarity hybrid	0.6221	0.6699	0.6298	0.7656
LightGBM	Hybrid	0.5993	0.6303	0.6152	0.7336
	Similarity hybrid	0.6169	0.6417	0.6288	0.7593
SVM	Hybrid	0.5806	0.6189	0.5855	0.7244
	Similarity hybrid	0.5811	0.6223	0.5870	0.7350

Table 6.7: Held-out RMSE in pK_i (lower is better) on hERG K_i with 10% of the training labels clipped, mean of 3 scaffold-group folds. One row is one downstream model on one hybrid, and one column is one version of the training labels. Original labels: the original training labels, on hybrid features chosen from the clipped labels. Clipped, then lifted: every training label at the limit raised by the shared lift of Section 6.2.3. Clipped, deleted: the training molecules at the limit removed.

Model	Representation	Training labels			
		Never clipped	Clipped	Clipped, then lifted	Clipped, deleted
Random forest	ECFP4	0.6032	0.6558	0.6139	0.7721
	MHG-GNN	0.6103	0.6650	0.6204	0.7717
LightGBM	ECFP4	0.6088	0.6454	0.6246	0.7620
	MHG-GNN	0.6079	0.6412	0.6091	0.7520
SVM	ECFP4	0.6122	0.6483	0.6165	0.7663
	MHG-GNN	0.5979	0.6335	0.6012	0.7516

Table 6.8: Held-out RMSE in pK_i (lower is better) on hERG K_i with the clipped labels of Table 6.7, mean of the same 3 scaffold-group folds. One row is one downstream model on one of its two best single representations. Never clipped: the original training labels; the other columns are as in Table 6.7.

Representation	Hybrid			Similarity hybrid		
	Fold 1	Fold 2	Fold 3	Fold 1	Fold 2	Fold 3
MHG-GNN	✓	✓	✓	✓		✓
Mordred	✓	✓	✓		✓	
WL graph kernel	✓	✓	✓		✓	
Avalon						
GROVER				✓		✓
PDV		✓	✓	✓	✓	
ECFP4						✓
Atom pairs	✓	✓	✓	✓	✓	✓
ChemBERT				✓	✓	
SMI-TED	✓					✓

Table 6.9: Representations in the hybrids of Table 6.7. One row is one of the ten hERG K_i representations; a check mark means the hybrid of that fold draws on it.

dicted and measured melting temperature on held-out nanobodies (higher is better), as the mean of 3 folds. The folds are grouped by 95% sequence identity cluster (309 clusters). Gaussian noise with a standard deviation of 0.5 or 1.5 times that of all 672 measured melting temperatures (10.12 °C) was added to the training labels, with one noise draw per level, and the held-out labels carry no added noise. Table 6.10 gives the score of each of the twelve methods, whose settings are listed in Additional file 17, beside a plain neural network trained without any noise handling. The plain network is the fully connected network of that Additional file, and every method except the heteroscedastic GP trains this same network. The heteroscedastic GP predicts with a Gaussian process (RBF kernel, at most 1,000 training nanobodies), and a small network estimates each nanobody's noise variance. The twelve methods include the four removal methods of the exploratory study in Section 4.4.1. One of them, small-loss selection, trains a new network only on the 70% of training nanobodies with the smallest error after a warm-up period. In these runs, removal by distance to neighbors and MentorNet found neighbors by Euclidean distance on the standardized features.

The inputs were one-hot (all 5,600 features), the hybrid, and the weighted hybrid, each standardized on the training fold. In each fold the hybrid was built from that fold's noisy training labels before the network was trained on it. Its budget of 50 features, linear allocation and LightGBM gain importance were fixed. The only setting searched was which representations it draws on: the three with the highest standalone scores in the fold, or all ten of Table 5.15. A random forest on 3 inner folds of the training nanobodies made this choice in each fold and at each noise level. At 1.5 times the label standard deviation, folds 1 and 2 used the three and fold 3 used all ten; which representations were used at 0.5 times could not be recovered. The 50 features were divided among the chosen representations in proportion to each representation's Spearman score with the same random forest, which served as the internal evaluator. The weighted hybrid multiplies the standardized hybrid features by the square root of each feature's LightGBM gain importance in that fold, with the multipliers rescaled to a mean of 1, and is not rescaled afterwards.

With no added noise, the plain network scored 0.4167 on one-hot, 0.5418 on the hybrid and 0.5857 on the weighted hybrid (Spearman, mean of the same 3 folds). At this level the hybrids are built from the unnoised labels, and the twelve methods were not run. At 0.5 times the label standard deviation, the plain network scored 0.4534 on one-hot, 0.5991 on the hybrid and 0.5833 on the weighted hybrid. At that

	Spearman correlation with measured melting temperature (higher is be					
	Noise SD 5.06 °C (0.5× label SD)			Noise SD 15.18 °C (1.5× label S		
Method	One-hot	Hybrid	Weighted hybrid	One-hot	Hybrid	Weighted h
Plain network	0.4534	0.5991	0.5833	0.2729	0.4859	0.5118
Small-loss selection	0.3282	0.5820	0.5615	0.3568	0.5029	0.5161
Confident learning	0.3938	0.5850	0.5674	0.2329	0.4759	0.5316
Removal by distance to neighbors	0.1790	0.5725	0.5602	0.1432	0.4834	0.5112
Staged removal	0.2993	0.5818	0.5805	0.1903	0.4693	0.4606
Per-epoch down-weighting	0.3671	0.5852	0.5954	0.2805	0.5153	0.5092
Curriculum by confidence	0.3808	0.5825	0.5949	0.3010	0.4996	0.5101
DivideMix	0.3850	0.5942	0.5862	0.3158	0.5077	0.5247
Meta-Weight-Net	0.3349	0.5764	0.5879	0.2778	0.4726	0.5049
MentorNet	0.4234	0.5749	0.5755	0.2965	0.5063	0.5368
Contrast to Divide	0.3421	0.5887	0.5894	0.2921	0.5153	0.5278
Heteroscedastic GP	0.3469	0.5814	0.5766	0.2662	0.5141	0.5157
Deep evidential regression	0.3435	0.5909	0.5803	0.2812	0.5012	0.5089

Table 6.10: Spearman correlation (higher is better) between predicted and measured melting temperature of held-out nanobodies on nanobody thermostability, mean of 3 folds, with Gaussian noise added to the training labels. One row is one training method, and one column is one input at one noise level; SD is standard deviation. Plain network: the neural network trained without noise handling. Weighted hybrid: the hybrid with each feature multiplied by the square root of its LightGBM gain importance.

level, none of the twelve methods scored above the plain network on one-hot or on the hybrid, and 5 of the twelve did on the weighted hybrid. At 1.5 times, the plain network scored 0.2729 on one-hot, 0.4859 on the hybrid and 0.5118 on the weighted hybrid. At that level, 8 of the twelve methods scored above the plain network on one-hot, 8 on the hybrid and 6 on the weighted hybrid. At both noise levels, the plain network and every one of the twelve methods scored higher on the hybrid than any method did on one-hot. At 1.5 times, the best method on one-hot, small-loss selection at 0.3568, scored below the plain network on the hybrid at 0.4859.

These numbers rest on 3 folds and one noise draw per level. With no added noise, the plain network on one-hot scored 0.6442, 0.2573 and 0.3484 Spearman on folds 1 to 3. The run used 3 of the planned 5 folds, 1 of 3 noise draws, 2 of 6 noise levels and one of three budgets. In these runs the hybrid's features were searched with a random forest at a budget of 50 features, whereas every method trains a neural network. One-hot is one of the ten representations of this dataset. The full runs search the hybrid with the neural network, average three noise draws per fold, and replace one-hot with each model's top two single representations.

6.5.3 Real Labels: Cleaning without Injected Noise

Preliminary results from local runs on a few datasets with 3 folds. These may change when the full runs finish.

With nothing injected, we tested four methods on the recorded hERG K_i labels with random forest, SVM and LightGBM, in runs with the default configuration and the limits set out in Section 6.3. The four ran on different folds, so their results are in three tables, and each method is compared only with the unchanged labels of its own table. Pair down-weighting ran on all 3 folds, on each model's top two single representations, the hybrid and the similarity hybrid (Table 6.11). It found pairs in each feature space, by Tanimoto similarity for ECFP4 and by Euclidean distance on z-scored features otherwise. ECFP4 and MHG-GNN were the top two single representations of all three models, ranked by cross-validated RMSE over all molecules, held-out ones included. In each fold, one hybrid and one similarity hybrid were built on that fold's training molecules and used by all three models. Both have 250 features, linear allocation and LightGBM gain importance, and draw on five of the ten hERG K_i representations chosen in that fold. The hybrid has a random forest internal evaluator, as in the default hybrid of Section 5.4. It draws on atom pairs, WL graph kernel, MHG-GNN, Mordred and SMI-TED in fold 1, and on PDV in place of SMI-TED in folds 2 and 3. In fold 1, the similarity hybrid draws on atom pairs, ChemBERT, GROVER, MHG-GNN and PDV. In fold 2, it draws on atom pairs, ChemBERT, WL graph kernel, Mordred and PDV. In fold 3, it draws on atom pairs, ECFP4, GROVER, MHG-GNN and SMI-TED.

With LightGBM on MHG-GNN, pair down-weighting lowered the mean RMSE from 0.6079 to 0.5966 pK_i. Per fold it scored 0.6094, 0.5967 and 0.5836 against 0.6159, 0.6241 and 0.5838. Down-weighting random molecules gave 0.6102. With LightGBM, pair down-weighting raised the RMSE on ECFP4, the hybrid and the similarity hybrid, for example from 0.5938 to 0.6086 on the hybrid. With SVM it raised the RMSE in all four feature spaces. With random forest it lowered the RMSE in all four; the largest change was on the similarity hybrid, from 0.6204 to 0.6154. On ECFP4, down-weighting random molecules gave a lower RMSE than pair down-weighting, 0.6007 against 0.6016.

Across the three folds, pair down-weighting flagged 310 pairs in ECFP4. Of these, 305 (77 of 82, 96 of 96 and 132 of 132) had a Tanimoto similarity of at least 0.55 and a label gap of more than twice the median absolute deviation of the

Model	Feature space	Unchanged	Down-weighting	
			Pair	Random
Random forest	ECFP4	0.6032	0.6016	0.6007
	MHG-GNN	0.6103	0.6097	0.6126
	Hybrid	0.6107	0.6063	0.6111
	Similarity hybrid	0.6204	0.6154	0.6188
SVM	ECFP4	0.6122	0.6214	0.6184
	MHG-GNN	0.5979	0.6134	0.5989
	Hybrid	0.5752	0.5891	0.5776
	Similarity hybrid	0.5766	0.5871	0.5786
LightGBM	ECFP4	0.6088	0.6156	0.6270
	MHG-GNN	0.6079	0.5966	0.6102
	Hybrid	0.5938	0.6086	0.6139
	Similarity hybrid	0.6037	0.6137	0.6214

Table 6.11: Held-out RMSE in pK_i (lower is better) on hERG K_i with nothing injected, mean of the same 3 scaffold-group folds. One row is one downstream model and one feature space. Unchanged: the model fitted on the recorded training labels. Pair: pair down-weighting (Section 6.2.3), with pairs found in the row's feature space. Random: weight 0.25 on the same number of randomly chosen training molecules. The similarity hybrid is that of Section 6.2.1.

training labels. That is the activity-cliff rule of this run, so the down-weighted pairs are mostly activity cliffs rather than label errors.

C-Mixup and target-noise bagging ran on folds 1 and 2 only, on each model's top two single representations (Table 6.12), ranked using all molecules, held-out ones included. The hybrid results of this run are left out, because its hybrids used representations chosen with the held-out molecules included. C-Mixup gave a lower mean RMSE for all six combinations of model and representation. The largest changes were with SVM, from 0.6095 to 0.5885 on MHG-GNN and from 0.6230 to 0.6033 on ECFP4. Target-noise bagging lowered the RMSE of LightGBM on MHG-GNN from 0.6200 to 0.5942. For the other five combinations it lowered the RMSE by at most 0.0026 pK_i or raised it.

Down-weighting suspect molecules ran on fold 1 only, with all models on MHG-GNN, at 11 settings of the noise score, the number of neighbors and the strength (Table 6.13). The cross-validation residual used out-of-fold predictions of a random forest of 100 trees within the training molecules, and the two neighbor-based scores found neighbors in the MHG-GNN features. Each training molecule got the inverse weight of Table 6.2, $1/(1 + \alpha\,\tilde{s}_i)$, so the most suspect molecule weighed one half, one quarter or one tenth of the least suspect at $\alpha = 1$, 3 or 9. The settings are

Model	Representation	Unchanged	C-Mixup	Target-noise bagging
Random forest	ECFP4	0.6055	0.6041	0.6085
	MHG-GNN	0.6075	0.5991	0.6049
SVM	ECFP4	0.6230	0.6033	0.6272
	MHG-GNN	0.6095	0.5885	0.6085
LightGBM	ECFP4	0.6039	0.5952	0.6028
	MHG-GNN	0.6200	0.5965	0.5942

Table 6.12: Held-out RMSE in pK_i (lower is better) on hERG K_i with nothing injected, mean of folds 1 and 2 of the 3 scaffold-group folds. One row is one downstream model and one of its top two single representations. Unchanged: the model fitted on the recorded training labels.

compared on the same fold they are scored on, so the lowest value in each column is optimistic. With LightGBM, the scored weights gave a lower RMSE than the unchanged labels (0.6255) at all 11 settings, and lower than the random weights at 10 of the 11. With random forest, the scored weights gave a lower RMSE than the unchanged labels (0.6211) at 10 of the 11 settings, and so did the random weights; the random weights were lower than the scored weights at 7 of the 11. With SVM, the scored weights gave a higher RMSE than the unchanged labels (0.6052) at all 11 settings.

Table 6.14 compares three cleanups of the humanness training labels, applied before the hybrid's features were chosen, with random forest, SVM and LightGBM each fitted on the resulting hybrid at 250 features. In each fold, the hybrid was searched and built on that fold's training labels, uncleaned or cleaned, and each downstream model was fitted on the same labels as its hybrid; all three models used the same hybrid. Every hybrid has the settings of the default hybrid of Section 5.4: linear allocation, LightGBM gain importance, a random forest internal evaluator, 5 representations and 250 features. Unlike that default, its five representations were chosen from the ten humanness representations by each fold's search. They were AbLang-2, Ab-RoBERTa, AntiBERTy CDR, BLOSUM62 and IgT5, with these exceptions. With the score from the top-ranked representation, ESM C took the place of BLOSUM62 for removal in fold 2 and for neighbor majority relabeling in fold 3. With the score from the second-ranked representation, ESM C took its place for neighbor majority relabeling in folds 2 and 3. ProtT5 took its place for forest-proximity majority relabeling in fold 3 with the score from the top-ranked representation. Every row, including no cleaning, starts from training labels that

		Random forest		SVM		LightGBM	
k	α	Scored	Random	Scored	Random	Scored	Random
Unchanged labels		0.6211		0.6052		0.6255	
Cross-validation residual							
–	1	0.6142	0.6141	0.6090	0.6068	0.6153	0.6340
–	3	0.6157	0.6192	0.6144	0.6065	0.6027	0.6183
–	9	0.6221	0.6138	0.6206	0.6008	0.6201	0.6089
Neighbor residual							
5	1	0.6084	0.6090	0.6076	0.6052	0.6176	0.6311
5	3	0.6195	0.6100	0.6119	0.6077	0.6059	0.6183
5	9	0.6138	0.6096	0.6172	0.6093	0.5957	0.6062
9	1	0.6060	0.6214	0.6080	0.6068	0.6083	0.6269
9	3	0.6090	0.6161	0.6142	0.6064	0.6078	0.6120
Activity-cliff-aware							
5	1	0.6142	0.6111	0.6069	0.6048	0.6106	0.6291
5	3	0.6208	0.6196	0.6101	0.6059	0.6116	0.6319
5	9	0.6191	0.6154	0.6150	0.6116	0.5967	0.6192

Table 6.13: Held-out RMSE in pK_i (lower is better) on hERG K_i with nothing injected, fold 1 of the 3 scaffold-group folds, with all three models fitted on MHG-GNN. One row is one setting: the noise score (Table 6.1), the number of neighbors k (a dash marks a score that uses no neighbors) and the strength α. Scored: weights from the noise score. Random: the same weights dealt in random order to the same number of randomly chosen training molecules.

differ slightly from the recorded ones, because the noise injector changed some labels with no noise requested; the uncleaned random forest scored 0.9564 AUROC, against 0.9576 on the recorded labels.

In each fold, the training antibodies at or above the 90th percentile of the cross-validation residual (Table 6.1) were cleaned. For each antibody, this residual is one minus the probability of its recorded class from a 100-tree random forest, trained in 5 group-aware inner folds on the training antibodies outside the antibody's own inner fold. The residual was computed in either the top-ranked or the second-ranked single representation of that fold, AntiBERTy CDR or IgT5, ranked by random forest score in the fold's search on the uncleaned labels. Removal drops the flagged antibodies. Neighbor majority relabeling replaces each flagged label with the majority class of its 9 nearest training neighbors in the same single representation. Forest-proximity majority relabeling does the same with neighbors given by random forest proximity in that representation. Neither relabeling is the label smoothing of Table 6.2. With the score from the top-ranked representation, removal and neighbor majority relabeling gave a lower mean AUROC than no cleaning for all three

models. Forest-proximity majority relabeling gave a higher mean AUROC for all three models (SVM 0.9591 against 0.9560, LightGBM 0.9565 against 0.9521), and was higher on each of the 3 folds with SVM and LightGBM. With the score from the second-ranked representation, all three cleanups gave a higher mean AUROC than no cleaning for all three models, for example 0.9639 against 0.9564 for neighbor majority relabeling with random forest. Four of these nine combinations were higher on all 3 folds: removal with random forest and with LightGBM, and both relabeling methods with LightGBM. The uncleaned random forest scored 0.9552, 0.9502 and 0.9637 AUROC on folds 1 to 3.

	Held-out AUROC (higher is better)		
Cleanup	Random forest	SVM	LightGBM
No cleaning	0.9564	0.9560	0.9521
Noise score in the top-ranked representation (AntiBERTy CDR in fold 1, IgT5 in folds 2 and 3)			
Removal	0.9550	0.9554	0.9487
Neighbor majority relabeling	0.9513	0.9512	0.9449
Forest-proximity majority relabeling	0.9579	0.9591	0.9565
Noise score in the second-ranked representation (IgT5 in fold 1, AntiBERTy CDR in folds 2 and 3)			
Removal	0.9606	0.9619	0.9618
Neighbor majority relabeling	0.9639	0.9625	0.9625
Forest-proximity majority relabeling	0.9615	0.9605	0.9615

Table 6.14: Held-out AUROC (higher is better) on humanness, mean of 3 folds, with the training labels cleaned before the hybrid is built. One row is one cleanup, grouped by the single representation in which the noise score was computed, and one column is one downstream model fitted on the hybrid. No cleaning: the hybrid and the models built on the uncleaned training labels.

6.5.4 Comparison across Noise Types

The noise types differ in what they allow a method to correct. Under censoring, the location of the error (the limit) and its direction are known and only its size is unknown. Raising every clipped label by one shared amount gave each of four models on ECFP4 an RMSE closer to its never-clipped value than to its clipped one (Table 6.6). Outlier noise affects few labels, so the widened labels should stand out against their neighbors, although detection results are pending; removing or down-weighting them would still not recover the true values. Under grouped-shifted noise, molecules of the same scaffold family share one offset, so comparing a molecule with

its neighbors cannot reveal it. Gaussian noise, the standard assumption in studies of noise robustness, is the hardest to correct, because every label carries an error of unknown size and sign, the neighbors used to judge a label are noisy too, and a large disagreement with them may be an activity cliff rather than an error. In the exploratory study of Chapter 4, 8 of 860 cleaning configurations scored better than training on the noisy labels at both noise levels (Gaussian noise with a standard deviation of 0.3 and 0.6 eV on the QM9 HOMO–LUMO gap). A better similarity can make neighbors closer in true activity, but it cannot remove the noise on their labels.

6.6 Conclusions

Preliminary: rewrite from the full runs.

In the preliminary runs on hERG K_i, predicting each molecule's pK_i from its five nearest neighbors gave a lower RMSE in the similarity hybrid than in any of the five single representations. Noise detection and estimation await the full runs. Under censoring at 10% of the hERG K_i training labels, raising every clipped label by one shared amount gave a lower mean RMSE than the clipped labels for every model and feature space tested, whereas deleting the clipped molecules gave the highest. Under Gaussian noise on nanobody thermostability, every one of twelve methods scored higher on the hybrid than any of them did on one-hot. With nothing injected, whether a method helped depended on the model, the representation and, on humanness, the representation in which the noise score was computed. Which method helps therefore depends on the type of noise, the dataset and the feature space, and Chapter 7 returns to this.

Chapter 7

Discussion and Conclusions

Real assay labels are not noisy in the way most studies assume. Measurements pooled across laboratories differ mainly between laboratories [77], and labels near an assay's detection limit are clipped to it [78] (Table 2.1). In Chapter 3, these two systematic kinds of error, simulated as grouped-shifted noise and censoring, gave the lowest $\mathrm{AUC}_{\mathrm{norm}}$. Gaussian noise, the usual assumption, harmed the models less, but we argue it is the hardest case to correct, because every label is off by an unknown amount in an unknown direction; in the exploratory study on QM9, cleaning usually left a model worse than no cleaning (Sections 4.4.1 and 4.4.2).

7.1 Summary

Chapter 3 found, on all four datasets, that the representation explained a larger share of the variance in predictive performance than in robustness ($\mathrm{AUC}_{\mathrm{norm}}$), where model family dominated and the representation explained at most 11.2% ("Conclusions", Chapter 3). Changing the shape of the noise distribution barely changed performance, while grouped-shifted noise and censoring, the two conditions with systematic error, gave the lowest $\mathrm{AUC}_{\mathrm{norm}}$. RF and QRF were reliably robust on every dataset, although their predictive performance on clean labels was middling, and on the three assay datasets a Bayesian network gained robustness over its deterministic counterpart at a cost in clean predictive performance. Mean predicted uncertainty rose with noise for almost every probabilistic model on QM9 and for most of them on the assay datasets. The Bayesian networks with a variance head and the heteroscedastic VBLL networks mostly attributed the added noise to the aleatoric component, whereas QRF attributed much of it to the epistemic component. Out-

side censoring, no model's per-molecule uncertainty identified the corrupted labels, and under censoring the models were most confident on the clipped labels.

Chapter 4 showed that on any one dataset many fingerprint features do not vary or do not relate to the property, yet a distance sums over all of them, and that some representations give different molecules identical feature vectors. In the exploratory study of LNLs, configurations that used a distance did worst, and judging a label by its neighbors usually made the model worse. A better representation would have few features, chosen for the task from several representations and traceable to their source.

Chapter 5 introduced the adaptive hybrid representation, which chooses features for one task from several representations under a budget in features, inside each fold. Its results are preliminary, mostly from local runs that built one default hybrid rather than searching its settings. On hERG K_i, the default hybrid of 250 features had a lower RMSE than the best single representation with SVM and LightGBM on all three folds, and a mean RMSE 0.0006 pK_i higher with random forest. The best budget differed between downstream models, from 100 features for random forest to 1,000 for SVM; whether datasets prefer different search settings needs the full runs **[RESULT NEEDED]**. Search settings found on LogD, with the features re-selected on hERG K_i, gave a lower RMSE than the best single representation of hERG K_i for three of five downstream models. With molecules kept at three separate hERG sources, the selected features gave a higher MCC than the 2,048 FCFP4 bits on all three folds, although the two settings differed in more than the selection. **[RESULT NEEDED: antibody.]** Importance weighting **[RESULT NEEDED: helped?]**.

Chapter 6 tested whether a hybrid chosen for the task repairs nearest-neighbor prediction, noise detection and learning with noisy labels, again in preliminary runs. On hERG K_i, predicting each molecule's pK_i from its five nearest training molecules gave a lower RMSE in the similarity hybrid than in any of the five single representations, and when both were chosen on the other folds the hybrid was lower on all three folds. Noise detection: **[RESULT NEEDED]**. Under censoring, raising every clipped label by one constant improved on the clipped labels in all 48 comparisons on ECFP4, on hERG K_i and Caco-2 at several clipping levels, and in 33 of 36 in a second run that included the hybrid, while deleting the clipped molecules made every model worse. Under Gaussian noise on nanobody thermostability, none of the twelve LNL methods beat the plain network at 0.5 times the label standard

deviation on one-hot or the hybrid, where the noise had not lowered its score, and eight of the twelve did at 1.5 times, but every method scored higher on the hybrid than any method did on one-hot, the weakest input. With nothing injected, some methods lowered the RMSE on hERG K_i for some models and representations, and on humanness whether cleaning helped depended on the representation in which the noise score was computed.

Chapter 3 shows which models held up best when labels are noisy, and Chapters 4 to 6 give a procedure for choosing the features to give a model. The contributions of this thesis are the NoiseInject package, the adaptive hybrid representation and its blueprints, and evidence that distance-based noise handling on standard representations usually made models worse. The resulting publications and presentations are listed in the Summary of Research Contributions.

7.2 Dataset Dependence of the Best Method

No value of any of the six settings of the hybrid search (Table 5.6) was best on every dataset **[RESULT NEEDED: confirm on the full runs]**, and which LNL method helps, if any, depends on the data **[RESULT NEEDED: confirm on the full runs]**. Noise mitigation, the LNL methods that change, drop or reweight the training labels before any model is trained, is the harder part (Chapter 6). We therefore report no overall winner averaged across datasets.

The best choice depends on at least five factors: the type and amount of label noise, the number of molecules, whether any feature describes the property, and the downstream model. A small dataset leaves few molecules in a fold to score each setting on, so the chosen setting is less stable (Chapter 5 gives the size threshold in its Limitations section). The search cannot add signal its inputs lack. The internal evaluator is searched per downstream model, since the best hybrid for a tree ensemble need not suit a neural network.

In Chapter 3, the most robust representation changed between Gaussian and grouped-shifted noise for NGBoost (Avalon to PDV), SVM (ChemBERTa to ECFP4) and BNN-α (PDV to MHG-GNN) on QM9, and the model with the highest predictive performance under noise differed between the four datasets. In Chapter 5, the winning allocation method, number of base representations, importance method and budget were **[RESULT NEEDED: per dataset]**, and the winning internal evaluator **[RESULT NEEDED: per dataset and model]**;

in the preliminary runs on hERG K_i the best budget already differed between downstream models. A fixed share of features per representation won on **[RESULT NEEDED: datasets]**, one pooled ranking on **[RESULT NEEDED: datasets]**. In the preliminary runs of Chapter 6, raising every clipped label by one constant beat the clipped labels in all 48 comparisons on ECFP4 and in 33 of 36 in a second run that included the hybrid, and under Gaussian noise the number of the twelve LNL methods beating the plain network changed with the amount of noise, from none at 0.5 times the label standard deviation to eight at 1.5 times, on both one-hot and the hybrid. The LNL methods that helped under each noise type were **[RESULT NEEDED: per noise type, full runs]**.

The thesis therefore offers no fixed recipe or new architecture tuned to one problem, but a set of methods that might work and a procedure that chooses the features for each supervised task. A practitioner has no default to trust and should ask which settings to try and how to compare them fairly.

7.3 Simple Models and Small Feature Sets

Under label noise, the simpler choice was a random forest rather than a neural network (Chapter 3). On QM9 under Gaussian noise, the forests, XGBoost, LightGBM, SVM and the GP were accurate on clean labels and robust. The four plain and Bayesian networks were among the most accurate but the four least robust, and their $\mathrm{AUC}_{\mathrm{norm}}$ varied most across representations; the VBLL networks sat in the middle. Robustness is not accuracy: NGBoost had the highest $\mathrm{AUC}_{\mathrm{norm}}$ on all six representations, yet the lowest clean R^2 on four. XGBoost and LightGBM lost the most robustness on Caco-2 and hERG K_i, whereas under Gaussian noise RF ranked between first and seventh of the thirteen base models for $\mathrm{AUC}_{\mathrm{norm}}$ on every dataset and representation. On QM9 with ECFP4, RF also rose from seventh for predictive performance on clean labels to first at a noise level of 1.0.

The hybrid search sweeps the budget rather than assuming that more features are better. In the preliminary runs on hERG K_i, every downstream model did best below the largest budget of 2,000 features, random forest at 100, and the RMSE changed by 0.008 pK_i or less between 250 and 2,000 features (Chapter 5); how often this holds across datasets **[RESULT NEEDED: full runs]**. An ensemble of single-representation models **[RESULT NEEDED: matched the hybrid at lower cost?]**. When predicting each hERG K_i molecule's pK_i from its five near-

est neighbors, a similarity hybrid of 250 features gave an RMSE of 0.6128 against 0.6452 for atom pairs, the best of the five single representations at full width with unweighted distances, both picked on the held-out folds; with each chosen on the other two folds, the hybrid was still lower on all three folds (Chapter 6, preliminary). The model stays simple, so any gain comes from the input.

The LNL methods that used a distance between molecules did worst in Chapter 4, and judging a label by its neighbors usually made the model worse (Sections 4.4.1 and 4.4.2). The preliminary results of Chapter 6 are mixed. Of twelve LNL methods compared with a plain neural network under Gaussian noise ("Gaussian Noise: Methods for Learning with Noisy Labels"), none beat it at 0.5 times the label standard deviation on one-hot or the hybrid, where the noise had not lowered its score, and eight beat it at 1.5 times on one-hot and on the hybrid. On real labels ("Real Labels: Cleaning without Injected Noise"), some methods lowered the RMSE on hERG K_i for some models and representations only, and on humanness the direction depended on the representation in which the noise score was computed. For labels clipped at a detection limit, raising every clipped label by one constant beat the clipped labels in all 48 comparisons on ECFP4 and in 33 of 36 in a second run that included the hybrid. At separate sites with different noise levels, removing flagged molecules **[RESULT NEEDED: beat pooling?]**.

LNL methods from other fields carry over poorly to assay data. Most were built for images with flipped class labels [47] (Sections 2.5.2 and 4.4.1), whereas the three assay datasets of Chapter 3 have 1,415 to 5,039 molecules and continuous labels, all with some error. Many LNL methods rely on a sample's neighbors or on how easily it is learned; both need a space where close molecules have similar activity, which standard representations lack (Chapter 4). Most LNL methods were also built for neural networks [47], whereas tree models still lead on medium-sized tabular data [256] ("Models and metrics", Chapter 5).

A method tuned closely to one dataset loses generality. The tasks in this thesis span binding affinity, ADME, quantum-chemical properties and antibodies, and this range is the reason learned models are used at all. Simple methods do not generalize automatically either: feature selection needs tuning for each dataset, which is why the search sweeps its settings, and the search itself can overfit (Discussion, Chapter 5). The balance is to fix the simple, robust parts, such as a tree ensemble or nearest neighbors on a small feature set, and search only what varies by dataset.

7.4 Limitations

The limits of the hybrid search itself are in the Limitations section of Chapter 5. This thesis set out to mitigate label noise, and we conclude that correcting it without ground truth is a hard problem with no clear general solution. Every experimental label carries an unknown amount of noise, so there is no clean copy of the labels to check a correction against, and finding a noisy label may be easier than correcting it **[RESULT NEEDED: Chapter 6 detection results]**. Systematic errors of known structure may still be detectable: under censoring, dividing a molecule's error by its predicted uncertainty picked out the clipped labels better than the error alone (Chapter 3). Whether outliers and shifts shared by one source can be detected is **[RESULT NEEDED: Chapter 6 detection results]**. In the assay experiments, "clean" means no noise was added, not that the labels are true; only QM9 is close to noise-free. Real data mix the injected noise conditions (Table 2.2) in unknown amounts, so a method that works under one condition may fail on real data. **[RESULT NEEDED: per noise condition, methods beating the uncorrected model; negative recovery.]**

Noise mitigation changes, drops or reweights the labels once before training, so a wrong estimate stays for every later model; it is cheaper, since it is done once and reused by any model. Noise-robust training instead leaves the labels unchanged, adapts throughout training and uses all the data [47] (Section 2.5.2). Correcting labels directly is justified when the noise has a known direction, as in censoring, or affects a clearly separate subset, such as gross outliers or an offset tied to one source. In the exploratory study of Chapter 4 (Section 4.4.1), the methods that held up estimated noise from the model's own loss or uncertainty, and they included one noise mitigation method, small-loss selection. Outside censoring and clearly separate subsets, we therefore recommend methods that judge labels by the model's own loss or uncertainty rather than by neighbors **[RESULT NEEDED: confirm against the Chapter 6 full runs]**. Either approach can be paired with uncertainty estimation where it is reliable. Mean predicted uncertainty rose with noise, and the aleatoric component can compare noise between datasets or noise levels, although it does not estimate a dataset's own error (Chapter 3). Outside censoring, however, per-molecule uncertainty did not identify the noisy labels, and under censoring the models were most confident on the clipped labels. Uncertainty should therefore be used to judge how noisy a dataset is, not which labels are wrong.

The label-noise experiments are almost all regression; only the Gaussian-noise experiments of Chapter 6 flip class labels, on classification datasets. The controlled label-noise experiments of Chapter 3 use QM9 and three small assay datasets, LogD, Caco-2 and hERG K_i, and the preliminary runs of Chapter 6 add nanobody thermostability and humanness. No dataset here has repeated measurements with a known true value for every molecule, as a direct test of label correction would need. Compute limited how many noise conditions, noise levels, folds and datasets could be crossed. Chapter 3 uses 10 replicates on QM9 and five scaffold folds on each assay dataset, whereas the results of Chapters 5 and 6 are per-fold scores and their spread over 3 to 5 folds, without significance tests, so some patterns show only a direction. **[RESULT NEEDED: final folds and datasets per chapter.]**

7.5 Future Directions

The first next step is classification, where a noisy label is a flipped class. Many QSAR class labels come from cutting a continuous value at a threshold, so flips concentrate near the cutoff and depend on the molecule. NoiseInject provides six flipping strategies (Section 2.6.1) and the search handles classification, but molecule-dependent flips have not been studied at the scale of the regression study of Chapter 3. Most noise-robust training methods were designed for flipped labels, so whether the regression findings carry over is open.

The search does not yet include transformed features (normalized or binarized continuous features), features derived from distances in the molecular graph, or importance methods that treat the model as a black box, such as KernelSHAP and permutation importance. No search objective yet rewards robustness to label noise as well as accuracy.

Chapter 6 uses outlier and grouped-wider noise only to grade noise detection, and grouped-shifted noise, in which every scaffold family receives its own offset as the labels of one laboratory might, only to grade noise estimation (Table 2.2). It remains open whether neighbor-based cleaning, which usually made the model worse under Gaussian noise (Section 4.4.2), does better on outliers, and whether label smoothing and sample weighting, which usually hurt there, behave differently on these noise types.

Hybrid features could also serve as inputs to generative models, such as the model that scores molecules for a generator. What the search should optimize there

is unclear, since held-out accuracy on known molecules may not reward the right properties in new ones. The hybrid features of every generated molecule must also be computed.

The full search is too expensive for routine use. In this research each dataset searched a smaller grid cut from an earlier search, which on hERG K_i kept the best configuration at 1.4% of the fitting time (Section 5.2.4). Adaptive pruning, which drops configurations early within one search, is a second way to cut its cost (Section 5.2.4). The next step is a warm start near the blueprint of a similar dataset; Chapter 5 already reuses one dataset's search settings on another without search. A faster search must still be able to reach every configuration in the search space, while scoring fewer of them. **[RESULT NEEDED: hours and model fits, full search against the per-dataset grid.]**

Three smaller directions remain open: whether the hybrid improves a weak new embedding as well as absorbing a strong one; transfer of blueprints across domains, which would have to work at the level of representation families, since features differ between domains; and a fuller federated-learning deployment with Lhasa Limited, extending the separate-sites setting of Chapter 5 to real sites that run feature selection on their own data [262].

External benchmarks not run here could be added later: activity-cliff prediction [53], antibody developability with a public competition's cluster splits [270], and antibody–antigen binding free-energy change, comparing sequence representations with a structure-based graph network [271].

7.6 Conclusions

Both halves of this thesis lead to the same advice. Before choosing a model, representation or LNL method, a practitioner should ask where the labels came from and what shape their noise is likely to have, then choose features for the task rather than take one representation whole. Since no single model or representation was best on every dataset in Chapter 3, and the best budget already differed between downstream models in the preliminary runs, the choices that vary by dataset should be searched and compared fairly.

Simple choices were often enough. Random forests were the most consistent choice under label noise across the four datasets of Chapter 3, while the plain and Bayesian neural networks were among the least robust. In the preliminary runs on

hERG K_i, a hybrid of a few hundred features chosen for the task matched or beat the best single representation used whole, as a model input and for nearest-neighbor prediction (Chapters 5 and 6) **[RESULT NEEDED: full runs]**.

A new foundation model appears every few months in molecular, protein and antibody modeling. With the hybrid search, a researcher need not choose between their current representation and a new one: they add the new embeddings to the representations searched and re-run the search, which decides whether the new embedding contributes features and, if so, how many and which. A choice often set by familiarity or laboratory convention becomes one measured on each task, repeatable as new representations appear.

Summary of Research Contributions

Research Papers

In preparation: Punt, A., Morris, G., Werner, S., & Hanser, T. (2026). Improving noise prediction robustness and mitigation with feature-importance hybrid representations in multi-modal contexts
A draft is in preparation, and the experiments of Chapters 5 and 6 will be extended before it is submitted.

Conference Presentations

Punt, A., Morris, G., Werner, S., & Hanser, T. (2024). Investigating the impact and mitigation of noise in QSAR models. Presented at the Autumn UK-QSAR 2024 Meeting.

Blog Posts

The author has contributed to *Blopig*, the blog of OPIG:

- Are you addicted to dopamine?
- Demystifying Git and merge conflicts
- AI in Academic Writing: Ethical Considerations and Best Practices
- Memory-mapped files for efficient data processing
- Exploring multilingual programming

Bibliography

[1] Bruno J. Neves, Rodolpho C. Braga, and Cleber C. Melo-Filho et al. QSAR-based virtual screening: Advances and applications in drug discovery. *Frontiers in Pharmacology*, 9:1275, 2018. ISSN 1663-9812.

[2] María Virginia Sabando, Ignacio Ponzoni, Evangelos E Milios, and Axel J Soto. Using molecular embeddings in QSAR modeling: Does it make a difference? *Briefings in Bioinformatics*, 23(1):bbab365, 2022. ISSN 1477-4054.

[3] David Rogers and Mathew Hahn. Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5):742–754, 2010. doi: 10.1021/ci100050t. PMID: 20426451.

[4] Luís David, Abhijit Thakkar, Raul Mercado, et al. Molecular representations in AI-driven drug discovery: a review and practical guide. *Journal of Cheminformatics*, 12(1):56, 2020.

[5] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Sort & slice: a simple and superior alternative to hash-based folding for extended-connectivity fingerprints. *Journal of Cheminformatics*, 16(1), December 2024. ISSN 1758-2946. doi: 10.1186/s13321-024-00932-y.

[6] Peter Gedeck, Bernhard Rohde, and Christian Bartels. Qsar – how good is it in practice? comparison of descriptor sets on an unbiased cross section of corporate data sets. *Journal of Chemical Information and Modeling*, 46(5): 1924–1936, 2006. doi: 10.1021/ci050413p.

[7] Akihiro Kishimoto, Hiroshi Kajino, Masataka Hirose, Junta Fuchiwaki, Indra Priyadarsini, Lisa Hamada, Hajime Shinohara, Daiju Nakano, and Seiji Takeda. Mhg-gnn: Combination of molecular hypergraph grammar with graph neural network, 2023.

[8] Scott S. Kolmar and Christopher M. Grulke. The effect of noise on the predictive limit of QSAR models. *Journal of Cheminformatics*, 13(1):92, 2021.

[9] Kjell Jorner, Tore Brinck, Per-Ola Norrby, and David Buttar. Machine learning meets mechanistic modelling for accurate prediction of experimental activation energies. *Chem. Sci.*, 12:1163–1175, 2021. doi: 10.1039/D0SC04896H.

[10] Christian Kramer, Tuomo Kalliokoski, Peter Gedeck, and Anna Vulpetti. The experimental uncertainty of heterogeneous public Ki data. *Journal of Medicinal Chemistry*, 55(11):5165–5173, 2012. doi: 10.1021/jm300131x.

[11] Tuomo Kalliokoski, Christian Kramer, Anna Vulpetti, and Peter Gedeck. Comparability of mixed IC50 data – a statistical analysis. *PLoS ONE*, 8 (4):e61007, 2013. doi: 10.1371/journal.pone.0061007.

[12] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify: On the importance of chemical structure curation in cheminformatics and qsar modeling research. *Journal of Chemical Information and Modeling*, 50 (7):1189–1204, 2010. doi: 10.1021/ci100176x. PMID: 20572635.

[13] W. Patrick Walters. We need better benchmarks for machine learning in drug discovery. Practical Cheminformatics Blog, 2023. URL http://practicalcheminformatics.blogspot.com/2023/08/we-need-better-benchmarks-for-machine.html. Accessed: 2026-04-12.

[14] Gregory A. Landrum and Sereina Riniker. Combining IC50 or Ki values from different sources is a source of significant noise. *Journal of Chemical Information and Modeling*, 64(5):1560–1567, 2024. doi: 10.1021/acs.jcim.4c00049. PMID: 38394344.

[15] Isidro Cortes-Ciriano, Andreas Bender, and Thérèse E. Malliavin. Comparing the influence of simulated experimental errors on 12 machine learning algorithms in bioactivity modeling using 12 diverse data sets. *Journal of Chemical Information and Modeling*, 55(7):1413–1425, 2015. doi: 10.1021/acs.jcim.5b00101. PMID: 26038978.

[16] Davide Bassani, Alessandro Brigo, and Andrea Andrews-Morger. Federated

learning in computational toxicology: An industrial perspective on the effiris hackathon. *Chemical Research in Toxicology*, 36(9):1503–1517, 2023.

[17] Thierry Hanser. Federated learning for molecular discovery. *Current Opinion in Structural Biology*, 79:102545, 2023. ISSN 0959-440X.

[18] David Weininger. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1):31–36, 1988.

[19] Michael Backenköhler, Paula Linh Kramer, and Joschka et al. Groß. TeachOpenCADD goes Deep Learning: Open-source Teaching Platform Exploring Molecular DL Applications. *ChemRxiv preprint*, 2023. doi: 10.26434/chemrxiv-2023-kz1pb.

[20] Alexandre V. Fassio, Laura Shub, and Luca Ponzoni et al. Prioritizing virtual screening with interpretable interaction fingerprints. *Journal of Chemical Information and Modeling*, 62(18):4300–4318, 2022.

[21] Liangxu Xie, Lei Xu, Ren Kong, Shan Chang, and Xiaojun Xu. Improvement of prediction performance with conjoint molecular fingerprint in deep learning. *Frontiers in Pharmacology*, 11, 2020. doi: 10.3389/fphar.2020.606668.

[22] Isidro Cortés-Ciriano, Ctibor Škuta, Andreas Bender, and Daniel Svozil. QSAR-derived affinity fingerprints (part 2): Modeling performance for potency prediction. *Journal of Cheminformatics*, 12(1):41, 2020.

[23] Sabrina Jaeger, Simone Fulle, and Samo Turk. Mol2vec: Unsupervised machine learning approach with chemical intuition. *Journal of Chemical Information and Modeling*, 58(1):27–35, 2018. doi: 10.1021/acs.jcim.7b00616.

[24] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta: Large-scale self-supervised pretraining for molecular property prediction. *arXiv preprint arXiv:2010.09885*, 2020.

[25] Benedek Fabian, Thomas Edlich, and Héléna et al. Gaspar. Molecular representation learning with language models and domain-relevant auxiliary tasks. *arXiv preprint arXiv:2011.13230*, 2020.

[26] Walid Ahmad, Elana Simon, Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. Chemberta-2: Towards chemical foundation models. *arXiv preprint arXiv:2209.01712*, 2022.

[27] Roberto Todeschini and Viviana Consonni. *Molecular Descriptors for Chemoinformatics*. Wiley-VCH, Weinheim, 2nd edition, 2009. doi: 10.1002/9783527628766.

[28] Andrea Mauri, Viviana Consonni, Manuela Pavan, and Roberto Todeschini. DRAGON software: An easy approach to molecular descriptor calculations. *MATCH Communications in Mathematical and in Computer Chemistry*, 56 (2):237–248, 2006.

[29] Artem Cherkasov, Eugene N. Muratov, and Denis et al. Fourches. QSAR modeling: Where have you been? where are you going to? *Journal of Medicinal Chemistry*, 57(12):4977–5010, 2014. doi: 10.1021/jm4004285.

[30] Greg Landrum. Rdkit: Open-source cheminformatics. https://www.rdkit.org. Accessed: 2026-01-09.

[31] Zhichun Guo, Kehan Guo, and Bozhao Nan et al. Graph-based molecular representation learning, 2023. Accessed: 2024-11-30.

[32] Stefan M.N. Hönig, Carsten Lemmen, and Matthias Rarey. Small molecule superposition: A comprehensive overview on pose scoring of the latest methods. *WIREs Computational Molecular Science*, 13(2):e1640, 2023.

[33] Ge Yan, Huaijin Wu, and Junchi Yan. Quantum 3D graph learning with applications to molecule embedding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 39126–39137. PMLR, 2023.

[34] Ziduo Yang, Weihe Zhong, Qiujie Lv, Tiejun Dong, and Calvin Yu-Chian Chen. Geometric interaction graph neural network for predicting protein–ligand binding affinities from 3D structures (GIGN). *The Journal of Physical Chemistry Letters*, 14(8):2020–2033, 2023. doi: 10.1021/acs.jpclett.2c03906.

[35] Wenjie Du, Xiaoting Yang, and Di et al. Wu. Fusing 2D and 3D molecular graphs as unambiguous molecular descriptors for conformational and chiral

stereoisomers. *Briefings in Bioinformatics*, 24(1):bbac560, 2022. ISSN 1477-4054.

[36] Denis Kuzminykh, Daniil Polykovskiy, and Artur Kadurin et al. 3D molecular representations based on the wave transform for convolutional neural networks. *Molecular Pharmaceutics*, 15(10):4378–4385, 2018.

[37] Giorgos Bouritsas, Fabrizio Frasca, Stefanos Zafeiriou, and Michael M. Bronstein. Improving graph neural network expressivity via subgraph isomorphism counting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(1):657–668, 2023.

[38] Jerret Ross, Brian Belgodere, Vijil Chenthamarakshan, Inkit Padhi, Youssef Mroueh, and Payel Das. Large-scale chemical language representations capture molecular structure and properties, 2022.

[39] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Wadsworth & Brooks/Cole, Monterey, CA, 1984. ISBN 978-0-412-04841-8.

[40] Vladimir Svetnik, Andy Liaw, Christopher Tong, John Culberson, Robert Sheridan, and Bradley Feuston. Random forest: A classification and regression tool for compound classification and qsar modeling. *Journal of chemical information and computer sciences*, 43:1947–58, 11 2003. doi: 10.1021/ci034160g.

[41] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

[42] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. doi: 10.1038/323533a0.

[43] Junshui Ma, Robert P Sheridan, Andy Liaw, George E Dahl, and Vladimir Svetnik. Deep neural nets as a method for quantitative structure-activity relationships. *Journal of Chemical Information and Modeling*, 55(2):263–274, 2015. doi: 10.1021/ci500747n.

[44] Igor I. Baskin, Vladimir A. Palyulin, and Nikolai S. Zefirov. Neural networks in building QSAR models. In *Methods in Molecular Biology*, volume 458, pages 137–158. Humana Press, 2008. doi: 10.1007/978-1-60327-101-1_8.

[45] Alexios Koutsoukas, Keith J. Monaghan, Xiaoli Li, and Jun Huan. Deep-learning: investigating deep neural networks hyper-parameters and comparison of performance to shallow methods for modeling bioactivity data. *Journal of Cheminformatics*, 9:42, 2017. doi: 10.1186/s13321-017-0226-y.

[46] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations (ICLR)*, 2017.

[47] Hwanjun Song, Minseok Kim, Dongmin Park, Yooju Shin, and Jae-Gil Lee. Learning from noisy labels with deep neural networks: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 34(11):8135–8153, 2023. doi: 10.1109/TNNLS.2022.3152527.

[48] Curtis G Northcutt, Lu Jiang, and Isaac L Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021. doi: 10.1613/jair.1.12125.

[49] Hongyi Zhang, Moustapha Cisse, Yann N. Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

[50] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jonathon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision, 2015. Section 7 introduces label-smoothing regularization.

[51] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*, 2017.

[52] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks?, 2019.

[53] Markus Dablander, Thierry Hanser, Renaud Lambiotte, and Garrett M. Morris. Exploring QSAR models for activity-cliff prediction. *Journal of Cheminformatics*, 15(1):47, 2023. doi: 10.1186/s13321-023-00708-w.

[54] Derek van Tilborg, Alisa Alenicheva, and Francesca Grisoni. Exposing the limitations of molecular machine learning with activity cliffs. *Jour-*

nal of Chemical Information and Modeling, 62(23):5938–5951, 2022. doi: 10.1021/acs.jcim.2c01073.

[55] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.

[56] Ismail Babajide Mustapha and Faisal Saeed. Bioactive molecule prediction using extreme gradient boosting. *Molecules*, 21(8), 2016. ISSN 1420-3049. doi: 10.3390/molecules21080983.

[57] Hao Tian, Rajas Ketkar, and Peng Tao. Accurate admet prediction with xgboost, 2022.

[58] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154, 2017.

[59] Tony Duan, Anand Avati, Daisy Yi Ding, Khanh K. Thai, Sanjay Basu, Andrew Y. Ng, and Alejandro Schuler. Ngboost: Natural gradient boosting for probabilistic prediction, 2020.

[60] Vladimir N. Vapnik. *The Nature of Statistical Learning Theory*. Springer, New York, 1995. doi: 10.1007/978-1-4757-2440-0.

[61] Alex J. Smola and Bernhard Schölkopf. A tutorial on support vector regression. *Statistics and Computing*, 14(3):199–222, 2004. doi: 10.1023/B:STCO.0000035301.49549.88.

[62] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.

[63] Chih-Chung Chang and Chih-Jen Lin. LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3): 1–27, 2011. doi: 10.1145/1961189.1961199.

[64] Ryan-Rhys Griffiths and Leo Klarner et al. Gauche: A library for gaussian processes in chemistry, 2022. Accessed: 2024-08-21.

[65] L. Ralaivola, S. Joshua Swamidass, Hiroto Saigo, and Pierre Baldi. Graph kernels for chemical informatics. *Neural networks : the official journal of the International Neural Network Society*, 18:1093–110, 11 2005. doi: 10.1016/j.neunet.2005.07.009.

[66] Henry Moss and Ryan-Rhys Griffiths. Gaussian process molecule property prediction with flowmo, 10 2020.

[67] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles, 2017.

[68] Glenn Palmer, Siqi Du, Alexander Politowicz, Joshua Emory, Xiyu Yang, Anupraas Gautam, Grishma Gupta, Zhelong Li, Ryan Jacobs, and Dane Morgan. Calibration after bootstrap for accurate uncertainty quantification in regression models. *npj Computational Materials*, 8:115, 05 2022. doi: 10.1038/s41524-022-00794-8.

[69] Mark A. Johnson and Gerald M. Maggiora, editors. *Concepts and Applications of Molecular Similarity*. Wiley, New York, 1990.

[70] Gerald Maggiora, Martin Vogt, Dagmar Stumpfe, and Jürgen Bajorath. Molecular similarity in medicinal chemistry. *Journal of Medicinal Chemistry*, 57(8):3186–3204, 2014. doi: 10.1021/jm401411z.

[71] Dávid Bajusz, Anita Rácz, and Károly Héberger. Why is tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7(1), May 2015. ISSN 1758-2946. doi: 10.1186/s13321-015-0069-3.

[72] Gerald M. Maggiora. On outliers and activity cliffs—why QSAR often disappoints. *Journal of Chemical Information and Modeling*, 46(4):1535, 2006. doi: 10.1021/ci060117s.

[73] Dagmar Stumpfe and Jürgen Bajorath. Exploring activity cliffs in medicinal chemistry. *Journal of Medicinal Chemistry*, 55(7):2932–2942, 2012. doi: 10.1021/jm201706b.

[74] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is "nearest neighbor" meaningful? In *Database Theory – ICDT'99*, volume

1540 of *Lecture Notes in Computer Science*, pages 217–235, Berlin, Heidelberg, 1999. Springer. doi: 10.1007/3-540-49257-7_15.

[75] Felix A. Krüger and John P. Overington. Global analysis of small molecule binding to related protein targets. *PLoS Computational Biology*, 8(1): e1002333, 2012. doi: 10.1371/journal.pcbi.1002333.

[76] Esther Heid, Charles J. McGill, Florence H. Vermeire, and William H. Green. Characterizing uncertainty in machine learning for chemistry. *Journal of Chemical Information and Modeling*, 63(13):4012–4029, 2023. doi: 10.1021/acs.jcim.3c00373. PMID: 37338239.

[77] Joe Bentz, Michael P. O'Connor, Dallas Bednarczyk, JoAnn Coleman, Caroline Lee, Johan Palm, Y. Anne Pak, Elke S. Perloff, Eric Reyner, Praveen Balimane, et al. Variability in P-glycoprotein inhibitory potency (IC50) using various in vitro experimental systems: Implications for universal digoxin drug-drug interaction risk assessment decision criteria. *Drug Metabolism and Disposition*, 41(7):1347–1366, 2013. doi: 10.1124/dmd.112.050500.

[78] Emma Svensson, Hannah Rosa Friesacher, Susanne Winiwarter, Lewis Mervin, Adam Arany, and Ola Engkvist. Enhancing uncertainty quantification in drug discovery with censored regression labels. *Artificial Intelligence in the Life Sciences*, 7:100128, 2025. doi: 10.1016/j.ailsci.2025.100128.

[79] James Inglese, Douglas S. Auld, and Ajit et a.. Jadhav. Quantitative high-throughput screening: A titration-based approach that efficiently identifies biological activities in large chemical libraries. *Proceedings of the National Academy of Sciences*, 103(31):11473–11478, 2006. doi: 10.1073/pnas.0604348103.

[80] Christine Brideau, Bert Gunter, Bill Pikounis, and Andy Liaw. Improved statistical methods for hit selection in high-throughput screening. *Journal of Biomolecular Screening*, 8(6):634–647, 2003. doi: 10.1177/1087057103258285.

[81] Ji-Hu Zhang, Thomas D. Y. Chung, and Kevin R. Oldenburg. A simple statistical parameter for use in evaluation and validation of high throughput screening assays. *Journal of Biomolecular Screening*, 4(2):67–73, 1999. doi: 10.1177/108705719900400206.

[82] Anton Simeonov, Ajit Jadhav, and Craig J. et al. Thomas. Fluorescence spectroscopic profiling of compound libraries. *Journal of Medicinal Chemistry*, 51(8):2363–2371, 2008. doi: 10.1021/jm701301m.

[83] Christos Hatzis, Philippe L. Bedard, Nicolai J. Birkbak, Andrew H. Beck, Hugo J.W.L. Aerts, David F. Stern, Leming Shi, Robert Clarke, John Quackenbush, and Benjamin Haibe-Kains. Enhancing reproducibility in cancer drug screening: How do we move forward? *Cancer Research*, 74(15):4016–4023, July 2014. ISSN 1538-7445. doi: 10.1158/0008-5472.can-14-0725.

[84] Elena Postnikova, Yu Cong, Lisa Evans DeWald, and Julie et al. Dyall. Testing therapeutics in cell-based assays: Factors that influence the apparent potency of drugs. *PLOS ONE*, 13(3):e0194880, March 2018. ISSN 1932-6203. doi: 10.1371/journal.pone.0194880.

[85] Anna Gaulton, Anne Hersey, and Michał et al. Nowotka. The ChEMBL database in 2017. *Nucleic Acids Research*, 45(D1):D945–D954, 2017. doi: 10.1093/nar/gkw1074.

[86] Michael K Gilson, Tiqing Liu, Michael Baitaluk, and et al. BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1):D1045–D1053, 2016. doi: 10.1093/nar/gkv1072.

[87] Denis Fourches, Eugene Muratov, and Alexander Tropsha. Trust, but verify II: A practical guide to chemogenomics data curation. *Journal of Chemical Information and Modeling*, 56(7):1243–1252, 2016. doi: 10.1021/acs.jcim.6b00129.

[88] Douglas Young, Todd Martin, Raghuraman Venkatapathy, and Phil Harten. Are the chemical structures in your QSAR correct? *QSAR & Combinatorial Science*, 27(11-12):1337–1345, 2008. doi: 10.1002/qsar.200810084.

[89] A Patrícia Bento, Anne Hersey, Eloy Félix, and Greg et al. Landrum. An open source chemical structure curation pipeline using RDKit. *Journal of Cheminformatics*, 12(1):51, 2020. doi: 10.1186/s13321-020-00456-1.

[90] Insha Ullah and A. H. Welsh. On the effect of noise on fitting linear regression models, 2024.

[91] Alexander Camuto, Matthew Willetts, Umut Şimşekli, Stephen Roberts, and Chris Holmes. Explicit regularisation in gaussian noise injections, 2021.

[92] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

[93] Frank R. Hampel. Robust statistics: a brief introduction and overview. Research Report 94, Seminar für Statistik, Eidgenössische Technische Hochschule (ETH) Zürich, 2001.

[94] Alex Avdeef. Multi-lab intrinsic solubility measurement reproducibility in CheqSol and shake-flask methods. *ADMET and DMPK*, 7(3):210–219, 2019. doi: 10.5599/admet.698.

[95] Bo Han, Quanming Yao, and Xingrui Yu et al. Co-teaching: Robust training of deep neural networks with extremely noisy labels. 2018. Accessed: 2024-09-01.

[96] Jun Shu, Qi Xie, Lixuan Yi, Qian Zhao, Sanping Zhou, Zongben Xu, and Deyu Meng. Meta-weight-net: Learning an explicit mapping for sample weighting. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[97] David H. Wolpert. The lack of a priori distinctions between learning algorithms. *Neural Computation*, 8(7):1341–1390, 1996. doi: 10.1162/neco.1996.8.7.1341.

[98] Dennis L. Wilson. Asymptotic properties of nearest neighbor rules using edited data. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-2(3):408–421, 1972. doi: 10.1109/TSMC.1972.4309137.

[99] Adelaide Punt. NoiseInject: A Python package for label-noise robustness benchmarking in QSAR models. https://github.com/adpunt/noise_inject, 2026. Accessed: 2026-06-01.

[100] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, 2016.

[101] Jianyuan Deng, Zhibo Yang, and Hehe Wang et al. Unraveling key elements underlying molecular property prediction: A systematic study, 2023.

[102] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, et al. Quantum chemistry structures and properties of 134 kilo molecules. *Scientific Data*, 1: 140022, 2014.

[103] Open ADMET Consortium. OpenADMET-ExpansionRx challenge dataset, 2026. URL https://huggingface.co/datasets/openadmet/openadmet-expansionrx-challenge-data. CC-BY-4.0. ADMET data from drug discovery campaigns by Expansion Therapeutics targeting RNA-mediated diseases. Challenge closed January 19, 2026.

[104] Barbara Zdrazil, Eloy Felix, Fiona Hunter, et al. The ChEMBL database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research*, 52(D1):D1180–D1192, 11 2023. ISSN 0305-1048. doi: 10.1093/nar/gkad1004.

[105] Nicolai Meinshausen. Quantile regression forests. *Journal of Machine Learning Research*, 7:983–999, 06 2006.

[106] Olga Obrezanova, Joelle M.R. Gola, and Matthew D. Segall. Gaussian processes: A method for automatic qsar and adme modelling. Presentation at ACS Boston, 22 August 2007, 2007.

[107] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. The MIT Press, 2005. ISBN 9780262256834.

[108] Joaquin Quiñonero-Candela and Carl Edward Rasmussen. A unifying view of sparse approximate gaussian process regression. *Journal of Machine Learning Research*, 6(65):1939–1959, 2005.

[109] Jacob R. Gardner, Geoff Pleiss, Kilian Q. Weinberger, David Bindel, and Andrew Gordon Wilson. GPyTorch: Blackbox matrix-matrix Gaussian process inference with GPU acceleration. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[110] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang,

Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://papers.nips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[111] Alex Kendall and Yarin Gal. What uncertainties do we need in bayesian deep learning for computer vision? *CoRR*, abs/1703.04977, 2017.

[112] James Harrison, John Willes, and Jasper Snoek. Variational bayesian last layers, 2024.

[113] Iurie Caraus, Abdulaziz A. Alsuwailem, Robert Nadon, and Vladimir Makarenkov. Detecting and overcoming systematic bias in high-throughput screening technologies: a comprehensive review of practical issues and methodological solutions. *Briefings in Bioinformatics*, 16(6):974–986, 03 2015. ISSN 1467-5463. doi: 10.1093/bib/bbv004.

[114] Daniel Crusius, Flaviu Cipcigan, and Philip Biggin. Are we fitting data or noise? analysing the predictive power of commonly used datasets in drug-, materials-, and molecular-discovery. *Faraday Discuss.*, 2024. doi: 10.26434/chemrxiv-2024-z0pz7.

[115] Markus Orsi and Jean-Louis Reymond. One chiral fingerprint to find them all. *Journal of Cheminformatics*, 16:53, 2024. doi: 10.1186/s13321-024-00849-6.

[116] Lien Ai Nguyen, Hua He, and Chuong Pham-Huy. Chiral drugs: an overview. *International Journal of Biomedical Science*, 2(2):85–100, 2006. PMID: 23674971; PMCID: PMC3614593.

[117] Alexander Golbraikh, Eugene Muratov, Denis Fourches, and Alexander Tropsha. Data set modelability by QSAR. *Journal of Chemical Information and Modeling*, 54(1):1–4, 2014. doi: 10.1021/ci400572x.

[118] Robert P. Sheridan, Prabha Karnachi, Matthew Tudor, Yuting Xu, Andy Liaw, Falgun Shah, Alan C. Cheng, Elizabeth Joshi, Meir Glick, and Juan Alvarez. Experimental error, kurtosis, activity cliffs, and methodology: What limits the predictivity of quantitative structure–activity relationship models? *Journal of Chemical Information and Modeling*, 60(4):1969–1982, 2020. doi: 10.1021/acs.jcim.9b01067.

[119] José L. Medina-Franco. Activity cliffs: Facts or artifacts? *Chemical Biology & Drug Design*, 81(5):553–556, 2013. doi: 10.1111/cbdd.12115.

[120] Sereina Riniker and Gregory A. Landrum. Open-source platform to benchmark fingerprints for ligand-based virtual screening. *Journal of Cheminformatics*, 5:26, 2013. doi: 10.1186/1758-2946-5-26.

[121] Mateusz Praski, Jakub Adamczyk, and Wojciech Czech. Benchmarking pretrained molecular embedding models for molecular representation learning, 2025.

[122] Florian Huber and Julian Pollmann. Count your bits: fingerprint benchmarking to assess broad chemical space representation. *Journal of Cheminformatics*, 18:83, 2026. doi: 10.1186/s13321-026-01245-y.

[123] Pawel Dabrowski-Tumanski, Bartosz Topolski, Dariusz Plewczynski, and Tomasz Jetka. The geometry of activity cliffs: Representation dependence and multi-scale characterization of activity landscapes, 2026.

[124] David E. Coupry and Péter Pogány. Application of deep metric learning to molecular graph similarity. *Journal of Cheminformatics*, 14(1):11, 2022.

[125] Weifan Zheng and Alexander Tropsha. Novel variable selection quantitative structure–property relationship approach based on the k-nearest-neighbor principle. *Journal of Chemical Information and Computer Sciences*, 40(1): 185–194, 2000. doi: 10.1021/ci980033m.

[126] Peter Itskowitz and Alexander Tropsha. k nearest neighbors QSAR modeling as a variational problem: Theory and applications. *Journal of Chemical Information and Modeling*, 45(3):777–785, 2005. doi: 10.1021/ci049628+.

[127] Rajarshi Guha and John H. Van Drie. Structure–activity landscape index: Identifying and quantifying activity cliffs. *Journal of Chemical Information and Modeling*, 48(3):646–658, 2008. doi: 10.1021/ci7004093.

[128] Kenneth López-Pérez, Juan F. Avellaneda-Tamayo, Lexin Chen, Edgar López-López, K. Eurídice Juárez-Mercado, José L. Medina-Franco, and Ramón Alain Miranda-Quintana. Molecular similarity: Theory, applications, and perspectives. *Artificial Intelligence Chemistry*, 2(2):100077, 2024. doi: 10.1016/j.aichem.2024.100077.

[129] Alice Capecchi, Daniel Probst, and Jean-Louis Reymond. One molecular fingerprint to rule them all: drugs, biomolecules, and the metabolome. *Journal of Cheminformatics*, 12(1):43, 2020. doi: 10.1186/s13321-020-00445-4.

[130] Noel M. O'Boyle and Roger A. Sayle. Comparing structural fingerprints using a literature-based similarity benchmark. *Journal of Cheminformatics*, 8:36, 2016. doi: 10.1186/s13321-016-0148-0.

[131] Charu C. Aggarwal, Alexander Hinneburg, and Daniel A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 420–434, Berlin, Heidelberg, 2001. Springer. doi: 10.1007/3-540-44503-X_27.

[132] Miloš Radovanović, Alexandros Nanopoulos, and Mirjana Ivanović. Hubs in space: Popular nearest neighbors in high-dimensional data. *Journal of Machine Learning Research*, 11:2487–2531, 2010. URL https://jmlr.org/papers/v11/radovanovic10a.html.

[133] David E. Patterson, Richard D. Cramer, Allen M. Ferguson, Robert D. Clark, and Laurence E. Weinberger. Neighborhood behavior: A useful concept for validation of "molecular diversity" descriptors. *Journal of Medicinal Chemistry*, 39(16):3049–3059, 1996. doi: 10.1021/jm960290n.

[134] Prasanta Chandra Mahalanobis. On the generalized distance in statistics. *Proceedings of the National Institute of Sciences of India*, 2(1):49–55, 1936.

[135] Alexander Tropsha. Best practices for QSAR model development, validation, and exploitation. *Molecular Informatics*, 29(6-7):476–488, 2010. doi: 10.1002/minf.201000061.

[136] Darko Butina. Unsupervised data base clustering based on daylight's fingerprint and tanimoto similarity: A fast and automated way to cluster small and large data sets. *Journal of Chemical Information and Computer Sciences*, 39 (4):747–750, 1999. doi: 10.1021/ci9803381.

[137] Thierry Hanser, Ernst Ahlberg, Alexander Amberg, Lennart T. Anger, Chris Barber, Richard J. Brennan, Alessandro Brigo, Annie Delaunois, Susanne

Glowienke, Nigel Greene, Laura Johnston, Daniel Kuhn, Lara Kuhnke, Jean-François Marchaland, Wolfgang Muster, Jeffrey Plante, Friedrich Rippmann, Yogesh Sabnis, Friedemann Schmidt, Ruud van Deursen, Stéphane Werner, Angela White, Joerg Wichard, and Tomoya Yukawa. Data-driven federated learning in drug discovery with knowledge distillation. *Nature Machine Intelligence*, 7(3):423–436, 2025. doi: 10.1038/s42256-025-00991-2.

[138] Junnan Li, Richard Socher, and Steven C.H. Hoi. Dividemix: Learning with noisy labels as semi-supervised learning. In *International Conference on Learning Representations*, 2020.

[139] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, ICML '09, pages 41–48. ACM, 2009.

[140] Kristian Kersting, Christian Plagemann, Patrick Pfaff, and Wolfram Burgard. Most likely heteroscedastic gaussian process regression. In *Proceedings of the 24th International Conference on Machine Learning*, pages 393–400, 2007.

[141] Carla E. Brodley and Mark A. Friedl. Identifying mislabeled training data. *Journal of Artificial Intelligence Research*, 11:131–167, 1999. doi: 10.1613/jair.606.

[142] Benoît Frénay and Michel Verleysen. Classification in the presence of label noise: A survey. *IEEE Transactions on Neural Networks and Learning Systems*, 25(5):845–869, 2014. doi: 10.1109/TNNLS.2013.2292894.

[143] Juan Martín, José A. Sáez, and Emilio Corchado. On the regressand noise problem: Model robustness and synergy with regression-adapted noise filters. *IEEE Access*, 9:145800–145816, 2021. doi: 10.1109/ACCESS.2021.3123151.

[144] Dejun Jiang, Zhenxing Wu, Chang-Yu Hsieh, Guangyong Chen, Ben Liao, Zhe Wang, Chao Shen, Dongsheng Cao, Jian Wu, and Tingjun Hou. Could graph neural networks learn better molecular representation for drug discovery? A comparison study of descriptor-based and graph-based models. *Journal of Cheminformatics*, 13(1):12, 2021. doi: 10.1186/s13321-020-00479-8.

[145] Andreas Mayr, Günter Klambauer, Thomas Unterthiner, Marvin Steijaert, Jörg K. Wegner, Hugo Ceulemans, Djork-Arné Clevert, and Sepp Hochre-

iter. Large-scale comparison of machine learning methods for drug target prediction on ChEMBL. *Chemical Science*, 9(24):5441–5451, 2018. doi: 10.1039/C8SC00148K.

[146] Henry Kenlay, Frédéric A. Dreyer, Aleksandr Kovaltsuk, Dom Miketa, Douglas Pires, and Charlotte M. Deane. Large scale paired antibody language models. *PLOS Computational Biology*, 20(12):e1012646, 2024. doi: 10.1371/journal.pcbi.1012646.

[147] Peter Neidlinger, Omar S. M. El Nahhas, Hannah Sophie Muti, Tim Lenz, Michael Hoffmeister, Hermann Brenner, Marko van Treeck, Rupert Langer, Bastian Dislich, Hans Michael Behrens, Christoph Röcken, Sebastian Foersch, Daniel Truhn, Antonio Marra, Oliver Lester Saldanha, and Jakob Nikolas Kather. Benchmarking foundation models as feature extractors for weakly supervised computational pathology. *Nature Biomedical Engineering*, 10(6): 1113–1123, 2025. doi: 10.1038/s41551-025-01516-3.

[148] Gordon F. Hughes. On the mean accuracy of statistical pattern recognizers. *IEEE Transactions on Information Theory*, 14(1):55–63, 1968. doi: 10.1109/TIT.1968.1054102.

[149] Richard E. Bellman. *Adaptive Control Processes: A Guided Tour*. Princeton University Press, Princeton, NJ, 1961. doi: 10.1515/9781400874668.

[150] Sören Richard Stahlschmidt, Benjamin Ulfenborg, and Jane Synnergren. Multimodal deep learning for biomedical data fusion: a review. *Briefings in Bioinformatics*, 23(2):bbab569, 2022. doi: 10.1093/bib/bbab569.

[151] Sandra Steyaert, Marija Pizurica, Divya Nagaraj, Priya Khandelwal, Tina Hernandez-Boussard, Andrew J. Gentles, and Olivier Gevaert. Multimodal data fusion for cancer biomarker discovery with deep learning. *Nature Machine Intelligence*, 5(4):351–362, 2023. doi: 10.1038/s42256-023-00633-5.

[152] Tongxin Wang, Wei Shao, Zhi Huang, Haixu Tang, Jie Zhang, Zhengming Ding, and Kun Huang. MOGONET integrates multi-omics data using graph convolutional networks allowing patient classification and biomarker identification. *Nature Communications*, 12(1):3445, 2021. doi: 10.1038/s41467-021-23774-w.

[153] Aurélien Beaude, Franck Augé, Farida Zehraoui, and Blaise Hanczar. CrossAttOmics: multiomics data integration with cross-attention. *Bioinformatics*, 41 (6):btaf302, 2025. doi: 10.1093/bioinformatics/btaf302.

[154] Mehmet Gönen and Ethem Alpaydın. Multiple kernel learning algorithms. *Journal of Machine Learning Research*, 12:2211–2268, 2011. URL https://www.jmlr.org/papers/v12/gonen11a.html.

[155] Anna Cichonska, Tapio Pahikkala, Sandor Szedmak, Heli Julkunen, Antti Airola, Markus Heinonen, Tero Aittokallio, and Juho Rousu. Learning with multiple pairwise kernels for drug bioactivity prediction. *Bioinformatics*, 34 (13):i509–i518, 2018. doi: 10.1093/bioinformatics/bty277.

[156] Oliver Laufkötter, Noé Sturm, Jürgen Bajorath, Hongming Chen, and Ola Engkvist. Combining structural and bioactivity-based fingerprints improves prediction performance and scaffold hopping capability. *Journal of Cheminformatics*, 11(1), August 2019. ISSN 1758-2946. doi: 10.1186/s13321-019-0376-1.

[157] Viktoriia Turkina, Melanie R.W. Messih, Etienne Kant, and Jelle T. et al. Gringhuis. Enhancing predictive modeling with molecular fingerprint fusion strategies. July 2025. doi: 10.26434/chemrxiv-2025-b3lfq.

[158] Cecile Valsecchi, Francesca Grisoni, Viviana Consonni, and Davide Ballabio. Consensus versus individual QSARs in classification: Comparison on a large-scale case study. *Journal of Chemical Information and Modeling*, 60(3):1215–1223, 2020. doi: 10.1021/acs.jcim.9b01057.

[159] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: a review and recent developments. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 374(2065):20150202, 2016. doi: 10.1098/rsta.2015.0202.

[160] Ian T. Jolliffe. A note on the use of principal components in regression. *Journal of the Royal Statistical Society Series C: Applied Statistics*, 31(3):300–303, 1982. doi: 10.2307/2348005.

[161] Svante Wold, Michael Sjöström, and Lennart Eriksson. PLS-regression: a basic tool of chemometrics. *Chemometrics and Intelligent Laboratory Systems*, 58 (2):109–130, 2001. doi: 10.1016/S0169-7439(01)00155-1.

[162] Isabelle Guyon and André Elisseeff. An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3:1157–1182, 2003. URL https://www.jmlr.org/papers/v3/guyon03a.html.

[163] Hanchuan Peng, Fuhui Long, and Chris Ding. Feature selection based on mutual information criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27(8):1226–1238, 2005. doi: 10.1109/TPAMI.2005.159.

[164] Ron Kohavi and George H. John. Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2):273–324, 1997. doi: 10.1016/S0004-3702(97)00043-X.

[165] Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1):267–288, 1996. doi: 10.1111/j.2517-6161.1996.tb02080.x.

[166] Isabelle Guyon, Jason Weston, Stephen Barnhill, and Vladimir Vapnik. Gene selection for cancer classification using support vector machines. *Machine Learning*, 46:389–422, 2002. doi: 10.1023/A:1012487302797.

[167] Ming Yuan and Yi Lin. Model selection and estimation in regression with grouped variables. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 68(1):49–67, 2006. doi: 10.1111/j.1467-9868.2005.00532.x.

[168] Marcus Olivecrona, Thomas Blaschke, Ola Engkvist, and Hongming Chen. Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1):48, 2017. doi: 10.1186/s13321-017-0235-x.

[169] Christophe Ambroise and Geoffrey J. McLachlan. Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10):6562–6566, 2002. doi: 10.1073/pnas.102102699.

[170] Andrius Vabalas, Emma Gowen, Ellen Poliakoff, and Alexander J. Casson. Machine learning algorithm validation with a limited sample size. *PLOS ONE*, 14(11):e0224365, 2019. doi: 10.1371/journal.pone.0224365.

[171] Sudhir Varma and Richard Simon. Bias in error estimation when using cross-validation for model selection. *BMC Bioinformatics*, 7:91, 2006. doi: 10.1186/1471-2105-7-91.

[172] Gavin C. Cawley and Nicola L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11(70):2079–2107, 2010. URL https://www.jmlr.org/papers/v11/cawley10a.html.

[173] Jiayun Dong and Cynthia Rudin. Exploring the cloud of variable importance for the set of all good models. *Nature Machine Intelligence*, 2(12):810–824, 2020. doi: 10.1038/s42256-020-00264-0.

[174] Carolin Strobl, Anne-Laure Boulesteix, Achim Zeileis, and Torsten Hothorn. Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8:25, 2007. doi: 10.1186/1471-2105-8-25.

[175] Avanti Shrikumar, Peyton Greenside, and Anshul Kundaje. Learning important features through propagating activation differences. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3145–3153, 2017. URL https://proceedings.mlr.press/v70/shrikumar17a.html.

[176] Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1):56–67, 2020. doi: 10.1038/s42256-019-0138-9.

[177] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR)*, 2015. URL https://arxiv.org/abs/1412.6980.

[178] Narine Kokhlikyan, Vivek Miglani, Miguel Martin, Edward Wang, Bilal Alsallakh, Jonathan Reynolds, Alexander Melnikov, Natalia Kliushkina, Carlos Araya, Siqi Yan, and Orion Reblitz-Richardson. Captum: A unified and generic model interpretability library for PyTorch. *arXiv preprint arXiv:2009.07896*, 2020. URL https://arxiv.org/abs/2009.07896.

[179] Marco Ancona, Enea Ceolini, Cengiz Öztireli, and Markus Gross. Towards better understanding of gradient-based attribution methods for deep neural networks. In *International Conference on Learning Representations (ICLR)*, 2018. URL https://arxiv.org/abs/1711.06104.

[180] Malkeet Singh Bahia, Omer Kaspi, Meir Touitou, Idan Binayev, Seema Dhail, Jacob Spiegel, Netaly Khazanov, Abraham Yosipof, and Hanoch Senderowitz. A comparison between 2D and 3D descriptors in QSAR modeling based on bio-active conformations. *Molecular Informatics*, 42(4):e2200186, 2023. doi: 10.1002/minf.202200186.

[181] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

[182] Bradley Efron, Trevor Hastie, Iain Johnstone, and Robert Tibshirani. Least angle regression. *The Annals of Statistics*, 32(2):407–499, 2004. doi: 10.1214/009053604000000067.

[183] Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*, volume 51 of *Proceedings of Machine Learning Research*, pages 240–248, 2016. URL https://proceedings.mlr.press/v51/jamieson16.html.

[184] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18(185):1–52, 2018. URL https://jmlr.org/papers/v18/16-558.html.

[185] Cheng Fang, Ye Wang, and Richard et al. Grater. Prospective validation of machine learning algorithms for absorption, distribution, metabolism, and excretion prediction: An industrial perspective. *Journal of Chemical Information and Modeling*, 63(11):3263–3274, 2023. doi: 10.1021/acs.jcim.3c00160. PMID: 37216672.

[186] Cas Wognum et al. Polaris: A benchmarking platform for drug discovery, 2024. URL https://doi.org/10.5281/zenodo.13652587.

[187] Raghunathan Ramakrishnan, Pavlo O. Dral, Matthias Rupp, and O. Anatole von Lilienfeld. Quantum chemistry structures and properties of 134 kilo molecules (QM9 dataset). https://doi.org/10.6084/m9.figshare.978904, 2014. Accessed: 2026-06-02.

[188] Derek T. Ahneman, Jesús G. Estrada, Shishi Lin, Spencer D. Dreher, and Abigail G. Doyle. Predicting reaction performance in C–N cross-coupling using machine learning. *Science*, 360(6385):186–190, 2018. doi: 10.1126/science.aar5169.

[189] Patrick Koenig, Chingwei V. Lee, Benjamin T. Walters, Vasantharajan Janakiraman, Jeremy Stinson, Thomas W. Patapoff, and Germaine Fuh. Mutational landscape of antibody variable domains reveals a switch modulating the interdomain conformational dynamics and antigen binding. *Proceedings of the National Academy of Sciences*, 114(4):E486–E495, 2017. doi: 10.1073/pnas.1613231114.

[190] Michael Chungyoun, Jeffrey Ruffolo, and Jeffrey Gray. FLAb: Benchmarking deep learning methods for antibody fitness prediction. *bioRxiv*, 2024. doi: 10.1101/2024.01.13.575504.

[191] Mario S. Valdés-Tresanco, Mario E. Valdés-Tresanco, Esteban Molina-Abad, and Ernesto Moreno. NbThermo: a new thermostability database for nanobodies. *Database*, 2023:baad021, 2023. doi: 10.1093/database/baad021.

[192] David Prihoda, Jad Maamary, Andrew Waight, Veronica Juan, Laurence Fayadat-Dilman, Daniel Svozil, and Danny A. Bitton. BioPhi: A platform for antibody design, humanization, and humanness evaluation based on natural antibody repertoires and deep learning. *mAbs*, 14(1):2020203, 2022. doi: 10.1080/19420862.2021.2020203.

[193] Lewis Chinery, Alissa M. Hummer, Brij Bhushan Mehta, Rahmad Akbar, Puneet Rawat, Andrei Slabodkin, Khang Le Quy, Fridtjof Lund-Johansen, Victor Greiff, Jeliazko R. Jeliazkov, and Charlotte M. Deane. Baselining the buzz. Trastuzumab-HER2 affinity, and beyond. *bioRxiv*, 2024. doi: 10.1101/2024.03.26.586756. Version 1; later versions retitled "Simple computational methods can outperform deep learning in designing diverse, binder-enriched antibody libraries".

[194] Hugo Dalla-Torre, Liam Gonzalez, Javier Mendoza-Revilla, Nicolas Lopez Carranza, Adam Henryk Grzywaczewski, Francesco Oteri, Christian Dallago, Evan Trop, Bernardo P. de Almeida, Hassan Sirelkhatim, Guillaume Richard, Marcin Skwark, Karim Beguir, Marie Lopez, and Thomas

Pierrot. Nucleotide transformer: building and evaluating robust foundation models for human genomics. *Nature Methods*, 22(2):287–297, 2025. doi: 10.1038/s41592-024-02523-z.

[195] The Cancer Genome Atlas Research Network. Comprehensive molecular profiling of lung adenocarcinoma. *Nature*, 511(7511):543–550, 2014. doi: 10.1038/nature13385.

[196] The Cancer Genome Atlas Research Network. Comprehensive genomic characterization of squamous cell lung cancers. *Nature*, 489(7417):519–525, 2012. doi: 10.1038/nature11404.

[197] Mary J. Goldman, Brian Craft, Mim Hastie, Kristupas Repečka, Fran McDade, Akhil Kamath, Ayan Banerjee, Yunhai Luo, Dave Rogers, Angela N. Brooks, Jingchun Zhu, and David Haussler. Visualizing and interpreting cancer genomics data via the Xena platform. *Nature Biotechnology*, 38(6):675–678, 2020. doi: 10.1038/s41587-020-0546-8.

[198] The Cancer Genome Atlas Network. Comprehensive molecular portraits of human breast tumours. *Nature*, 490(7418):61–70, 2012. doi: 10.1038/nature11412.

[199] Joel S. Parker, Michael Mullins, Maggie C. U. Cheang, Samuel Leung, David Voduc, Tammi Vickery, Sherri Davies, Christiane Fauron, Xiaping He, Zhiyuan Hu, John F. Quackenbush, Inge J. Stijleman, Juan Palazzo, J. S. Marron, Andrew B. Nobel, Elaine Mardis, Torsten O. Nielsen, Matthew J. Ellis, Charles M. Perou, and Philip S. Bernard. Supervised risk predictor of breast cancer based on intrinsic subtypes. *Journal of Clinical Oncology*, 27(8):1160–1167, 2009. doi: 10.1200/JCO.2008.18.1370.

[200] Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, Andrew Palmer, Volker Settels, et al. Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8):3370–3388, 2019. doi: 10.1021/acs.jcim.9b00237.

[201] Izhar Wallach and Abraham Heifets. Most ligand-based classification benchmarks reward memorization rather than generalization. *Journal of Chemi-*

cal Information and Modeling, 58(5):916–932, 2018. doi: 10.1021/acs.jcim.7b00403.

[202] Luiz C. Vieira, Sophia Lin, and Claus O. Wilke. Intrinsic dataset features drive mutational effect prediction by protein language models. *bioRxiv*, 2026. doi: 10.64898/2026.03.08.710389. Preprint.

[203] Frank Hoonakker, Nicolas Lachiche, Alexandre Varnek, and Alain Wagner. A representation to apply usual data mining techniques to chemical reactions – illustration on the rate constant of SN2 reactions in water. *International Journal on Artificial Intelligence Tools*, 20(2):253–270, 2011. doi: 10.1142/S0218213011000140.

[204] Philippe Schwaller, Benjamin Hoover, Jean-Louis Reymond, Hendrik Strobelt, and Teodoro Laino. Extraction of organic chemistry grammar from unsupervised learning of chemical reactions. *Science Advances*, 7(15):eabe4166, 2021. doi: 10.1126/sciadv.abe4166.

[205] Philippe Schwaller, Daniel Probst, Alain C. Vaucher, Vishnu H. Nair, David Kreutter, Teodoro Laino, and Jean-Louis Reymond. Mapping the space of chemical reactions using attention-based neural networks. *Nature Machine Intelligence*, 3(2):144–152, 2021. doi: 10.1038/s42256-020-00284-w.

[206] Tatsuya Sagawa and Ryosuke Kojima. Reactiont5: a large-scale pre-trained model towards application of limited reaction data, 2023. URL https://arxiv.org/abs/2311.06708.

[207] Kangway V. Chuang and Michael J. Keiser. Comment on "predicting reaction performance in C–N cross-coupling using machine learning". *Science*, 362 (6416):eaat8603, 2018. doi: 10.1126/science.aat8603.

[208] S. Henikoff and J. G. Henikoff. Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22):10915–10919, 1992. doi: 10.1073/pnas.89.22.10915.

[209] Marie-Paule Lefranc, Christelle Pommié, Manuel Ruiz, Véronique Giudicelli, Elodie Foulquier, Lisa Truong, Valérie Thouvenin-Contet, and Gérard Lefranc. Imgt unique numbering for immunoglobulin and t cell receptor variable do-

mains and ig superfamily v-like domains. *Developmental & Comparative Immunology*, 27(1):55–77, 2003. doi: 10.1016/S0145-305X(02)00039-3.

[210] James Dunbar and Charlotte M. Deane. Anarci: antigen receptor numbering and receptor classification. *Bioinformatics*, 32(2):298–300, 2016. doi: 10.1093/bioinformatics/btv552.

[211] Susana Vinga and Jonas Almeida. Alignment-free sequence comparison—a review. *Bioinformatics*, 19(4):513–523, 2003. doi: 10.1093/bioinformatics/btg005.

[212] Zhihan Zhou, Yanrong Ji, Weijian Li, Pratik Dutta, Ramana Davuluri, and Han Liu. Dnabert-2: Efficient foundation model and benchmark for multi-species genome, 2023. URL https://arxiv.org/abs/2306.15006.

[213] Eric Nguyen, Michael Poli, Marjan Faizi, Armin Thomas, Callum Birch-Sykes, Michael Wornow, Aman Patel, Clayton Rabideau, Stefano Massaroli, Yoshua Bengio, Stefano Ermon, Stephen A. Baccus, and Chris Ré. Hyenadna: Long-range genomic sequence modeling at single nucleotide resolution, 2023. URL https://arxiv.org/abs/2306.15794.

[214] The Cancer Genome Atlas Research Network, John N Weinstein, Eric A Collisson, Gordon B Mills, Kenna R Mills Shaw, Brad A Ozenberger, Kyle Ellrott, Ilya Shmulevich, Chris Sander, and Joshua M Stuart. The cancer genome atlas pan-cancer analysis project. *Nature Genetics*, 45(10):1113–1120, 2013. doi: 10.1038/ng.2764.

[215] Raymond E. Carhart, Dennis H. Smith, and R. Venkataraghavan. Atom pairs as molecular features in structure-activity studies: definition and applications. *Journal of Chemical Information and Computer Sciences*, 25(2):64–73, 1985. doi: 10.1021/ci00046a002.

[216] Ramaswamy Nilakantan, Norman Bauman, J. Scott Dixon, and R. Venkataraghavan. Topological torsion: a new molecular descriptor for sar applications. comparison with other descriptors. *Journal of Chemical Information and Computer Sciences*, 27(2):82–85, 1987. doi: 10.1021/ci00054a008.

[217] Joseph L. Durant, Burton A. Leland, Douglas R. Henry, and James G. Nourse. Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*, 42(6):1273–1280, 2002. doi: 10.1021/ci010132r.

[218] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, and Karsten M. Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(77):2539–2561, 2011. URL https://www.jmlr.org/papers/v12/shervashidze11a.html.

[219] Hirotomo Moriwaki, Yu-Shi Tian, Norihito Kawashita, and Tatsuya Takagi. Mordred: a molecular descriptor calculator. *Journal of Cheminformatics*, 10 (1):4, 2018. doi: 10.1186/s13321-018-0258-y.

[220] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. Self-supervised graph transformer on large-scale molecular data, 2020. URL https://arxiv.org/abs/2007.02835.

[221] Shengchao Liu, Hanchen Wang, Weiyang Liu, Joan Lasenby, Hongyu Guo, and Jian Tang. Pre-training molecular graph representation with 3d geometry, 2022. URL https://arxiv.org/abs/2110.07728.

[222] Eduardo Soares, Emilio Vital Brazil, Victor Shirasuna, Dmitry Zubarev, Renato Cerqueira, and Kristin Schmidt. An open-source family of large encoder-decoder foundation models for chemistry. *Communications Chemistry*, 8, 2025. doi: 10.1038/s42004-025-01585-0.

[223] Xiao-Chen Zhang, Cheng-Kun Wu, Jia-Cai Yi, Xiang-Xiang Zeng, Can-Qun Yang, Ai-Ping Lu, Ting-Jun Hou, and Dong-Sheng Cao. Pushing the boundaries of molecular property prediction for drug discovery with multitask learning bert enhanced by smiles enumeration. *Research*, 2022, 2022. doi: 10.34133/research.0004.

[224] Atakan Yüksel, Erva Ulusoy, Atabey Ünlü, and Tunca Doğan. Selformer: molecular representation learning via selfies language models. *Machine Learning: Science and Technology*, 4(2):025035, 2023. doi: 10.1088/2632-2153/acdb30.

[225] Pedro J. Ballester and W. Graham Richards. Ultrafast shape recognition to search compound databases for similar molecular shapes. *Journal of Computational Chemistry*, 28(10):1711–1723, 2007. doi: 10.1002/jcc.20681.

[226] Adrian M. Schreyer and Tom Blundell. Usrcat: real-time ultrafast shape recognition with pharmacophoric constraints. *Journal of Cheminformatics*, 4 (1):27, 2012. doi: 10.1186/1758-2946-4-27.

[227] Matthias Rupp, Alexandre Tkatchenko, Klaus-Robert Müller, and O. Anatole von Lilienfeld. Fast and accurate modeling of molecular atomization energies with machine learning. *Physical Review Letters*, 108(5):058301, 2012. doi: 10.1103/PhysRevLett.108.058301.

[228] Zeming Lin, Halil Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, Wenting Lu, Nikita Smetanin, Robert Verkuil, Ori Kabeli, Yaniv Shmueli, Allan dos Santos Costa, Maryam Fazel-Zarandi, Tom Sercu, Salvatore Candido, and Alexander Rives. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023. doi: 10.1126/science.ade2574.

[229] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alexander Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. *bioRxiv*, 2021. doi: 10.1101/2021.07.09.450648.

[230] Thomas Hayes, Roshan Rao, Halil Akin, Nicholas J. Sofroniew, Deniz Oktay, Zeming Lin, Robert Verkuil, Vincent Q. Tran, Jonathan Deaton, Marius Wiggert, Rohil Badkundri, Irhum Shafkat, Jun Gong, Alexander Derry, Raul S. Molina, Neil Thomas, Yousuf A. Khan, Chetan Mishra, Carolyn Kim, Liam J. Bartie, Matthew Nemeth, Patrick D. Hsu, Tom Sercu, Salvatore Candido, and Alexander Rives. Simulating 500 million years of evolution with a language model. *Science*, 387(6736):850–858, 2025. doi: 10.1126/science.ads0018.

[231] Salvatore Candido, Thomas Hayes, Alexander Derry, Roshan Rao, Zeming Lin, Robert Verkuil, Bryan Z. Wu, Jin Sub Lee, Elise S. Bruguera, Jehan A. Keval, Mykhailo Kopylov, John E. Pak, Wesley Wu, Neil Thomas, Samson Mataraso, Alvin Hsu, Ashton C. Trotman-Grant, Kilian Fatras, Allan dos Santos Costa, Rohil Badkundri, Halil Akın, Deniz Oktay, Jonathan Deaton, Elizabeth Montabana, Hrishita Sitwala, Yue Yu, Marius Wiggert, Dylan Alexander

Carlin, Anthony W. Goering, Tomasz Blazejewski, McCullen Sandora, Michael Hla, Tina Z. Jia, Leon H. Kloker, Nicholas J. Sofroniew, Masatoshi Uehara, Jassi Pannu, Sharrol Bachas, Daniel S. Liu, Tom Sercu, and Alexander Rives. Language modeling materializes a world model of protein biology. *bioRxiv*, 2026. doi: 10.64898/2026.06.03.729735. Preprint, posted 4 June 2026.

[232] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Ghalia Rehawi, Yu Wang, Llion Jones, Tom Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, Debsindhu Bhowmik, and Burkhard Rost. Prottrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2022. doi: 10.1109/TPAMI.2021.3095381.

[233] Jin Su, Chenchen Han, Yuyang Zhou, Junjie Shan, Xibin Zhou, and Fajie Yuan. Saprot: Protein language modeling with structure-aware vocabulary. *bioRxiv*, 2023. doi: 10.1101/2023.10.01.560349.

[234] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. Addressing the antibody germline bias and its effect on language models for improved antibody design. *Bioinformatics*, 40(11), 2024. doi: 10.1093/bioinformatics/btae618.

[235] Jeffrey A. Ruffolo, Jeffrey J. Gray, and Jeremias Sulam. Deciphering antibody affinity maturation with language models and weakly supervised learning, 2021. URL https://arxiv.org/abs/2112.07782.

[236] Eunna Huh, Hyeonsu Lee, and Hyunjin Shin. Antibody foundational model: Ab-RoBERTa, 2025.

[237] Melissa Sanabria, Jonas Hirsch, Pierre M. Joubert, and Anna R. Poetsch. Dna language model grover learns sequence context in the human genome. *Nature Machine Intelligence*, 6(8):911–923, 2024. doi: 10.1038/s42256-024-00872-0.

[238] Raphael Mourad. Mistral-DNA: Large language model Mistral for DNA. https://github.com/raphaelmourad/Mistral-DNA, 2024. Code and model release, no paper; model RaphaelMourad/Mistral-DNA-v1-422M-hg38 on Hugging Face.

[239] Paul M. Sharp, Therese M. F. Tuohy, and Krzysztof R. Mosurski. Codon usage in yeast: cluster analysis clearly differentiates highly and lowly expressed

genes. *Nucleic Acids Research*, 14(13):5125–5143, 1986. doi: 10.1093/nar/14.13.5125.

[240] Christina V. Theodoris, Ling Xiao, Anant Chopra, Mark D. Chaffin, Zeina R. Al Sayed, Matthew C. Hill, Helene Mantineo, Elizabeth M. Brydon, Zexian Zeng, X. Shirley Liu, and Patrick T. Ellinor. Transfer learning enables predictions in network biology. *Nature*, 618(7965):616–624, 2023. doi: 10.1038/s41586-023-06139-9.

[241] Maxence Gélard, Guillaume Richard, Thomas Pierrot, and Paul-Henry Cournède. Bulkrnabert: Cancer prognosis from bulk rna-seq based language models. *bioRxiv*, 2024. doi: 10.1101/2024.06.18.599483.

[242] Craig H Mermel, Steven E Schumacher, Barbara Hill, Matthew L Meyerson, Rameen Beroukhim, and Gad Getz. Gistic2.0 facilitates sensitive and confident localization of the targets of focal somatic copy-number alteration in human cancers. *Genome Biology*, 12(4):R41, 2011. doi: 10.1186/gb-2011-12-4-r41.

[243] Jun Li, Yiling Lu, Rehan Akbani, Zhenlin Ju, Paul L Roebuck, Wenbin Liu, Ji-Yeon Yang, Bradley M Broom, Roeland G W Verhaak, David W Kane, Chris Wakefield, John N Weinstein, Gordon B Mills, and Han Liang. Tcpa: a resource for cancer functional proteomics data. *Nature Methods*, 10(11):1046–1047, 2013. doi: 10.1038/nmeth.2650.

[244] Tong Ding, Sophia J. Wagner, Andrew H. Song, Richard J. Chen, Ming Y. Lu, Andrew Zhang, Anurag J. Vaidya, Guillaume Jaume, Muhammad Shaban, Ahrong Kim, Drew F. K. Williamson, Bowen Chen, Cristina Almagro-Perez, Paul Doucet, Sharifa Sahai, Chengkuan Chen, Daisuke Komura, Akihiro Kawabe, Shumpei Ishikawa, Georg Gerber, Tingying Peng, Long Phi Le, and Faisal Mahmood. Multimodal whole slide foundation model for pathology, 2024. URL https://arxiv.org/abs/2411.19666.

[245] Hanwen Xu, Naoto Usuyama, Jaspreet Bagga, Sheng Zhang, Rajesh Rao, Tristan Naumann, Cliff Wong, Zelalem Gero, Javier González, Yu Gu, Yanbo Xu, Mu Wei, Wenhui Wang, Shuming Ma, Furu Wei, Jianwei Yang, Chunyuan Li, Jianfeng Gao, Jaylen Rosemon, Tucker Bower, Soohee Lee, Roshanthi Weerasinghe, Bill J. Wright, Ari Robicsek, Brian Piening, Carlo Bifulco,

Sheng Wang, and Hoifung Poon. A whole-slide foundation model for digital pathology from real-world data. *Nature*, 630(8015):181–188, 2024. doi: 10.1038/s41586-024-07441-w.

[246] Xiyue Wang, Sen Yang, Jun Zhang, Minghui Wang, Jing Zhang, Wei Yang, Junzhou Huang, and Xiao Han. Transformer-based unsupervised contrastive learning for histopathological image classification. *Medical Image Analysis*, 81: 102559, 2022. doi: 10.1016/j.media.2022.102559.

[247] Alexandre Filiot, Ridouane Ghermi, Antoine Olivier, Paul Jacob, Lucas Fidon, Alice Mac Kain, Charlie Saillard, and Jean-Baptiste Schiratti. Scaling self-supervised learning for histopathology with masked image modeling. *medRxiv*, 2023. doi: 10.1101/2023.07.21.23292757.

[248] Charlie Saillard, Rodolphe Jenatton, Felipe Llinares-López, Zelda Mariet, David Cahané, Eric Durand, and Jean-Philippe Vert. H-optimus-0, 2024. URL https://github.com/bioptimus/releases/tree/main/models/h-optimus/v0.

[249] Theo A. Knijnenburg, Linghua Wang, Michael T. Zimmermann, Nyasha Chambwe, Galen F. Gao, Andrew D. Cherniack, et al. Genomic and molecular landscape of DNA damage repair deficiency across The Cancer Genome Atlas. *Cell Reports*, 23(1):239–254.e6, 2018. doi: 10.1016/j.celrep.2018.03.076.

[250] Kyle Ellrott, Matthew H. Bailey, Gordon Saksena, Kyle R. Covington, Cyriac Kandoth, Chip Stewart, et al. Scalable open science approach for mutation calling of tumor exomes using multiple genomic pipelines. *Cell Systems*, 6(3): 271–281.e7, 2018. doi: 10.1016/j.cels.2018.03.002.

[251] Scott L. Carter, Kristian Cibulskis, Elena Helman, Aaron McKenna, Hui Shen, Travis Zack, Peter W. Laird, Robert C. Onofrio, Wendy Winckler, Barbara A. Weir, Rameen Beroukhim, David Pellman, Douglas A. Levine, Eric S. Lander, Matthew Meyerson, and Gad Getz. Absolute quantification of somatic DNA alterations in human cancer. *Nature Biotechnology*, 30(5):413–421, 2012. doi: 10.1038/nbt.2203.

[252] Vésteinn Thorsson, David L. Gibbs, Scott D. Brown, Denise Wolf, Dante S. Bortone, Tai-Hsien Ou Yang, et al. The immune landscape of cancer. *Immunity*, 48(4):812–830.e14, 2018. doi: 10.1016/j.immuni.2018.03.023.

[253] Aaron M. Newman, Chih Long Liu, Michael R. Green, Andrew J. Gentles, Weiguo Feng, Yue Xu, Chuong D. Hoang, Maximilian Diehn, and Ash A. Alizadeh. Robust enumeration of cell subsets from tissue expression profiles. *Nature Methods*, 12(5):453–457, 2015. doi: 10.1038/nmeth.3337.

[254] Tathiane M. Malta, Artem Sokolov, Andrew J. Gentles, Tomasz Burzykowski, Laila Poisson, John N. Weinstein, et al. Machine learning identifies stemness features associated with oncogenic dedifferentiation. *Cell*, 173(2):338–354.e15, 2018. doi: 10.1016/j.cell.2018.03.034.

[255] Minoru Toyota, Nita Ahuja, Mutsumi Ohe-Toyota, James G. Herman, Stephen B. Baylin, and Jean-Pierre J. Issa. CpG island methylator phenotype in colorectal cancer. *Proceedings of the National Academy of Sciences*, 96(15):8681–8686, 1999. doi: 10.1073/pnas.96.15.8681.

[256] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022. URL https://arxiv.org/abs/2207.08815.

[257] Davide Chicco and Giuseppe Jurman. The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation. *BMC Genomics*, 21(1):6, 2020. doi: 10.1186/s12864-019-6413-7.

[258] Yoshua Bengio and Yves Grandvalet. No unbiased estimator of the variance of K-fold cross-validation. *Journal of Machine Learning Research*, 5:1089–1105, 2004. URL https://www.jmlr.org/papers/v5/grandvalet04a.html.

[259] Sarah Nogueira, Konstantinos Sechidis, and Gavin Brown. On the stability of feature selection algorithms. *Journal of Machine Learning Research*, 18(174): 1–54, 2018. URL https://jmlr.org/papers/v18/17-514.html.

[260] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Series in Statistics. Springer, New York, 2 edition, 2009. doi: 10.1007/978-0-387-84858-7.

[261] Ivan Olier, Noureddin Sadawi, G. Richard Bickerton, Joaquin Vanschoren, Crina Grosan, Larisa Soldatova, and Ross D. King. Meta-QSAR: a large-scale

application of meta-learning to drug design and discovery. *Machine Learning*, 107(1):285–311, 2018. doi: 10.1007/s10994-017-5685-x.

[262] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 1273–1282, 2017. URL https://proceedings.mlr.press/v54/mcmahan17a.html.

[263] Wouter Heyndrickx, Lewis Mervin, Tobias Morawietz, Noé Sturm, Lukas Friedrich, Adam Zalewski, Anastasia Pentina, Lina Humbeck, Martijn Oldenhof, Ritsuya Niwayama, Peter Schmidtke, Nikolas Fechner, Jaak Simm, Adam Arany, Nicolas Drizard, Rama Jabal, Arina Afanasyeva, Regis Loeb, Shlok Verma, Simon Harnqvist, Matthew Holmes, Balazs Pejo, Maria Telenczuk, Nicholas Holway, Arne Dieckmann, Nicola Rieke, Friederike Zumsande, Djork-Arné Clevert, Michael Krug, Christopher Luscombe, Darren Green, Peter Ertl, Peter Antal, David Marcus, Nicolas Do Huu, Hideyoshi Fuji, Stephen Pickett, Gergely Acs, Eric Boniface, Bernd Beck, Yax Sun, Arnaud Gohier, Friedrich Rippmann, Ola Engkvist, Andreas H. Göller, Yves Moreau, Mathieu N. Galtier, Ansgar Schuffenhauer, and Hugo Ceulemans. MELLODDY: Cross-pharma federated learning at unprecedented scale unlocks benefits in QSAR without compromising proprietary information. *Journal of Chemical Information and Modeling*, 64(7):2331–2344, 2024. doi: 10.1021/acs.jcim.3c00799.

[264] Pietro Cassarà, Alberto Gotta, and Lorenzo Valerio. Federated feature selection for cyber-physical systems of systems. *IEEE Transactions on Vehicular Technology*, 71(9):9937–9950, 2022. doi: 10.1109/TVT.2022.3178612.

[265] Paul Czodrowski. herg me out. *Journal of Chemical Information and Modeling*, 53(9):2240–2251, 2013.

[266] Ryan J. Tibshirani. Degrees of freedom and model search. *Statistica Sinica*, 25(3):1265–1296, 2015. doi: 10.5705/ss.2014.147.

[267] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jörg Sander. Lof: identifying density-based local outliers. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, SIGMOD '00,

page 93–104, New York, NY, USA, 2000. Association for Computing Machinery. ISBN 1581132174. doi: 10.1145/342009.335388.

[268] Maximilian Seitzer, Arash Tavakoli, Dimitrije Antic, and Georg Martius. On the pitfalls of heteroscedastic uncertainty estimation with probabilistic neural networks, 2022.

[269] Jing Lei, Max G'Sell, Alessandro Rinaldo, Ryan J. Tibshirani, and Larry Wasserman. Distribution-free predictive inference for regression. *Journal of the American Statistical Association*, 113(523):1094–1111, 2018. doi: 10.1080/01621459.2017.1307116.

[270] Lood van Niekerk, Joshua Moller, Seth Ritter, Porfirio Quintero-Cadena, Ariel Cohen, Alexander Channing, Michael Chungyoun, Ulfert Rand, Ian Smith, Nikhil Bhatt, Emmanuel Pierre, Casey Harris, Yi Ao, Vincent Grippo, Jasmin Schwenk, Amos Rosenbaum, Lucas Allen, Naif Asi, Jiaqi Zhu, Ritika Singh, Sonia Sammi, Vikram Jadhav, Jan Dušek, Sarath Chandra, Andreea Badea, Nels Thorsteinson, Marissa Blalock, Jared Kim, Oliver Turnbull, Shreyas Kulkarni, Vivek Kohar, Abiy Gebremedhin, Charlotte M. Deane, Peter M. Tessier, and Ammar Arsiwala. 2025 Ginkgo Datapoints antibody developability competition outcomes: limited model performance and a call for data standardization. *mAbs*, 18(1):2634216, 2026. doi: 10.1080/19420862.2026.2634216.

[271] Alissa M. Hummer, Constantin Schneider, Lewis Chinery, and Charlotte M. Deane. Investigating the volume and diversity of data needed for generalizable antibody–antigen $\delta\delta g$ prediction. *Nature Computational Science*, 5(8):635–647, 2025. doi: 10.1038/s43588-025-00823-8.